Cheatsheet · 51 chapters on one page

Python cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

255 key ideas | 301 formulas | 357 tables | 284 tips and traps

STEP 1 Core Python · Getting started

1.1 What a program is and your first Python program

Programs, compiler and interpreter, CPython, REPL and scripts, print() with sep and end, keywords, indentation, error kinds, PEP 8.

KEY IDEAS

- **Program and language** A program is a list of exact instructions. Lexis is the vocabulary, syntax the grammar, semantics the meaning.
- **How CPython runs a file** Source is checked and compiled to bytecode, then the virtual machine runs it top to bottom. A SyntaxError anywhere stops everything.
- **print()** print(*values, sep=" ", end="\n"). str() on each value, sep between, end after. N = L + (n − 1)s + e.
- **Keywords and blocks** 35 keywords in 3.14, soft keywords match, case, type, _. # starts a comment. Indent blocks with 4 spaces.
- **Three errors** Syntax error before running, runtime error at the bad line, logic error never reported.

FORMULAS

The full form of print() Exam favourite

$$\text{print(*values, sep=" ", end="\n", file=sys.stdout, flush=False)}$$

values: any number of values. sep: text between them. end: text after the last one. file: where to write. flush: write at once.

Lesson: print(): showing results on the screen

Characters written by one print() Exam favourite

$$N = \sum \text{len(str}(v_i)) + (n - 1) \cdot \text{len(sep)} + \text{len(end)}$$

n values v_1 to v_n. There are n − 1 gaps between n values.

Lesson: print(): showing results on the screen

Characters from one print() Exam favourite

$$N = L + (n - 1) \cdot s + e$$

L = total length of all values as text, n = number of values, s = len(sep), e = len(end).

Lesson: The maths: counting what print() writes

Bytes in a plain English text file

$$\text{bytes} = \text{characters}$$

Each English letter, digit, space and new line is 1 byte in UTF-8. Letters from other scripts can take 2 to 4 bytes (Core 3.1).

Lesson: The maths: counting what print() writes

TABLES

Table: word · meaning · in everyday language

Word	Meaning	In everyday language
Lexis	The words and symbols a language allows (print, if, +, "text")	The vocabulary of English
Syntax	The grammar rules: how words may be put together	"I rice eat" breaks English grammar
Semantics	What a correct sentence means	"The stone ate the rice" has good grammar but makes no sense

Lesson: What a program is

Table: · compiler · interpreter

	Compiler	Interpreter
How it works	Translates the whole program first into a separate machine code file, then you run that file	Reads the program and runs it straight away, statement by statement
Like	Translating a full book into Hindi, printing it, then reading it	A live interpreter at a meeting who translates each sentence as it is spoken
When errors show	Most mistakes are found before anything runs	A mistake is found when Python reaches that line (except grammar mistakes, see below)
Speed	Usually faster to run	Usually slower, but you see results at once
Examples	C, C++, Go, Rust	Python, JavaScript in old browsers, shell scripts

Lesson: Compiler, interpreter and how Python runs your file

Table: python implementation · written in · where it is used

Python implementation	Written in	Where it is used
CPython	C	The standard one from python.org. This course uses CPython 3.14
PyPy	Python (RPython)	Has a just in time compiler, often several times faster on long loops
MicroPython	C	Small boards like the Raspberry Pi Pico and ESP32
Pyodide	CPython compiled to WebAssembly	Runs Python inside your web browser. The Run buttons on this page use it

Lesson: Compiler, interpreter and how Python runs your file

Table: way to run · what it is · good for

Way to run	What it is	Good for
Interactive mode (REPL)	Type python3 with no file. The prompt >>> waits for one line, runs it and shows the result at once	Quick tests, like a calculator
Script mode	Save lines in a .py file and run python3 file.py	Real programs you want to keep and run again
An editor or IDE	IDLE (comes with Python), VS Code, PyCharm, Thonny	Writing, running and debugging bigger programs
A notebook	Jupyter or Google Colab: code cells and notes in one page	Data science and learning

Lesson: Install Python and run your first program

Table: you write · the screen shows

You write	The screen shows
print("A", "B")	A B
print("A", "B", sep="")	AB
print("A", end=""), then print("B")	AB
print("A", "B", sep="-", end="!")	A-B!
print()	an empty line
print(3 + 4, "3 + 4")	7 3 + 4

Lesson: print(): showing results on the screen

Table: part · what it is · example

Part	What it is	Example
Keyword	A reserved word with a fixed job. Python 3.14 has 35 of them	if, else, for, while, def, class, return, True, None
Soft keyword	A word that is a keyword only in one place, and a normal name everywhere else	match, case, type, _
Statement	One complete instruction	price = 120
Comment	Text after # that Python ignores. It is a note for people	# add the tax
Indentation	Spaces at the start of a line. They show which lines belong to a block	4 spaces under an if
Docstring	A text in triple quotes at the top of a file or function that explains it	"""Works out the bill."""

Lesson: Keywords, statements, comments and indentation

Table: kind · when python finds it · example · what you see

Kind	When Python finds it	Example	What you see
Syntax error	Before anything runs, while checking the grammar of the whole file	print("Hi"	SyntaxError, and not one line has run
Runtime error (exception)	Only when the bad line runs	10 / 0, a misspelt name	A traceback. The lines before it have already run
Logic error	Never. Python cannot know what you meant	price + 2 instead of price * 2	A wrong answer and no message at all

Lesson: Three kinds of errors

Table: pep 8 rule · good · bad

PEP 8 rule	Good	Bad
Indent with 4 spaces	four spaces before print	a tab, or 2 spaces
Variable and function names: lower case with underscores (snake_case)	total_price	TotalPrice, totalprice
Class names: CapWords	BankAccount	bank_account
Constants: all capitals	MAX_SIZE = 100	maxsize = 100
One space around = and + in normal code	x = a + b	x=a+b
Lines no longer than 79 characters	short lines	one very long line
Two blank lines between top level functions	a gap you can see	everything packed together

Lesson: Writing code other people can read: PEP 8 and PEP 20

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2235	Add Two Integers	Easy	Arithmetic	Return num1 + num2. The point is to get used to a function that returns a value instead of printing it.
2469	Convert the Temperature	Easy	Formula	Kelvin = Celsius + 273.15 and Fahrenheit = Celsius × 1.80 + 32.00. Return both in a list.
2413	Smallest Even Multiple	Easy	Arithmetic	If n is already even, n itself works. If n is odd, the answer is 2 × n.
1108	Defanging an IP Address	Easy	String replace	Replace every dot with [.] using the string method replace().

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER Python is often called an interpreted language, but CPython does compile your code to bytecode first. Both words are partly true. The Advanced Python step 7 chapter "How Python runs" opens every one of these stages with an animation.

Lesson: Compiler, interpreter and how Python runs your file

COMMON MISTAKE In the REPL, typing 2 + 3 shows 5 without print(). In a .py file, nothing appears unless you call print(). Beginners often expect the REPL behaviour inside a file.

Lesson: Install Python and run your first program

IN THE EXAM sep and end must be given by name (keyword arguments). print("A", "B", "-") prints A B - with the dash as a third value, it does not change sep.

Lesson: print(): showing results on the screen

COMMON MISTAKE Python uses indentation where many languages use curly brackets. Mixing tabs and spaces, or using 3 spaces on one line and 4 on the next, gives IndentationError. PEP 8 says: always 4 spaces.

Lesson: Keywords, statements, comments and indentation

IN THE EXAM PCEP asks which names follow PEP 8 conventions. snake_case for variables and functions, CapWords for classes, UPPER_CASE for constants.

Lesson: Writing code other people can read: PEP 8 and PEP 20

1.2 Variables, values, types and numeral systems

Names and objects, the basic types and literals, numeral systems, type conversion, naming rules, multiple assignment.

KEY IDEAS

- **Names and objects** name = value binds a name to an object. b = a gives a second name to the same object. Objects have id, type, value.
- **Basic types** int (unlimited), float (about 15 to 17 digits), complex, bool (subclass of int), str, None. All immutable.
- **Bases** 0b, 0o, 0x literals. bin(), oct(), hex() to text. int(text, base) back. N = sum of $d_i\, b^i$.
- **Conversions** int() cuts towards zero, int("3.5") is ValueError, round() ties to even, bool("0") is True.
- **Naming** Letters, digits, _ , no leading digit, no keyword, case sensitive, snake_case, do not shadow built ins.

FORMULAS

Scientific notation

Exam favourite

$$a\ e\ b = a \times 10^b$$

1.5e3 = 1.5×10^3 = 1500.0. The result is always a float.

Lesson: The basic data types and their literals

Underscores in numbers

$$1_000_000 == 1000000$$

Since Python 3.6 (PEP 515) underscores can group digits. Python ignores them.

Lesson: The basic data types and their literals

Value of a number written in base b

Exam favourite

$$value = d_k b^k + ... + d_1 b^1 + d_0 b^0$$

Each digit is multiplied by the base to the power of its position, counting from 0 on the right.

Lesson: Numeral systems: binary, octal and hexadecimal

Digits needed in base 2

$$bits(n) = \lfloor \log_2 n \rfloor + 1$$

For n > 0. Python gives it as n.bit_length(). 156 needs 8 bits.

Lesson: Numeral systems: binary, octal and hexadecimal

Base b to decimal

Exam favourite

$$N = \sum_{i=0}^{k} d_i \cdot b^i$$

d_i is the digit at position i from the right.

Lesson: The maths: place value and conversions

Decimal to base b

Exam favourite

$$d_0 = N \bmod b,\ N \leftarrow N \mathbin{//} b, \text{repeat}$$

The remainders in reverse order are the digits.

Lesson: The maths: place value and conversions

Largest number in k bits

$$2^k - 1$$

8 bits hold 0 to 255. 16 bits hold 0 to 65,535.

Lesson: The maths: place value and conversions

One hex digit

$$= 4 \text{ bits, one octal digit} = 3 \text{ bits}$$

So 0xFF is 1111 1111 and 0o7 is 111.

Lesson: The maths: place value and conversions

TABLES

Table: type · what it holds · literals · can it change?

Type	What it holds	Literals	Can it change?
int	Whole numbers of any size	7, -3, 1_000_000, 0b101, 0o17, 0xFF	No (immutable)
float	Numbers with a decimal point, about 15 to 17 correct digits	3.14, -0.5, 2.0, 1.5e3, 1e-9	No
complex	Numbers with a real and an imaginary part	2 + 3j, 1j	No
bool	True or False	True, False	No
str	Text, any number of characters	"hi", 'hi', """many lines"""	No
NoneType	The single value meaning "nothing here"	None	No

Lesson: The basic data types and their literals

Table: base · digits · python literal · turn into text

Base	Digits	Python literal	Turn into text	Example
2 (binary)	0, 1	0b1010	bin(10)	0b1010 == 10
8 (octal)	0 to 7	0o17	oct(15)	0o17 == 15
10 (decimal)	0 to 9	42	str(42)	42
16 (hexadecimal)	0 to 9, A to F	0xFF	hex(255)	0xFF == 255

Lesson: Numeral systems: binary, octal and hexadecimal

Table: call · result · rule

Call	Result	Rule
int("42")	42	Text with only digits (and a sign, spaces around)
int(3.99)	3	Cuts towards zero, it does not round
int("3.5")	ValueError	A decimal point in the text is not allowed
int("ff", 16)	255	A second argument gives the base
float("3.5")	3.5	Also accepts "1e3", "inf", "nan"
str(7)	"7"	Anything can become text
bool(0), bool("")	False	Zero and empty things are False
bool("0"), bool(" ")	True	Any non empty text is True, even "0"
round(2.5), round(3.5)	2, 4	Ties go to the even number (bankers rounding)

Lesson: Changing one type into another

Table: rule · valid · not valid

Rule	Valid	Not valid
Letters, digits and underscores only	total_2	total-2, total 2
Must not start with a digit	price2	2price
Must not be a keyword	klass, class_	class, for, None
Case matters	Total and total are two different names	
Letters from any language are allowed	año, naam	

Lesson: Naming rules and good names

Table: · good · poor

	Good	Poor
Says what it holds	total_price, student_count	x, tp, data2
Follows PEP 8	snake_case for variables	TotalPrice for a variable (that style is for classes)
Avoids built in names	items, values	list, str, sum (they hide the built in functions)

Lesson: Naming rules and good names

Table: form · meaning

Form	Meaning
a, b = 1, 2	Two names at once. The right side is a tuple that is unpacked
a, b = b, a	Swap. The right side is worked out fully before any name changes
x = y = 0	Both names point to the same object 0
n += 1	Augmented assignment, n = n + 1. Also -=, *=, /=, //=, %=, **=
total: int = 0	A type hint. Python does not check it, tools like mypy do (Advanced 3.1)

Lesson: More ways to assign

Table: · python · c · java

	Python	C	Java
Type belongs to	The object (dynamic typing)	The variable, fixed when declared	The variable, fixed when declared
Declare first?	No, the first assignment creates the name	Yes: `int age;`	Yes: `int age;`
What a variable holds	A reference to an object	The value itself, in a box of fixed size	A value (primitives) or a reference (objects)
Int size	Unlimited	32 or 64 bits, can overflow	32 bits (int) or 64 bits (long)
Type errors found	When the line runs, or earlier with mypy	When compiling	When compiling

Lesson: How Python variables compare with other languages

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
`1281`	Subtract the Product and Sum of Digits of an Integer	Easy	Digits with % and //	Take the last digit with n % 10 and drop it with n // 10 until n is 0. Keep a product and a sum.
`191`	Number of 1 Bits	Easy	Binary	bin(n) gives the binary text. Count the "1" characters in it.
`1290`	Convert Binary Number in a Linked List to Integer	Easy	Place value	Start with 0. For each bit, value = value * 2 + bit. Here the list is given as a Python list of bits.
`405`	Convert a Number to Hexadecimal	Easy	Base conversion	For negative numbers add 2 ** 32 first (two's complement). Then repeat: digit = num % 16, num = num // 16.
`504`	Base 7	Easy	Base conversion	Same as base 2 but divide by 7. Handle the sign separately.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER — In Python a variable has no type of its own. The object has the type. The same name can point to an int now and to a str later. This is called dynamic typing.

Lesson: What a variable really is

IN THE EXAM — bool is a kind of int: True == 1 and False == 0, so True + True is 2. This surprises many people in exams.

Lesson: The basic data types and their literals

COMMON MISTAKE — A leading zero is not allowed on a decimal literal in Python 3: 017 is a SyntaxError. Write 0o17 for octal.

Lesson: Numeral systems: binary, octal and hexadecimal

IN THE EXAM — int(-3.99) is -3, not -4. int() cuts towards zero. math.floor(-3.99) is -4 because floor goes down.

Lesson: Changing one type into another

COMMON MISTAKE — Naming a variable list or str hides the built in. After list = [1, 2], calling list("abc") fails with TypeError: 'list' object is not callable.

Lesson: Naming rules and good names

1.3 Operators, expressions and the order Python works them out

Arithmetic, // and %, precedence, comparisons, float precision, logic and short circuit, bitwise operators.

KEY IDEAS

- **Division** / gives float. // floors (towards minus infinity). % has the sign of the divisor. a == (a // b) * b + a % b.
- **Priority** brackets, **, unary, * / // %, + -, shifts, & ^ |, comparisons, not, and, or. ** right to left. -2 ** 2 = -4.
- **Comparisons** Chained 1 < x < 10. Floats: use math.isclose(). 0.1 + 0.2 == 0.3 is False.
- **Logic** and, or short circuit and return an operand. not > and > or.
- **Bits** ~x = -x - 1. x << k = x * 2**k. x & 1 tests odd. x & (x - 1) == 0 tests power of 2.

FORMULAS

True division

$$a / b$$

Always gives a float: 6 / 3 = 2.0.

Lesson: Arithmetic: the seven number operators

Floor division

Exam favourite

$$a // b = \lfloor a \div b \rfloor$$

Divide, then round down towards minus infinity. 7 // 2 = 3, -7 // 2 = -4.

Lesson: Arithmetic: the seven number operators

Remainder (modulo)

Exam favourite

$$a \% b = a - b \cdot (a // b)$$

Has the same sign as b. -7 % 2 = 1.

Lesson: Arithmetic: the seven number operators

The division identity

Exam favourite

$$a == (a // b) * b + a \% b$$

Always true in Python for b ≠ 0. Use it to check your answers.

Lesson: Arithmetic: the seven number operators

Safe float comparison

Exam favourite

$$|a - b| \le \max(\text{rel} \cdot \max(|a|, |b|), \text{abs_tol})$$

This is math.isclose(a, b). The default rel is 1e-09 and abs_tol is 0.

Lesson: Comparisons and the trouble with floats

Machine epsilon

$$\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$$

The gap between 1.0 and the next float. sys.float_info.epsilon.

Lesson: Comparisons and the trouble with floats

Bitwise not

Exam favourite

$$\sim x = -x - 1$$

Python ints act like two's complement numbers with endless bits.

Lesson: Bitwise operators

Shifts

Exam favourite

$$x << k = x \cdot 2^k,\ x >> k = x // 2^k$$

Shifting is a fast way to multiply or divide by powers of 2.

Lesson: Bitwise operators

Useful tricks

$$x \,\&\, 1 \text{ (odd?)},\ x \,\&\, (x - 1) == 0 \text{ (power of 2?)},\ x \,\hat{}\, x == 0$$

Used in LeetCode problems such as Single Number and Power of Two.

Lesson: Bitwise operators

Euclidean style division in Python

Exam favourite

$$a = q \cdot b + r,\ q = \lfloor a/b \rfloor,\ 0 \le r < b\ (b > 0)$$

For b < 0, b < r ≤ 0.

Lesson: The maths behind the operators

Last digit and dropping it

Exam favourite

$$n \% 10,\ n // 10$$

Used to loop over the digits of a number.

Lesson: The maths behind the operators

Sum of digits test for 9

$$n \% 9 == \text{digit_sum}(n) \% 9$$

Because 10 % 9 = 1, every place value leaves remainder 1.

Lesson: The maths behind the operators

Float spacing near x

$$\text{ulp}(x) \approx |x| \cdot 2^{-52}$$

math.ulp(x). Bigger numbers have bigger gaps between floats.

Lesson: The maths behind the operators

TABLES

Table: group · operators · example · result

Group	Operators	Example	Result
Arithmetic	+ - * / // % **	17 // 5	3
Comparison	== != < <= > >=	3 < 5	True
Logical	and or not	True and False	False
Bitwise	& | ^ ~ << >>	12 & 10	8
Assignment	= += -= *= /= //= %= **=	n += 1	n grows by 1
Membership	in, not in	"a" in "cat"	True
Identity	is, is not	x is None	True or False

Lesson: Operators: the verbs of a program

Table: operand types · + - * // % ** · /

Operand types	+ - * // % **	/
int and int	int (but int ** negative int is float)	float
int and float	float	float
bool and int	int (True counts as 1)	float

Lesson: Arithmetic: the seven number operators

Table: priority (high to low) · operators · direction

Priority (high to low)	Operators	Direction
1	( ) brackets	innermost first
2	**	right to left
3	+x, -x, ~x (unary)	
4	* / // %	left to right
5	+ -	left to right
6	<< >>	left to right
7	&, then ^, then |	left to right
8	comparisons, in, not in, is, is not	chained
9	not	
10	and	
11	or	
12	if else (conditional expression), lambda, :=	

Lesson: Which operation goes first

Table: · float · decimal.decimal · fractions.fraction

	float	decimal.Decimal	fractions.Fraction
Stores	base 2, 53 bits	base 10, as many digits as you set	an exact top and bottom number
0.1 + 0.2 == 0.3	False	True (with Decimal("0.1"))	True
Speed	Fastest, done by the chip	Slower	Slowest
Use for	Science, graphics, machine learning	Money and bills	Exact maths with ratios

Lesson: Comparisons and the trouble with floats

Table: a · b · a and b · a or b

a	b	a and b	a or b	not a
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Lesson: Logical operators and short circuit

Table: operator · with text · example

Operator	With text	Example
+	Joins two texts	"ab" + "cd" is "abcd"
*	Repeats a text a whole number of times	"ha" * 3 is "hahaha"
in	Is one text inside another?	"py" in "python" is True
== < >	Compare letter by letter by Unicode number	"Zebra" < "apple" is True

Lesson: Text operators and assignment shortcuts

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1009	Complement of Base 10 Integer	Easy	Bitwise xor with a mask	Make a mask of 1s as long as n in binary, (1 << n.bit_length()) - 1, then n ^ mask flips every bit. n = 0 gives 1.
231	Power of Two	Easy	Bit trick	A power of two has exactly one 1 bit, so n & (n - 1) removes it and leaves 0. n must be above 0.
258	Add Digits	Easy	Maths (digital root)	Without a loop: 0 stays 0, otherwise the answer is 1 + (num - 1) % 9.
461	Hamming Distance	Easy	xor then count bits	x ^ y has a 1 exactly where the bits differ. Count the 1s.
29	Divide Two Integers	Medium	Shifts and truncation	Work with absolute values. Subtract the biggest divisor << k that fits, add 1 << k to the answer. LeetCode truncates towards zero and clamps to 32 bits.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM — A unary operator has one operand (-x, not x, ~x). A binary operator has two (a + b). The same symbol - can be either.

Lesson: Operators: the verbs of a program

IN THE EXAM — C and Java cut integer division towards zero, so -7 / 2 is -3 there. Python floors, so -7 // 2 is -4. Exams love this difference.

Lesson: Arithmetic: the seven number operators

COMMON MISTAKE -2 ** 2 is -4, not 4. The power is done first, then the minus. Write (-2) ** 2 when you mean minus two squared.

Lesson: Which operation goes first

IN THE EXAM not has a higher priority than and, and and is higher than or. So True or False and False is True or (False and False) = True.

Lesson: Logical operators and short circuit

COMMON MISTAKE "5" + 5 is a TypeError. Python never turns a number into text by itself. Use str(5) or an f-string.

Lesson: Text operators and assignment shortcuts

1.4 Input, output and formatting text

input() and conversion, split and map, f-strings, the format spec, tables, older formatting, safe input.

KEY IDEAS

- **input()** Shows a prompt, waits, returns text. Convert with int() or float(). a, b = map(int, input().split()).
- **f-strings** f"{expr}", {x!r}, {x = }, {x:spec}. {{ and }} for real brackets.
- **Format spec** [[fill]align][sign][#][0][width][grouping][.precision][type]. Width is a minimum. .2f rounds half to even.
- **Older formatting** "%s %d" % (a, b) and "{} {}".format(a, b). Prefer f-strings.
- **Safe input** while True + try int(text) except ValueError + range check + break.

FORMULAS

Format spec grammar Exam favourite

[[fill]align][sign][#][0][width][grouping][.precision][type]

Every part is optional, but the order is fixed. align is < > ^ or =.

Lesson: The format spec: width, alignment, decimals and more

Common types

d (int), f (fixed), e (scientific), % (percent), b o x X (bases), s (text)

No type means the same as str() for most values.

Lesson: The format spec: width, alignment, decimals and more

Padding added by a width Exam favourite

$$\text{pad} = \max(0, \text{width} - \text{len}(\text{text}))$$

Width is a minimum. It never cuts the text.

Lesson: The maths: widths and rounding

Centre alignment Exam favourite

$$\text{left} = \lfloor \text{pad} / 2 \rfloor, \ \text{right} = \text{pad} - \text{left}$$

With an odd pad the extra character goes on the right.

Lesson: The maths: widths and rounding

Rounding to p places

round half to even on the exact binary value

f"{0.125:.2f}" is 0.12 because 0.125 is exact and 2 is even.

Lesson: The maths: widths and rounding

Digits of a number

$$\text{len}(\text{str}(n)) = \lfloor \log_{10} n \rfloor + 1$$

For a whole number n > 0.

Lesson: The maths: widths and rounding

TABLES

Table: you want · write · user types · you get

You want	Write	User types	You get
Text	name = input("Name: ")	Asha	"Asha"
A whole number	n = int(input("N: "))	42	42
A decimal number	x = float(input("X: "))	3.5	3.5
Several numbers on one line	a, b = map(int, input().split())	4 9	4 and 9
Yes or no	ok = input("y/n? ").strip().lower() == "y"	Y	True

Lesson: input() always returns text

Table: inside the brackets · meaning · example · gives

Inside the brackets	Meaning	Example	Gives
{x}	The value as str(x)	f"{3 * 4}"	12
{x!r}	repr(x), shows quotes on text	f"{'hi'!r}"	'hi'
{x = }	The expression text, then the value (Python 3.8+)	f"{price = }"	price = 45.5
{x:spec}	Formatted with a format spec	f"{3.14159:.2f}"	3.14
{{ and }}	A real curly bracket	f"{{x}}"	{x}

Lesson: f-strings: the modern way to build text

Table: · % formatting · str.format() · f-string

	% formatting	str.format()	f-string
Looks like	"%s is %d" % (name, age)	"{} is {}".format(name, age)	f"{name} is {age}"
Since	Python 1 (copied from C printf)	Python 2.6	Python 3.6
Readability	Values far from their places	Better, numbered or named fields	Best, values sit in place
Speed	Medium	Slowest	Fastest
Still useful for	logging messages: log.info("%s", x)	templates stored as data, built before the values exist	almost everything else

Lesson: Older ways: % formatting and str.format()

Table: stream · python name · normally · used by

Stream	Python name	Normally	Used by
Standard input	sys.stdin	The keyboard	input()
Standard output	sys.stdout	The screen	print()
Standard error	sys.stderr	The screen, often in red	Tracebacks, print(..., file=sys.stderr)

Lesson: Where input and output really go

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
412	Fizz Buzz	Easy	Modulo and building text	Check divisible by 15 first, then 3, then 5. Build a list of texts.
709	To Lower Case	Easy	Character codes	Without lower(): for "A" to "Z" add 32 to the character code with ord() and chr().
1678	Goal Parser Interpretation	Easy	Text replace	Replace "()" with "o" and then "(al)" with "al".
2864	Maximum Odd Binary Number	Easy	Counting and building text	Keep one "1" for the end (odd). Put all other 1s at the front, then the 0s.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER The Run button above feeds the typed answers (Asha and 25) for you. On your own computer, Python waits for you to type them.

Lesson: Input and output: how a program talks to people

COMMON MISTAKE input() returns text even when the user types digits. "5" + "5" is "55". Convert first, then calculate.

Lesson: input() always returns text

REMEMBER Since Python 3.12 (PEP 701) you may reuse the same quotes inside an f-string expression, for example f"{", ".join(names)}". Older versions need the other kind of quote inside.

Lesson: f-strings: the modern way to build text

IN THE EXAM .2f rounds the stored binary value, and exact halves go to the even digit: f"{2.5:.0f}" gives 2 and f"{2.675:.2f}" gives 2.67.

Lesson: The format spec: width, alignment, decimals and more

REMEMBER Loops arrive in Core 2.2 and try/except in Core 5.1. You can use this pattern now and understand every line later.

Lesson: Reading input safely

STEP 2 Core Python · Control flow

2.1 Making decisions: if, elif, else and match

if, elif, else, truthiness, combining conditions, conditional expression, walrus, match and case, De Morgan.

KEY IDEAS

- **Chains** if, any number of elif, optional else. First truthy condition wins, the rest are skipped. Strictest test first.
- **Truthiness** Falsy: False, None, 0, 0.0, "", empty collections. "0" and [0] are truthy.
- **Combining** and, or, not, 13 <= x <= 19, x in (a, b). x == a or b is always truthy.
- **One liners** a if cond else b. (n := len(x)) assigns inside an expression.
- **match** Literal, capture, _, |, [a, *rest], {"k": v}, Class(), guard if. A bare name captures.

FORMULAS

Conditional expression

Exam favourite

value_if_true if condition else value_if_false

An expression, so it gives a value: kind = "even" if n % 2 == 0 else "odd".

Lesson: One line choices: the conditional expression and the walrus

Assignment expression (walrus)

(name := expression)

Python 3.8+. Assigns and gives the value in one go, useful inside if and while conditions.

Lesson: One line choices: the conditional expression and the walrus

De Morgan laws

Exam favourite

not (a and b) == (not a) or (not b), not (a or b) == (not a) and (not b)

Use them to simplify or flip a condition.

Lesson: The maths: boolean algebra

Number of paths

Exam favourite

k independent if statements give up to 2^k combinations

An elif chain with k conditions and an else has exactly k + 1 paths.

Lesson: The maths: boolean algebra

Distributive law

a and (b or c) == (a and b) or (a and c)

Like a × (b + c) = ab + ac in school algebra.

Lesson: The maths: boolean algebra

TABLES

Table: · if / elif / else · separate if statements · nested if

	if / elif / else	separate if statements	nested if
How many blocks can run	Exactly one (or none without else)	Any number	Depends on the tree
Use when	The cases exclude each other (a grade)	Several things can be true together (flags)	A second question only makes sense after the first
Risk	Wrong order catches cases too early	Several outputs when you wanted one	Deep indentation that is hard to read

Lesson: if, elif and else chains

Table: falsy (counts as false) · truthy (counts as true)

Falsy (counts as False)	Truthy (counts as True)
False, None	True
0, 0.0, 0j	any other number, even -2 and 0.001
"" (empty text)	"0", " ", "False" (non empty text)
[], (), {}, set(), range(0)	[0], (None,), {"a": 1} (non empty collections)

Lesson: Truthy and falsy values

Table: need · write · instead of

Need	Write	Instead of
Both must hold	a and b	a nested if inside if
Either is enough	a or b	two ifs with the same block
A range	13 <= age <= 19	age >= 13 and age <= 19
One of several values	day in ("Sat", "Sun")	day == "Sat" or day == "Sun"
The opposite	not done	done == False

Lesson: Combining conditions

Table: pattern · fits · example

Pattern	Fits	Example
Literal	A value equal to it	case 404:
Capture	Anything, and binds it to the name	case code:
Wildcard	Anything, binds nothing	case _:
Or	Any of the alternatives	case 200 | 201:
Sequence	A list or tuple of that length	case ["go", direction]:
Star	Collects the rest of a sequence	case ["drop", *things]:
Mapping	A dict with those keys (others allowed)	case {"x": x, "y": y}:
Class	An instance of the class, with attributes	case Point(x=0, y=y):
Guard	The pattern plus a condition	case int(n) if n > 0:

Lesson: match and case: pattern matching

Table: a · b · not (a and b) · not a or not b

a	b	not (a and b)	not a or not b
True	True	False	False
True	False	True	True
False	True	True	True
False	False	True	True

Lesson: The maths: boolean algebra

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1455	Check If a Word Occurs As a Prefix of Any Word in a Sentence	Easy	Condition inside a loop	Split the sentence. Return the 1 based position of the first word that starts with searchWord, else -1.
2129	Capitalize the Title	Easy	if else on length	Words of length 1 or 2 become all lower case. Longer words get a capital first letter and the rest lower.
1672	Richest Customer Wealth	Easy	Keep the biggest with if	Add up each row. Keep the biggest sum seen so far with an if.
2678	Number of Senior Citizens	Easy	Condition on a slice	Characters 11 and 12 of each passenger text are the age. Count ages above 60.
1344	Angle Between Hands of a Clock	Medium	Formula and min	Minute hand at 6 × minutes degrees, hour hand at 30 × (hour % 12) + 0.5 × minutes. Return the smaller of the gap and 360 minus the gap.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER Write if items: instead of if len(items) > 0:, and if not name: instead of if name == "":. PEP 8 recommends this short form. But write if x is None: when you really mean None, because 0 and "" are falsy too.

Lesson: Truthy and falsy values

COMMON MISTAKE if day == "Sat" or "Sun": is always True. Python reads it as (day == "Sat") or "Sun", and "Sun" is truthy. Write day in ("Sat", "Sun").

Lesson: Combining conditions

REMEMBER Do not chain many conditional expressions in one line. a if x else b if y else c is legal but hard to read. Use an if, elif, else statement instead.

Lesson: One line choices: the conditional expression and the walrus

COMMON MISTAKE A plain name in a case is a capture, not a comparison. case RED: catches everything and binds it to RED. To compare with a constant, use a dotted name such as case Color.RED:.

Lesson: match and case: pattern matching

2.2 Loops: while, for, range, break and continue

while, for, range, break, continue, pass, loop else, loop patterns, nested loops, enumerate and zip, loop maths.

KEY IDEAS

- **while and for** while repeats while a condition is True. for repeats once per item of an iterable.
- **range** range(start, stop, step) excludes stop. len = max(0, ceil((stop - start)/step)). step 0 is an error.
- **Loop control** break leaves, continue skips, pass does nothing, loop else runs when no break happened.
- **Patterns** Accumulator, counter, running max, search with break, build a list. Nested loops: outer x inner turns.
- **Pythonic loops** enumerate(items, start=1), zip(a, b) stops at the shortest, reversed(), sorted().

FORMULAS

range(start, stop, step) Exam favourite

$$\text{start, start + step, start + 2·step, ... while still before stop}$$

stop is never included. start defaults to 0, step to 1. step must not be 0.

Lesson: for loops and range()

How many numbers range gives Exam favourite

$$\text{len} = \max(0, \lceil (\text{stop} - \text{start}) / \text{step} \rceil)$$

range(1, 10, 3) has ⌈9/3⌉ = 3 numbers: 1, 4, 7.

Lesson: for loops and range()

Turns of a nested loop Exam favourite

$$\text{outer} \times \text{inner}$$

3 rows × 5 columns = 15 prints. Three nested loops of n each run n^3 times.

Lesson: Loops inside loops

Triangle loop Exam favourite

$$1 + 2 + \ldots + n = n(n + 1)/2$$

When the inner loop runs i times on turn i, the total is a triangular number.

Lesson: Loops inside loops

Length of a range Exam favourite

$$\text{len(range(a, b, s))} = \max(0, \lceil (b - a)/s \rceil)$$

Works for negative steps too: range(10, 0, -3) has ⌈-10/-3⌉ = 4 numbers.

Lesson: The maths of loops

Last number of a range

$$a + (\text{len} - 1) \cdot s$$

range(0, 100, 7)[-1] = 0 + 14 × 7 = 98.

Lesson: The maths of loops

Sum of an arithmetic sequence

Exam favourite

$$S = n(\text{first} + \text{last})/2$$

sum(range(1, 101)) = 100 × 101 / 2 = 5050 (the young Gauss story).

Lesson: The maths of loops

Turns to halve n down to 1

$$\lfloor \log_2 n \rfloor$$

while n > 1: n //= 2 runs about $\log_2 n$ times. This is why binary search is fast.

Lesson: The maths of loops

Digits of n (turns of the digit loop)

$$\lfloor \log_{10} n \rfloor + 1$$

The digit loop above ran 5 times for 90731.

Lesson: The maths of loops

TABLES

Table: statement · what it does · typical use

Statement	What it does	Typical use
break	Leaves the innermost loop at once	Stop searching once you found what you wanted
continue	Skips the rest of this turn and starts the next	Ignore bad rows and carry on
pass	Does nothing. A placeholder where a statement is needed	An empty block you will write later
else (on a loop)	Runs once after the loop if it was not stopped by break	Say "not found" after a search

Lesson: break, continue, pass and else

Table: pattern · start with · each turn · built in shortcut

Pattern	Start with	Each turn	Built in shortcut
Accumulator (sum)	total = 0	total += x	sum(items)
Counter	count = 0	if test: count += 1	sum(1 for x in items if test)
Running maximum	best = first item	if x > best: best = x	max(items)
Search	found = None	if test: found = x and break	next((x for x in items if test), None)
Build a new list	result = []	result.append(f(x))	[f(x) for x in items] (Core 4.3)
Sentinel	read a value	while value != "stop": ... read again	

Lesson: Loop patterns every programmer uses

Table: · index loop · enumerate

	Index loop	enumerate
Code	for i in range(len(names)): print(i, names[i])	for i, name in enumerate(names): print(i, name)
Reads	Two steps: index, then look up	Says what you mean
Mistakes	Off by one, wrong list in names[i]	Fewer
When you still need indexes	Changing items in place, comparing names[i] with names[i + 1]	

Lesson: enumerate, zip, reversed and sorted

Table: · for · while

	for	while
Number of turns	Known: one per item	Unknown: until a condition changes
Stops by itself	Yes, when items run out	Only if your code makes the condition False
Examples	Each student, each line of a file, range(10)	Digits of a number, asking until the input is valid, a game loop, Collatz

Lesson: When to use while

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1480	Running Sum of 1d Array	Easy	Accumulator	Keep a total. Each turn add the current number and append the total.
9	Palindrome Number	Easy	Digit loop with % and //	Negative numbers are not palindromes. Build the reversed number digit by digit and compare.
1295	Find Numbers with Even Number of Digits	Easy	Counter	Count the digits of each number with len(str(n)) and count the even ones.
7	Reverse Integer	Medium	Digit loop and range check	Reverse the absolute value with a digit loop, put the sign back, and return 0 if it leaves the 32 bit range.
202	Happy Number	Easy	while loop with a seen set	Replace n by the sum of the squares of its digits. Stop at 1 (happy) or when a number repeats (a cycle).
1137	N-th Tribonacci Number	Easy	Loop with three running values	Keep the last three numbers a, b, c and move them forward n times.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM range(5) is 0, 1, 2, 3, 4: five numbers, but 5 itself is not included. This "stop is excluded" rule matches list indexes, so range(len(items)) gives every valid index.

Lesson: for loops and range()

IN THE EXAM The loop else is not "else if the loop did not run". It means "no break happened". A for loop over an empty list runs its else block.

Lesson: break, continue, pass and else

REMEMBER Learn the loops first, then the shortcuts. In an interview you may be asked to write max() yourself, and the running maximum pattern is the answer.

Lesson: Loop patterns every programmer uses

COMMON MISTAKE break only leaves the inner loop. To leave two loops at once, put them in a function and use return, or set a flag and check it in the outer loop.

Lesson: Loops inside loops

COMMON MISTAKE zip stops at the shortest list. zip([1, 2, 3], "ab") gives only two pairs. Python 3.10 added zip(..., strict=True) to raise an error when the lengths differ.

Lesson: enumerate, zip, reversed and sorted

REMEMBER An infinite loop is not always a bug: servers and games run while True: with a break or return inside. In the REPL or a terminal, press Ctrl+C to stop a runaway loop.

Lesson: When to use while

3.1 Strings: text, slicing, methods and Unicode

str, quotes, escapes, indexing, slicing, immutability, methods, split and join, Unicode, UTF-8, string maths.

KEY IDEAS

- **Making strings** Quotes: single, double or triple. Escapes start with a backslash. r"..." is a raw string. len() counts characters.
- **Index and slice** s[0] first, s[-1] last. s[a:b:c] includes a, excludes b. s[::-1] reverses. Slices never raise IndexError.
- **Immutable** You cannot change a character. Methods return new strings: write s = s.upper().
- **Key methods** strip, split, join, replace, find (-1) vs index (ValueError), count, startswith, isdigit, isalpha, upper, lower.
- **Unicode** ord("A") = 65, chr(97) = "a". UTF-8 uses 1 to 4 bytes per character. Compare strings by code point.

FORMULAS

Index of a character Exam favourite

$$0, 1, \ldots, \text{len}(s) - 1 \text{ from the left, and } -1, -2, \ldots, -\text{len}(s) \text{ from the right}$$

s[-k] is the same as s[len(s) − k].

Lesson: Indexing and slicing

Slice Exam favourite

$$s[\text{start:stop:step}]$$

start is included, stop is not. Leave out start for "from the beginning" and stop for "to the end".

Lesson: Indexing and slicing

Length of a simple slice

$$\text{len}(s[a:b]) = b - a$$

when $0 \le a \le b \le \text{len}(s)$. "PYTHON"[1:4] is "YTH", 3 characters.

Lesson: Indexing and slicing

Number of substrings of a text of length n

Exam favourite

$$n(n + 1) / 2$$

Pick a start and an end. "abc" has 6: a, b, c, ab, bc, abc.

Lesson: The maths of strings

Number of subsequences (keep or drop each letter)

$$2^n$$

Including the empty one. "abc" has 8.

Lesson: The maths of strings

Number of different arrangements (anagrams)

Exam favourite

$$n! / (c_1! \cdot c_2! \cdot \ldots)$$

c_i is how often each letter repeats. "banana": 6! / (3! · 2! · 1!) = 60.

Lesson: The maths of strings

Caesar shift of a lower case letter

$$\text{new} = (\text{ord}(\text{ch}) - 97 + k) \bmod 26 + 97$$

mod 26 wraps z round to a.

Lesson: The maths of strings

Polynomial hash of a text (used in Rabin Karp search)

$$h = (c_0 \cdot b^{n-1} + c_1 \cdot b^{n-2} + \ldots + c_{n-1}) \bmod m$$

Lets a program compare a window of text with a pattern in O(1) by sliding the hash one letter at a time.

Lesson: The maths of strings

TABLES

Table: you write · you get · name

You write	You get	Name
`\n`	a new line	newline
`\t`	a tab	tab
`\\`	one backslash	backslash
`\'` and `\"`	a quote inside quotes of the same kind	quote
`\u20b9`	the character with code point 20B9 (the rupee sign)	Unicode escape

Lesson: Escape characters and raw strings

Table: · str (immutable) · list of characters (mutable)

	str (immutable)	list of characters (mutable)
Change one character	Not allowed: build a new string	chars[0] = "R" works
Safe to share	Yes, nobody can change it behind your back	No, every name sees the change
Can be a dict key or set item	Yes	No
Best for	Storing and passing text around	Many small edits, then "".join(chars) at the end

Lesson: Strings never change

Table: operation · example · result

Operation	Example	Result
+ joins	"ab" + "cd"	"abcd"
* repeats	"ha" * 3	"hahaha"
in searches	"cat" in "concatenate"	True
< > compare	"apple" < "banana"	True
ord()	ord("A")	65
chr()	chr(97)	"a"

Lesson: Joining, repeating, searching and comparing

Table: group · methods · note

Group	Methods	Note
Clean	strip(), lstrip(), rstrip(), removeprefix(), removesuffix()	strip() with no argument removes spaces, tabs and new lines at both ends
Case	upper(), lower(), title(), capitalize(), swapcase(), casefold()	Use casefold() to compare text without caring about case
Search	find(), rfind(), index(), rindex(), count(), startswith(), endswith()	find() gives −1 when not found, index() raises ValueError
Test	isdigit(), isalpha(), isalnum(), isspace(), isupper(), islower()	Return True or False. An empty string gives False
Change	replace(old, new, count)	Returns a new string
Pad	zfill(), center(), ljust(), rjust()	Make columns of the same width

Lesson: The most used string methods

Table: · adding in a loop (s += piece) · "sep".join(pieces)

	Adding in a loop (s += piece)	"sep".join(pieces)
Code	result = "" then result += w + ", " each turn	", ".join(words)
Extra separator at the end	Yes, you must remove it	No
Speed	Each += may copy the whole text so far: up to $O(n^2)$ work	Works out the size once and copies each piece once: O(n)
Pieces must be	Anything you convert with str()	All strings already

Lesson: split and join

Table: code point range · utf-8 bytes · examples

Code point range	UTF-8 bytes	Examples
0 to 127	1	A, z, 5, space
128 to 2,047	2	é, ñ, Greek and Arabic letters
2,048 to 65,535	3	₹, Hindi and most Indian scripts, Chinese
65,536 to 1,114,111	4	emojis, old scripts

Lesson: Unicode, code points and UTF-8

Table: · str · bytes · list of str

	str	bytes	list of str
Holds	Unicode characters	Numbers 0 to 255	Any items, here single characters
Written as	"café"	b"caf\xc3\xa9"	["c", "a", "f", "é"]
Changeable	No	No (bytearray is)	Yes
len("café")	4 characters	5 bytes in UTF-8	4 items
Used for	Text you read and show	Files, images, network data	Editing text one character at a time

Lesson: str, bytes and lists of characters

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
344	Reverse String	Easy	Two pointers	The input is a list of characters. Swap the first and last, then move both ends inward.
125	Valid Palindrome	Easy	Clean then compare	Keep only letters and digits in lower case, then compare the text with its reverse.
58	Length of Last Word	Easy	split	split() with no argument ignores extra spaces. Take the last word.
1768	Merge Strings Alternately	Easy	Index loop over the longer length	Loop i up to the longer length and add word1[i] and word2[i] when they exist.
14	Longest Common Prefix	Easy	Compare column by column	Take the first word. For each position, check that every other word has the same character there.
28	Find the Index of the First Occurrence in a String	Easy	Sliding window of slices	Try every start i and compare haystack[i:i + len(needle)] with needle. str.find does this for you.
13	Roman to Integer	Easy	Look ahead one character	Add each value, but subtract it when the next symbol is bigger (IV, IX, XL).
151	Reverse Words in a String	Medium	split and join	split() removes the extra spaces. Reverse the list of words and join with one space.
6	Zigzag Conversion	Medium	Simulate rows	Keep one string per row. Walk down and up the rows, adding each character to the current row.
5	Longest Palindromic Substring	Medium	Expand around the centre	Every palindrome has a centre: a letter or the gap between two letters. Grow outwards from each of the 2n − 1 centres.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER Single and double quotes make exactly the same string. Pick one style and keep it. Use the other kind when the text itself contains a quote.

Lesson: What a string is

IN THE EXAM A backslash starts an **escape sequence**: the backslash and the next character together stand for one special character. That is why len of the new line escape is 1. In a raw string (r before the quote) Python keeps every backslash, which is handy for Windows paths and regular expressions (Advanced 6.1).

Lesson: Escape characters and raw strings

IN THE EXAM A single index outside the string raises IndexError. A slice outside the string never fails: it just gives what is there, or an empty string.

Lesson: Indexing and slicing

COMMON MISTAKE Strings are compared letter by letter by code point, not by dictionary order. All upper case letters (65 to 90) come before all lower case letters (97 to 122), so "Zebra" < "apple" is True. "10" < "9" is True too, because "1" comes before "9". Turn text into numbers before comparing numbers.

Lesson: Joining, repeating, searching and comparing

COMMON MISTAKE Every method returns a new string. s.upper() alone on a line does nothing useful. Write s = s.upper() to keep the result.

Lesson: The most used string methods

IN THE EXAM split() with no argument and split(" ") are different. "a b".split() gives ["a", "b"], but "a b".split(" ") gives ["a", "", "b"] because there are two spaces.

Lesson: split and join

REMEMBER Most text problems follow one plan: walk through the characters with for ch in text, test each one (isalpha, isdigit, a comparison), and collect or count the ones you want. The Caesar cipher above uses ord() and chr() to do arithmetic on letters.

Lesson: Working through text with loops

COMMON MISTAKE Always say the encoding when you open a text file: open(name, encoding="utf-8"). Otherwise Python uses the system default, which is not UTF-8 on every Windows computer (Core 5.3).

Lesson: Unicode, code points and UTF-8

3.2 Lists: storing, changing, sorting and copying

list methods, sort vs sorted, aliases and copies, matrices, bubble sort, unpacking, list growth and costs.

KEY IDEAS

- **Lists** Ordered, changeable, any types. Index from 0, slice like strings. len, min, max, sum, in.
- **Add and remove** append (one item), extend (many), insert(i, x). remove(x) first match, pop() last, pop(i), del items[i].
- **Sorting** sort() changes the list and returns None. sorted() returns a new list. key= and reverse=. Stable.
- **Copies** b = a is the same list. a[:] or a.copy() is a shallow copy. copy.deepcopy for nested lists. [[0]*n for _ in range(m)].
- **Costs** append, pop(), index access O(1). insert(0), pop(0), in, remove O(n). Bubble sort n(n-1)/2 comparisons.

FORMULAS

Item of a matrix Exam favourite

matrix[row][col]

The first index picks the row, the second the column.

Lesson: Lists in lists: matrices

Transpose

T[c][r] = M[r][c]

An r × c matrix becomes c × r.

Lesson: Lists in lists: matrices

Number of items

rows × cols

A 3D list (a cube) has rows × cols × depth items.

Lesson: Lists in lists: matrices

Comparisons in the worst case Exam favourite

$(n - 1) + (n - 2) + \ldots + 1 = n(n - 1)/2$

5 items need at most 10 comparisons, 1,000 items about 500,000. That is $O(n^2)$.

Lesson: Bubble sort, step by step

Best case with the early stop

n − 1 comparisons

An already sorted list needs one pass.

Lesson: Bubble sort, step by step

Average cost of an append (amortised)

Exam favourite

(n + total copies) / n < a small constant

With growth by a factor g > 1 the copies form a geometric series $n/g + n/g^2 + \ldots < n/(g - 1)$.

Lesson: The maths of lists

Bubble sort comparisons Exam favourite

$n(n - 1)/2$

The same sum as the triangle loop of Core 2.2.

Lesson: The maths of lists

Number of slices items[a:b] with a < b

$C(n + 1, 2) = n(n + 1)/2$

Choose 2 of the n + 1 cut points.

Lesson: The maths of lists

Mean of a list

$$\text{sum(items)} / \text{len(items)}$$

Undefined for an empty list: guard against ZeroDivisionError.

Lesson: The maths of lists

Binary search steps on a sorted list

$$\lfloor \log_2 n \rfloor + 1$$

A sorted list of 1,000,000 items needs at most 20 checks with bisect (Advanced 4.2).

Lesson: The maths of lists

TABLES

Table: method · what it does · cost

Method	What it does	Cost
append(x)	Add x at the end	O(1) on average
insert(i, x)	Add x before index i, the rest move right	O(n)
extend(items)	Add every item of another iterable at the end	O(k) for k new items
items[i] = x	Replace one item	O(1)
items[a:b] = other	Replace a slice with any number of items	O(n)

Lesson: Adding and changing items

Table: way · removes · gives back · if missing

Way	Removes	Gives back	If missing
remove(x)	The first item equal to x	None	ValueError
pop()	The last item	The item	IndexError on an empty list
pop(i)	The item at index i	The item	IndexError
del items[i] or del items[a:b]	By index or slice	Nothing (it is a statement)	IndexError for a single index
clear()	Everything	None	Never fails

Lesson: Removing items

Table: · items.sort() · sorted(items)

	items.sort()	sorted(items)
Changes the original	Yes	No
Returns	None	A new list
Works on	Lists only	Any iterable: tuples, strings, sets, dict keys
Options	key=..., reverse=True	key=..., reverse=True

Lesson: Sorting: sort() and sorted()

Table: way · new outer list · new inner lists

Way	New outer list	New inner lists
b = a	No	No
a[:], a.copy(), list(a)	Yes	No (shallow copy)
copy.deepcopy(a)	Yes	Yes, at every level

Lesson: Shallow copy, deep copy and lists of lists

Table: · list · tuple · set

	list	tuple	set	dict
Order kept	Yes	Yes	No	Yes (insertion order)
Can change	Yes	No	Yes	Yes
Repeats allowed	Yes	Yes	No	Keys no, values yes
Find an item	O(n)	O(n)	O(1) average	O(1) average by key
Written as	[1, 2]	(1, 2)	{1, 2}	{"a": 1}
Best for	A changing sequence	A fixed record	Unique items and membership	Looking up a value by a key

Lesson: Lists compared with other collections

Table: operation · time

Operation	Time
items[i], items[i] = x, len, append, pop()	O(1)
insert(0, x), pop(0), remove(x), x in items, index(x)	O(n)
sort(), sorted()	O(n log n)
items[a:b] (copy of k items)	O(k)

Lesson: Lists compared with other collections

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1929	Concatenation of Array	Easy	List repetition	The answer is the list followed by itself.
66	Plus One	Easy	Walk from the right with a carry	Start at the last digit. A 9 becomes 0 and the carry moves left. If every digit was 9, put a 1 in front.
27	Remove Element	Easy	Two pointers (read and write)	Keep a write position. Copy every item that is not val to the write position. Return how many you kept.
26	Remove Duplicates from Sorted Array	Easy	Two pointers	The list is sorted, so repeats sit next to each other. Write an item only when it differs from the last one written.
283	Move Zeroes	Easy	Two pointers	Move every non zero item forward to a write position, then fill the rest with zeros.
121	Best Time to Buy and Sell Stock	Easy	Running minimum	Keep the lowest price so far. Each day, check the profit if you sold today.
88	Merge Sorted Array	Easy	Three pointers from the back	Fill nums1 from its end. Each time place the bigger of the two last items.
118	Pascal's Triangle	Easy	Build each row from the one above	Each row starts and ends with 1. Every inner number is the sum of the two numbers above it.
189	Rotate Array	Medium	Three reversals	Reverse the whole list, then reverse the first k items and the rest. Use k % n first.
75	Sort Colors	Medium	Dutch national flag (three pointers)	Keep low, mid and high. A 0 swaps to low, a 2 swaps to high, a 1 just moves mid on.
48	Rotate Image	Medium	Transpose, then reverse each row	Swap matrix[r][c] with matrix[c][r] above the diagonal, then reverse every row.
54	Spiral Matrix	Medium	Shrinking borders	Keep top, bottom, left and right borders. Walk right, down, left and up, moving a border in after each side.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM append adds one item, even if that item is a list. [1, 2].append([3, 4]) gives [1, 2, [3, 4]]. extend adds the items one by one and gives [1, 2, 3, 4].

Lesson: Adding and changing items

COMMON MISTAKE Do not remove items from a list while a for loop is walking over it. The loop skips the item after each removal. Loop over a copy (for x in items[:]) or build a new list instead.

Lesson: Removing items

REMEMBER x in items checks each item from the left until it finds a match, so it takes O(n) time. For many lookups in a big collection use a set or a dict (Core 3.3 and 3.4), which answer in O(1) on average.

Lesson: Finding items

IN THE EXAM [[0] * 3] * 3 makes one inner list and three references to it, so changing one row changes all three. Build a grid with a comprehension: [[0] * 3 for _ in range(3)].

Lesson: Shallow copy, deep copy and lists of lists

REMEMBER Never use bubble sort in real code. sorted() is far faster. Learn it to understand sorting and to answer exam questions.

Lesson: Bubble sort, step by step

IN THE EXAM The number of names must match the number of items, unless one name has a star. a, b = [1, 2, 3] raises ValueError: too many values to unpack.

Lesson: Unpacking, swapping and a first look at comprehensions

REMEMBER pop(0) and insert(0, x) move every other item. For a queue that adds at one end and removes at the other, use collections.deque, which does both in O(1) (Advanced 4.2).

Lesson: Lists compared with other collections

3.3 Tuples and sets: fixed records and unique items

tuples, packing and unpacking, hashable keys, namedtuple, sets, set algebra, deduplication, O(1) membership, set maths.

KEY IDEAS

- **Tuples** Immutable ordered sequence. (5,) is a tuple, (5) is not. count and index only. Unpack with a, b = t.
- **Hashable** Dict keys and set items must be hashable: numbers, strings, tuples of hashables, frozensets. Not lists, sets or dicts.
- **Sets** Unique items, no order. set() is empty. add, remove (KeyError), discard (safe), pop (any item).
- **Set algebra** | union, & intersection, - difference, ^ symmetric difference, <= subset. Methods accept any iterable.
- **Counting** |A u B| = |A| + |B| - |A n B|. A set of n items has 2^n subsets. x in set is O(1) on average.

FORMULAS

Size of a union (inclusion and exclusion)

Exam favourite

$$|A \cup B| = |A| + |B| - |A \cap B|$$

The middle part was counted twice, so take it away once.

Lesson: The maths of sets

Three sets

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

Add singles, take away pairs, add back the triple.

Lesson: The maths of sets

Symmetric difference

$$|A \Delta B| = |A| + |B| - 2|A \cap B|$$

A ^ B in Python.

Lesson: The maths of sets

Number of subsets (the power set)

Exam favourite

$$2^n$$

Each item is in or out. {1, 2, 3} has 8 subsets.

Lesson: The maths of sets

De Morgan's laws

$$\text{not } (A \text{ or } B) = (\text{not } A) \text{ and } (\text{not } B)$$

For sets inside a universe U: U − (A ∪ B) = (U − A) ∩ (U − B).

Lesson: The maths of sets

TABLES

Table: you can · example

You can	Example
Index and slice	t[0], t[-1], t[1:]
Join and repeat	(1, 2) + (3,), (0,) * 4
Search	x in t, t.index(x), t.count(x)
Loop and unpack	for a, b in pairs, x, y = point
Not: append, remove, sort, item assignment	Use sorted(t) to get a sorted list

Lesson: Making and using tuples

Table: · tuple · list

	tuple	list
Brackets	( )	[ ]
Can change	No	Yes
Hashable (dict key, set item)	Yes, if every item is hashable	No
Memory for (1, 2, 3)	72 bytes on CPython 3.14	88 bytes
Methods	count, index	about 11, including append and sort
Meaning	A fixed record: (name, age), (x, y)	Many items of the same kind that grows or shrinks

Lesson: Tuple or list

Table: method · does · if the item is missing

Method	Does	If the item is missing
add(x)	Adds x (nothing happens if it is already there)	
remove(x)	Removes x	KeyError
discard(x)	Removes x	Nothing happens
pop()	Removes and returns some item (you cannot choose which)	KeyError on an empty set
update(items)	Adds every item of an iterable	
clear()	Empties the set	

Lesson: Making sets and changing them

Table: operator · method · means

Operator	Method	Means
A | B	A.union(B)	In A or B or both
A & B	A.intersection(B)	In both
A – B	A.difference(B)	In A but not in B
A ^ B	A.symmetric_difference(B)	In exactly one of them
A <= B	A.issubset(B)	Every item of A is in B
A < B		Subset and not equal
	A.isdisjoint(B)	No item in common

Lesson: Union, intersection, difference

Table: · x in list · x in set

	x in list	x in set
Average time	O(n)	O(1)
Keeps order and repeats	Yes	No
Items must be	Anything	Hashable
Extra memory	Less	More (the table has empty slots)

Lesson: Why sets are fast

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
217	Contains Duplicate	Easy	Seen set	Add each number to a set. If it is already there, you found a duplicate.
349	Intersection of Two Arrays	Easy	Set intersection	Turn both lists into sets and take the intersection.
771	Jewels and Stones	Easy	Set membership	Put the jewel letters in a set, then count the stones that are in it.
1832	Check if the Sentence Is Pangram	Easy	Set size	A pangram uses every letter a to z. The set of its letters has 26 items.
268	Missing Number	Easy	Gauss sum	The numbers 0 to n add up to n(n + 1)/2. The missing one is that total minus the sum of the list.
2215	Find the Difference of Two Arrays	Easy	Set difference	A – B and B – A answer the two parts.
1436	Destination City	Easy	Set difference	The destination is a city that appears as an end but never as a start.
645	Set Mismatch	Easy	Set and Gauss sum	The repeated number is sum(nums) – sum(set(nums)). The missing number is n(n + 1)/2 – sum(set(nums)).
3	Longest Substring Without Repeating Characters	Medium	Sliding window with a set	Grow the window on the right. While the new letter is already in the window, drop letters from the left.
128	Longest Consecutive Sequence	Medium	Set, start only at the beginning of a run	Put all numbers in a set. Start counting only from x where x – 1 is not in the set, so each run is walked once: O(n).

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM The comma makes a tuple, not the brackets. (5) is the number 5. (5,) is a tuple with one item. This is a favourite exam trick.

Lesson: Making and using tuples

COMMON MISTAKE A tuple holds references. You cannot change which objects it holds, but if one of them is a list, that list can still change. Such a tuple is not hashable, so it cannot be a dict key.

Lesson: What "cannot change" means

REMEMBER A named tuple gives each position a name, so p.x is clearer than p[0]. For records with many fields or that need to change, use a dataclass (Advanced 1.3).

Lesson: Tuples as keys, and named tuples

IN THE EXAM {} is an empty dict, not an empty set. Write set() for an empty set. Also, a set can only hold hashable items: numbers, strings, tuples of them, but not lists or other sets.

Lesson: Making sets and changing them

IN THE EXAM The operators need two sets. The methods accept any iterable, so A.union([1, 2]) works but A | [1, 2] raises TypeError.

Lesson: Union, intersection, difference

REMEMBER set(items) is the fastest way to remove repeats, but the order is lost. If order matters, use list(dict.fromkeys(items)), which keeps the first time each item appears.

Lesson: Removing duplicates

REMEMBER A frozenset is a set that cannot change. Because it never changes it is hashable, so it can be a dict key or an item of another set.

Lesson: frozenset and set comprehensions

3.4 Dictionaries: keys, values and hash tables

dict basics, get and setdefault, keys values items, counting, grouping, nesting and JSON, merging, hash table internals.

KEY IDEAS

- **Dicts** Key to value mapping. Keys unique and hashable, values anything. Ordered by insertion since 3.7.
- **Read safely** d[k] raises KeyError. d.get(k, default) does not. k in d checks keys only.
- **Change** d[k] = v adds or replaces. del d[k], d.pop(k, default), d.popitem() (last added), update, |, |=.
- **Loop** for k in d (keys), d.values(), d.items(). Do not add or remove keys while looping, use list(d).
- **Patterns** Count: d[k] = d.get(k, 0) + 1 or Counter. Group: dict of lists or defaultdict(list). Two Sum: dict of seen values.

FORMULAS

Slot of a key — Exam favourite

$$\text{slot} = \text{hash}(\text{key}) \bmod \text{table size}$$

Table sizes are powers of 2, so this is just the last few bits of the hash.

Lesson: The maths of hashing

Load factor — Exam favourite

$$\alpha = \text{used slots} / \text{table size} \leq 2/3$$

A CPython dict with 8 slots holds at most 5 keys. The 6th key makes it grow.

Lesson: The maths of hashing

New table size

$$\text{smallest power of } 2 \geq 3 \times (\text{number of keys})$$

When the 6th key arrives, 5 keys are stored: 3 × 5 = 15, so the new size is 16.

Lesson: The maths of hashing

Expected probes for a lookup that hits (uniform hashing)

$$\text{about } (1/\alpha)\ \ln(1/(1-\alpha))$$

At α = 2/3 this is about 1.6 probes, so lookups stay fast.

Lesson: The maths of hashing

Chance that two of k keys share a slot of m (birthday problem)

$$1 - e^{-k(k-1)/(2m)}$$

With 23 people and 365 days the chance of a shared birthday is above 50 percent. Collisions are normal, so tables must handle them.

Lesson: The maths of hashing

TABLES

Table: code · does · if the key is missing

Code	Does	If the key is missing
d[k] = v	Adds k, or replaces its value	Adds it
del d[k]	Removes k	KeyError
d.pop(k)	Removes k and returns its value	KeyError
d.pop(k, default)	Same, with a default	Returns default
d.popitem()	Removes and returns the last added pair	KeyError on an empty dict
d.update(other)	Adds or replaces every pair of other	Adds them
d.clear()	Empties the dict	

Lesson: Adding, changing and removing

Table: · d[k] · d.get(k, default) · k in d first

	d[k]	d.get(k, default)	k in d first
Missing key	KeyError	Returns default (None if not given)	You decide in an if
Use when	The key must be there and a missing key is a bug	A missing key is normal	You need different code for the two cases

Lesson: Safe lookups: get and setdefault

Table: way · result

Way	Result
a | b	A new dict. For equal keys the value from b wins
a |= b or a.update(b)	Changes a in place
{**a, **b}	A new dict, works before Python 3.9
{k: v for ...}	A dict comprehension
dict.fromkeys(keys, value)	Every key gets the same value
dict(zip(keys, values))	Pairs keys and values from two lists

Lesson: Merging, comprehensions and other ways to build

Table: · two lists (names, marks) · list of pairs · dict

	two lists (names, marks)	list of pairs	dict
Get Ravi's mark	Find the index in names, then marks[i]: O(n)	Loop until the name matches: O(n)	marks["Ravi"]: O(1)
Add a student	Append to both lists and keep them in step	Append a pair	marks["Zara"] = 80
Repeated names	Allowed (can be a bug)	Allowed	Not possible, the key is unique
Keeps order	Yes	Yes	Yes (insertion order)

Lesson: dict compared with list and other mappings

Table: tool · use it for

Tool	Use it for
dict	Any key to value lookup
collections.Counter	Counting things
collections.defaultdict	Grouping, when missing keys need a starting value
collections.OrderedDict	Moving keys to the end (move_to_end), an LRU cache
types.MappingProxyType	A read only view of a dict

Lesson: dict compared with list and other mappings

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1	Two Sum	Easy	Dict of seen values	For each number x, the partner is target − x. Keep a dict from value to index of the numbers seen so far.
242	Valid Anagram	Easy	Count letters	Two words are anagrams when every letter appears the same number of times.
387	First Unique Character in a String	Easy	Count, then scan again	First count every letter. Then walk the text again and return the first index whose letter count is 1.
383	Ransom Note	Easy	Count and spend	Count the magazine letters. Each letter of the note uses one up.
1512	Number of Good Pairs	Easy	Count as you go	Each new copy of a value makes a pair with every earlier copy, so add the count so far before increasing it.
169	Majority Element	Easy	Count, or Boyer Moore voting	Count each value and return the one seen more than n/2 times. The O(1) memory trick is the Boyer Moore vote.
205	Isomorphic Strings	Easy	Two dicts, one per direction	Each letter of s must always map to the same letter of t, and no two letters may map to the same one.
290	Word Pattern	Easy	Two dicts, one per direction	Split the text into words. Then it is Isomorphic Strings with letters on one side and words on the other.
49	Group Anagrams	Medium	Group by a sorted key	Anagrams have the same letters in sorted order. Use that sorted text as the dict key and a list of words as the value.
347	Top K Frequent Elements	Medium	Count, then bucket by count	Count each number. Put numbers into buckets by their count, then read the buckets from the highest count down.
560	Subarray Sum Equals K	Medium	Prefix sums in a dict	A part from i to j sums to k when prefix[j] − prefix[i] = k. Count how often each prefix sum has appeared.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM for x in d gives the keys, not the values and not the pairs. Use d.values() for values and d.items() for (key, value) pairs. x in d also checks keys only.

Lesson: Looping over a dict

COMMON MISTAKE keys(), values() and items() return **views**: windows onto the dict that update by themselves. Adding or removing keys while you loop over a dict raises RuntimeError. Loop over list(d) instead.

Lesson: Views, and changing a dict while looping

REMEMBER Counting is the most common use of a dict. Learn the get(word, 0) + 1 line by heart, then use collections.Counter in real code. Counter also adds, subtracts and finds the most common items.

Lesson: Counting with a dict

REMEMBER A dict of dicts and lists looks exactly like JSON, the text format that web APIs use. json.dumps turns a dict into JSON text and json.loads turns it back. The web part of this subject uses this all the time.

Lesson: Nested dicts and JSON

COMMON MISTAKE dict.fromkeys(keys, []) gives every key the same list object. Appending to one changes all of them. Use a comprehension {k: [] for k in keys} instead.

Lesson: Merging, comprehensions and other ways to build

IN THE EXAM Keys are compared with ==. Because 1 == 1.0 == True and they have the same hash, they are the same key: the dict keeps the first key object and the last value. Use one type for your keys.

Lesson: What can be a key

STEP 4 Core Python · Functions

4.1 Functions: def, return and every kind of argument

def, return, None, argument kinds, *args, **kwargs, / and *, unpacking, mutable defaults, docstrings, first-class functions.

KEY IDEAS

- **def and return** def name(params): body. return value leaves the function. No return gives None. print is not return.
- **Argument kinds** Positional by order, keyword by name, positional before keyword. Defaults make parameters optional.
- **Star parameters** *args tuple of extra positionals, **kwargs dict of extra keywords. Bare * means keyword-only after it, / means positional-only before it.
- **Mutable default** Defaults are made once at def time. Use None, then create the list inside.
- **Functions are objects** Assign them, store them, pass them: sorted(key=f), map(f, items). Docstring on the first line, type hints are not enforced.

FORMULAS

A function maps each input to one output

Exam favourite

$$f: X \to Y, x \mapsto f(x)$$

area(width, height) maps each pair of numbers to one number.

Lesson: Functions in maths and in code

Composition Exam favourite

$$(f \circ g)(x) = f(g(x))$$

apply_twice(f, x) is f(f(x)). len(str(n)) is len ∘ str.

Lesson: Functions in maths and in code

Idempotent function

$$f(f(x)) = f(x)$$

abs, str.strip and sorted are idempotent: doing them twice changes nothing more.

Lesson: Functions in maths and in code

How many functions from a set of m inputs to n outputs

$$n^m$$

Each of the m inputs can go to any of the n outputs. From {1, 2, 3} to {a, b}: $2^3 = 8$.

Lesson: Functions in maths and in code

Arguments a call may give with p positional parameters, d of them with defaults and no *args

between p − d and p positional arguments

describe has p = 3, d = 1, so 2 or 3 positional arguments.

Lesson: Functions in maths and in code

TABLES

Table: · print(x) · return x

	print(x)	return x
What it does	Shows x on the screen	Gives x back to the code that called the function
Can the caller use the value	No, it is gone	Yes: store it, add it, pass it on
The call evaluates to	None	x
Stops the function	No	Yes, at once

Lesson: return, print and None

Table: kind of argument · example call · rule

Kind of argument	Example call	Rule
Positional	describe("Ravi", 23)	Matched by position, left to right
Keyword	describe(age=30, name="Meera")	Matched by name, any order
Mixed	describe("John", city="Goa", age=40)	All positional arguments must come before any keyword argument
Default value	def describe(..., city="Pune")	Used when the caller gives nothing for that parameter

Lesson: Positional, keyword and default arguments

Table: in the def line · meaning · inside the function it is

In the def line	Meaning	Inside the function it is
a	Positional or keyword	The value
a=1	Optional, with a default	The value or the default
*args	Collects extra positional arguments	A tuple
**kwargs	Collects extra keyword arguments	A dict
* alone	Every parameter after it is keyword-only	
/	Every parameter before it is positional-only (Python 3.8+)	

Lesson: *args, **kwargs, keyword-only and positional-only

Table: · copy and paste · function · method (core 6.1)

	Copy and paste	Function	Method (Core 6.1)	Module (Core 5.2)
Defined with	Nothing	def	def inside a class	a .py file
Fix a bug in	Every copy	One place	One place	One place
Called as		name(args)	obj.name(args)	module.name(args)
Best for	Never	One job	A job tied to an object	Sharing many functions between programs

Lesson: Kinds of reusable code compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1342	Number of Steps to Reduce a Number to Zero	Easy	Loop inside a function	While num > 0: halve it if even, else take away 1. Count the steps and return the count.
1920	Build Array from Permutation	Easy	Index into a list	ans[i] is nums[nums[i]]. Build a new list and return it.
1470	Shuffle the Array	Easy	Two halves side by side	Walk i from 0 to n − 1 and add nums[i] then nums[i + n].
2011	Final Value of Variable After Performing Operations	Easy	A helper decides plus or minus	Every operation contains either "++" or "--". Add 1 or take away 1.
1365	How Many Numbers Are Smaller Than the Current Number	Easy	Helper function per item	Write a small helper that counts the numbers smaller than x, then call it for every item.
1431	Kids With the Greatest Number of Candies	Easy	Find the max once	Work out max(candies) once. Kid i can be the greatest if candies[i] + extra reaches it.
2652	Sum Multiples	Easy	Small test function	Add every number from 1 to n that divides by 3, 5 or 7.
1323	Maximum 69 Number	Easy	Change the first 6	Turn the number into text, replace the first 6 with a 9, and turn it back.
2160	Minimum Sum of Four Digit Number After Splitting Digits	Easy	Sort the digits	Sort the 4 digits a ≤ b ≤ c ≤ d. The best split is (10a + c) + (10b + d).
1641	Count Sorted Vowel Strings	Medium	Maths in a function	A sorted string of length n over 5 vowels is a choice of n items from 5 with repeats: C(n + 4, 4).

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM — A **parameter** is the name in the def line (name, width). An **argument** is the value you pass in the call ("Asha", 4). Exams use both words, so learn the difference.

Lesson: What a function is

IN THE EXAM — A function with no return, or with a bare return, gives back **None**. That is why a = show_double(5) printed 10 but left a as None.

Lesson: return, print and None

IN THE EXAM — describe(name="Sara", 25) is a SyntaxError: a positional argument may not follow a keyword argument. Python refuses to even start the program.

Lesson: Positional, keyword and default arguments

REMEMBER — The full order is: positional-only, /, normal, *args (or a bare *), keyword-only, **kwargs. The names args and kwargs are only a habit. What matters is the * and **.

Lesson: *args, **kwargs, keyword-only and positional-only

IN THE EXAM

In a def line, * and ** **collect** arguments. In a call, * and ** **spread** a list or dict out into separate arguments. Same symbols, opposite jobs.

Lesson: Spreading lists and dicts into a call

REMEMBER

A **docstring** is a string on the first line of the body. help(bmi) and editors show it. **Type hints** such as height_m: float and -> float tell readers and tools like mypy what you expect, but Python itself does not check them when the program runs (Advanced 3.1).

Lesson: Docstrings and type hints

REMEMBER

A function name is just a variable that holds a function object. You can give it another name, store it in a list or dict, and pass it to another function. Functions that take or return functions are called **higher-order functions**. Core 4.3 uses this with sorted, map, filter and closures.

Lesson: Functions are objects too

4.2 Scope and recursion: LEGB, closures and the call stack

scope, LEGB, global, nonlocal, closures, recursion, call stack, recursion limit, memoisation, recurrences.

KEY IDEAS

- **Scopes** Local (inside a call), Enclosing, Global (module), Built-in: LEGB. Assigning makes a name local in the whole function.
- **global and nonlocal** global x assigns the module x. nonlocal x assigns the enclosing function x. Reading needs neither.
- **Closures** An inner function that remembers enclosing names (cells). Used for counters, factories and decorators.
- **Recursion** Base case + smaller recursive case. Each call is a frame. Limit about 1000, RecursionError.
- **Costs** T(n) = T(n-1) + c is O(n). Naive fib 2F(n+1) - 1 calls, lru_cache makes it n + 1. Hanoi 2^n - 1. Fast power O(log n).

FORMULAS

Recurrence for a simple recursion — Exam favourite

$$T(n) = T(n-1) + c \text{ gives } T(n) = O(n)$$

factorial and countdown do constant work and then one smaller call.

Lesson: The maths of recursion

Calls made by naive fib(n) — Exam favourite

$$C(n) = C(n-1) + C(n-2) + 1 = 2F(n+1) - 1$$

fib(5) makes $2 \times 8 - 1 = 15$ calls, as the animation counts.

Lesson: The maths of recursion

Golden ratio growth

$$F(n) \approx \varphi^n / \sqrt{5},\ \varphi = (1+\sqrt{5})/2 \approx 1.618$$

So naive fib is exponential: about 1.6^n calls.

Lesson: The maths of recursion

Tower of Hanoi moves — Exam favourite

$$M(n) = 2M(n-1) + 1 = 2^n - 1$$

3 discs need 7 moves, 64 discs need about 1.8×10^{19}.

Lesson: The maths of recursion

Fast power

$$T(n) = T(n/2) + c \text{ gives } T(n) = O(\log n)$$

power(2, 1000) needs only about 10 levels of calls.

Lesson: The maths of recursion

Master theorem (divide and conquer)

$$T(n) = a \cdot T(n/b) + O(n^d)$$

If $d > \log_b a$: $O(n^d)$. If equal: $O(n^d \log n)$. If less: $O(n^{\log_b a})$. Merge sort: a = 2, b = 2, d = 1 gives O(n log n).

Lesson: The maths of recursion

TABLES

Table: letter · scope · made by

Letter	Scope	Made by
L	Local	Assignments and parameters inside the current function call
E	Enclosing	The local names of any outer function, for a function inside a function
G	Global	Assignments at the top level of the module (the .py file)
B	Built-in	Names Python gives you: print, len, range, int, ValueError ...

Lesson: The LEGB rule

Table: · global variable · pass in and return

	global variable	pass in and return
Code	global visits then visits += 1	total = next_count(total)
Who can change it	Any function, from anywhere	Only code that gets the returned value
Testing	Hard: the result depends on hidden state	Easy: same input, same output
Good for	Settings read in many places, small scripts	Almost everything else

Lesson: The global statement

Table: part · factorial · question to ask

Part	factorial	Question to ask
Base case	if n <= 1: return 1	What is the smallest input I can answer at once?
Recursive case	return n * factorial(n - 1)	How does the answer use the answer for a smaller input?
Progress	n − 1 is smaller than n	Does every call get closer to the base case?

Lesson: Recursion: base case and recursive case

Table: · recursion · loop

	Recursion	Loop
Fits best	Nested data: trees, folders, JSON, divide and conquer	Walking a list, counting, simple repeats
Memory	One frame per pending call: O(depth)	Usually O(1)
Depth limit in Python	About 1,000	None
Speed in CPython	Slower (each call has a cost)	Faster
Readability	Very clear when the problem is recursive	Very clear when it is a sequence

Lesson: Recursion or a loop

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
509	Fibonacci Number	Easy	Recursion with memo or a loop	F(0) = 0, F(1) = 1. Use two names a and b and move them forward n times.
70	Climbing Stairs	Easy	Recurrence ways(n) = ways(n − 1) + ways(n − 2)	The last step was 1 or 2 stairs. It is Fibonacci shifted by one.
326	Power of Three	Easy	Divide while you can	Keep dividing by 3 while n % 3 == 0. A power of three ends at 1.
342	Power of Four	Easy	Recursive halving	Base cases 1 (True) and anything not divisible by 4 (False). Otherwise ask the same question for n // 4.
1688	Count of Matches in Tournament	Easy	Recursion, then a maths shortcut	Each round plays n // 2 matches and n − n // 2 teams go on. In the end every team but one loses once: n − 1 matches.
50	Pow(x, n)	Medium	Fast power by halving	$x^n = (x^{n/2})^2$, times x when n is odd. For negative n use $1 / x^{-n}$.
22	Generate Parentheses	Medium	Backtracking	Build the text one bracket at a time. Add "(" while you have some left, and ")" only when it closes an open one.
46	Permutations	Medium	Backtracking	Pick each unused number for the next place, recurse, then undo the choice.
78	Subsets	Medium	Take it or leave it	For each number, make two recursive calls: one that takes it and one that skips it.
779	K-th Symbol in Grammar	Medium	Recursion on the parent	Symbol k of row n comes from symbol (k + 1) // 2 of row n − 1. An odd k copies the parent, an even k flips it.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM — Assigning to a name inside a function makes a new local name, even if a global with the same name exists. This is called **shadowing**. Changing an object (items.append) is different from rebinding a name (items = [...]): the first is seen by the caller, the second is not.

Lesson: Local names hide global names

IN THE EXAM — Python decides which names are local when it compiles the function, not while it runs. If a name is assigned anywhere in the function, it is local in the whole function.

Lesson: The LEGB rule

REMEMBER — The fix is either to rename the local variable, to pass count in as a parameter, or, if you really mean the global, to add global count at the top of the function.

Lesson: UnboundLocalError

IN THE EXAM — global reaches the module scope. nonlocal reaches the nearest enclosing function scope and is a SyntaxError at the top level. Decorators (Advanced 1.2) are built from closures.

Lesson: nonlocal and closures

COMMON MISTAKE — Each call uses a frame on the call stack. To protect the program, CPython stops at about 1,000 frames deep with RecursionError. sys.setrecursionlimit can raise the limit, but a recursion that deep is usually better written as a loop.

Lesson: The recursion limit

REMEMBER

total(items[1:]) is clear but copies the list each time, which makes it O(n^2). Real code uses sum(). Fast power is the opposite: halving the exponent makes it O(log n), which is how pow(a, b, m) handles huge numbers in cryptography.

Lesson: Classic recursive functions

4.3 lambda, map, filter and comprehensions

lambda, key functions, map, filter, reduce, list, dict and set comprehensions, generator expressions, higher-order functions.

KEY IDEAS

- **lambda** lambda args: expression. One expression, no statements. Best as an argument, def otherwise.
- **key functions** sorted/min/max(items, key=f). Tuple keys sort by several fields. -x reverses a number key.
- **map, filter, reduce** Lazy iterators: list(map(f, a)), list(filter(test, a)). functools.reduce(f, a, start) folds to one value.
- **Comprehensions** [expr for x in a if test], [a if t else b for x in s], {k: v for ...}, {x for ...}. Outer for first.
- **Generator expressions** (expr for x in a): lazy, one use, tiny memory. sum(x*x for x in a), any(...), all(...).

FORMULAS

List comprehension Exam favourite

[expression for item in iterable if condition]

Read it as: the expression, for each item, but only if the condition holds.

Lesson: List comprehensions

With if-else Exam favourite

[a if test else b for item in iterable]

The if-else is part of the expression, so it comes before for. A filter if comes after.

Lesson: List comprehensions

Set builder notation Exam favourite

$$\{ f(x) \mid x \in S, p(x) \}$$

Maths wrote "the set of f(x) for x in S where p(x)" long before Python. {f(x) for x in S if p(x)} is the same idea.

Lesson: The maths behind comprehensions

map is f applied to each item

$$\text{map}(f, [a_1, ..., a_n]) = [f(a_1), ..., f(a_n)]$$

The length does not change.

Lesson: The maths behind comprehensions

reduce is a fold Exam favourite

$$\text{reduce}(\oplus, [a_1, a_2, a_3], s) = ((s \oplus a_1) \oplus a_2) \oplus a_3$$

With + this is the sum, with max it is the maximum.

Lesson: The maths behind comprehensions

Size of a comprehension with two for clauses

$$|A| \times |B|$$

[(x, y) for x in range(1, 4) for y in "ab"] has 3 × 2 = 6 pairs, the Cartesian product A × B.

Lesson: The maths behind comprehensions

Sum of squares

$$1^2 + 2^2 + ... + n^2 = n(n + 1)(2n + 1)/6$$

sum(n * n for n in range(1, 11)) = 10 × 11 × 21 / 6 = 385.

Lesson: The maths behind comprehensions

TABLES

Table: · lambda · def

	lambda	def
Body	One expression, its value is returned	Any statements, return when you want
Name	Anonymous (shown as <lambda>)	Has a real name for errors and tracebacks
Docstring and type hints	No	Yes
Best for	A tiny throw away function passed to sorted, max, map	Anything you name, reuse or test

Lesson: lambda: a function in one expression

Table: tool · takes · gives · comprehension version

Tool	Takes	Gives	Comprehension version
map(f, items)	A function and one or more iterables	A lazy iterator of f(x)	[f(x) for x in items]
filter(test, items)	A test function (or None) and an iterable	A lazy iterator of the x where test(x) is true	[x for x in items if test(x)]
reduce(f, items, start)	A function of two values	One value	A loop with an accumulator, or sum, max, min

Lesson: map, filter and reduce

Table: brackets · builds · example

Brackets	Builds	Example
[ ]	list	[w.upper() for w in words]
{ } with key: value	dict	{w: len(w) for w in words}
{ } with one value	set	{w[0] for w in words}
( )	generator (lazy, not a tuple)	(len(w) for w in words)

Lesson: Dict and set comprehensions

Table: · [... for ...] · (... for ...)

	[... for ...]	(... for ...)
Makes	The whole list now	One value at a time, when asked
Memory for a million items	Megabytes	A couple of hundred bytes
Use again	Yes, as often as you like	No, once only
Index, len, slicing	Yes	No
Best for	Results you keep or reuse	Feeding sum, max, any, all, join, or a for loop once

Lesson: Generator expressions: comprehensions without the list

Table: · for loop · comprehension · map and filter

	for loop	comprehension	map and filter
Code	result = [] then for n in nums: if n % 2 == 0: result.append(n * n)	[n * n for n in nums if n % 2 == 0]	list(map(lambda n: n * n, filter(lambda n: n % 2 == 0, nums)))
Lines	4	1	1
Reads like	Instructions	A sentence	Nested calls, inside out
Speed in CPython	Slowest	Fast	Fast with built-in functions, slower with lambdas
Good when	Several steps, prints, try and except in the body	One expression and an optional filter	You already have a named function: map(str.upper, words)

Lesson: Loop, comprehension or map

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
977	Squares of a Sorted Array	Easy	Comprehension and sorted	Square every number with a comprehension, then sort. (A two pointer version does it in O(n).)
867	Transpose Matrix	Easy	Nested comprehension	Row c of the answer is column c of the matrix.
1572	Matrix Diagonal Sum	Easy	Generator expression with sum	Add mat[i][i] and mat[i][n − 1 − i]. When n is odd the middle cell was added twice.
2942	Find Words Containing Character	Easy	Comprehension with enumerate	Keep the index i of every word that contains x.
1748	Sum of Unique Elements	Easy	Count, then sum with a filter	Count each value, then add the values whose count is 1.
1913	Maximum Product Difference Between Two Pairs	Easy	sorted	Biggest two times each other, minus smallest two times each other.
1636	Sort Array by Increasing Frequency	Easy	sorted with a tuple key	Sort by (count, −value): rarer numbers first, and for equal counts the bigger number first.
451	Sort Characters By Frequency	Medium	Count, sort by count, join	Count each letter, sort the letters by count from high to low, and repeat each one count times.
791	Custom Sort String	Medium	sorted with a dict key	Give each letter of order its position. Letters not in order can go anywhere, here at the end.
179	Largest Number	Medium	Custom comparison with cmp_to_key	Put a before b when a + b is bigger than b + a as text. Join, and turn "000" into "0".

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM A lambda cannot contain statements such as if blocks, for loops, return or assignment with =. It can contain a conditional expression: lambda n: "even" if n % 2 == 0 else "odd".

Lesson: lambda: a function in one expression

REMEMBER key is a function that is called once per item. Python sorts by what it returns. To sort by two things, return a tuple: tuples compare item by item. To reverse just one of two numeric keys, make it negative.

Lesson: Sorting and choosing with key

COMMON MISTAKE map and filter return iterators, not lists. They give each value once, only when asked. Wrap them in list() to see all the values, and do not loop over the same map object twice.

Lesson: map, filter and reduce

IN THE EXAM [n for n in nums if n % 2 == 0 else 0] is a SyntaxError. An if after for only filters and has no else.
Lesson: List comprehensions

REMEMBER In [v for row in matrix for v in row] the for clauses are in the same order as nested loops: outer first. A comprehension inside the expression ([[... for c] for r]) builds a list of lists. If it needs more than two for clauses, a normal loop is easier to read.
Lesson: Nested comprehensions

REMEMBER A **higher-order function** takes a function or returns one: sorted, map, filter, make_greeter, compose and partial are all examples. functools.partial is often clearer than a lambda that only fixes some arguments.
Lesson: Higher-order functions, compose and partial

STEP 5 Core Python · Errors, modules and files

5.1 Exceptions: try, except, else, finally and raise

try, except, else, finally, exception hierarchy, propagation, raise, custom exceptions, assert, EAFP, with.

KEY IDEAS

- **try and except** try: risky code. except X as err: handles X and its children. First matching handler only.
- **else and finally** else runs if no exception. finally always runs (cleanup). Order: try, except..., else, finally.
- **Hierarchy** BaseException > Exception > ArithmeticError > ZeroDivisionError. LookupError > KeyError, IndexError. KeyboardInterrupt is not an Exception.
- **raise** raise ValueError("msg"). Bare raise re-raises. raise New() from err sets __cause__. Custom: class MyError(Exception).
- **Style** Catch specific classes, never bare except. assert is for programmer checks (-O removes it). EAFP. with closes resources.

FORMULAS

Handlers checked before a match Exam favourite

position of the first matching except clause

Python stops at the first match, so order decides the cost and the meaning.

Lesson: Counting and reasoning about failures

Chance that at least one of n independent steps fails

Exam favourite

$$1 - (1 - p)^n$$

With p = 1 percent per network call, 50 calls fail at least once with chance $1 - 0.99^{50} \approx 39$ percent. Errors are normal at scale, so handle them.

Lesson: Counting and reasoning about failures

Expected tries until success (retry loop)

$$1 / q$$

If each try succeeds with chance q = 0.8, you need 1.25 tries on average (a geometric distribution).

Lesson: Counting and reasoning about failures

Exponential back off before retry k

$$\text{wait} = \text{base} \times 2^k$$

1, 2, 4, 8 seconds. Used by web clients so a busy server is not flooded (Web 2.2).

Lesson: Counting and reasoning about failures

TABLES

Table: · syntaxerror · exception at run time

	SyntaxError	Exception at run time
When	Before the program starts, while Python reads the file	While the program runs
Example	print("hi" (a missing bracket)	int("seven"), 10 / 0, {}["x"]
Can try and except catch it	Not in the same file, which never starts	Yes
Fix	Correct the code	Handle it, or prevent the cause

Lesson: try and except

Table: exception · typical cause

Exception	Typical cause
ValueError	Right type, wrong value: int("seven")
TypeError	Wrong type: "2" + 3, len(5)
ZeroDivisionError	Dividing by zero with / // or %
IndexError	List index out of range: [1, 2][5]
KeyError	Missing dict key
NameError	A name that was never defined (UnboundLocalError is its child)
AttributeError	A missing attribute or method: (5).append(1)
FileNotFoundError	Opening a file that does not exist (a child of OSError)
ImportError and ModuleNotFoundError	A module that cannot be found or loaded
RecursionError	Too deep recursion (a child of RuntimeError)
KeyboardInterrupt	Ctrl+C. A child of BaseException, not of Exception

Lesson: Several handlers, and the exception object

Table: clause · runs when

Clause	Runs when
try	Always first
except X	An exception of class X (or a child) was raised in try
else	try finished with no exception
finally	Always last: after try, except or else, even after return, break or an unhandled exception

Lesson: else and finally

Table: form · meaning

Form	Meaning
raise ValueError("message")	Raise a new exception with a message
raise ValueError	Python makes the object for you, with no message
raise (alone, inside except)	Raise the same exception again, after logging or cleaning up
raise NewError(...) from err	Raise a new exception and record err as its cause (__cause__)
raise NewError(...) from None	Hide the original exception from the traceback

Lesson: Raising exceptions yourself

Table: · assert condition, message · if not condition: raise ...

	assert condition, message	if not condition: raise ...
Meant for	Things that can only fail if the code itself has a bug	Bad input, missing files, anything the outside world causes
Turned off by python -O	Yes	No
Exception	AssertionError	Any class you choose

Lesson: assert: checks for programmers

Table: · lbyl (look before you leap) · eafp (ask forgiveness)

	LBYL (look before you leap)	EAFP (ask forgiveness)
Code	if key in d: use d[key]	try: use d[key] then except KeyError
Fast when	Errors are common	Errors are rare (no extra check on the normal path)
Race conditions	Possible: a file can disappear between the check and the open	None: the action itself is the check
Python style	Fine for simple checks	Preferred by most Python code

Lesson: Two styles: EAFP and LBYL

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
20	Valid Parentheses	Easy	Stack	Push every opening bracket. A closing bracket must match the top of the stack. At the end the stack must be empty.
1614	Maximum Nesting Depth of the Parentheses	Easy	Counter	Add 1 for "(" and take 1 for ")". The answer is the largest value the counter reaches.
1021	Remove Outermost Parentheses	Easy	Depth counter	Keep a bracket only when it is not at depth 0 before an opening or after a closing.
682	Baseball Game	Easy	Stack of scores	"+" adds the last two, "D" doubles the last, "C" removes the last, anything else is a number.
8	String to Integer (atoi)	Medium	Careful parsing	Skip leading spaces, read one optional sign, read digits until something else, then clamp to the 32 bit range.
150	Evaluate Reverse Polish Notation	Medium	Stack	Numbers go on a stack. An operator pops two numbers, applies itself and pushes the result. Division truncates toward zero.
468	Validate IP Address	Medium	Validate every part	IPv4: 4 parts of digits, 0 to 255, no leading zero. IPv6: 8 parts of 1 to 4 hex digits. Check characters yourself, because int() also accepts signs and spaces.
71	Simplify Path	Medium	Stack of folder names	Split on "/". Skip "" and ".", pop for "..", push anything else. Join with "/".
227	Basic Calculator II	Medium	Stack with the previous operator	Read a number, then apply the operator before it: + and – push, * and / combine with the top of the stack. Sum the stack.
65	Valid Number	Hard	Scan with a few flags	Track whether you saw a digit, a dot and an exponent. A sign is allowed only at the start or right after e. A dot is not allowed after e.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER Read a traceback from the bottom up. The last line names the exception and the reason. The lines above show where it happened, with the newest call last. Python 3.11 and later mark the exact part of the line with ^ and ~ characters.

Lesson: When things go wrong: exceptions

COMMON MISTAKE Avoid a bare except: or except BaseException:. They also catch KeyboardInterrupt and SystemExit, so Ctrl+C stops working. Catch Exception at most, and only where you log or report the error.

Lesson: The exception hierarchy and the order of handlers

IN THE EXAM The order is fixed: try, then except clauses, then else, then finally. You need at least one except or a finally. else keeps code out of the try block, so you do not accidentally catch errors from it.

Lesson: else and finally

REMEMBER A low level function often cannot know what the right reaction is. average should not print "No marks yet" because it may be used in a web page or a report. Let it raise, and handle the exception at the level that knows what to tell the user. This is called delegating responsibility.

Lesson: Exceptions travel up through function calls

REMEMBER Make your own exceptions by subclassing Exception (never BaseException). Give your program one base class, such as BankError, and make specific errors its children. Callers can then catch one exact problem or every problem of your program. Classes are explained in Core 6.1.

Lesson: Your own exception classes

REMEMBER A with statement uses a context manager that closes the file (or releases a lock, or ends a database transaction) even when an exception happens. It is the short, safe form of try and finally. Advanced 2.2 shows how to write your own.

Lesson: Cleaning up: finally and with

5.2 Modules and packages: import, the standard library and pip

import forms, sys.modules and sys.path, math, random, platform, own modules and packages, __all__, main guard, pip and venv.

KEY IDEAS

- **import forms** import m, import m as a, from m import x, from m import x as y. Avoid import *. Relative: from . import x.
- **How import works** Check sys.modules, search sys.path, run the file once, cache it, bind the name. importlib.reload re-runs.
- **math and random** ceil, floor, trunc, sqrt, isqrt, hypot, factorial, comb, gcd, log, isclose. randint(a, b) includes b, randrange excludes stop, seed repeats.
- **Packages** Folder + __init__.py. _name is private by convention. __all__ controls import *. __pycache__ holds .pyc files.
- **Main guard and pip** if __name__ == "__main__": main(). pip install inside a venv. pip freeze > requirements.txt.

FORMULAS

Combinations and permutations Exam favourite

$$C(n, k) = n! / (k!(n - k)!), P(n, k) = n! / (n - k)!$$

math.comb(5, 2) = 10, math.perm(5, 2) = 20.

Lesson: Maths you get from math and random

Euclid: greatest common divisor Exam favourite

$$\gcd(a, b) = \gcd(b, a \bmod b), \gcd(a, 0) = a$$

gcd(84, 36) = gcd(36, 12) = gcd(12, 0) = 12. lcm(a, b) = a × b / gcd(a, b).

Lesson: Maths you get from math and random

Change of base for logarithms

$$\log_b x = \ln x / \ln b$$

math.log(8, 2) = 3.0. Floats can make this 2.9999999999999996 for other numbers, so use log2 for base 2.

Lesson: Maths you get from math and random

Expected value of a fair die (randint(1, 6))

$$E = (1 + 2 + \ldots + 6)/6 = 3.5$$

The mean of many rolls gets close to 3.5 (law of large numbers).

Lesson: Maths you get from math and random

Monte Carlo estimate of π

$$\pi \approx 4 \times (\text{points with } x^2 + y^2 \le 1) / (\text{all points})$$

Random points in a square, counted inside a quarter circle. The error shrinks like 1/√n.

Lesson: Maths you get from math and random

TABLES

Table: form · binds · use as

Form	Binds	Use as
import math	math	math.sqrt(2)
import math as m	m	m.sqrt(2)
from math import sqrt, pi	sqrt and pi	sqrt(2)
from math import sqrt as root	root	root(2)
from math import *	every public name (or those in __all__)	sqrt(2), pi, ... (avoid in real code)
import os.path	os (and loads os.path)	os.path.join(...)
from . import circle	circle, from the same package	circle.area(1) (relative import, inside packages only)

Lesson: Every way to write import

Table: · import math · from math import sqrt

	import math	from math import sqrt
Where a name comes from	Clear: math.sqrt	You must look at the top of the file
Name clashes	None: math.log and your own log can live together	Possible: a later def log replaces the imported one
Typing	Longer	Shorter

Lesson: Every way to write import

Table: function · does · example

Function	Does	Example
ceil(x), floor(x)	Round up, round down to a whole number	ceil(4.2) = 5, floor(-4.2) = −5
trunc(x)	Cut off the fraction, toward zero	trunc(−4.8) = −4
sqrt(x), isqrt(n)	Square root as float, exact whole number square root	isqrt(20) = 4
hypot(x, y)	Length of the hypotenuse, $\sqrt{(x^2 + y^2)}$	hypot(3, 4) = 5.0
factorial, comb, perm	n!, C(n, k), P(n, k)	comb(5, 2) = 10
gcd, lcm	Greatest common divisor, least common multiple	gcd(84, 36) = 12
log, log2, log10, exp	Logarithms and e^x	log(8, 2) = 3.0
isclose(a, b)	Compare floats safely	isclose(0.1 + 0.2, 0.3) is True

Lesson: The math module

Table: function · gives

Function	Gives
random()	A float with 0.0 ≤ x < 1.0
randint(a, b)	A whole number with a ≤ n ≤ b (b included!)
randrange(start, stop, step)	Like range: stop is not included
choice(seq)	One item
sample(seq, k)	k different items, as a new list
shuffle(list)	Mixes the list in place and returns None
seed(n)	Restart the sequence, so results repeat (useful for tests)

Lesson: The random module

Table: name · meaning by convention

Name	Meaning by convention
slug	Public: part of what the module offers
_clean	Internal: may change without warning. from module import * skips it
__all__	The list of names that from module import * takes
__name__, __file__, __doc__	Set by Python for every module

Lesson: Public and private names, and __all__

Table: module · for · chapter

Module	For	Chapter
math, statistics, random, decimal, fractions	Numbers	Core 1.3 and here
datetime, time, calendar	Dates and times	Here
collections, itertools, functools, heapq, bisect	Data structures and iteration	Core 3.4, 4.3, Advanced 2.1 and 4.2
os, os.path, pathlib, shutil, glob	Files and folders	Core 5.3
json, csv, sqlite3, xml.etree, configparser, logging	Data formats and logs	Advanced 6.2
re	Regular expressions	Advanced 6.1
threading, multiprocessing, asyncio	Doing several things at once	Advanced 5.1
socket, http, urllib	Networks and the web	Web 1.1 to 1.2
unittest, doctest	Testing	Advanced 3.2
tkinter	Desktop windows	Advanced 6.3

Lesson: A short tour of the standard library

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1979	Find Greatest Common Divisor of Array	Easy	math.gcd	Take the smallest and the largest number and return math.gcd of the two.
1071	Greatest Common Divisor of Strings	Easy	math.gcd on lengths	If str1 + str2 differs from str2 + str1 there is no answer. Otherwise the answer is the first gcd(len1, len2) characters.
69	Sqrt(x)	Easy	Binary search (or math.isqrt)	Find the largest m with m × m ≤ x by halving the search range.
367	Valid Perfect Square	Easy	math.isqrt	math.isqrt(n) is the exact whole square root rounded down. Square it back and compare.
1185	Day of the Week	Easy	datetime	datetime.date(year, month, day).weekday() gives 0 for Monday. Map it to the names.
1360	Number of Days Between Two Dates	Easy	datetime subtraction	Turn both texts into dates with date.fromisoformat and take abs of the difference in days.
1154	Day of the Year	Easy	datetime	Subtract 1 January of the same year and add 1.
172	Factorial Trailing Zeroes	Medium	Count factors of 5	Each trailing zero needs a 10 = 2 × 5, and 5s are rarer. Add n // 5 + n // 25 + n // 125 + ...
384	Shuffle an Array	Medium	random and the Fisher Yates shuffle	Keep a copy of the original. To shuffle, swap item i with a random item from i to the end.
528	Random Pick with Weight	Medium	Prefix sums, random and bisect	Build running totals. Pick a random number up to the total and find its slot with bisect.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

floor goes down and trunc goes toward zero. They agree for positive numbers and differ for negative ones: floor(−4.8) is −5, trunc(−4.8) is −4. round() is a built-in, not in math, and rounds halves to the even number.

Lesson: The math module

COMMON MISTAKE random gives **pseudo-random** numbers from the Mersenne Twister algorithm: good for games and simulations, but predictable. For passwords, tokens and anything secret use the secrets module (secrets.token_hex(16)).

Lesson: The random module

REMEMBER The output above came from the build machine. Press Run on this page and platform.system() says Emscripten, because the Python on this page runs inside your browser (Pyodide). The same code gives Windows or Darwin on other computers.

Lesson: platform and sys: where am I running

COMMON MISTAKE Any file name.py that Python can find is a module called name. Use lower case names with underscores, and never name your file after a standard module: a file called random.py in your folder hides the real random module, and import random gives you your own file.

Lesson: Writing your own module

REMEMBER Since Python 3.3 a folder without __init__.py can still be imported as a namespace package, but an __init__.py makes the intent clear and lets you run setup code and choose what the package exports.

Lesson: Packages: folders of modules

REMEMBER Run pip as python -m pip install ... when you are not sure which Python a plain pip belongs to. Modern projects describe their dependencies in a pyproject.toml file, and tools such as uv and poetry manage environments for you.

Lesson: pip, PyPI and virtual environments

5.3 Files: reading, writing, modes, paths and streams

open modes, with, read and write methods, binary files, encodings, errno, pathlib, os and shutil, streams, csv and json.

KEY IDEAS

- **open and with** with open(path, mode, encoding="utf-8") as f: closes automatically. Modes r, w (empties), a, x (new only), +, b.
- **Reading** read(), read(n), readline() ("" at end), readlines(), for line in f (best for big files). Lines keep their new line.
- **Writing** write(s) and writelines(list) add no new lines. print(x, file=f) does. write returns characters written.
- **Binary and cursor** "rb"/"wb" use bytes. bytearray + readinto. seek(n) moves, tell() reports.
- **Errors and paths** OSError family: FileNotFoundError (ENOENT), FileExistsError, PermissionError. pathlib.Path / "x", glob, read_text.

FORMULAS

File size in bytes of an ASCII text Exam favourite

sum of len(line) for all lines, new lines included

"milk\n" is 5 bytes. On Windows a text file stores each new line as 2 bytes (\r\n).

Lesson: Sizes, bytes and cursors

UTF-8 size Exam favourite

1 byte for code points 0 to 127, 2 up to 2,047, 3 up to 65,535, 4 above

"नमस्ते ₹100 café\n" has 16 characters but 31 bytes.

Lesson: Sizes, bytes and cursors

Units

1 KB = 1,000 bytes, 1 KiB = 1,024 bytes

Disks are sold in KB, MB, GB (powers of 10). Operating systems often show KiB, MiB, GiB (powers of 2).

Lesson: Sizes, bytes and cursors

Cursor after a read in text mode (ASCII)

new position = old position + characters read

In binary mode tell() is always a byte offset. In text mode it is an opaque number, so only seek() to values that tell() gave you.

Lesson: Sizes, bytes and cursors

Lines in memory

readlines(): O(file size). for line in f: O(longest line)

A 10 GB log can be processed with a for loop on a laptop with 8 GB of memory.

Lesson: Sizes, bytes and cursors

TABLES

Table: method · returns · at the end of the file

Method	Returns	At the end of the file
read()	The rest of the file as one str	""
read(n)	Up to n characters	""
readline()	One line, including its new line character	""
readlines()	A list of all remaining lines	[]
for line in f	One line per turn, without loading the whole file	The loop ends

Lesson: Reading a text file

Table: mode · read · write · file must exist

Mode	Read	Write	File must exist	Empties the file	Cursor starts
r (default)	Yes	No	Yes (else FileNotFoundError)	No	Start
w	No	Yes	No, it is created	**Yes**	Start
a	No	Yes, always at the end	No, it is created	No	End
x	No	Yes	Must NOT exist (else FileExistsError)	New file	Start
r+	Yes	Yes	Yes	No	Start
w+	Yes	Yes	No	**Yes**	Start
a+	Yes	Yes, at the end	No	No	End

Lesson: Open modes

Table: · f = open(...) then f.close() · with open(...) as f:

	f = open(...) then f.close()	with open(...) as f:
If an exception happens in between	close() is skipped, the file may stay open and data may not be saved	The file is closed anyway
If you forget close()	Data may sit in a buffer and never reach the disk	Not possible
Lines of code	More, plus try and finally to be safe	Fewer

Lesson: Always use with

Table: · text ("r", "w") · binary ("rb", "wb")

	text ("r", "w")	binary ("rb", "wb")
read() returns	str	bytes
write() takes	str	bytes or bytearray
Encoding	Decodes and encodes (say encoding="utf-8")	None, raw bytes
New line characters	Changed to and from the system style	Never changed
Use for	Text, CSV, JSON, code	Images, audio, zip, PDF, anything not text

Lesson: Binary files, bytes and bytearray

Table: exception (all are oserror) · errno name · when

Exception (all are OSError)	errno name	When
FileNotFoundError	ENOENT	The file or folder does not exist
FileExistsError	EEXIST	Mode "x" or mkdir on something that exists
PermissionError	EACCES or EPERM	Not allowed to read or write it
IsADirectoryError	EISDIR	open() on a folder
NotADirectoryError	ENOTDIR	A path part is a file, not a folder
OSError with ENOSPC	ENOSPC	The disk is full

Lesson: File errors and errno

Table: · os.path (strings) · pathlib.path (objects)

	os.path (strings)	pathlib.Path (objects)
Join	os.path.join("a", "b.txt")	Path("a") / "b.txt"
Parts	os.path.basename, splitext	p.name, p.stem, p.suffix, p.parent
Read and write	open(...) then read()	p.read_text(), p.write_text()
Find files	glob.glob("*.csv")	p.glob("*.csv"), p.rglob("*.csv")
Style	Older, everywhere in old code	Modern, clearer

Lesson: Paths with pathlib

Table: stream · python name · used by

Stream	Python name	Used by
Standard input	sys.stdin	input() reads from it
Standard output	sys.stdout	print() writes to it
Standard error	sys.stderr	Tracebacks and warnings, print(..., file=sys.stderr)

Lesson: stdin, stdout and stderr

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1816	Truncate Sentence	Easy	split and join	Split on spaces, keep the first k words, join them again.
819	Most Common Word	Easy	Clean text, count words	Turn every non letter into a space, lower case everything, count the words that are not banned and take the most common.
884	Uncommon Words from Two Sentences	Easy	Count words of both texts	A word is uncommon if it appears exactly once in the two sentences together.
1592	Rearrange Spaces Between Words	Easy	Count spaces, divide	Count the spaces and words. Put spaces // (words − 1) between words and the rest at the end.
2042	Check if Numbers Are Ascending in a Sentence	Easy	Parse tokens	Split into tokens. Every token made of digits must be bigger than the previous number.
609	Find Duplicate File in System	Medium	Group paths by content	Each entry is "folder name(content) ...". Group full paths by content in a dict and keep groups with more than one path.
388	Longest Absolute File Path	Medium	Depth by tabs, a stack of lengths	Split into lines. The number of tabs is the depth. Keep the path length at each depth. A name with a dot is a file.
1233	Remove Sub-Folders from the Filesystem	Medium	Sort, then compare with the last kept folder	After sorting, a sub folder comes right after its parent. Keep a folder unless it starts with the last kept folder plus "/".
1047	Remove All Adjacent Duplicates In String	Easy	Stack	Push each character. If it equals the top of the stack, pop instead.
68	Text Justification	Hard	Greedy lines, spread spaces	Fill each line with as many words as fit. Spread the spaces, giving the left gaps one extra when they do not divide evenly. The last line is left aligned.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

All the programs in this chapter first move into a new empty temporary folder with tempfile.mkdtemp(), so they never touch your own files. Drop that line when you work in your project folder.

Lesson: Files: data that lasts

IN THE EXAM

Lines keep their new line character. Use line.strip() or line.rstrip("\n") before you compare or split them. For big files loop with for line in f, which reads one line at a time instead of loading gigabytes into memory.

Lesson: Reading a text file

COMMON MISTAKE

Add t (text, the default) or b (binary) to any mode: "rb", "wb", "ab", "r+b". The most common accident is opening a file with "w" when you meant "a": the old contents are gone at once.

Lesson: Open modes

COMMON MISTAKE

Without encoding=..., open() uses a default that depends on the computer. It is UTF-8 on Linux and macOS but can be an old code page on Windows, so a file written on one computer may not read on another. Since Python 3.15 the default becomes UTF-8 everywhere (PEP 686), but writing encoding="utf-8" is still the safe habit.

Lesson: Encodings: always say utf-8

COMMON MISTAKE

os.remove, os.rename and shutil.rmtree do not use a recycle bin: deleted files are gone. Print the paths and check them before you delete anything in a loop.

Lesson: Folders with os and shutil

REMEMBER The page shows only stdout, so the stderr line above is not in the output. In a terminal both appear, but they are separate streams: python3 app.py > out.txt saves only stdout and still shows errors on the screen.

Lesson: stdin, stdout and stderr

COMMON MISTAKE Never split CSV lines with line.split(","). A value like "Ravi, Jr" contains a comma and is stored in quotes. The csv module handles quotes, commas and new lines inside values correctly. Advanced 6.2 covers csv, json, xml and sqlite3 in depth.

Lesson: CSV and JSON files

STEP 6 Core Python · Object oriented programming

6.1 Classes and objects: attributes, methods, self and encapsulation

classes, objects, __init__, self, instance and class attributes, __str__ and __repr__, introspection, name mangling, is and ==.

KEY IDEAS

- **Class and object** class C: defines a blueprint. C(...) makes an object and runs __init__(self, ...). self is the object.
- **Two kinds of attribute** self.x is per object (in obj.__dict__). x = ... in the class body is shared. Read looks in the object, then the class. Write goes to the object.
- **Shared list trap** A list or dict class attribute is shared by all objects. Create it in __init__.
- **Text forms** __str__ for print and str, __repr__ for repr, lists and debugging. Write __repr__ at least.
- **Introspection and privacy** type, isinstance, hasattr, getattr, setattr, vars, __name__, __module__, __bases__. _x is a convention, __x becomes _Class__x. is = same object.

FORMULAS

Length of a vector (Pythagoras) Exam favourite

$$|v| = \sqrt{(x^2 + y^2)}$$

Vector(3, 4).length() is √(9 + 16) = 5.

Lesson: The maths inside a class

Dot product Exam favourite

$$a \cdot b = a_x b_x + a_y b_y$$

(3, 4) · (1, 2) = 3 + 8 = 11.

Lesson: The maths inside a class

Angle between two vectors

$$\cos \theta = (a \cdot b) / (|a|\,|b|)$$

cos θ = 11 / (5 × 2.236) = 0.984, so θ = 10.3 degrees.

Lesson: The maths inside a class

Counting objects with a class attribute

count after n objects = n

One class attribute is shared, so 3 students give Student.count = 3.

Lesson: The maths inside a class

Name mangling

__name inside class C becomes _C__name

__balance in Account becomes _Account__balance (17 characters).

Lesson: The maths inside a class

Memory

n objects store n copies of each instance attribute but 1 copy of each class attribute and method

1,000 dogs: 1,000 names, 1 species, 1 bark function.

Lesson: The maths inside a class

TABLES

Table: word · meaning · in the program

Word	Meaning	In the program
class	The blueprint	Dog
object or instance	One thing made from the class	rex, moti
attribute	A value stored on an object or a class	name, age, species
method	A function defined inside the class	bark, birthday
constructor	The code that sets up a new object	__init__
self	The object a method is working on	self.name

Lesson: Classes and objects

Table: you write · python does

You write	Python does
rex = Dog("Rex")	Makes an empty object, then calls Dog.__init__(that object, "Rex"), then names it rex
rex.bark()	Finds bark on the class and calls Dog.bark(rex)
Dog.bark	A plain function
rex.bark	A bound method: the function plus the object, ready to call

Lesson: Methods, self and __init__

Table: · instance attribute · class attribute

	instance attribute	class attribute
Made by	self.name = ... (usually in __init__)	name = ... in the class body
Stored in	obj.__dict__	Class.__dict__
How many copies	One per object	One for the whole class
Good for	Data that differs between objects: name, age, balance	Constants and data every object shares: species, bank name, a counter

Lesson: Instance attributes and class attributes

Table: · __str__ · __repr__

	__str__	__repr__
Used by	print(obj), str(obj), f"{obj}"	repr(obj), f"{obj!r}", the REPL, items inside a list
Written for	Users	Developers
Good style	Short and friendly: (3, 4)	Clear and exact, ideally valid code: Point(3, 4)
If missing	Python uses __repr__	Python shows the class name and memory address

Lesson: __str__ and __repr__

Table: tool · answers

Tool	Answers
type(obj), obj.__class__	Which class made this object
isinstance(obj, C)	Is obj a C (or a child class of C, see 6.2)
hasattr(obj, "x")	Can obj.x be read without an error
getattr(obj, "x", default)	Read an attribute by its name as a string
setattr(obj, "x", v), delattr(obj, "x")	Set or delete an attribute by name
vars(obj), obj.__dict__	The instance attributes as a dict
C.__name__, C.__module__, C.__bases__, C.__doc__	The class name, its module, its parent classes and its docstring
dir(obj)	Every attribute name, including the inherited ones

Lesson: Introspection: looking inside objects

Table: name · meaning · enforced

Name	Meaning	Enforced
balance	Public: anyone may use it	Not needed
_balance	Internal: please use it only inside the class	No, a convention only
__balance	Name mangled to _Account__balance so child classes cannot clash with it by accident	Partly: the short name fails outside the class
__init__	Special (dunder) name used by Python itself	Do not invent your own

Lesson: Encapsulation, _private and name mangling

Table: · separate variables · a dict · a class

	separate variables	a dict	a class
Example	name1, mark1, name2, mark2	{"name": "Asha", "mark": 88}	Student("Asha", 88)
Typing mistakes	Easy to mix up	s["nmae"] fails only when run	s.nmae fails, and editors warn early
Behaviour	Separate functions	Separate functions	Methods next to the data
Rules (no negative marks)	Checked everywhere by hand	Checked everywhere by hand	Checked once, in the class
Many of them	Impossible	A list of dicts	A list of objects
Best for	A few unrelated values	Data from JSON, flexible keys	Things with data AND behaviour

Lesson: Classes compared with other ways to group data

Table: · procedural (functions and data apart) · object oriented (classes)

	procedural (functions and data apart)	object oriented (classes)
Organised around	Steps and functions	Things and their behaviour
Data protection	Any function can change any data	Methods guard their own object
Reuse	Copy or call functions	Inheritance and composition (6.2)
Good for	Short scripts, data pipelines	Bigger programs, games, GUIs, web frameworks

Lesson: Classes compared with other ways to group data

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
155	Min Stack	Medium	Keep a second stack of minimums	Next to the normal stack, keep a stack whose top is always the smallest value so far. Push to it when the new value is not bigger.
232	Implement Queue using Stacks	Easy	Two stacks	Push onto an inbox stack. To pop, pour the inbox into an outbox stack (which reverses it) only when the outbox is empty.
225	Implement Stack using Queues	Easy	Rotate one queue	After adding x to the back, move every older item from the front to the back, so x is at the front.
705	Design HashSet	Easy	Buckets of lists	Keep 1,000 small lists. A key goes in bucket key % 1000, so each search looks at only one short list.
706	Design HashMap	Easy	Buckets of key and value pairs	Like the hash set, but each bucket holds [key, value] pairs. put updates a pair or adds one.
1603	Design Parking System	Easy	One counter per car type	Store the free spaces in a list indexed by car type. A car parks only if its counter is above zero.
1472	Design Browser History	Medium	A list and a position	Keep the pages in a list and the index of the current page. visit cuts off the forward pages, back and forward move the index within limits.
622	Design Circular Queue	Medium	Fixed list, head index and count	The back of the queue is at (head + count) % k. Moving the head with % k makes the list wrap around.
2043	Simple Bank System	Medium	Check, then change	Every method first checks that the account numbers exist and the money is there, and only then changes the balances.
146	LRU Cache	Medium	A dict keeps insertion order	Python dicts remember the order keys were added. Move a key to the end when it is used, and remove the first key when the cache is full.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM self is not a keyword. It is only the name every Python programmer uses for the first parameter of a method. Python fills it in for you: rex.bark() becomes Dog.bark(rex). Forget self in the def line and the call fails with "takes 0 positional arguments but 1 was given".

Lesson: Methods, self and __init__

IN THE EXAM Writing obj.attr = value never goes to the class. It always creates or changes the attribute on that one object. That is why a.species = "wolf" left b.species and Dog.species alone.

Lesson: Instance attributes and class attributes

COMMON MISTAKE A class attribute that holds a list, dict or set is ONE object for all instances. Changing it with append or update through any object changes it for every object. Create such data in __init__ with self.items = [] instead. This is the same trap as a mutable default argument (4.1).

Lesson: The shared list trap

IN THE EXAM Inside a method, change a class attribute through the class name: Student.count += 1. Writing self.count += 1 reads the class value but then stores the result on the object, so the class counter would never change.

Lesson: Class attributes as counters

REMEMBER getattr with a name in a variable is how frameworks such as Django read a field whose name comes from a settings file or a URL.

Lesson: Introspection: looking inside objects

IN THE EXAM **Encapsulation** means an object guards its own data and offers methods to work with it, so outside code cannot put it into a bad state (like a negative balance). Python trusts programmers: it has no real private keyword like Java. Use one underscore for internal things, and properties (Advanced 1.3) when you need checks on reading or writing.

Lesson: Encapsulation, _private and name mangling

IN THE EXAM a is b asks "are these the same object". a == b asks "are these equal". For your own classes == means the same as is until you write __eq__. Writing __eq__ sets __hash__ to None, so the objects can no longer be set items or dict keys unless you also write __hash__ (Advanced 1.1).

Lesson: Identity and equality of objects

REMEMBER Each method keeps the object valid: deposit refuses amounts that are not positive and withdraw refuses to go below zero. Code outside the class never has to repeat these checks. That is the main gain of object oriented design.

Lesson: Putting it together: a bank account

6.2 Inheritance: super(), overriding, polymorphism and the MRO

inheritance, overriding, super(), isinstance, polymorphism, duck typing, abstract methods, mixins, MRO, composition.

KEY IDEAS

- **Inherit and override** class Child(Parent): reuses Parent. Writing a method again overrides it. Lookup: object, class, then parents in MRO order, ending at object.
- **super()** super().__init__(...) runs parent setup. super().m() extends a method. super() means the next class in the MRO of the object.
- **Checks** isinstance counts children, type(x) is C does not. issubclass(bool, int) is True. __bases__ and __mro__ show the family.
- **Polymorphism** One call works on many types. Duck typing needs only the method. Abstract methods: NotImplementedError or abc with @abstractmethod.
- **MRO and composition** C3: child before parents, parents in written order, each once. Impossible order is TypeError. Prefer has a over is a.

FORMULAS

C3 linearisation Exam favourite

$$L[C] = C + \text{merge}(L[B_1], ..., L[B_n], [B_1, ..., B_n])$$

B_1 ... B_n are the parents of C in the order written.

Lesson: The MRO and the diamond problem

The merge rule

take the first head that is not in the tail of any list, remove it everywhere, repeat

If every head is in some tail, there is no valid order and Python raises TypeError.

Lesson: The MRO and the diamond problem

Circle Exam favourite

$$A = \pi r^2$$

Circle(2): 3.14159 × 4 = 12.57.

Lesson: The maths: areas, MROs and lookup cost

Heron for a triangle Exam favourite

$$s = (a + b + c) / 2,\ A = \sqrt{(s(s - a)(s - b)(s - c))}$$

Triangle(3, 4, 5): s = 6, A = √(6 × 3 × 2 × 1) = 6.

Lesson: The maths: areas, MROs and lookup cost

Total area by polymorphism

total = sum of shape.area() for every shape

12.57 + 12 + 6 = 30.57, one loop for every kind of shape.

Lesson: The maths: areas, MROs and lookup cost

Length of an MRO

len(C.__mro__) = number of distinct ancestors + 1 (C itself), object included

Diamond D(B, C): D, B, C, A, object is 5.

Lesson: The maths: areas, MROs and lookup cost

Lookup cost

worst case one dict check per class in the MRO: O(len(MRO))

CPython caches the result per type, so repeated lookups are close to O(1).

Lesson: The maths: areas, MROs and lookup cost

TABLES

Table: · super().method(...) · parent.method(self, ...)

	super().method(...)	Parent.method(self, ...)
self	Passed for you	You pass it by hand
If the parent class is renamed	Nothing to change	Every call must be edited
Multiple inheritance	Follows the MRO, so every class runs once	Can run a class twice or skip one
Seen in	Modern code	Old code

Lesson: super(): use the parent version

Table: check · true when

Check	True when
isinstance(obj, C)	obj was made by C or by any child of C
type(obj) is C	obj was made by exactly C (children do not count)
issubclass(A, B)	A is B or a child (at any depth) of B
C.__bases__	A tuple of the direct parents of C
C.__mro__	The full search order, ending with object

Lesson: isinstance, issubclass and __bases__

Table: · through inheritance · duck typing

	through inheritance	duck typing
Needs a shared parent	Yes	No
Checked	When the code runs	When the code runs
Error if a method is missing	NotImplementedError from the parent, or a TypeError from abc	AttributeError at the call
Seen in	Frameworks: Django models, views	Everywhere: len, for loops, files, +

Lesson: Polymorphism and duck typing

Table: · raise notimplementederror · abc.abc with @abstractmethod

	raise NotImplementedError	abc.ABC with @abstractmethod
When the mistake shows	Only when the missing method is called	As soon as you try to make the object
Parent can be made	Yes	No
Import needed	No	from abc import ABC, abstractmethod

Lesson: Abstract methods and a shape family

Table: · inheritance (is a) · composition (has a)

	inheritance (is a)	composition (has a)
Example	class Dog(Animal)	self.engine = Engine(...)
Link	Fixed when the class is written	Can be swapped while running
Child gets	Every parent method, wanted or not	Only what you pass on
Risk	A change in the parent can break children	More small methods that pass work on
Use when	The child really is a kind of the parent	The object uses another object to do a job

Lesson: Composition compared with inheritance

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
284	Peeking Iterator	Medium	Wrap an object and look one step ahead	Keep the next value in an attribute. peek returns it, next returns it and fetches a new one. This is composition: the class holds another object.
341	Flatten Nested List Iterator	Medium	Flatten once with recursion	Walk the nested list with a recursive helper and store every integer in a plain list. Then next and hasNext only move an index.
707	Design Linked List	Medium	Node objects and a dummy head	Each node object holds a value and a link to the next node. A dummy node before the first one means adding at index 0 needs no special case.
208	Implement Trie (Prefix Tree)	Medium	A tree of node objects	Each node has a dict of children, one per next letter, and a flag that marks the end of a word.
641	Design Circular Deque	Medium	Fixed list with a moving front	Like the circular queue, but the front can also move back by one: (front − 1) % k.
933	Number of Recent Calls	Easy	A queue of times	Add each new time to the back. Remove times from the front that are older than t − 3000. The length is the answer.
901	Online Stock Span	Medium	Stack of price and span pairs	Keep a stack of [price, span]. While the top price is not bigger than today, pop it and add its span to today.
1352	Product of the Last K Numbers	Medium	Prefix products, reset at zero	Keep running products. The product of the last k is last / products[−k − 1]. A zero clears the list, and asking past it gives 0.
2241	Design an ATM Machine	Medium	Greedy from the biggest note	Store how many of each note there are. To withdraw, take as many big notes as possible, then smaller ones. If money is left over, refuse and change nothing.
1396	Design Underground System	Medium	Two dicts	One dict remembers where and when each card checked in. Another adds up total time and trip count for each pair of stations.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER The child is a special kind of the parent: a Dog is an Animal. If you cannot say "a Child is a Parent" in plain words, inheritance is probably the wrong tool (see composition below).

Lesson: Inheritance: reuse a class

IN THE EXAM An overriding method normally keeps the same parameters as the parent one, so code that works with an Animal also works with a Dog. This rule is called the Liskov substitution principle.

Lesson: Overriding methods

COMMON MISTAKE A child __init__ replaces the parent one completely. If it does not call super().__init__(...), the attributes the parent sets are never created.

Lesson: super(): use the parent version

IN THE EXAM Prefer isinstance over type(x) == C, because a child object is also a valid parent object. Every class inherits from object in the end, and bool is a child of int, which is why True + True is 2.

Lesson: isinstance, issubclass and __bases__

REMEMBER A **mixin** is a small class that adds one ability (like to_json) and is not meant to be used alone. Django uses mixins a lot, for example LoginRequiredMixin on views. Put mixins first or last consistently and keep them free of __init__ when you can.

Lesson: Multiple inheritance and mixins

IN THE EXAM super() does not mean "my parent". It means "the next class in the MRO of the object I am working on". Inside B, super().hello() went to C for a D object but to A for a B object. That is what lets every class in a diamond run exactly once.

Lesson: The MRO and the diamond problem

STEP 7 Core Python · Exam practice

7.1 Exam practice: PCEP and PCAP style papers

Four practice papers, a bug hunt, the traps table, exam technique and a mock quiz.

KEY IDEAS

- **Exam facts** PCEP-30-02: 30 questions, 40 min, 70 %. PCAP-31-03: 40 questions, 65 min, 70 %. OOP is 34 % of PCAP.
- **Operator traps** -2 ** 2 is -4, 2 ** 3 ** 2 is 512, -7 // 2 is -4, -7 % 3 is 2, round(2.5) is 2, 0.1 + 0.2 != 0.3.
- **Collection traps** b = a aliases, [[0] * 3] * 3 shares a row, (1,) is a tuple, {} is a dict, sort returns None.
- **Function traps** Mutable defaults are shared, assignment makes a name local, lambdas in loops bind late, finally always runs.
- **Object traps** obj.x = v never changes the class, __x becomes _Class__x, a child __init__ must call super().__init__().

FORMULAS

Time per question Exam favourite

$$\text{minutes} \times 60 / \text{questions}$$

PCEP: 40 × 60 / 30 = 80 seconds. PCAP: 65 × 60 / 40 = 97.5 seconds.

Lesson: Exam technique and a study plan

Questions to pass (if every item counts the same)

Exam favourite

$$\text{correct needed} = \text{ceil}(\text{pass mark} \times \text{questions})$$

PCEP: 0.7 × 30 = 21. PCAP: 0.7 × 40 = 28.

Lesson: Exam technique and a study plan

Points by block

block share of the exam = block weight

Object oriented programming is 34 % of PCAP, more than any other block, so it deserves the most practice.

Lesson: Exam technique and a study plan

TABLES

Table: · pcep-30-02 (entry) · pcap-31-03 (associate)

	PCEP-30-02 (entry)	PCAP-31-03 (associate)
Questions	30	40
Time	40 minutes, plus 5 for the agreement and tutorial	65 minutes, plus 10 for the agreement and tutorial
Pass mark	70 % overall	70 % overall
Question types	Single and multiple choice, drag and drop, gap fill, sort, code fill, code insertion	Single and multiple choice, plus interactive and scenario items
Blocks and weights	Fundamentals 18 %, control flow 29 %, data collections 25 %, functions and exceptions 28 %	Modules and packages 12 %, exceptions 14 %, strings 18 %, object oriented programming 34 %, miscellaneous (comprehensions, lambdas, closures, files) 22 %
Chapters here	1.1 to 4.3 and 5.1	3.1, 4.1 to 6.2

Lesson: The exams and how to use this chapter

Table: paper · covers · lesson

Paper	Covers	Lesson
A	Operators, numbers, loops, booleans, print	2
B	Lists, tuples, slices, dicts, sets, strings	3
C	Arguments, defaults, scope, closures, exceptions	4
D	Classes, inheritance, strings, modules, generators	5

Lesson: The exams and how to use this chapter

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	-4 512	** binds tighter than the minus sign, so -2 ** 2 is -(2 ** 2). ** groups from the right: 2 ** 9 (1.3)
2	-4 2 -2	// rounds down (towards minus infinity). The result of % takes the sign of the divisor (1.3)
3	False 2 4	0.1 and 0.2 are not exact in binary. round() goes to the even neighbour on a tie (1.2)
4	True True	Chained comparisons mean 3 > 2 and 2 > 1 and 1 == 1 (1.3)
5	done 12	The loop ends normally when x reaches 12, so the else block runs (2.2)
6	i is 1	break skips the else block. The loop variable keeps its last value (2.2)
7	31 15 5 1000	int(text, 16) reads hexadecimal. 0o is octal, 0b is binary, _ is only for reading (1.2)
8	True False True False	Any non empty string is true, even "False". [0] is a non empty list (1.2)
9	ababab 55 10	* repeats a string, + joins strings, + adds numbers (1.2)
10	a-b!	sep goes between the values, end replaces the new line (1.4)

Lesson: Paper A: fundamentals and control flow

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	[1, 2, 3, 4]	b is the same list as a, and += changes a list in place (3.2)
2	(1, 2, 3) (1, 2, 3, 4)	A tuple cannot change, so += builds a new tuple and only u points to it (3.3)
3	[[1, 0, 0], [1, 0, 0], [1, 0, 0]]	[row] * 3 repeats the SAME inner list three times. Use a comprehension (3.2)
4	[4, 2] [4, 3, 2] []	A negative step walks backwards. The stop index is never included. Slices never raise IndexError (3.2)
5	['a', 'b'] 0 2	Changing a value keeps the key in its place. get gives the default for a missing key (3.4)
6	yth hon -1 P	Slices stop before the end index. find gives -1 when nothing is found (3.1)
7	tuple int	The comma makes a tuple, not the brackets (3.3)
8	{1, 2, 3, 4} {2}	| is union, & is intersection, and a set keeps one copy of each value (3.3)
9	[1, 2, 3] None	sort changes the list and returns None. sorted would return a new list (3.2)
10	1 {True: 'c'}	True == 1 == 1.0 with equal hashes, so they are one key. The first key stays, the last value wins (3.4)

Lesson: Paper B: data collections

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	(1, 3, (4, 5), 6, {'d': 7})	Positional values fill a and b, the rest go to *args. c is keyword only. Unknown keywords go to **kw (4.1)
2	[1, 2]	The default list is made once, when def runs, and shared by every call (4.1)
3	UnboundLocalError	x = 5 anywhere in g makes x local in all of g, so print(x) reads it too early (4.2)
4	[2, 2, 2]	Each lambda looks up i when it is called, and by then i is 2 (4.3)
5	finally runs, then try	finally runs before the function really returns (5.1)
6	V, then fin	The first matching except wins. else runs only when no error happened. finally always runs (5.1)
7	None	A function without return gives None (4.1)
8	KeyError 'k'	KeyError is a child of LookupError. str of a KeyError shows the key with quotes (5.1)
9	[2, 6, 10]	filter keeps 1, 3 and 5 (n % 2 is 1, which is true), map doubles them (4.3)
10	2	nonlocal lets inner change n in outer. The closure keeps n between calls (4.2)

Lesson: Paper C: functions and exceptions

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	1 2 True	Assigning through an instance adds an instance attribute and leaves the class alone (6.1)
2	False True	__secret is name mangled to _B__secret (6.1)
3	QP True False	super() calls the parent version. A child is an instance of its parent, not the other way round (6.2)
4	True True True 65 a	Strings compare code point by code point. "Z" is 90, "a" is 97, and "1" comes before "9" (3.1)
5	['a', 'b', '', 'c'] ['x', 'y'] a-b-c	split(",") keeps empty parts. split() with no argument splits on runs of spaces and drops the ends (3.1)
6	**Hello** Abc Def True True	center pads both sides. title capitalises each word (3.1)
7	-3 -2 -2 4.0	floor goes down, ceil goes up, trunc cuts towards zero. sqrt always returns a float (5.2)
8	__main__	A file run directly gets the name __main__. Imported, it would get its module name (5.2)
9	C! C!? C!	print, str and f-strings all use __str__ (6.1)
10	0 14	next takes 0. sum uses what is left: 1 + 4 + 9 (4.3)
11	[0, 3, 6, 9] {0: 0, 1: 1, 2: 4}	A list comprehension with if, and a dict comprehension (4.3)
12	12 abab	make returns a closure that remembers k. * works on strings too (4.2)
13	AttributeError	Child.__init__ never calls super().__init__(), so name is never set (6.2)

Lesson: Paper D: objects, strings and modules

Table: trap · looks like · really · chapter

Trap	Looks like	Really	Chapter
Power and minus	-2 ** 2 is 4	-4	1.3
Floor division	-7 // 2 is -3	-4	1.3
Rounding	round(2.5) is 3	2 (round half to even)	1.2
Integer from text	int("3.5") is 3	ValueError (use int(float("3.5")))	1.2
Boolean of text	bool("False") is False	True	1.2
is and ==	a is b means equal	is means the same object	1.3
Copy	b = a copies a list	b is another name for the same list	3.2
Nested repeat	[[0] * 3] * 3 makes 3 rows	It makes 3 names for one row	3.2
One item tuple	(5) is a tuple	It is the int 5, write (5,)	3.3
Empty set	{} is an empty set	It is an empty dict, write set()	3.3
sort	x = x.sort() sorts	x becomes None	3.2
Default argument	def f(x=[]) gets a new list each call	One list for all calls	4.1
Scope	Reading a global inside a function that also assigns it works	UnboundLocalError	4.2
Lambda in a loop	Each lambda keeps its own i	They all see the last i	4.3
else on loops	Runs when the loop runs	Runs when the loop was not stopped by break	2.2
finally	return skips finally	finally always runs	5.1
Class attribute	obj.x = 1 changes the class	It adds an instance attribute	6.1
Forgotten super()	Parent attributes exist anyway	AttributeError	6.2

Lesson: The traps in one table

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
11	Container With Most Water	Medium	Two pointers	Start at both ends. The area is width times the shorter wall. Move the shorter wall inwards, because only that can make the area bigger.
12	Integer to Roman	Medium	Greedy table	Walk a table from 1000 (M) down to 1 (I), including 900, 400, 90, 40, 9 and 4. Take each value as many times as it fits.
15	3Sum	Medium	Sort, then two pointers	Sort the list. For each first number, look for two more with two pointers. Skip equal neighbours so no triple repeats.
17	Letter Combinations of a Phone Number	Medium	Grow the list digit by digit	Start with [""] and, for every digit, add each of its letters to every string made so far.
36	Valid Sudoku	Medium	Sets for rows, columns and boxes	Keep a set for each row, column and 3 by 3 box. A digit seen twice in any of them makes the board invalid. The box number is (r // 3) * 3 + c // 3.
38	Count and Say	Medium	Describe the previous term	Read the previous string in runs of equal digits and write the run length followed by the digit.
56	Merge Intervals	Medium	Sort by start	Sort the intervals by start. If the next one starts before the last merged one ends, stretch the end, otherwise start a new interval.
67	Add Binary	Easy	Add from the right with a carry	Walk both strings from the end, add the digits and the carry, write total % 2 and carry total // 2.
238	Product of Array Except Self	Medium	Products from the left and from the right	answer[i] is the product of everything left of i times everything right of i. Two passes, no division.
448	Find All Numbers Disappeared in an Array	Easy	Set difference	The numbers should be 1 to n. Put the list in a set and keep the numbers from 1 to n that are not in it.

Lesson: LeetCode review set

TIPS AND TRAPS

COMMON MISTAKE The Python Institute has announced newer versions, PCEP-30-03 (35 questions, 50 minutes, 70 %) and PCAP-31-04 (42 questions, 65 minutes, 72 %), released on 31 October 2026. The skills tested are the same core Python, but check which version your voucher is for before you book.

Lesson: The exams and how to use this chapter

REMEMBER Cover the answers, write down what you think each line prints, then open the answers. A wrong guess tells you exactly which chapter to reread.

Lesson: The exams and how to use this chapter

IN THE EXAM In the exam: read the whole snippet before the options, trace it line by line on scratch paper, watch for indentation and for a missing comma or bracket that turns the answer into "the code raises an error". Mark hard questions and come back, every question is worth the same.

Lesson: Exam technique and a study plan

STEP 1 Advanced Python · Advanced object oriented programming

1.1 Special methods and operator overloading

special methods, operator overloading, reflected operators, total_ordering, the hash contract, container protocol, __call__, __format__.

KEY IDEAS

- **How operators work** a + b calls type(a).__add__(a, b). Missing or NotImplemented: try type(b).__radd__(b, a), then TypeError. Lookup is on the class.
- **Text** __repr__ (developers, lists, REPL), __str__ (print), __format__ (f-string specs).
- **Compare and hash** __eq__ plus __lt__ with total_ordering. Equal objects need equal hashes. __eq__ alone makes a class unhashable.
- **Containers** __len__, __getitem__ (ints and slices), __setitem__, __contains__, __iter__. __bool__ falls back to __len__.
- **Other hooks** __call__ for callable objects, __iadd__ returns self, __getattr__ only on failed lookup, __setattr__ on every assignment.

FORMULAS

Adding fractions

Exam favourite

$$a/b + c/d = (ad + cb) / bd$$

1/2 + 1/3 = (3 + 2) / 6 = 5/6.

Lesson: The maths: fractions and vectors with operators

Lowest terms

Exam favourite

$$\text{divide top and bottom by } g = \gcd(\text{top}, \text{bottom})$$

2/4: gcd(2, 4) = 2, so 1/2.

Lesson: The maths: fractions and vectors with operators

Euclid

$$\gcd(a, b) = \gcd(b, a \bmod b), \gcd(a, 0) = a$$

gcd(48, 18) = gcd(18, 12) = gcd(12, 6) = gcd(6, 0) = 6.

Lesson: The maths: fractions and vectors with operators

Vector length

$$|v| = \sqrt{(x^2 + y^2)}$$

abs(Vector(3, 4)) = 5.0, using math.hypot.

Lesson: The maths: fractions and vectors with operators

Scaling

$$k(x, y) = (kx, ky)$$

2 * Vector(3, 4) = Vector(6, 8), through __rmul__.

Lesson: The maths: fractions and vectors with operators

Dot product with @

$$a \mathbin{@} b = a_x b_x + a_y b_y$$

Python 3.5 added @ (__matmul__) for exactly this kind of product.

Lesson: The maths: fractions and vectors with operators

TABLES

Table: you write · python calls · group

You write	Python calls	Group
repr(x), str(x), format(x, s)	__repr__, __str__, __format__	Text
x + y, x - y, x * y, x / y, x // y, x % y, x ** y, x @ y	__add__, __sub__, __mul__, __truediv__, __floordiv__, __mod__, __pow__, __matmul__	Arithmetic
y + x when y does not know x	__radd__ and the other r methods on x	Reflected
x += y	__iadd__ (or __add__ then assign)	In place
-x, +x, abs(x), ~x	__neg__, __pos__, __abs__, __invert__	Unary
x == y, x != y, x < y, x <= y, x > y, x >= y	__eq__, __ne__, __lt__, __le__, __gt__, __ge__	Comparison
hash(x), set and dict keys	__hash__	Hashing
bool(x), if x	__bool__, else __len__	Truth
len(x), x[i], x[i] = v, del x[i], v in x	__len__, __getitem__, __setitem__, __delitem__, __contains__	Container
for v in x, iter(x), next(it)	__iter__, __next__	Iteration (2.1)
x(...)	__call__	Callable
with x:	__enter__, __exit__	Context manager (2.3)
x.name when it is missing, x.name = v	__getattr__, __setattr__	Attributes
int(x), float(x), round(x)	__int__, __float__, __round__	Conversion

Lesson: The main special methods

Table: · all six methods by hand · @total_ordering with __eq__ and __lt__

	all six methods by hand	@total_ordering with __eq__ and __lt__
Lines of code	Many, easy to make inconsistent	Two methods
Speed	Fastest	Slightly slower for the generated ones
Needed for sorted, min, max	__lt__ is enough	__lt__ is enough
Good for	Hot loops	Almost everything else

Lesson: Comparisons and total_ordering

Table: · __getattr__ · __getattribute__ · __setattr__

	__getattr__	__getattribute__	__setattr__
Called when	Normal lookup failed	EVERY attribute read	EVERY attribute assignment
Typical use	Defaults, proxies, lazy loading	Rare: logging every access	Validation, read only objects
Danger	Small	Infinite recursion if you read self.x inside it	Infinite recursion if you write self.x inside it

Lesson: Attribute hooks: __getattr__ and __setattr__

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
592	Fraction Addition and Subtraction	Medium	Parse, add, reduce with gcd	Read each signed fraction, add it to a running a/b with (ad + cb) / bd, then divide by the gcd.
166	Fraction to Recurring Decimal	Medium	Long division, remember remainders	Do long division. When a remainder comes back, the digits since its first appearance repeat, so wrap them in brackets.
1656	Design an Ordered Stream	Easy	A list and a pointer	Store each value at its id. After each insert, return the run of filled slots starting at the pointer and move the pointer past them.
1845	Seat Reservation Manager	Medium	A min heap of free seats	Keep the free seats in a heap so the smallest one is always on top.
2336	Smallest Number in Infinite Set	Medium	A counter plus a set of returned numbers	Numbers from next_new upwards are all present. Numbers added back below it go into a heap and a set.
1381	Design a Stack With Increment Operation	Medium	A list with a size limit	push only when there is room. increment adds val to the bottom k items.
1286	Iterator for Combination	Medium	Make the combinations up front	itertools.combinations gives the combinations in order. Store them and walk through them with an index.
900	RLE Iterator	Medium	Use up counts	Walk the pairs (count, value). Take n from the current count, moving to the next pair when one runs out.
1670	Design Front Middle Back Queue	Medium	A plain list	A list with insert and pop at the right index is simple and fast enough for the limits. The middle index is len // 2 for push and (len - 1) // 2 for pop.
2349	Design a Number Container System	Medium	Dict plus a lazy heap per number	Remember the number at each index. For each number keep a heap of indexes and drop stale ones when you look at the top.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM — Special methods are looked up on the **class**, not on the object. Setting obj.__len__ = ... on one object does nothing for len(obj). Also, never invent your own __names__: they are reserved for Python.

Lesson: The main special methods

IN THE EXAM — Return the special value NotImplemented (not raise NotImplementedError) when your method cannot handle the other type. That gives the other object a chance to answer. Raising stops Python from trying.

Lesson: Arithmetic, reflected and unary operators

REMEMBER — If you write __eq__ but not __ne__, Python gives != for free by inverting __eq__.

Lesson: Comparisons and total_ordering

REMEMBER — A class with only __len__ and __getitem__ already works with for, in, reversed and slicing. That is why Python calls this the **sequence protocol**. Inheriting from collections.abc.Sequence (or MutableSequence) also gives you index, count and more for free.

Lesson: Emulating containers

IN THE EXAM — x += y first tries x.__iadd__(y) and then ASSIGNS the result to x. That is why __iadd__ must return self. A tuple has no __iadd__, so += makes a new tuple and other names still see the old one.

Lesson: Callable objects, text formats and in place operators

REMEMBER

0.1 + 0.2 is 0.30000000000000004 with floats, but Fraction(1, 10) + Fraction(2, 10) is exactly 3/10. Use Fraction or Decimal when exact answers matter, such as money.

Lesson: The maths: fractions and vectors with operators

1.2 Extended arguments, closures and decorators

parameters, closures, decorators, wraps, decorator factories, stacking, class decorators, cache.

KEY IDEAS

- **Parameters** a, / positional only. *args tuple. after * keyword only. **kwargs dict. f(*seq, **d) unpacks at a call.
- **Closures** Functions are objects. An inner function remembers the variables of the outer one (__closure__), even after it returned.
- **Decorator shape** @d def f is f = d(f). def d(func): @wraps(func) def wrapper(*args, **kwargs): ... return func(*args, **kwargs). return wrapper.
- **Arguments and stacking** @d(x) is f = d(x)(f): three levels. @a @b def f is a(b(f)): applied bottom up, run top down.
- **Built ins** @cache, @lru_cache(maxsize), @wraps, @property, @classmethod, @staticmethod, @dataclass, @contextmanager. Cache only pure functions.

FORMULAS

Calls of plain recursive Fibonacci Exam favourite

$$C(n) = 1 + C(n-1) + C(n-2),\ C(0) = C(1) = 1,\ \text{so}\ C(n) = 2F(n+1) - 1$$

fib(25): 2 × 121,393 − 1 = 242,785 calls, as the program counted.

Lesson: The maths: memoisation and backoff

Calls with @cache Exam favourite

$$\text{each n from 0 to N is computed once: } N + 1 \text{ misses}$$

fib(25): 26 misses and 23 hits.

Lesson: The maths: memoisation and backoff

Growth

$$F(n) \approx \varphi^n / \sqrt{5},\ \varphi = (1 + \sqrt{5}) / 2 \approx 1.618$$

Plain recursion is exponential, $O(\varphi^n)$. The cached version is linear, O(n).

Lesson: The maths: memoisation and backoff

Exponential backoff

$$\text{wait before retry k} = \text{base} \times 2^k,\ \text{total for n retries} = \text{base}(2^n - 1)$$

base 1 s, 4 retries: 1 + 2 + 4 + 8 = 15 s.

Lesson: The maths: memoisation and backoff

LRU hit rate

$$\text{hit rate} = \text{hits} / (\text{hits} + \text{misses})$$

square with maxsize 2 on 1, 2, 1, 3, 2: 1 hit, 4 misses, 20 %.

Lesson: The maths: memoisation and backoff

TABLES

Table: part of def · meaning · example call

Part of def	Meaning	Example call
name, /	Positional only: cannot be passed by keyword (PEP 570)	describe("Asha", ...)
age	Positional or keyword	describe("Asha", 30) or age=30
*hobbies	Collects extra positional values into a tuple	"chess", "music"
city="Pune" after *	Keyword only: must be passed by name (PEP 3102)	city="Delhi"
**extra	Collects extra keyword values into a dict	lang="hi"
f(*seq, **mapping)	At a CALL, * and ** unpack instead of collecting	describe(*values, **options)

Lesson: Extended argument syntax

Table: · attribute on the wrapper function · a class with __call__ · a closure variable with nonlocal

	attribute on the wrapper function	a class with __call__	a closure variable with nonlocal
Example	wrapper.calls	self.calls	nonlocal calls
Readable from outside	Yes: greet.calls	Yes: square.calls	No
Works on methods	Yes	Needs extra work (__get__)	Yes
Best for	Simple counters and flags	Complex state	Private state

Lesson: Decorators that keep state

Table: decorator · module · what it does · chapter

Decorator	Module	What it does	Chapter
@functools.cache, @lru_cache(maxsize)	functools	Remembers results for arguments seen before	1.2
@functools.wraps(func)	functools	Copies name and docstring onto a wrapper	1.2
@functools.total_ordering	functools	Fills in comparison methods	1.1
@property, @x.setter	built in	Methods that look like attributes	1.3
@classmethod, @staticmethod	built in	Methods that get the class, or nothing	1.3
@abc.abstractmethod	abc	Methods every child must write	1.3
@dataclasses.dataclass	dataclasses	Writes __init__, __repr__ and __eq__ for you	1.3
@contextlib.contextmanager	contextlib	Turns a generator into a with statement helper	2.3
@app.route("/")	Flask	Registers a function as a web page	Web 3.1

Lesson: Built in decorators and caching

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
746	Min Cost Climbing Stairs	Easy	Top down with @cache	best(i) is the cheapest way to stand past the top from step i: cost[i] + min(best(i + 1), best(i + 2)). Start from step 0 or 1.
62	Unique Paths	Medium	Paths from a cell, cached	paths(r, c) = paths(r + 1, c) + paths(r, c + 1), and 1 on the last row or column.
64	Minimum Path Sum	Medium	Cheapest from a cell, cached	cost(r, c) is the grid value plus the cheaper of moving down or right.
91	Decode Ways	Medium	Ways from a position, cached	From position i, take one digit (if not 0) or two digits (if 10 to 26) and add the ways for the rest.
139	Word Break	Medium	Can the rest be split, cached	From position i, try every dictionary word that matches there and ask the same question for the rest.
198	House Robber	Medium	Rob or skip, cached	best(i) = max(rob house i and jump two, or skip to i + 1).
322	Coin Change	Medium	Fewest coins for an amount, cached	fewest(a) = 1 + the smallest fewest(a - coin). Use infinity for impossible amounts.
343	Integer Break	Medium	Best split, cached	For each first piece k, either keep the rest whole or break it further: max(k * (n - k), k * best(n - k)).
96	Unique Binary Search Trees	Medium	Catalan numbers, cached	Choose each value as the root. The trees on the left and right are independent, so multiply their counts.
494	Target Sum	Medium	Ways from (index, total), cached	At each number add it or subtract it. Count the ways that land exactly on target.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

The order in a def is always: positional only, /, normal, *args (or a bare *), keyword only, **kwargs. *args, **kwargs in a wrapper accept any call and pass it on unchanged, which is why every general decorator uses them. Notice the fourth line of output: name="Asha" did not fill name. Because describe has **extra, it went into extra, so Python reports name as missing.

Lesson: Extended argument syntax

REMEMBER

A **closure** is a function that remembers the variables of the function it was made in, even after that function has returned. greet remembers greeting. That memory, stored in __closure__, is what lets a wrapper keep hold of the original function.

Lesson: Functions are objects, closures remember

COMMON MISTAKE

Without @wraps every decorated function is called "wrapper" and loses its docstring, which breaks help(), debuggers, logging and frameworks that look up functions by name (Flask uses the name for its routes). Always put @wraps(func) on the wrapper.

Lesson: functools.wraps: keep the name and docstring

COMMON MISTAKE

A decorator with arguments has three nested functions: the factory (settings), the decorator (the function) and the wrapper (the call). Forgetting the brackets, @retry instead of @retry(3), passes the function as attempts and fails in a confusing way.

Lesson: Decorators with arguments

IN THE EXAM

Decorators are applied from the bottom up (nearest the def first) and run from the top down at call time. @a then @b above def f means f = a(b(f)).

Lesson: Stacking decorators

COMMON MISTAKE Only cache **pure** functions, whose result depends only on the arguments. Arguments must be hashable. @cache never forgets, so for many different arguments use lru_cache with a maxsize.

Lesson: Built in decorators and caching

1.3 Class design: methods, properties, abstract classes and dataclasses

class and static methods, properties, cached_property, ABCs, dataclasses, __slots__, descriptors.

KEY IDEAS

- **Method kinds** Instance method gets self. @classmethod gets cls (alternative constructors, returns cls(...)). @staticmethod gets nothing.
- **Properties** @property getter, @x.setter with checks, no setter means read only. Store the value as self._x. @cached_property computes once.
- **Abstract classes** class C(ABC) with @abstractmethod: C() and incomplete children raise TypeError. collections.abc offers ready contracts.
- **Dataclasses** @dataclass writes __init__, __repr__, __eq__. field(default_factory=list), frozen, order, __post_init__, kw_only, slots, asdict, replace.
- **Slots and descriptors** __slots__ removes __dict__. Descriptors have __get__ and __set__. Data descriptors beat the instance dict.

FORMULAS

Celsius to Fahrenheit (a computed property)

Exam favourite

$$F = C \times 9 / 5 + 32$$

25 °C is 25 × 1.8 + 32 = 77 °F.

Lesson: The maths: conversions, ordering and memory

The other way

$$C = (F - 32) \times 5 / 9$$

212 °F is 180 × 5 / 9 = 100 °C.

Lesson: The maths: conversions, ordering and memory

Absolute zero

$$C \geq -273.15, \text{ so } F \geq -459.67$$

The setter rejects anything colder.

Lesson: The maths: conversions, ordering and memory

order=True compares as tuples

Exam favourite

$(a_1, a_2, \ldots) < (b_1, b_2, \ldots)$ at the first position where they differ

Version(3, 9, 7) < Version(3, 14, 0) because 9 < 14, even though "9" > "14" as text.

Lesson: The maths: conversions, ordering and memory

Memory for n objects

$$\text{total} \approx n \times \text{bytes per object}$$

1,000,000 objects at 40 bytes less each save about 40 MB.

Lesson: The maths: conversions, ordering and memory

Age from a birth year (ignoring the birthday)

$$\text{age} = \text{current year} - \text{birth year}$$

2026 − 1996 = 30.

Lesson: The maths: conversions, ordering and memory

TABLES

Table: · instance method · @classmethod · @staticmethod

	instance method	@classmethod	@staticmethod
First argument	self (the object)	cls (the class)	none
Can read or change	The object and its class	The class only	Only its arguments
Called as	obj.m()	Cls.m() or obj.m()	Cls.m() or obj.m()
Typical use	Almost everything	Alternative constructors like from_text, dict.fromkeys, datetime.now	A helper that belongs to the topic, like is_adult
With inheritance	self is the child object	cls is the child class	Nothing changes

Lesson: Instance, class and static methods

Table: · a public attribute · get_x() and set_x() methods · @property

	a public attribute	get_x() and set_x() methods	@property
Caller writes	t.celsius = 5	t.set_celsius(5)	t.celsius = 5
Checks values	No	Yes	Yes
Can add checks later without changing callers	No	No	Yes
Feels like Python	Yes	No (Java style)	Yes

Lesson: Properties: attributes with rules

Table: option · what it adds

Option	What it adds
@dataclass	__init__, __repr__ and __eq__ from the annotated fields
field(default_factory=list)	A new default object for every instance (a plain [] default is refused)
frozen=True	Assignment raises FrozenInstanceError, and __hash__ is generated
order=True	<, <=, >, >= comparing the fields as a tuple, in order
field(init=False) and __post_init__	Computed fields and checks after __init__
kw_only=True	Every field must be passed by name
slots=True	Generates __slots__ for less memory and faster access
asdict, astuple, replace	Convert to a dict or tuple, or copy with changes

Lesson: Dataclasses: classes for data

Table: · tuple · namedtuple · dict

	tuple	namedtuple	dict	@dataclass
Fields by name	No	Yes	Yes (keys)	Yes
Can change	No	No	Yes	Yes (or frozen)
Methods and properties	No	Some	No	Yes
Type hints and defaults	No	Defaults only	No	Yes
Best for	Small fixed groups	Light records	JSON style data	Most records in an app

Lesson: Dataclasses: classes for data

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1146	Snapshot Array	Medium	Per index history plus binary search	For each index keep a list of (snap id, value) changes. get looks for the last change at or before the snap id with bisect.
981	Time Based Key-Value Store	Medium	Lists of times per key, bisect	Timestamps arrive in increasing order, so each key keeps a sorted list. get finds the last time not after the asked time.
1797	Design Authentication Manager	Medium	A dict of expiry times	Store when each token expires. renew only works while the token is alive. Count the tokens that expire after the current time.
1500	Design a File Sharing System	Medium	Reuse the smallest free id with a heap	Hand out ids from a heap of freed ids first, then new ids. Each user has a set of chunks.
1244	Design A Leaderboard	Medium	A dict of scores	addScore adds to a player, top sorts the scores and adds the K biggest, reset sets a player back to 0.
2526	Find Consecutive Integers from a Data Stream	Medium	Count the current run	Keep how many values in a row have equalled value. consec returns whether that run has reached k.
2296	Design a Text Editor	Hard	Two stacks around the cursor	Keep the text left of the cursor and the text right of it (reversed) as two lists. Every operation moves characters between them.
1825	Finding MK Average	Hard	Keep the last m values, sort when asked	A deque keeps the last m values. To answer, sort them, drop k from each end and average the middle. Simple, and fine for small m.
2034	Stock Price Fluctuation	Medium	Dict plus two lazy heaps	Store the price at each time. Push into a max heap and a min heap, and skip heap tops whose price was later corrected.
1600	Throne Inheritance	Medium	A family tree and a preorder walk	Keep each person's children in birth order and a set of the dead. The order of succession is a preorder walk that skips the dead.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM Student.from_text returned a Student because the method calls cls(...), not Person(...). That is why alternative constructors should be class methods.

Lesson: Instance, class and static methods

COMMON MISTAKE Store the real value under another name (self._celsius). If the setter wrote self.celsius = value it would call itself forever and raise RecursionError.

Lesson: Properties: attributes with rules

REMEMBER An abstract base class is a contract: "every Payment can pay and has a name". Python refuses to create an object until every abstract method is filled in, so a forgotten method fails at once instead of in front of a customer. collections.abc gives ready made contracts like Sized, Iterable, Sequence and Mapping.

Lesson: Abstract base classes

REMEMBER __slots__ removes the per object __dict__, so objects are smaller and adding a misspelt attribute fails at once. In Python 3.14 a normal object already keeps its attributes compactly until you touch __dict__, so the saving is smaller than older guides say: about a third in this run. Use slots for classes with very many objects.

Lesson: __slots__ and descriptors

IN THE EXAM

A **descriptor** is an object with __get__, __set__ or __delete__, stored as a class attribute. property, classmethod, staticmethod and every normal method are descriptors. Write your own when the same rule (like "must be positive") applies to many attributes in many classes.

Lesson: __slots__ and descriptors

1.4 Exceptions in depth, copying and serialisation

exception hierarchies, chaining, tracebacks, notes, exception groups, warnings, shallow and deep copy, pickle and shelve.

KEY IDEAS

- **Exception design** class AppError(Exception) plus children. super().__init__(message). Catch the child for one problem, the base for all.
- **Chaining** raise New() from err sets __cause__. Raising inside except sets __context__. from None hides it. err.__traceback__ has the frames.
- **3.11 tools** err.add_note(text) and err.__notes__. ExceptionGroup(msg, [errors]) caught with except* Type.
- **Copies** b = a no copy. a.copy(), a[:], list(a), dict(a), copy.copy: shallow. copy.deepcopy: everything, loops handled by memo.
- **Saving objects** pickle.dumps/loads (bytes), dump/load (files in "wb"/"rb"). Never unpickle untrusted data. shelve: dict on disk, returns copies.

FORMULAS

Chance that k independent attempts all fail

Exam favourite

$$P(\text{all fail}) = p^k,\ P(\text{success}) = 1 - p^k$$

A call fails 20 % of the time. With 3 attempts: $1 - 0.2^3 = 99.2$ % success.

Lesson: The maths: retries, copies and sizes

Objects created by copies of an n by m nested list

Exam favourite

$$\text{shallow: } 1 \text{ list. deep: } 1 + n \text{ lists}$$

For [[1, 2], [3, 4]]: shallow makes 1 new list, deep makes 3. The ints are immutable and shared either way.

Lesson: The maths: retries, copies and sizes

Attempts needed for a target

$$k \geq \log(1 - \text{target}) / \log(p)$$

p = 0.2, target 99.99 %: $k \geq \log(0.0001) / \log(0.2) \approx 5.7$, so 6 attempts.

Lesson: The maths: retries, copies and sizes

Expected attempts until success

$$E = 1 / (1 - p)$$

p = 0.2: 1.25 attempts on average.

Lesson: The maths: retries, copies and sizes

TABLES

Table: you write · attribute set on the new error · traceback says

You write	Attribute set on the new error	Traceback says
raise New() from old	__cause__ = old	"The above exception was the direct cause of the following exception"
raise New() inside an except block	__context__ = the error being handled	"During handling of the above exception, another exception occurred"
raise New() from None	__cause__ = None, __suppress_context__ = True	Only the new error

Lesson: Chaining: __cause__ and __context__

Table: · raise one exception · add_note · exceptiongroup and except*

	raise one exception	add_note	ExceptionGroup and except*
Since	Always	Python 3.11	Python 3.11
Reports	The first problem	One problem with extra context	Many problems at once
Caught with	except	except (notes in err.__notes__)	except*, each matching part separately
Typical use	Most errors	Adding the row, file or user while an error passes up	Form checks, many parallel tasks (asyncio TaskGroup)

Lesson: Tracebacks, notes and exception groups

Table: · exception · warning · assert

	exception	warning	assert
Stops the program	Yes, unless caught	No, by default it prints once	Yes (AssertionError)
For	Errors the caller must handle	Old or risky use that still works	Checks of your own logic while developing
Turned off by	Never	warnings filters	python -O removes every assert
Example	raise ValueError	warnings.warn(..., DeprecationWarning)	assert total >= 0

Lesson: Warnings and assertions

Table: · b = a · shallow copy · deep copy

	b = a	shallow copy	deep copy
Written as	b = a	a.copy(), a[:], list(a), dict(a), copy.copy(a)	copy.deepcopy(a)
New outer object	No	Yes	Yes
New inner objects	No	No, shared	Yes, all the way down
Cost	Free	O(length)	O(every object inside)
Safe to change inner items	No	No	Yes

Lesson: Shallow and deep copies

Table: · json · pickle · shelve

	json	pickle	shelve
Format	Text, readable by any language	Python only bytes	A pickle per key in a small database
Saves	dict, list, str, numbers, bool, None	Almost any Python object	Almost any object, by string key
Safe with untrusted data	Yes	NO	NO
Good for	APIs, config, data exchange	Caches, saving program state, sending objects between Python processes	A simple persistent dict

Lesson: pickle and shelve: save objects

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
138	Copy List with Random Pointer	Medium	Deep copy with a memo dict	Map every old node to its new copy in a dict, exactly like copy.deepcopy does with its memo. Then set next and random from the map.
133	Clone Graph	Medium	DFS with a memo, like deepcopy	Copy a node, remember the copy, then copy its neighbours. The memo stops the copy from looping forever around cycles.
297	Serialize and Deserialize Binary Tree	Hard	Preorder with markers	Write the tree in preorder with # for an empty child. Read it back with an iterator, building the left subtree before the right. This is pickling by hand.
449	Serialize and Deserialize BST	Medium	Preorder values and bounds	A BST is fixed by its preorder alone. Rebuild it by giving each value a range (low, high) it must fit in.
535	Encode and Decode TinyURL	Medium	A counter and two dicts	Give each new long URL the next number as its short code, and remember both directions.
606	Construct String from Binary Tree	Easy	Preorder with brackets	Write the value, then the left child in brackets, then the right child in brackets. Keep () for a missing left child only when a right child exists.
331	Verify Preorder Serialization of a Binary Tree	Medium	Count open slots	Start with one slot. Every token fills a slot, and a number opens two more. The string is valid if slots never go below zero and end at zero.
652	Find Duplicate Subtrees	Medium	Serialise every subtree	Turn each subtree into a string (its own little pickle). Count the strings and report a root the second time its string appears.
394	Decode String	Medium	A stack of unfinished parts	Read the text once. A number before [says how many times to repeat. On [save the text so far and the count on a stack. On] repeat the inner text and glue it to the saved text. This is how a loader rebuilds nested data from a flat string.
1106	Parsing A Boolean Expression	Hard	A stack of values and operators	Push every character except commas. On) pop the values back to the (, take the operator in front of it, and push the single result. Nested expressions are solved from the inside out.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER Give your package one base exception (ShopError) and children for each problem. Callers can then catch one specific error, or everything from your package with one except.

Lesson: Exceptions, copies and saved objects

IN THE EXAM Use raise ... from err when you turn a low level error into a meaningful one, so the original is never lost. Use from None only when the original adds nothing for the reader.

Lesson: Chaining: __cause__ and __context__

COMMON MISTAKE Never unpickle data from a source you do not trust. pickle.loads can run any code the data asks for, so a crafted file can take over your computer. For data from users or the internet use JSON.

Lesson: pickle and shelve: save objects

IN THE EXAM A shelf returns copies. db["ravi"]["tags"].append("work") changes a copy that is thrown away. Read the value, change it and assign it back (or open with writeback=True).

Lesson: pickle and shelve: save objects

1.5 Metaprogramming: type, class hooks and metaclasses

classes as objects, type(), introspection, __init_subclass__, class decorators, metaclasses, creation order.

KEY IDEAS

- **Classes are objects** type(x) gives the class. type(Dog) is the metaclass (normally type). type(type) is type. Classes can be stored, passed and returned.
- **type() with three arguments** type(name, bases, namespace) builds a class. A class statement does the same. The body runs once and fills the namespace.
- **Introspection** getattr(obj, name, default), setattr, hasattr, delattr, callable, vars, dir, __class__, __bases__, __mro__, __subclasses__(), inspect.signature.
- **Hooks** __init_subclass__ on the parent runs for every child (registries, checks). Class decorators get the finished class.
- **Metaclasses** class Meta(type): __new__ builds the class, __call__ makes objects. Inherited by children. Conflicts raise TypeError. Prefer lighter tools.

FORMULAS

Classes made by a factory

$$\text{classes} = n_1 \times n_2 \times \ldots \times n_k$$

make_record for 3 colours and 4 sizes builds 3 × 4 = 12 classes.

Lesson: The maths: counting classes, lookups and steps

Classes in a full hierarchy below one base

Exam favourite

$$b + b^2 + \ldots + b^d = b(b^d - 1) / (b - 1)$$

With b = 3 children each and d = 2 levels: 3 + 9 = 12 subclasses, each one seen by __init_subclass__.

Lesson: The maths: counting classes, lookups and steps

Average checks in an if and elif chain

Exam favourite

$$\text{average} = (n + 1) / 2 \text{ for } n \text{ equally likely names}$$

A chain of 9 actions needs 5 checks on average. A dict or getattr needs about 1 lookup.

Lesson: The maths: counting classes, lookups and steps

Worst case attribute lookup

$$\text{dicts checked} \le 1 + \text{len}(C.__mro__)$$

The instance dict, then each class in the MRO. For Kitten, Cat, object: at most 1 + 3 = 4.

Lesson: The maths: counting classes, lookups and steps

Objects made by a singleton

$$\text{objects} = 1 \text{ for any } n \ge 1 \text{ calls, against } n \text{ for a normal class}$$

50 calls of Config() make 1 object.

Lesson: The maths: counting classes, lookups and steps

Steps from an object up to type

$$\text{type(rex)} = \text{Dog, type(Dog)} = \text{Meta, type(Meta)} = \text{type, type(type)} = \text{type}$$

Three steps reach type, and then the chain repeats itself.

Lesson: The maths: counting classes, lookups and steps

TABLES

Table: · class statement · type(name, bases, dict) · collections.namedtuple or dataclasses.make_dataclass

	class statement	type(name, bases, dict)	collections.namedtuple or dataclasses.make_dataclass
Readable	Best	Hard for big classes	Good
Name and fields known only at run time	No	Yes	Yes
Methods	Written normally	Functions in the dict	Generated for you, add more by hand
Typical use	Almost always	Factories, plugins, tests	Record types built from data

Lesson: Building classes with type()

Table: tool · what it gives you

Tool	What it gives you
getattr(obj, "name", default)	The attribute with that name, or the default instead of AttributeError
setattr(obj, "name", value), delattr(obj, "name")	Set or remove an attribute when the name is in a string
hasattr(obj, "name")	True if getattr would work
callable(x)	True if x can be called: functions, methods, classes, objects with __call__
vars(obj) or obj.__dict__	The attributes stored in the object itself
dir(obj)	Every name you can reach from obj, including inherited ones, sorted
obj.__class__, C.__name__, C.__bases__, C.__mro__	The class of an object, its name, its parents and its lookup order
C.__subclasses__()	The direct children that exist right now
inspect.signature(f)	The parameters of a function, with defaults

Lesson: Introspection: looking inside while the program runs

Table: · if and elif chain · getattr(obj, name) · a dict of name to function

	if and elif chain	getattr(obj, name)	a dict of name to function
Code to add a new action	A new branch every time	Only the new method	One new dict entry
Speed	Checks branches one by one	One lookup	One lookup
Safety with user input	Safe, only listed names	Can reach any attribute, so check the name first	Safe, only listed names

Lesson: Introspection: looking inside while the program runs

Table: · class decorator · __init_subclass__ · metaclass

	class decorator	__init_subclass__	metaclass
Applies to	Only the decorated class	Every child (and grandchild) of the parent	Every class it builds, and their children
When it runs	After the class exists	While the class is being built	Before, during and after building, and on every call of the class
Can change the namespace before the class exists	No	No	Yes, in __new__ or __prepare__
Can change how objects are made	Only by replacing methods	Only by replacing methods	Yes, with __call__
Mixes with other metaclasses	Always	Always	Can clash (metaclass conflict)
How hard to read	Easy	Easy	Hard

Lesson: __init_subclass__ and class decorators

Table: · a module level object · a metaclass __call__ · __new__ that returns a saved object

	a module level object	a metaclass __call__	__new__ that returns a saved object
Code needed	One line: config = Config()	A metaclass class	A few lines in the class
Easy to understand	Yes	No	Medium
__init__ runs again on each call	Not called again	No	Yes, a common surprise
Easy to test and replace	Yes	Hard	Hard

Lesson: Metaclasses

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
211	Design Add and Search Words Data Structure	Medium	A trie of dicts, search with recursion	Store each word letter by letter in nested dicts. A dot must try every child, so search with a small recursive helper.
380	Insert Delete GetRandom O(1)	Medium	A list plus a dict of positions	The list allows a random pick. The dict maps each value to its index. To remove, move the last value into the hole and pop.
2502	Design Memory Allocator	Medium	Scan a list of owners	Keep one owner id per memory unit (0 means free). allocate looks for the first run of free units that is long enough.
2353	Design a Food Rating System	Medium	A heap per cuisine with lazy deletion	Push (-rating, name) into the heap of the cuisine. When asked, pop entries whose rating is out of date before reading the top.
2671	Frequency Tracker	Medium	Two counters	Count each number, and also count how many numbers have each frequency. Then hasFrequency is one dict lookup.
1993	Operations on Tree	Medium	Parents list, children lists, a lock dict	lock and unlock check one node. upgrade checks that no ancestor is locked and at least one descendant is, then unlocks every descendant.
1357	Apply Discount Every n Orders	Medium	A counter and a price dict	Count customers. Every n-th bill is multiplied by (100 – discount) / 100.
1476	Subrectangle Queries	Medium	Keep a list of updates	Instead of painting every cell, remember each update. getValue checks the updates from newest to oldest and falls back to the original grid.
2166	Design Bitset	Medium	A list of bits plus a flipped flag	flip only toggles a flag, so it is O(1). Keep a count of ones and fix it up when flip happens.
2424	Longest Uploaded Prefix	Medium	A set and a pointer that only moves forward	Remember uploaded videos in a set. The prefix pointer moves right while the next video is there. Each video is passed once in total.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

type has two jobs. With one argument, type(x) tells you the class of x. With three arguments, type(name, bases, namespace) builds a new class. And type(type) is type: type is its own class, which is where the chain ends.

Lesson: Classes are objects too

REMEMBER

A class statement and a type() call give exactly the same kind of class. Python itself turns every class statement into a call like type("Cat", (object,), namespace). You only need type() directly when the name or the fields are known only while the program runs, for example from a config file.

Lesson: Building classes with type()

COMMON MISTAKE

The class body runs once, at the moment Python reaches the class statement, not each time you make an object. Names it creates (tax, price, total) end up in the namespace dict, which becomes the class __dict__. That is why a class body can hold any normal code, even print.

Lesson: Building classes with type()

IN THE EXAM

__init_subclass__ (PEP 487, Python 3.6) runs on the PARENT every time a child class is created. It is a class method automatically. Keyword arguments in the class line, like name="csv", are passed to it. Always call super().__init_subclass__(**kwargs) so other parents in a multiple inheritance chain get their turn.

Lesson: __init_subclass__ and class decorators

REMEMBER A class decorator is a function that takes a finished class and returns a class, usually the same one after changing it. @dataclass, @functools.total_ordering and @enum.unique are class decorators from the standard library. Stacked decorators run from the bottom up, so auto_repr ran before route.

Lesson: __init_subclass__ and class decorators

IN THE EXAM A metaclass is a subclass of type. Its __new__(mcs, name, bases, namespace) gets everything the class statement collected and must return the new class, normally from super().__new__. Children of Job use CheckMeta too, because Python uses the metaclass of the base class when none is given. The check fires when the class is DEFINED, long before anyone uses it.

Lesson: Metaclasses

COMMON MISTAKE Calling a class, Config(...), really calls type(Config).__call__. The normal type.__call__ runs __new__ and then __init__. Overriding __call__ in a metaclass changes what calling the class does: here it returns the saved object. Notice that the second call did not even run __init__, so mode stayed "prod".

Lesson: Metaclasses

REMEMBER You already use metaclasses: ABC is built by ABCMeta (that is how abstract methods block objects) and every Enum by EnumType (that is why len(Colour) and for c in Colour work on the class itself). A class whose bases have two unrelated metaclasses cannot be made, because Python cannot pick one metaclass that is a child of both.

Lesson: Metaclasses

STEP 2 Advanced Python · Iterators, generators and context managers

2.1 Iterators and generators

iterator protocol, iterator classes, yield, pipelines, yield from, send, throw, close, lazy built ins.

KEY IDEAS

- **Iterator protocol** iter(x) gives an iterator. next(it) gives a value or raises StopIteration. next(it, default) avoids the error. A for loop does this for you.
- **Iterable versus iterator** Iterables (list, range, dict) give a fresh iterator each time. Iterators (zip, map, files, generators) are used up after one pass.
- **Generators** A function with yield returns a generator. Nothing runs until next(). It pauses at each yield and keeps its locals. Return ends it.
- **Pipelines and laziness** Chain generators: values are made only when asked, memory stays constant, and islice or break stops work early.
- **Advanced control** yield from delegates and returns the inner return value. send(v), throw(e), close() (GeneratorExit). Prime with next() before send.

FORMULAS

Memory of a list against a generator

Exam favourite

list: about 8n bytes of pointers plus the items,

generator: a constant (about 200 bytes)

100,000 squares: the list needs about 800 KB of pointers, the generator stays near 200 bytes.

Lesson: The maths: memory, early stopping and counting

Work done by a lazy search Exam favourite

items read = position of the first match (k), not n

Looking for the first even square above 50 in 1, 2, 3, ... reads 8 numbers and stops (8 × 8 = 64).

Lesson: The maths: memory, early stopping and counting

Running average after k values

$$\text{average}_k = (x_1 + x_2 + \ldots + x_k) / k$$

After 10, 20 and 60: (10 + 20 + 60) / 3 = 30.

Lesson: The maths: memory, early stopping and counting

Number of next() calls to see StopIteration

values + 1

countdown(4) gives 4 values, and the 5th call raises StopIteration.

Lesson: The maths: memory, early stopping and counting

Subsets and combinations (LeetCode 77 and 90)

subsets = 2^n, combinations of k from n = n! / (k! (n − k)!)

From 5 numbers choose 2: 5! / (2! × 3!) = 10.

Lesson: The maths: memory, early stopping and counting

A pipeline of s stages over n items

time grows with n × s, memory with s only

10^6 lines through 4 stages: 4 × 10^6 stage steps, but only 4 paused generators in memory.

Lesson: The maths: memory, early stopping and counting

TABLES

Table: · iterable (like shelf, list) · iterator (like countdown)

	iterable (like Shelf, list)	iterator (like Countdown)
Has __iter__	Yes, returns a NEW iterator	Yes, returns itself
Has __next__	No	Yes
Loop over it twice	Works both times	Second loop gets nothing
Remembers a position	No	Yes
Examples	list, str, dict, range, set	iter(list), zip, map, enumerate, files, generators

Lesson: Writing iterators as classes

Table: · return a list · iterator class · generator function

	return a list	iterator class	generator function
Lines of code	Few	Many (__init__, __iter__, __next__)	Few
Memory for n items	Grows with n	Constant	Constant
Can be endless	No	Yes	Yes
Stops work early	No, everything is made first	Yes	Yes
Use twice, index, len	Yes	No	No

Lesson: Generator functions and yield

Table: · read everything into lists · a generator pipeline

	read everything into lists	a generator pipeline
Memory	The whole file, maybe gigabytes	One line at a time
First result appears	After the whole file is read	Right away
Stopping early	Too late, the work is done	No extra work
Code	One big loop with many ifs	Small reusable stages

Lesson: Generator pipelines

Table: method · what it does

Method	What it does
next(g)	Run to the next yield and return the value it yields
g.send(v)	Resume, and make the paused yield expression evaluate to v. The first call must be next(g) or g.send(None)
g.throw(exc)	Raise exc at the paused yield. The generator may catch it and go on
g.close()	Raise GeneratorExit at the paused yield so finally blocks run, then mark it closed

Lesson: yield from, send, throw and close

Table: built in · lazy? · use more than once?

Built in	Lazy?	Use more than once?
range	Yes, works out each value from start, step and index	Yes, it is a sequence
dict.keys(), values(), items()	Yes, live views of the dict	Yes
zip, map, filter, enumerate, reversed	Yes, iterators	No
open(...) file objects	Yes, one line per next()	No, unless you seek(0)
iter(function, sentinel)	Calls function until it returns sentinel	No
sorted, list, tuple, sum	No, they use up the whole input	They return a value

Lesson: Lazy tools you already use

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
94	Binary Tree Inorder Traversal	Easy	A recursive generator with yield from	Yield everything from the left subtree, then the node, then everything from the right subtree.
173	Binary Search Tree Iterator	Medium	Keep one generator inside the class	An inorder generator of a BST gives the values in sorted order. next takes the next value. To answer hasNext, look one value ahead and remember it.
1305	All Elements in Two Binary Search Trees	Medium	Merge two sorted generators	Each BST gives a sorted stream with an inorder generator. heapq.merge joins sorted streams lazily, like the merge step of merge sort.
23	Merge k Sorted Lists	Hard	Turn each list into a generator, then heapq.merge	A small generator walks one linked list. heapq.merge takes k sorted streams and always hands out the smallest next value, using a heap of size k.
77	Combinations	Medium	A backtracking generator	Choose the next number from start to n, yield when k numbers are chosen. itertools.combinations does the same in C.
90	Subsets II	Medium	Sort, then skip equal neighbours	Sort first so equal numbers sit together. At each depth, do not start a branch with the same number twice.
216	Combination Sum III	Medium	Filter a combinations stream	Every answer is a set of k different digits from 1 to 9. Generate the combinations lazily and keep the ones whose sum is n.
401	Binary Watch	Easy	Loop over every time, count the lit bits	There are only 12 × 60 times. For each one count the 1 bits of the hour and the minute with bin(x).count("1").
1415	The k-th Lexicographical String of All Happy Strings of Length n	Medium	Generate in order, stop at k	A generator makes happy strings in alphabetical order. islice jumps to the k-th one without building the rest.
119	Pascal's Triangle II	Easy	A generator of rows	Each row is made from the one before: add neighbours and put 1 at both ends. Yield rows one by one and keep the one you need.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM — iter(nums) gave a NEW iterator each time, so the first answer was False. An iterator returns itself from iter(), so the second was True. That is the whole difference: an iterable can make many iterators, an iterator is one walk that can be used once.

Lesson: Iterators and generators: one value at a time

COMMON MISTAKE — Raising StopIteration ends the loop. Do not raise it inside a generator body: since Python 3.7 (PEP 479) it is turned into RuntimeError. In a generator, simply return.

Lesson: Writing iterators as classes

IN THE EXAM — A function that contains yield anywhere is a generator function. Calling it runs nothing: the first line, print("start"), only ran at the first next(). After the last value the generator is closed, and list() on a used generator gives an empty list.

Lesson: Generator functions and yield

REMEMBER — Building the pipeline read nothing. Asking for 2 errors read only 4 lines and never touched "WARN slow" or the last line. Each value travels through all three stages before the next line is read. Unix pipes such as grep | cut | head work the same way.

Lesson: Generator pipelines

REMEMBER yield from other passes every value of other straight to whoever is asking, and when other finishes, its return value becomes the value of the yield from expression. It is the clean way to split a big generator into smaller ones, and it handles send and throw for you too.

Lesson: yield from, send, throw and close

COMMON MISTAKE A common bug: saving map(...) or zip(...) in a variable, looping over it once for a check, and then finding it empty the second time. Wrap it in list(...) if you need it twice.

Lesson: Lazy tools you already use

2.2 itertools, functools and operator

endless iterators, slicing, joining, groupby, combinatorics with formulas, reduce, partial, singledispatch, operator.

KEY IDEAS

- **Endless and slicing** count, cycle, repeat never stop: use islice, zip, takewhile. takewhile/dropwhile cut at the first failure. compress applies a mask.
- **Joining** chain, chain.from_iterable, zip_longest(fillvalue=), pairwise, batched, starmap, accumulate(func, initial=), tee.
- **groupby** Joins neighbours with equal keys only. Sort by the same key first. Use each group before moving on.
- **Combinatorics** product n**r, permutations n!/(n-r)!, combinations C(n,r), with replacement C(n+r-1,r). math.comb and math.perm count without listing.
- **functools and operator** reduce (left fold), partial, singledispatch, lru_cache and cache_info. itemgetter, attrgetter, methodcaller as key functions.

FORMULAS

product(items, repeat=r) Exam favourite

$$n^r$$

3 items, r = 2: $3^2 = 9$ tuples.

Lesson: The maths: counting, prefix sums and folds

permutations(items, r) Exam favourite

$$P(n, r) = n! / (n - r)!$$

4 items, r = 2: 24 / 2 = 12.

Lesson: The maths: counting, prefix sums and folds

combinations(items, r) Exam favourite

$$C(n, r) = n! / (r! (n - r)!)$$

4 items, r = 2: 24 / (2 × 2) = 6.

Lesson: The maths: counting, prefix sums and folds

combinations_with_replacement(items, r)

$$C(n + r - 1, r)$$

3 items, r = 2: C(4, 2) = 6.

Lesson: The maths: counting, prefix sums and folds

Pascal's rule

$$C(n, r) = C(n - 1, r - 1) + C(n - 1, r)$$

C(4, 2) = C(3, 1) + C(3, 2) = 3 + 3 = 6: tuples with the first item, plus tuples without it.

Lesson: The maths: counting, prefix sums and folds

Prefix sums with accumulate Exam favourite

P = accumulate(a, initial=0), sum of a[i] to a[j] = P[j + 1] − P[i]

a = [3, 1, 4, 1, 5], P = [0, 3, 4, 8, 9, 14], a[1] + a[2] + a[3] = 9 − 3 = 6.

Lesson: The maths: counting, prefix sums and folds

reduce is a left fold

reduce(f, [a, b, c]) = f(f(a, b), c)

reduce(operator.sub, [10, 3, 2]) = (10 − 3) − 2 = 5.

Lesson: The maths: counting, prefix sums and folds

TABLES

Table: · filter · takewhile · dropwhile

	filter	takewhile	dropwhile	compress
Looks at	Every item	Items until the test first fails	Items until the test first fails	Each item and its mask value
Keeps	Items that pass	The front part that passes	Everything from the first failure on	Items where the mask is true
Works on endless input	Yes, but never finishes	Yes, it stops by itself	Yes	Yes
Typical use	Clean a list	Read until a blank line	Skip a header	Apply a yes or no column

Lesson: Taking and dropping items

Table: tool · what it gives · instead of

Tool	What it gives	Instead of
chain(a, b, c)	All items of a, then b, then c	a + b + c, which builds new lists
chain.from_iterable(lists)	Flattens one level	A nested loop
zip_longest(a, b, fillvalue=x)	Pairs until the LONGEST input ends	zip, which stops at the shortest
pairwise(xs)	Each item with the next one	zip(xs, xs[1:])
batched(xs, n)	Tuples of n items, the last may be shorter	Slicing by hand
starmap(f, pairs)	f(*pair) for each pair	A lambda that unpacks
accumulate(xs, f, initial=)	Running results of f (sum by default)	A loop with a total variable
tee(it, n)	n independent iterators from one	list(it), when the data is small

Lesson: Joining, pairing and running totals

Table: · sorted then groupby · a dict of lists · collections.counter

	sorted then groupby	a dict of lists	collections.Counter
Needs sorted data	Yes	No	No
Memory	One group at a time	All groups at once	One count per key
Keeps runs (aaa then b then aa)	Yes, on unsorted data	No	No
Best for	Run length encoding and data that is already sorted	General grouping	Counting only

Lesson: groupby: runs of equal keys

Table: · nested for loops · itertools · math.comb and math.perm

	nested for loops	itertools	math.comb and math.perm
Number of choices fixed in the code	Yes, one loop each	No, repeat=r or r is a variable	Not needed
Gives the actual tuples	Yes	Yes, lazily	No, only the count
Speed	Python speed	C speed	Instant
Use when	Two or three simple loops	You need the tuples	You only need how many

Lesson: Combinations and permutations

Table: · partial(power, exponent=2) · lambda b: power(b, 2) · def square(b)

	partial(power, exponent=2)	lambda b: power(b, 2)	def square(b)
Code	Short	Short	Longest
Shows what it wraps	Yes: .func, .args, .keywords	No	No
Can be pickled and sent to another process	Yes	No	Yes
Has its own docstring and name	No	No	Yes

Lesson: functools: reduce, partial, singledispatch and caches

Table: · lambda r: (r[1], r[0]) · itemgetter(1, 0)

	lambda r: (r[1], r[0])	itemgetter(1, 0)
Readable	Yes	Yes, once you know it
Speed	Python function call	Faster, written in C
Works with attributes	Yes	Use attrgetter("pages")
Can be pickled	No	Yes

Lesson: operator: itemgetter, attrgetter, methodcaller

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
303	Range Sum Query - Immutable	Easy	Prefix sums with accumulate	Build P = accumulate(nums, initial=0) once. Then the sum from left to right is P[right + 1] − P[left], one subtraction per query.
1732	Find the Highest Altitude	Easy	Running totals, then max	The altitudes are the running totals of gain starting from 0, which is accumulate(gain, initial=0).
1310	XOR Queries of a Subarray	Medium	Prefix XOR with accumulate	XOR works like a sum that undoes itself: P[r + 1] ^ P[l] is the XOR of arr[l] to arr[r].
1313	Decompress Run-Length Encoded List	Easy	batched, repeat and chain	Read the list two numbers at a time with batched. Each pair (freq, val) becomes repeat(val, freq). chain joins them.
443	String Compression	Medium	groupby gives the runs	Each run of the same letter becomes the letter plus its count (only if the count is above 1). Write the result back into chars and return its length.
1446	Consecutive Characters	Easy	Longest run with groupby	groupby splits the text into runs of one letter. The answer is the length of the longest run.
830	Positions of Large Groups	Easy	groupby over enumerate	Group (index, letter) pairs by the letter. The first and last index of each group give its interval. Keep groups of 3 or more.
784	Letter Case Permutation	Medium	product of choices per character	A letter has two choices (lower, upper), a digit has one. product of all the choices gives every string.
47	Permutations II	Medium	permutations, then remove repeats	permutations treats equal numbers as different positions, so it repeats some orders. A set removes the repeats. Fine for small inputs, backtracking with a counter is faster.
1588	Sum of All Odd Length Subarrays	Easy	Prefix sums, then every odd window	With prefix sums each subarray sum is one subtraction. Try every start and every odd length.

Lesson: LeetCode practice

TIPS AND TRAPS

COMMON MISTAKE count, cycle and repeat never end on their own. Always stop them: with islice, zip with something finite, takewhile, or a break.

Lesson: The iterator toolbox

REMEMBER tee keeps every value that one copy has seen and the other has not, so if one copy runs far ahead it can use as much memory as a list. Use it when both copies move at a similar pace.

Lesson: Joining, pairing and running totals

COMMON MISTAKE groupby only joins items that are NEXT to each other. On unsorted data tea appeared twice. Sort by the same key first, or use a dict (collections.defaultdict(list)) when you cannot sort.

Lesson: groupby: runs of equal keys

IN THE EXAM singledispatch chose the int version for 21, the list version for [1, 2] and the base version for 2.5. It looks at the type of the FIRST argument only. For methods use functools.singledispatchmethod.

Lesson: functools: reduce, partial, singledispatch and caches

REMEMBER lru_cache(maxsize=2) keeps the 2 most recently used answers. Calling 1, 2, 1, 3, 1 gave 2 hits: when 3 arrived, 2 was the least recently used and was thrown away.

Lesson: functools: reduce, partial, singledispatch and caches

REMEMBER The itertools documentation ends with a long list of recipes like these: sliding_window, unique_everseen, first_true, roundrobin, partition and more. The third party package more-itertools has all of them ready to install with pip.

Lesson: Recipes built from the tools

2.3 Context managers and the with statement

with, __enter__ and __exit__, suppressing errors, @contextmanager, standard managers, exit order, ExitStack.

KEY IDEAS

- **The with protocol** with X as v: v = X.__enter__(), body, then X.__exit__(type, err, tb) always. Methods are looked up on the type.
- **Suppressing** __exit__ returning True hides the exception. contextlib.suppress(Error) hides only the listed types.
- **@contextmanager** Set up, then yield the as value inside try, then clean up in finally. Re raise errors you do not handle. Works as a decorator.
- **Standard managers** open, Lock, TemporaryDirectory, chdir, redirect_stdout, localcontext, closing, nullcontext, sqlite3 (transaction), mock.patch.
- **Several managers** Entered left to right, exited right to left. Bracketed form across lines. ExitStack: enter_context and callback for a dynamic number.

FORMULAS

Exit order of n nested managers Exam favourite

the k-th one entered is the (n − k + 1)-th one exited

with A, B, C: A is entered 1st and exited 3rd.

Lesson: The maths: leaks, failures and exit order

Chance that at least one of n steps fails

Exam favourite

$$1 - (1 - p)^n$$

Each of 10 steps fails with p = 0.01: $1 - 0.99^{10} \approx 9.6\ \%$.
Errors are common, so clean up must not depend on success.

Lesson: The maths: leaks, failures and exit order

Expected leaks without with

leaks = calls × p (chance a path forgets to close)

1000 calls with p = 0.02 leak about 20 files. With with: 0.

Lesson: The maths: leaks, failures and exit order

Time until a leaking server runs out of file handles

requests = free handles / leaks per request

1024 handles and 2 leaks per request: 512 requests.

Lesson: The maths: leaks, failures and exit order

Elapsed time with a timer context manager

$$\text{elapsed} = t_{\text{exit}} - t_{\text{enter}}$$

Entered at 1.50 s, exited at 2.75 s: 1.25 s.

Lesson: The maths: leaks, failures and exit order

TABLES

Table: method · called · gets · returns

Method	Called	Gets	Returns
__enter__(self)	When the with line runs	Nothing else	The value stored by as (often self, a file, or a connection)
__exit__(self, kind, err, tb)	When the block ends, for ANY reason	None, None, None, or the exception type, object and traceback	True to hide the exception, False or None to let it continue

Lesson: Writing a context manager class

Table: · clean up as the last line · try and finally · with and a context manager

	clean up as the last line	try and finally	with and a context manager
Runs after an error	No	Yes	Yes
Runs after return or break	No	Yes	Yes
Code at each use	One line, often forgotten	Four or more lines	One line
Set up and clean up kept together	No	No, they are far apart	Yes, in one class or function
Reusable	No	No	Yes

Lesson: Writing a context manager class

Table: · try / except / pass · with suppress(error)

	try / except / pass	with suppress(Error)
Lines	Four	Two
Says what it means	Not at a glance	Yes
Can run code on the error	Yes	No
Other errors	Pass through	Pass through

Lesson: Suppressing exceptions

Table: · a class with __enter__ and __exit__ · @contextmanager on a generator

	a class with __enter__ and __exit__	@contextmanager on a generator
Code	Longer	Short
Keeps state between uses	Yes, in attributes	Only for one with
Can decide to hide errors	Return True from __exit__	Catch the error around yield and do not re raise
Can be used as a decorator	With ContextDecorator	Yes, automatically
Best for	Reusable objects like connections	Quick set up and clean up pairs

Lesson: contextmanager: a context manager from a generator

Table: context manager · set up · clean up

Context manager	Set up	Clean up
open(...)	Open a file	Close it
threading.Lock()	Acquire the lock	Release it
tempfile.TemporaryDirectory()	Make a folder	Delete it with everything inside
contextlib.chdir(path)	Change the working folder	Change back
contextlib.redirect_stdout(f)	Send print to f	Send it back to the screen
decimal.localcontext()	A private copy of the decimal settings	Restore the old settings
contextlib.closing(obj)	Nothing	Call obj.close()
contextlib.nullcontext(v)	Nothing, gives v	Nothing
sqlite3 connection	Start a transaction	Commit, or roll back on error (it does not close)
unittest.mock.patch(...)	Replace an object for a test	Put the original back

Lesson: Context managers in the standard library

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
636	Exclusive Time of Functions	Medium	A stack of running functions	Logs open and close like nested with blocks. Keep a stack of running ids. When something starts or ends, give the time since the last event to the function on top.
946	Validate Stack Sequences	Medium	Replay the pushes and pops on a real stack	Push each value. After every push, pop while the top is the next value that should come out. It works if the stack ends empty, just as every with block must close in reverse order.
921	Minimum Add to Make Parentheses Valid	Medium	Count open brackets	Track how many "(" are still open. A ")" with nothing open needs an added "(". At the end every open one needs a ")".
1249	Minimum Remove to Make Valid Parentheses	Medium	A stack of open positions	Push the index of each "(". A ")" with an empty stack is removed. Any "(" left on the stack at the end is removed too.
856	Score of Parentheses	Medium	A stack of scores per level	Opening starts a new level with score 0. Closing turns the level into 1 if it was empty, or double its score otherwise, and adds it to the level outside.
1190	Reverse Substrings Between Each Pair of Parentheses	Medium	A stack of unfinished text	On "(" save the text so far and start fresh. On ")" reverse the inner text and glue it to the saved text.
32	Longest Valid Parentheses	Hard	A stack of indexes with a base	Keep the index of the last unmatched position at the bottom of the stack. After each match, the length is the current index minus the index on top.
735	Asteroid Collision	Medium	A stack of survivors	Only a left moving asteroid can hit the right moving ones on the stack. Pop the smaller ones, stop if it is destroyed.
844	Backspace String Compare	Easy	Build each text with a stack	Push letters, pop on "#". Compare the two results.
2390	Removing Stars From a String	Medium	A stack again	Each star removes the closest letter on its left: push letters and pop on "*".

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER A file object is its own context manager: its __exit__ calls close(). Always open files with with. In CPython an unclosed file is usually closed by garbage collection, but other Pythons (PyPy) and long running programs can run out of file handles first.
Lesson: Context managers and the with statement

COMMON MISTAKE Only return True from __exit__ on purpose and only for the exact errors you expect. Hiding every error makes bugs invisible. contextlib.suppress(SomeError) is the clear, ready made way to say "this error is fine here".
Lesson: Suppressing exceptions

IN THE EXAM Everything before yield is the set up, the yielded value goes into as, and everything after yield is the clean up. The try and finally around the yield is what makes the clean up run after an error: without it, an exception in the body would be raised at the yield and skip the rest.
Lesson: contextmanager: a context manager from a generator

REMEMBER nullcontext is handy when a with is sometimes needed and sometimes not: lock if threaded else nullcontext().
Lesson: Context managers in the standard library

IN THE EXAM Managers are entered left to right and exited in the reverse order, like a stack: the last one opened is the first one closed. That is what you want: an inner resource that depends on an outer one is cleaned up first.
Lesson: Several managers and ExitStack

REMEMBER ExitStack is for when the number of managers is only known while the program runs. enter_context adds a manager, callback adds any function to call at the end, and everything is cleaned up in reverse order, even if opening the third file fails.
Lesson: Several managers and ExitStack

STEP 3 Advanced Python · Professional code: style, testing and logging

3.1 Clean code: PEP 8, docstrings and type hints

Zen of Python, PEP 8 layout and names, bug preventing habits, docstrings, doctest, type hints, ruff, black, mypy.

KEY IDEAS

- **PEP 8 layout** 4 spaces, 79 characters, 2 blank lines around top level defs, imports one per line grouped stdlib, third party, local.
- **PEP 8 names** snake_case functions and variables, CapWords classes, UPPER_CASE constants, _private by convention.
- **Bug preventing habits** is None, if items:, isinstance, startswith and endswith, None instead of mutable defaults, small try blocks.
- **Docstrings** PEP 257: """One line summary.""" then details. __doc__ and help() read it. doctest runs >>> examples.
- **Type hints and tools** list[int], int | None, Callable, Protocol, TypedDict, Literal, Final, def f[T]. Not checked at run time: use mypy, ruff and black.

FORMULAS

Cyclomatic complexity (McCabe)

Exam favourite

$$M = E - N + 2 \text{ (edges and nodes of the control flow graph)} = \text{decision points} + 1$$

circle_area has one if, so M = 1 + 1 = 2. ruff rule C901 warns when M is above a limit, often 10.

Lesson: The maths: complexity, paths and line budgets

Paths through k independent if statements

Exam favourite

$$\text{paths} = 2^k$$

4 separate ifs give 16 paths. Smaller functions keep this testable.

Lesson: The maths: complexity, paths and line budgets

Minimum tests to cover every independent path

$$\text{tests} \geq M$$

A function with M = 6 needs at least 6 tests for basis path coverage.

Lesson: The maths: complexity, paths and line budgets

Characters left on a line

$$\text{room} = 79 - \text{indent}$$

Code nested 2 levels deep (8 spaces) has 71 characters left.

Lesson: The maths: complexity, paths and line budgets

Spaces of indentation

$$\text{spaces} = 4 \times \text{depth}$$

3 levels deep: 12 spaces. Deep nesting eats the line budget, a hint to split the function.

Lesson: The maths: complexity, paths and line budgets

TABLES

Table: rule · do · do not

Rule	Do	Do not
Indentation	4 spaces per level	Tabs, 2 spaces, or a mix
Line length	At most 79 characters (72 for comments and docstrings)	Lines that need side scrolling
Blank lines	2 around top level functions and classes, 1 between methods	None at all, or many
Imports	One per line, at the top: standard library, then third party, then your own, each group sorted	import sys, os in the middle of the file
Spaces	x = 1, a == b, f(a, b), d["k"]	x=1, f(a ,b), d ["k"]
Functions and variables	snake_case: circle_area, total_price	CircleArea, totalPrice, A
Classes	CapWords: ShoppingCart	shopping_cart
Constants	UPPER_CASE: MAX_USERS	maxUsers
Private by convention	_helper, _cache	__everything (name mangling, only when needed)
Long expressions	Break inside brackets, operators at the start of the new line	Backslashes at line ends

Lesson: PEP 8: layout and names

Table: · x == none · x is none · if not x

	x == None	x is None	if not x
Asks	Does x say it equals None (__eq__)?	Is x the None object?	Is x empty, zero, False or None?
Can a class change the answer	Yes	No	Yes, with __bool__ or __len__
Right for	Never for None	Checking for a missing value	Checking for "nothing in it"

Lesson: Rules that prevent bugs

Table: pep 257 rule · example

PEP 257 rule	Example
Always triple double quotes	"""Return the mean."""
First line: a short summary that ends with a full stop, written as a command	"""Return the mean of a list.""", not """Returns the mean"""
Longer docstring: summary, blank line, then details	Arguments, return value, errors raised, examples
Every public module, function, class and method gets one	Private helpers may skip it
A class docstring says what the class is, __init__ says how to make one	See Wallet above

Lesson: Docstrings and doctest (PEP 257)

Table: hint · means

Hint	Means
int, str, float, bool, None	That exact kind of value
list[int], dict[str, int], tuple[int, str], set[str]	Containers and what is inside
int | None (old: Optional[int])	An int or None
Callable[[int], str]	A function taking an int and returning a str
Literal["up", "down"]	Only these values
TypedDict	A dict with known keys and value types
Protocol	Any class that has these methods (structural, "duck" typing)
Final	Must not be changed after it is set
def f[T](x: list[T]) -> T and type Alias = ...	Generics and aliases with the Python 3.12 syntax

Lesson: Type hints

Table: · no hints · hints and mypy · checks with isinstance at run time

	no hints	hints and mypy	checks with isinstance at run time
When it is found	When a user hits that line	Before the program runs	When that line runs
Cost while running	None	None	A little on every call
Documents the code	No	Yes	Partly
Catches every mistake	No	Most, if hints are complete	Only the ones you check

Lesson: Type hints

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
520	Detect Capital	Easy	Three allowed shapes	Like a naming rule: all capitals, all small, or only the first letter capital.
482	License Key Formatting	Easy	Clean, then group from the right	Remove dashes, make it upper case, then cut groups of k from the end so only the first group may be shorter.
273	Integer to English Words	Hard	Groups of three digits with clear helper names	Split the number into groups of three from the right (ones, Thousand, Million, Billion). Turn each group under 1000 into words with a small helper, and skip groups that are zero.
824	Goat Latin	Easy	Follow the rules word by word	Vowel start: add "ma". Otherwise move the first letter to the end and add "ma". Then add one "a" per word position.
1859	Sorting the Sentence	Easy	Sort by the last character	Each word ends with its position. Sort by that digit and remove it.
2047	Number of Valid Words in a Sentence	Easy	Check each rule, like a linter	No digits, at most one hyphen with letters on both sides, at most one punctuation mark and only at the end.
929	Unique Email Addresses	Easy	Normalise, then count with a set	In the local part drop everything after "+" and remove dots. Keep the domain as it is.
1556	Thousand Separator	Easy	Format with a separator	Python can group digits with format(n, ","). LeetCode wants dots, so swap the commas.
1694	Reformat Phone Number	Easy	Blocks of 3, end with 2 and 2 if 4 are left	Keep only digits. Take blocks of 3 while more than 4 remain. Then 4 digits become 2 and 2, otherwise one last block.
2299	Strong Password Checker II	Easy	One check per rule	At least 8 characters, a lower case letter, an upper case letter, a digit, a special character, and no two equal characters side by side.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER — The Zen is a set of values, not rules. Lines like "Readability counts", "Explicit is better than implicit" and "Errors should never pass silently" explain WHY PEP 8 and good habits exist.

Lesson: Clean code: style, docstrings and type hints

IN THE EXAM — black fixed the layout but not the names, the unused import or == None: a formatter only moves spaces and line breaks, it never changes meaning. That is why teams use a formatter AND a linter. ruff can do both (ruff format and ruff check).

Lesson: PEP 8: layout and names

REMEMBER — PEP 8 also says: use isinstance instead of comparing types, use startswith and endswith instead of slicing, do not compare booleans to True with ==, and keep try blocks small so you only catch the error you expect.

Lesson: Rules that prevent bugs

REMEMBER — doctest finds >>> lines in docstrings, runs them and compares the output with the next line. TestResults(failed=0, attempted=2) means both examples still work. It keeps documentation honest. Real test suites come in 3.2.

Lesson: Docstrings and doctest (PEP 257)

IN THE EXAM — Hints are stored in __annotations__ but Python never checks them while running: first("hello") worked although the hint says list. In Python 3.14 annotations are evaluated lazily (PEP 649 and 749), so a hint can even name a class defined later in the file.

Lesson: Type hints

3.2 Testing: unittest, pytest and mocks

unittest, fixtures, failures and errors, mocks, pytest, coverage, property based testing, test strategy.

KEY IDEAS

- **unittest** class T(unittest.TestCase), methods test_..., assertEqual, assertTrue, assertIn, assertRaises (with), assertAlmostEqual, setUp, subTest, skip.
- **Outcomes** PASSED, FAILED (assert false), ERROR (other exception). Read the last line. Tests must not depend on each other.
- **Mocks** Mock().m.return_value, side_effect (list or exception), assert_called_once_with, call_count, patch and patch.object. Inject dependencies.
- **pytest** test_ functions with plain assert, fixtures by parameter name, parametrize, approx, raises(match=), tmp_path, monkeypatch, -v, -k.
- **Strategy** Edges and just outside, every fixed bug, coverage is not correctness, property tests with Hypothesis, the test pyramid, TDD.

FORMULAS

Line coverage Exam favourite

$$\text{coverage} = \text{lines run} / \text{total lines} \times 100\ \%$$

17 of 19 lines run: 17 / 19 ≈ 89.5 %.

Lesson: The maths: coverage, boundaries and binary search

Boundary value tests for a range a to b

Exam favourite

$$\text{test } a - 1,\ a,\ b,\ b + 1 \text{ (and one value inside)}$$

percent from 0 to 100: test −1, 0, 100, 101 and 50, five tests.

Lesson: The maths: coverage, boundaries and binary search

Combinations of k options with v values each

$$\text{all combinations} = v^k$$

3 settings with 4 values each: 64 tests. Pairwise testing covers every pair of values with far fewer, about $v^2 = 16$.

Lesson: The maths: coverage, boundaries and binary search

Binary search steps (the LeetCode set)

Exam favourite

$$\text{steps} \le \lceil \log_2(n + 1) \rceil$$

n = 1000 sorted items: at most 10 steps. This is also how git bisect finds the commit that broke a test.

Lesson: The maths: coverage, boundaries and binary search

Chance a random test hits a bug that needs one value out of N

$$1 - (1 - 1/N)^t \text{ after t tests}$$

N = 1000 and t = 100: about 9.5 %. Picking edge cases on purpose beats random luck.

Lesson: The maths: coverage, boundaries and binary search

TABLES

Table: tool · what it does

Tool	What it does
setUp(self) and tearDown(self)	Run before and after EVERY test, so each test starts fresh
setUpClass(cls) and tearDownClass(cls)	Run once for the whole class (for slow set up like a database)
self.subTest(...)	Inside a loop, report each case separately and keep going after a failure
@unittest.skip(reason), skipIf(cond, reason)	Do not run this test, and say why
@unittest.expectedFailure	A known bug: the test is expected to fail for now
python -m unittest discover	Find every test_*.py file in the folder and run it

Lesson: setUp, subTest and skip

Table: · passed · failed · error

	PASSED	FAILED	ERROR
Means	Every assert was true	An assert was false	The test crashed with another exception
Usually points to	Nothing to do	Wrong result: a bug in the code or in the test	A crash: bad input handling, a typo, a missing file
First thing to read		The two values in the message	The last line of the traceback

Lesson: Reading failures and errors

Table: mock feature · use

Mock feature	Use
Mock().method.return_value = x	The fake method returns x
side_effect = [a, b] or an exception	A different answer on each call, or raise an error to test failure handling
assert_called_once_with(...), call_count, call_args	Check HOW your code used the dependency
patch("module.name") and patch.object(Class, "attr")	Replace a real object for the length of a with block or a test, then put it back
MagicMock	A Mock that also supports special methods like len() and with

Lesson: Mocks: replacing what you cannot test

Table: · call the real service · a mock · a fake local server

	call the real service	a Mock	a fake local server
Speed	Slow	Instant	Fast
Works offline and in CI	No	Yes	Yes
Can test timeouts and errors	Hard	Easy with side_effect	Yes
Catches changes in the real service	Yes	No	No

Lesson: Mocks: replacing what you cannot test

Table: · unittest · pytest · doctest

	unittest	pytest	doctest
Install	Built in	pip install pytest	Built in
A test is	A method in a TestCase class	Any test_ function	A >>> example in a docstring
Checks with	self.assertEqual and friends	Plain assert, with detailed messages	Printed output
Set up	setUp and tearDown	Fixtures, shared and combined by name	None
Many inputs	subTest	parametrize	More examples
Runs the others	No	Yes, unittest and doctest tests too	No

Lesson: pytest: the most used test runner

Table: kind of test · checks · speed · how many

Kind of test	Checks	Speed	How many
Unit test	One function or class, dependencies mocked	Milliseconds	Most of them
Integration test	Several parts together: code plus a real database or file	Seconds	Some
End to end test	The whole program as a user uses it (for web apps: Selenium or Playwright)	Slow	A few for the main journeys

Lesson: What to test and how much

Table: always test · example for discount(price, percent)

Always test	Example for discount(price, percent)
A normal case	discount(80, 50) is 40
The edges of every range	percent 0 and 100, price 0
Just outside the range	percent −1 and 101 raise ValueError
Awkward values	floats like 0.3, very large prices
Every bug you fixed	A regression test that would have caught it

Lesson: What to test and how much

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
704	Binary Search	Easy	Halve the range each step	Keep low and high. Look at the middle. The famous bug is an off by one in the loop condition, so test arrays of length 0, 1 and 2.
35	Search Insert Position	Easy	Binary search for the first place not smaller	When the loop ends, low is where target belongs. Test before the first item and after the last.
278	First Bad Version	Easy	Binary search on a yes or no question	Exactly what git bisect does with a failing test. The is_bad check is passed in, so a test can give a fake one.
374	Guess Number Higher or Lower	Easy	Binary search against an API	The guess API answers -1, 1 or 0. In tests we inject a fake API, like a Mock with a side effect.
34	Find First and Last Position of Element in Sorted Array	Medium	Two binary searches	Find the first index not smaller than target and the first index bigger than target. bisect_left and bisect_right do exactly this.
162	Find Peak Element	Medium	Walk uphill with binary search	If the middle is lower than its right neighbour, a peak lies to the right. Otherwise it lies at the middle or to the left.
153	Find Minimum in Rotated Sorted Array	Medium	Compare the middle with the right end	If the middle is bigger than the last item, the drop (and the minimum) is to the right.
33	Search in Rotated Sorted Array	Medium	One half is always sorted	Check which half is in order, then whether target lies inside it. Many edge cases: test a rotation at every position.
875	Koko Eating Bananas	Medium	Binary search on the answer	Try a speed in the middle and count the hours needed. Too slow: go faster. Fast enough: try slower.
4	Median of Two Sorted Arrays	Hard	Binary search the cut in the shorter array	Cut both arrays so the left parts hold half of all items and every left item is at most every right item. Then the median comes from the four items around the cuts.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM unittest is built into Python. A test is a method whose name starts with test_ inside a class that inherits from unittest.TestCase. assertEqual, assertTrue, assertIn, assertRaises and assertAlmostEqual (for floats) check results and give a clear message when they fail.

Lesson: Why we test, and a first test suite

COMMON MISTAKE Tests must not depend on each other or on their order. test_add_more added a book, but test_total still saw only the pen, because setUp made a new cart for each test.

Lesson: setUp, subTest and skip

REMEMBER advice() receives the client as an argument (dependency injection), so a test can hand it a fake. Code that creates its own network client deep inside is much harder to test. Designing for testability means passing dependencies in.

Lesson: Mocks: replacing what you cannot test

COMMON MISTAKE 100 % coverage means every line ran during the tests, NOT that every case was checked. The next lesson shows a bug that survives full coverage.

Lesson: pytest: the most used test runner

IN THE EXAM All 8 example tests passed with 100 % coverage, yet price – price × 100 / 100 can come out a tiny bit below zero because of float rounding. The fix is price × (100 – percent) / 100, which gives exactly 0 at 100 %. The failing price changes from run to run, because Hypothesis tries random inputs.

Lesson: Property based testing

REMEMBER

This shape, many fast unit tests at the bottom and few slow end to end tests at the top, is called the test pyramid. Run the fast tests on every save and all tests in CI (see the Git and CI/CD subject).

Lesson: What to test and how much

3.3 Logging and debugging

log levels, loggers, handlers, formatters, habits, dictConfig, rotation, print, assert and pdb debugging.

KEY IDEAS

- **Levels** DEBUG 10, INFO 20, WARNING 30 (default), ERROR 40, CRITICAL 50. basicConfig(level=, format=, stream=, force=True).
- **Parts** getLogger(__name__), logger and handler levels, Formatter fields (%(asctime)s %(levelname)s %(name)s %(message)s), propagate to parents.
- **Habits** logger.info("x %s", x) not f-strings, logger.exception in except, no secrets in logs, libraries never configure logging.
- **Configuration** dictConfig once at start, RotatingFileHandler (maxBytes, backupCount), TimedRotatingFileHandler, QueueHandler.
- **Debugging** f"{x=}", assert (removed by -O, not for input checks), breakpoint() and python -m pdb: b, c, n, s, p, l, w, q.

FORMULAS

Log volume Exam favourite

$$\text{bytes} = \text{lines per second} \times \text{bytes per line} \times \text{seconds}$$

50 lines a second of 200 bytes for a day (86,400 s) is 864,000,000 bytes, about 864 MB.

Lesson: The maths: log volume, rotation and bisecting

Disk used by a rotating log Exam favourite

$$\text{max size} = (\text{backupCount} + 1) \times \text{maxBytes}$$

maxBytes 1 MB with backupCount 4: at most 5 MB, for ever.

Lesson: The maths: log volume, rotation and bisecting

Levels shown for a given level L

$$\text{shown} = \text{levels with number} \geq L$$

With WARNING (30): WARNING, ERROR and CRITICAL, 3 of 5.

Lesson: The maths: log volume, rotation and bisecting

Sampling

$$\text{lines kept} = \text{events} / N \text{ when keeping 1 in } N$$

1 in 100 of 2,000,000 requests: 20,000 lines.

Lesson: The maths: log volume, rotation and bisecting

Bisecting a bug (halving the suspect code)

$$\text{checks} = \lceil \log_2 n \rceil$$

64 suspect lines: 6 checks with well placed prints or breakpoints.

Lesson: The maths: log volume, rotation and bisecting

Error budget

$$\text{allowed failures} = \text{requests} \times (1 - \text{target})$$

1,000,000 requests at 99.99 % success allow 100 failures. 120 ERROR lines use 120 % of the budget.

Lesson: The maths: log volume, rotation and bisecting

TABLES

Table: level · number · use it for

Level	Number	Use it for
DEBUG	10	Details for developers: values, steps
INFO	20	Normal events: started, saved, user logged in
WARNING	30	Something odd that the program can handle (the default level)
ERROR	40	An operation failed
CRITICAL	50	The program may not be able to go on

Lesson: Logging: a diary for your program

Table: part · job · example

Part	Job	Example
Logger	Named entry point, first level check, part of a tree split by dots	logging.getLogger("shop.cart")
Handler	Sends records somewhere, with its own level	StreamHandler, FileHandler, RotatingFileHandler, SMTPHandler
Formatter	Turns a record into text	"%(asctime)s %(levelname)s %(name)s: %(message)s"
Filter	Extra yes or no rule for a logger or handler	Drop records that contain a password

Lesson: Loggers, handlers and formatters

Table: format field · gives

Format field	Gives
%(asctime)s	Date and time
%(levelname)s	DEBUG, INFO, ...
%(name)s	Logger name
%(message)s	Your message with its arguments filled in
%(funcName)s, %(lineno)d, %(filename)s	Where the call was made
%(process)d, %(threadName)s	Which process and thread

Lesson: Loggers, handlers and formatters

Table: · print · logging

	print	logging
Turn off without editing code	No	Yes, change the level
Time, module and line	Only if you add them by hand	Yes, from the format
Where it goes	The screen	Screen, files, servers, email, all at once
Importance	All the same	Five levels
Good for	Quick checks while learning and the normal output of a script	Everything in programs that run for real

Lesson: Good logging habits

Table: handler (logging.handlers) · keeps logs under control by

Handler (logging.handlers)	Keeps logs under control by
RotatingFileHandler(path, maxBytes, backupCount)	Starting a new file at maxBytes and keeping backupCount old ones
TimedRotatingFileHandler(path, when="midnight", backupCount=7)	Starting a new file every day (or hour) and keeping a week
QueueHandler and QueueListener	Writing from a background thread, so slow disks do not slow the program
SysLogHandler, HTTPHandler	Sending logs to a central server

Lesson: Configuring logging for a whole app

Table: pdb command · does

pdb command	Does
b 4 (break)	Stop at line 4. b alone lists breakpoints
c (continue)	Run until the next breakpoint
n (next)	Run the current line, stepping over calls
s (step)	Step into a function call
p expr, pp expr	Print (pretty print) a value
l, ll	List the code around here, or the whole function
w (where), u, d	Show the call stack, move up or down in it
r (return)	Run until the current function returns
cl 1 (clear)	Remove breakpoint 1
q (quit)	Stop debugging

Lesson: Debugging: print, assert and pdb

Table: · print / f"{x=}" · logging · pdb or an ide debugger

	print / f"{x=}"	logging	pdb or an IDE debugger
Set up	None	Once per project	One breakpoint() line
See values at any line	Only where you printed	Only where you logged	Yes, and change them
Works on a server with nobody watching	No	Yes	No
Best for	Quick checks	Problems in running programs	Puzzling bugs you can reproduce

Lesson: Debugging: print, assert and pdb

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
937	Reorder Data in Log Files	Medium	A sort key that splits two kinds of logs	Letter logs come first, sorted by content then identifier. Digit logs keep their order: sorted is stable, so give them all the same key.
1598	Crawler Log Folder	Easy	Track the depth	"../" goes up one level (never above the top), "./" stays, anything else goes one level down.
2933	High-Access Employees	Medium	Group times per person, then a window of 3	Turn "hhmm" into minutes, sort each person's times, and check whether any 3 in a row fit within 59 minutes.
1604	Alert Using Same Key-Card Three or More Times in a One Hour Period	Medium	The same window idea on access logs	Group by name, sort the minutes, and alert if times[i] − times[i − 2] is at most 60.
1817	Finding the Users Active Minutes	Medium	A set of minutes per user	Collect each user's different minutes in a set. The size of the set is that user's active minutes. Count users per size.
2747	Count Zero Request Servers	Medium	Sort logs and queries, slide a window	Sort logs by time. Answer queries in time order with two pointers that keep only logs inside [q − x, q] and a count per server.
1169	Invalid Transactions	Medium	Parse, then compare every pair	A transaction is invalid above 1000, or when the same name has another one in a different city within 60 minutes.
539	Minimum Time Difference	Medium	Turn log times into minutes and sort	Change each "HH:MM" into minutes, sort, and compare neighbours. Do not forget the gap that wraps around midnight: first + 1440 − last.
1418	Display Table of Food Orders in a Restaurant	Medium	Turn order records into a report table	Count orders in a dict of table number to a dict of food to count. The header is "Table" plus the sorted food names. Rows go in number order of the tables.
165	Compare Version Numbers	Medium	Compare number by number	Split on dots, turn each part into an int (so 01 equals 1) and treat missing parts as 0.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Without basicConfig the root logger shows only WARNING and above, on stderr. basicConfig sets the level, the format and the stream in one call, but only the first time, unless you pass force=True.

Lesson: Logging: a diary for your program

REMEMBER

Loggers form a tree by name: shop.cart is a child of shop, which is a child of root. A record goes to the handlers of its logger and then, unless propagate is False, up to the parents. Add handlers in one place (often the root or the top package logger) and just call getLogger(__name__) in every module.

Lesson: Loggers, handlers and formatters

IN THE EXAM

Pass values as arguments: logger.debug("total %s", total). The text is only built if the record passes the level checks. An f-string is always built, even when the message is thrown away, as the first slow __str__ line shows.

Lesson: Good logging habits

COMMON MISTAKE

Inside except, logger.exception("...") logs at ERROR level and adds the full traceback. Never log passwords, card numbers or personal data: logs are read by many people and kept for a long time.

Lesson: Good logging habits

REMEMBER Configure logging once, at program start, with dictConfig (or a file read into a dict, often YAML or TOML). Libraries must never call basicConfig: they only call getLogger(__name__), and the application decides what to show. dictConfig switches off loggers that already exist unless you set "disable_existing_loggers": False.

Lesson: Configuring logging for a whole app

REMEMBER On the second stop total is 70, not 70 + 0: the line replaces total instead of adding to it. The fix is total += s.

Lesson: Debugging: print, assert and pdb

COMMON MISTAKE assert checks a condition you believe is always true and raises AssertionError if not. Python started with -O skips every assert, so never use assert to check user input. Raise ValueError for that.

Lesson: Debugging: print, assert and pdb

STEP 4 Advanced Python · Speed: complexity and smart data structures

4.1 Complexity, timing and profiling

Big O, operation costs, timeit, cProfile, memory, amortized cost and caching.

KEY IDEAS

- **Big O** Drop constants, keep the biggest term. Loops in a row add, nested loops multiply. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.
- **Costs** list: index O(1), append O(1) amortized, x in list O(n), insert(0)/pop(0) O(n). dict and set: O(1) average. sort O(n log n).
- **Measure** timeit (python -m timeit -s setup stmt, best of several runs), time.perf_counter, realistic n.
- **Profile** python -m cProfile -s cumulative file.py, tottime vs cumtime, fix only the hot spot. tracemalloc for memory, getsizeof is shallow.
- **Trade offs** A better algorithm first. Sets for lookups, lru_cache for repeated work, generators to save memory.

FORMULAS

Definition of Big O Exam favourite

$$f(n) = O(g(n)) \text{ if there are numbers } c > 0 \text{ and } n_0 \text{ with } f(n) \le c \cdot g(n) \text{ for every } n \ge n_0$$

$3n + 5 = O(n)$: take $c = 4$ and $n_0 = 5$, since $3n + 5 \le 4n$ when $n \ge 5$.

Lesson: The maths behind Big O

Big Omega and Big Theta

$$\Omega(g) \text{ is a lower bound, } \Theta(g) \text{ means both } O(g) \text{ and } \Omega(g)$$

Comparison sorting is $\Omega(n \log n)$: no comparison sort can do better.

Lesson: The maths behind Big O

Sum of a loop inside a loop Exam favourite

$$1 + 2 + \ldots + n = n(n + 1) / 2 = O(n^2)$$

Comparing each item with the ones after it: 1000 items give 499,500 comparisons.

Lesson: The maths behind Big O

Geometric sum (doubling) Exam favourite

$$1 + 2 + 4 + \ldots + n = 2n - 1 < 2n$$

Copies for 1024 appends with doubling: $1 + 2 + \ldots + 512 = 1023$, about 1 per append.

Lesson: The maths behind Big O

Halving

$$\text{steps} = \lfloor \log_2 n \rfloor + 1$$

Binary search in 1,000,000 sorted items: at most 20 steps.

Lesson: The maths behind Big O

Master theorem

Exam favourite

$$T(n) = a \cdot T(n/b) + O(n^d) \text{ gives } O(n^d) \text{ if } d > \log_b a,$$

$$O(n^d \log n) \text{ if } d = \log_b a, O(n^{\log_b a}) \text{ if } d < \log_b a$$

Merge sort: a = 2, b = 2, d = 1. $\log_2 2 = 1 = d$, so O(n log n).

Lesson: The maths behind Big O

Time estimate

$$\text{seconds} \approx \text{steps} / (\text{about } 10^7 \text{ simple Python steps per second})$$

$n = 10^5$ with $O(n^2)$ is 10^{10} steps: about 1000 s. With O(n log n) it is 1.7×10^6: about 0.17 s.

Lesson: The maths behind Big O

TABLES

Table: big o · name · n = 1000 needs about · typical code

Big O	Name	n = 1000 needs about	Typical code
O(1)	constant	1 step	d[key], x in a set, list[i], append
O(log n)	logarithmic	10 steps	binary search, bisect, heap push
O(n)	linear	1,000 steps	one loop, x in a list, sum, max, copy
O(n log n)	linearithmic	10,000 steps	sorted(), list.sort()
$O(n^2)$	quadratic	1,000,000 steps	a loop inside a loop over the same data
$O(2^n)$	exponential	more than the atoms in the universe	trying every subset, naive recursion

Lesson: Big O: how the work grows

Table: operation · list · dict / set · collections.deque

Operation	list	dict / set	collections.deque
Get or set by index or key	O(1)	O(1) on average	O(n) in the middle, O(1) at the ends
x in container	O(n)	O(1) on average	O(n)
Add at the end	O(1) amortized (append)	O(1) on average (d[k] = v, add)	O(1) (append)
Add or remove at the front	O(n) (insert(0, x), pop(0))	not ordered by position	O(1) (appendleft, popleft)
Remove a given value	O(n) (remove)	O(1) on average (del d[k], discard)	O(n)
Sort	O(n log n)	sorted(d) O(n log n)	not supported in place
Copy, len	O(n), O(1)	O(n), O(1)	O(n), O(1)

Lesson: Finding the Big O of code

Table: · list · set

	list	set
x in c	O(n): looks at items one by one	O(1): jumps to the right place with the hash
Memory	Small	About 3 to 4 times more
Keeps order and duplicates	Yes	No
Items must be	Anything	Hashable (no lists or dicts)
Best for	Data you go through in order	Many "is it there" questions

Lesson: Finding the Big O of code

Table: rule for fair timing · why

Rule for fair timing	Why
Use timeit or time.perf_counter, not time.time	perf_counter is the most precise clock, and timeit repeats the code many times
Take the minimum of several runs	Other programs only ever slow a run down, so the fastest run is closest to the truth
Time with realistic sizes	With 10 items everything is fast. Differences show up with large n
Keep setup out of the timed part	Building test data is not what you want to measure
Compare on the same computer	Times are not comparable across machines, ratios are

Lesson: Measuring with timeit

Table: · time.time() · time.perf_counter() · timeit

	time.time()	time.perf_counter()	timeit
Precision	Can be coarse, and jumps if the clock is changed	The best clock available	Uses perf_counter
Repeats the code	No	No	Yes, many loops, several rounds
Best for	Dates and times of day	Timing one block in a real program	Comparing two small pieces of code

Lesson: Measuring with timeit

Table: column · means

Column	Means
ncalls	How many times the function was called
tottime	Time spent in the function itself, without the functions it calls
cumtime	Time in the function including everything it calls
percall	tottime / ncalls and cumtime / ncalls

Lesson: Finding the slow part with cProfile

Table: · cprofile (built in) · line_profiler · py-spy, scalene

	cProfile (built in)	line_profiler	py-spy, Scalene
Shows	Time per function	Time per line	A live sampling of a running program, Scalene also memory
Install	Nothing	pip install line_profiler	pip install py-spy or scalene
Slows the program	Somewhat	A lot, only on chosen functions	Hardly at all (sampling)
Best for	Finding the slow function	Finding the slow line inside it	Programs already running in production

Lesson: Finding the slow part with cProfile

Table: way to save memory · how

Way to save memory	How
Generators instead of lists	sum(x * x for x in data) makes one value at a time
__slots__ in classes with many objects	No per object __dict__, often half the memory
array.array or NumPy for numbers	Raw numbers in one block instead of separate int objects
Read files line by line	for line in f instead of f.read() or f.readlines()
del large objects you no longer need	The memory can be reused as soon as nothing refers to it

Lesson: Memory: getsizeof, tracemalloc and generators

Table: · a better algorithm or data structure · caching (lru_cache, a dict) · small tweaks (local variables, fewer function calls)

	A better algorithm or data structure	Caching (lru_cache, a dict)	Small tweaks (local variables, fewer function calls)
Typical gain	10 to 1,000,000 times	Big when the same inputs repeat	10 % to 2 times
Cost	Thinking	Memory, and care with changing data	Harder to read code
When	First	When profiling shows repeated work	Last, and only in the hot spot

Lesson: Amortized cost and trading memory for time

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
219	Contains Duplicate II	Easy	Remember the last index of each value	A dict from value to its last index answers "where did I see this before" in O(1), so one pass is enough: O(n) instead of comparing all pairs.
2006	Count Number of Pairs With Absolute Difference K	Easy	Count what you have seen	For each number, the partners seen so far are x − k and x + k. A dict of counts finds them in O(1), so the whole list takes O(n).
1010	Pairs of Songs With Total Durations Divisible by 60	Medium	Group by remainder	Two songs fit when their remainders mod 60 add up to 0 or 60. Count remainders seen so far: O(n) instead of $O(n^2)$.
454	4Sum II	Medium	Meet in the middle	Trying all four lists is $O(n^4)$. Count every a + b in a dict ($O(n^2)$), then look up −(c + d) for every pair of the other two lists ($O(n^2)$).
974	Subarray Sums Divisible by K	Medium	Prefix sums and remainders	A subarray sum is prefix[j] − prefix[i]. It is divisible by k when both prefixes have the same remainder. Count remainders: O(n) instead of trying all $O(n^2)$ subarrays.
209	Minimum Size Subarray Sum	Medium	A sliding window	Grow the window on the right, shrink it from the left while the sum is big enough. Each index enters and leaves once, so it is O(n).
1004	Max Consecutive Ones III	Medium	A window with at most k zeros	Move the right end one step at a time. When the window holds more than k zeros, move the left end until it does not. O(n) time, O(1) memory.
204	Count Primes	Medium	Sieve of Eratosthenes	Testing each number by division is about $O(n \sqrt{n})$. The sieve crosses out multiples of each prime starting at p^2, which is O(n log log n).
287	Find the Duplicate Number	Medium	Treat the list as a linked list	Without changing the list and with O(1) memory: i points to nums[i]. The repeated value is where a cycle starts. Floyd's slow and fast pointers find it in O(n).
41	First Missing Positive	Hard	Use the list itself as the table	O(n) time and O(1) extra memory: swap each value v into place v − 1. Then the first place i that does not hold i + 1 gives the answer.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Look at the table: for 1000 items linear search needs 1000 steps, binary search only 10, and comparing every pair needs 1,000,000. The same computer, the same data size, and a 100,000 times difference.

Lesson: Big O: how the work grows

COMMON MISTAKE

Hidden loops count too. x in my_list, my_list.index(x), my_list.remove(x), my_list.insert(0, x), "".join and slicing a[i:j] each walk through items. Putting one of them inside a loop is the most common way Python code becomes quadratic by accident.

Lesson: Finding the Big O of code

IN THE EXAM

Average case versus worst case: a dict lookup is O(1) on average, but if many keys had the same hash it could take O(n). Python adds random salt to str hashes so attackers cannot cause this on purpose. For interviews and exams, dict and set operations are O(1).

Lesson: Finding the Big O of code

REMEMBER

60.9 microseconds against 19.4 nanoseconds: the set is about 3,000 times faster for 10,000 numbers. -s is the setup, which runs once and is not timed. timeit chooses how many loops to run and reports the best of 5 rounds.

Lesson: Measuring with timeit

REMEMBER

slow_unique takes 0.136 of the 0.138 seconds, so that is the only function worth fixing. Speeding up anything else could save at most 0.002 seconds. Always profile before you optimise: the slow part is often not where you expect.

Lesson: Finding the slow part with cProfile

IN THE EXAM

sys.getsizeof is shallow. For a list it counts the list and its slots, not the objects inside, which is why a list holding a huge number has the same size as one holding a small number. tracemalloc counts every allocation made while it runs, so it shows the real memory a piece of code needed.

Lesson: Memory: getsizeof, tracemalloc and generators

REMEMBER

Most appends cost one step. Now and then the array is full and must copy everything, which costs n steps. But because the space doubles each time, the copies add up to less than n in total, so the average (amortized) cost per append stays about 1. That is why list.append is called O(1) amortized.

Lesson: Amortized cost and trading memory for time

REMEMBER

fib_slow(20) makes 21,891 calls because it works out the same values again and again. With lru_cache each value from 0 to 20 is computed once: 21 calls. We spend a little memory (21 stored answers) to save a huge amount of time, which is the idea behind dynamic programming.

Lesson: Amortized cost and trading memory for time

4.2 collections, heapq and bisect

Counter, deque, namedtuple, OrderedDict, ChainMap, heapq priority queues and bisect.

KEY IDEAS

- **Counter** Counter(items), c[x] is 0 if missing, most_common(k), total(), elements(), + - & | on counts.
- **deque** append, appendleft, pop, popleft all O(1), rotate(n), deque(maxlen=n). Queues and BFS use popleft.
- **Records** namedtuple("P", ["x", "y"]) with _replace, _asdict, _fields. OrderedDict.move_to_end, popitem(last=False). ChainMap(first, fallback). defaultdict(list).
- **heapq** heappush, heappop O(log n), heapify O(n), heap[0] smallest, nlargest and nsmallest O(n log k), merge, (priority, count, item), heappush_max in 3.14.
- **bisect** bisect_left, bisect_right, insort, key= since 3.10. Range count = right(high) − left(low). Only on sorted lists.

FORMULAS

Heap indexes Exam favourite

$$\text{parent}(i) = (i - 1) \,//\, 2, \text{ children } 2i + 1 \text{ and } 2i + 2$$

Index 4 has parent 1 and children 9 and 10.

Lesson: The maths: heap indexes, heights and costs

Height of a heap Exam favourite

$$h = \lfloor \log_2 n \rfloor$$

1,000,000 items: 19 levels below the top, so a push makes at most 19 swaps.

Lesson: The maths: heap indexes, heights and costs

heapify is O(n)

$$\text{work} \le \sum (n / 2^{h+1}) \cdot h < n$$

Most items are near the bottom and move only a little, so the total is less than n swaps.

Lesson: The maths: heap indexes, heights and costs

Top k with a heap Exam favourite

$$O(n \log k) \text{ instead of } O(n \log n) \text{ for a full sort}$$

k = 10 of $n = 10^6$: about 3.3×10^6 steps against 2×10^7.

Lesson: The maths: heap indexes, heights and costs

Binary search

$$\text{comparisons} \le \lceil \log_2(n+1) \rceil$$

1000 sorted items: at most 10 comparisons.

Lesson: The maths: heap indexes, heights and costs

Front removals from a list

$$\text{shifts} = (n-1) + (n-2) + \ldots + 1 = n(n-1)/2$$

Emptying 10,000 items with pop(0): about 5×10^7 shifts. With deque.popleft: 0.

Lesson: The maths: heap indexes, heights and costs

TABLES

Table: you need to · use · speed of the key operation

You need to	Use	Speed of the key operation
Count things	collections.Counter	O(1) per item
Group things by a key	collections.defaultdict(list)	O(1) per item
Add and remove at both ends (a queue)	collections.deque	O(1) at each end
Keep only the last n items	deque(maxlen=n)	O(1), old items drop off
A small read only record with names	namedtuple or a frozen dataclass	Same as a tuple
Always get the smallest (or largest) item	heapq	O(log n) push and pop, O(1) peek
The k largest or smallest of many	heapq.nlargest, heapq.nsmallest	O(n log k)
Search or insert in a sorted list	bisect	O(log n) search, O(n) insert
Look through several dicts in order	collections.ChainMap	O(number of dicts)

Lesson: The right container for the job

Table: counter operation · result

Counter operation	Result
c + d	Adds counts, keeps only positive ones
c - d	Subtracts counts, keeps only positive ones
c & d	The smaller count of each item (intersection)
c | d	The bigger count of each item (union)
c.update(items), c.subtract(items)	Change counts in place (subtract can go below 0)
c.elements()	Each item repeated count times
c.total()	Sum of all counts (3.10+)

Lesson: Counter: counting made easy

Table: · list with pop(0) · collections.deque · queue.queue

	list with pop(0)	collections.deque	queue.Queue
Remove from the front	O(n)	O(1)	O(1)
Index in the middle	O(1)	O(n)	Not supported
Safe between threads	No	append and popleft are thread safe	Yes, with blocking get and put
Best for	Stacks and arrays	Queues, BFS, sliding windows, last n items	Passing work between threads (step 5)

Lesson: deque: a fast queue with two ends

Table: · tuple · namedtuple · dataclass

	tuple	namedtuple	dataclass	dict
Fields by name	No, only p[0]	Yes, p.x	Yes, p.x	Yes, p["x"]
Can change	No	No (use _replace)	Yes, unless frozen=True	Yes
Memory	Smallest	Same as a tuple	More, unless slots=True	Most
Methods and defaults	No	defaults=, few methods	Yes, any	No
Best for	Two or three values that go together	Read only rows, like CSV lines	Objects with behaviour	Data whose keys are not known in advance

Lesson: namedtuple, OrderedDict, ChainMap and defaultdict

Table: · min(list) each time · keep the list sorted · heapq

	min(list) each time	Keep the list sorted	heapq
Add an item	O(1)	O(n) (insort shifts items)	O(log n)
Take the smallest	O(n)	O(1) from the end of a reverse sorted list	O(log n)
Good when	You ask only once or twice	You rarely add, often read in order	You keep adding and taking, as in a scheduler or Dijkstra

Lesson: heapq: a priority queue in a list

Table: function · returns

Function	Returns
bisect_left(a, x)	The first index where x could go: before any equal items
bisect_right(a, x) (also bisect)	The index after the last item equal to x
insort(a, x)	Puts x into a, keeping it sorted (search O(log n), insert O(n))
key=func (3.10+)	Compare func(item) instead of the item, without building a new list

Lesson: bisect: binary search in a sorted list

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1046	Last Stone Weight	Easy	A max-heap with negative numbers	Always smash the two heaviest stones. heapq is a min-heap, so store −weight. (Python 3.14 also has heappop_max, but LeetCode may run an older Python.)
1351	Count Negative Numbers in a Sorted Matrix	Easy	bisect on each row	Each row goes down from left to right. With key=lambda v: −v it goes up, so bisect finds where the negatives start in O(log n) per row.
215	Kth Largest Element in an Array	Medium	A min-heap of size k	Keep the k largest numbers seen so far in a min-heap. Its top is the kth largest. O(n log k) time and O(k) memory.
973	K Closest Points to Origin	Medium	nsmallest with a key	Compare squared distances $x^2 + y^2$ (no square root needed, the order is the same). heapq.nsmallest does it in O(n log k).
692	Top K Frequent Words	Medium	Counter, then sort by two keys	Count with Counter. Sort by count going down, then by the word going up for ties: key=(-count, word).
2300	Successful Pairs of Spells and Potions	Medium	Sort once, then bisect for each spell	A spell s needs a potion of at least ⌈success / s⌉. Sort the potions and bisect_left gives how many are too weak. O((n + m) log m).
300	Longest Increasing Subsequence	Medium	Patience sorting with bisect	tails[i] is the smallest last number of an increasing run of length i + 1. Each number replaces the first tail that is not smaller, found with bisect_left. O(n log n).
621	Task Scheduler	Medium	Count the most common task	The most common task (count m) needs m − 1 gaps of n slots after it. Tasks that share that top count fill the last row. The answer is never less than the number of tasks.
239	Sliding Window Maximum	Hard	A monotonic deque of indexes	Keep indexes whose values go down from front to back. Drop smaller values from the back before adding, and the front once it leaves the window. Each index enters and leaves once: O(n).
295	Find Median from Data Stream	Hard	Two heaps	A max-heap (negative numbers) holds the smaller half, a min-heap the bigger half. Keep their sizes equal or the low half one bigger. The median is at the tops.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Counter is a dict subclass, so everything a dict can do works too. Unlike a dict, a missing key gives 0 instead of KeyError. most_common(k) returns the k most common items with their counts, and items with the same count keep the order in which they were first seen.

Lesson: Counter: counting made easy

IN THE EXAM

A deque (say "deck") is a double ended queue. Adding and removing at either end is O(1). A list does the same at the end, but list.insert(0, x) and list.pop(0) must shift every other item, which is O(n).

Lesson: deque: a fast queue with two ends

REMEMBER

Breadth first search (BFS) always takes the oldest path from the front of the queue and adds new paths at the back. That is why it finds the path with the fewest steps first, and why it needs a deque, not a list.

Lesson: deque: a fast queue with two ends

IN THE EXAM

Since Python 3.7 every dict keeps insertion order. OrderedDict still has extras: move_to_end(key), popitem(last=False) for the oldest item, and == compares order too. That makes it the classic tool for an LRU cache.

Lesson: namedtuple, OrderedDict, ChainMap and defaultdict

REMEMBER

ChainMap does not copy anything. It searches the dicts one after another, so a change to user is seen at once. Writes go to the first dict only. It is how Python looks up names: locals, then globals, then builtins.

Lesson: namedtuple, OrderedDict, ChainMap and defaultdict

IN THE EXAM

A heap is a list where every item is smaller than or equal to its two children: heap[i] ≤ heap[2i + 1] and heap[i] ≤ heap[2i + 2]. So heap[0] is always the smallest. The rest is NOT sorted. heappush and heappop are O(log n), heapify is O(n), and looking at heap[0] is O(1).

Lesson: heapq: a priority queue in a list

REMEMBER

Put tuples like (priority, count, task) in a heap. Tuples compare item by item, so the priority decides first. The count (from itertools.count()) breaks ties, so two tasks with the same priority never have to be compared, which would fail for objects that cannot be compared.

Lesson: heapq: a priority queue in a list

REMEMBER

bisect_right(a, x) – bisect_left(a, x) is how many times x appears. Counting values in a range or looking up which band a number falls into (grades, tax brackets, prices) are classic uses.

Lesson: bisect: binary search in a sorted list

STEP 5

Advanced Python · Doing many things at once: threads, processes and asyncio

5.1 Threads and processes

threads, races and locks, worker queues, futures, the GIL, processes and free-threaded Python.

KEY IDEAS

- **When to use what** I/O bound: threads or asyncio. CPU bound: processes (or python3.14t). Measure first.
- **Threads** Thread(target, args), start, join, daemon. Shared memory means races: protect with with lock:.
- **Sync tools** Lock, RLock, Semaphore(n), Event, Condition, Barrier, queue.Queue. Same lock order everywhere to avoid deadlock.
- **Executors** ThreadPoolExecutor and ProcessPoolExecutor: submit, map (keeps order), as_completed (finish order), result() raises errors.
- **Processes** if __name__ == "__main__" guard, picklable functions and data, start methods fork, spawn, forkserver (Linux default in 3.14).

FORMULAS

Speedup and efficiency — Exam favourite

$$S = T_1 / T_n,\ E = S / n$$

1.30 s alone, 0.40 s with 4 processes: S = 3.25, E = 0.8125 (about 81 %).

Lesson: The maths: speedup, Amdahl and pool sizes

Amdahl's law — Exam favourite

$$S(n) = 1 / ((1 - p) + p / n)$$

p = 0.9 of the work can run in parallel, n = 4: S = 1 / (0.1 + 0.225) ≈ 3.08.

Lesson: The maths: speedup, Amdahl and pool sizes

The limit of Amdahl's law

$$S(\infty) = 1 / (1 - p)$$

With p = 0.9, even a million cores give at most 10 times.

Lesson: The maths: speedup, Amdahl and pool sizes

Gustafson's law

$$S(n) = n - (1 - p)(n - 1)$$

If the problem grows with the cores: n = 4, p = 0.9 gives 3.7.

Lesson: The maths: speedup, Amdahl and pool sizes

Threads for I/O work

Exam favourite

$$\text{threads} \approx \text{cores} \times (1 + \text{wait time} / \text{compute time})$$

4 cores, 90 ms waiting and 10 ms computing per job: 4 × 10 = 40 threads.

Lesson: The maths: speedup, Amdahl and pool sizes

Little's law

$$L = \lambda \times W$$

200 requests per second that each take 0.25 s: 50 requests in progress, so about 50 workers.

Lesson: The maths: speedup, Amdahl and pool sizes

Lost updates

$$\text{final} = \text{expected} - \text{lost}$$

4 threads × 100 additions should give 400. Every overwrite loses one.

Lesson: The maths: speedup, Amdahl and pool sizes

TABLES

Table: kind of work · example · best tool

Kind of work	Example	Best tool
I/O bound: mostly waiting	Downloading pages, calling APIs, reading many files, database queries	Threads (or asyncio, chapter 5.2)
CPU bound: mostly calculating	Image processing, prime numbers, simulations, parsing huge files	Processes (or the free-threaded build, or NumPy which releases the GIL)
A mix	A web crawler that also parses pages	Threads or asyncio for I/O, a process pool for the heavy parts

Lesson: Doing many things at once

Table: threading · does

threading	Does
Thread(target=f, args=(...), name=...)	Makes a thread that will run f(*args)
t.start()	Starts it. It runs next to the main thread
t.join(timeout=None)	Waits until t has finished
Thread(..., daemon=True)	A background thread that does not keep the program alive
threading.current_thread(), active_count()	Which thread is running this line, how many threads exist

Lesson: Threads: start, join and share

Table: tool · use it to

Tool	Use it to
Lock	Let only one thread at a time run a block (use with lock:)
RLock	A lock the same thread may take again, for recursive code
Semaphore(n)	Let at most n threads in at once, for example 5 connections
Event	Let threads wait until one sets a flag: event.set(), event.wait()
Condition	Wait until some state is true, with wait_for and notify_all
Barrier(n)	Wait until n threads have all arrived
queue.Queue	Pass data between threads safely, with no lock code of your own

Lesson: Race conditions and locks

Table: · threading.thread · queue.queue + workers · threadpoolexecutor

	threading.Thread	queue.Queue + workers	ThreadPoolExecutor
Get results back	By hand (shared dict or list)	A results queue	Futures: result(), map, as_completed
Errors in the thread	Printed and lost	You must catch them	Raised again by result()
Limit how many run at once	By hand	Number of workers	max_workers
Best for	One long background job	Long running pipelines	Most everyday jobs

Lesson: Worker threads with queue and futures

Table: · threads · processes

	Threads	Processes
Memory	Shared: easy to pass data, easy to break it	Separate: data is pickled and copied
Pure Python CPU work on many cores	No, with the GIL (yes in python3.14t)	Yes
Start up cost	Small	Bigger (a new interpreter)
A crash	Can take the whole program down	Stays inside one process
Best for	I/O bound work	CPU bound work

Lesson: The GIL and processes

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1114	Print in Order	Easy	Two Events	second waits for an Event that first sets, and third waits for one that second sets. The threads may start in any order.
1115	Print FooBar Alternately	Medium	Two Semaphores take turns	foo starts with one permit and bar with none. Each thread waits on its own semaphore and then gives a permit to the other one.
1116	Print Zero Even Odd	Medium	Three Semaphores	zero prints 0 and then wakes odd or even depending on the next number. odd and even print and wake zero again.
1117	Building H2O	Medium	Semaphores for places, a Barrier for the group	A Semaphore(2) lets two hydrogen in, a Semaphore(1) one oxygen. A Barrier(3) makes the three wait for each other before they bond.
1195	Fizz Buzz Multithreaded	Medium	One Condition and a shared counter	Every thread waits until the current number is its kind (or the end is reached), prints, moves the counter on and wakes the others with notify_all.
1226	The Dining Philosophers	Medium	Always take the lower numbered fork first	A deadlock needs a cycle of waiting. If every philosopher takes the lower numbered fork first, no cycle can form, so somebody can always eat.
1834	Single-Threaded CPU	Medium	A heap of ready tasks	Sort tasks by arrival. Push every task that has arrived into a heap of (time needed, index), run the shortest, and jump ahead in time when nothing is ready.
1882	Process Tasks Using Servers	Medium	A heap of free servers and a heap of busy ones	Free servers by (weight, index), busy servers by (free time, weight, index). Before each task, move servers that are free again. If none is free, jump to the time the first one is.
2402	Meeting Rooms III	Hard	Free rooms and busy rooms in two heaps	Take meetings by start time. Free every room whose meeting has ended. Use the lowest free room, or delay the meeting until the room that frees first is free.
1094	Car Pooling	Medium	Changes at each point, then a running total	At each pickup the load goes up and at each drop it goes down. Sort the points and add up the changes. Too many people at any point means False.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER Four downloads of 0.5 seconds take 2 seconds one by one but only 0.5 seconds with four threads, because the waiting happens at the same time. While a thread waits for I/O (sleep, network, files) it lets go of the GIL, so others can run.

Lesson: Doing many things at once

COMMON MISTAKE Every thread in a process shares the same global variables and objects. That makes it easy to pass data, and also easy to break data: two threads changing the same object at the same time is a race condition.

Lesson: Threads: start, join and share

COMMON MISTAKE counter = counter + 1 is really three steps: read, add, write. If another thread writes between your read and your write, your write wipes out its update. time.sleep(0) here forces a switch at the worst moment, but real programs hit the same bug by chance, often only under heavy load.

Lesson: Race conditions and locks

IN THE EXAM A **deadlock** happens when thread 1 holds lock A and waits for lock B while thread 2 holds B and waits for A. Neither can go on. Prevent it by always taking locks in the same order, holding them briefly, or using a timeout.

Lesson: Race conditions and locks

REMEMBER queue.Queue does the locking for you. Producers put jobs in, worker threads get them out. get() waits when the queue is empty. Putting one None per worker is a common way to tell workers to stop.

Lesson: Worker threads with queue and futures

IN THE EXAM With the GIL, 4 threads are no faster than one (1.37 s against 1.30 s). 4 processes are 3.3 times faster. The free-threaded build python3.14t (officially supported since 3.14) has no GIL, so there 4 threads are fast too.

Lesson: The GIL and processes

5.2 asyncio: async and await

coroutines, the event loop, gather and TaskGroup, timeouts, limits, queues and httpx.

KEY IDEAS

- **Basics** async def, await (only inside async def), asyncio.run(main()) once. Calling a coroutine function does not run it.
- **Together** gather (results in order), TaskGroup with except*, create_task (keep a reference), as_completed (finish order).
- **Control** asyncio.timeout(s) and wait_for, task.cancel() and CancelledError (raise it again), Semaphore(k) to limit.
- **Pipelines** asyncio.Queue(maxsize) with await put and get for back pressure. Async generators, async for, async with.
- **Never block** No time.sleep or requests in coroutines. Use asyncio.sleep, httpx.AsyncClient, asyncio.to_thread or run_in_executor.

FORMULAS

Time one by one — Exam favourite

$$T = n \times t$$

20 requests of 0.2 s: 4 s.

Lesson: The maths: waits that overlap

Time with all at once — Exam favourite

$$T \approx \max(t_i) + \text{overhead}$$

20 requests of 0.2 s: about 0.2 s (measured 0.26 s).

Lesson: The maths: waits that overlap

With a limit of k at once

Exam favourite

$$T \approx \lceil n / k \rceil \times t$$

20 requests, at most 5 at once, 0.2 s each: 4 rounds, 0.8 s.

Lesson: The maths: waits that overlap

Throughput

$$\text{requests per second} = k / t$$

5 at once, 0.2 s each: 25 requests per second.

Lesson: The maths: waits that overlap

Little's law for open connections

$$L = \lambda \times W$$

1000 requests per second with 0.5 s waits: 500 tasks waiting at any moment.

Lesson: The maths: waits that overlap

Memory for many waiters

$$\text{memory} \approx n \times \text{size of one waiter}$$

10,000 tasks of about 2 KB is about 20 MB. 10,000 threads reserve about 80 GB of address space for stacks.

Lesson: The maths: waits that overlap

TABLES

Table: word · means

Word	Means
async def f()	Defines a coroutine function. Calling it gives a coroutine object
await x	Pause this coroutine until x is done, and let the loop run others. Only allowed inside async def
Task	A coroutine scheduled on the loop. It runs on its own, next to the others
Event loop	The scheduler: runs ready tasks one at a time and wakes waiting ones
asyncio.run(main())	Start a loop, run main until it finishes, close the loop. Call it once, at the top

Lesson: asyncio: many waits in one thread

Table: tool · use it to

Tool	Use it to
asyncio.create_task(coro)	Start a coroutine in the background and get a Task you can await later
await asyncio.gather(a, b, ...)	Run several and get their results as a list, in the order given
async with asyncio.TaskGroup() as g	Start tasks with g.create_task. If one fails, the others are cancelled and errors come out as an ExceptionGroup
asyncio.as_completed(aws)	Get results in the order they finish
asyncio.wait(aws, return_when=...)	Wait for the first one, the first error, or all, and get done and pending sets

Lesson: Tasks, TaskGroup and as_completed

Table: · gather · taskgroup (3.11+)

	gather	TaskGroup (3.11+)
Results	A list in order	task.result() for each task
When one fails	The error is raised, the others keep running (unless you cancel them)	The others are cancelled and all errors arrive in an ExceptionGroup
Catching errors	try / except	try / except*
Best for	A fixed list of jobs, return_exceptions=True to collect errors	Structured code where no task may be left running

Lesson: Tasks, TaskGroup and as_completed

Table: · threads · asyncio

	Threads	asyncio
Switches between jobs	Any time (the OS decides)	Only at await
Shared data	Needs locks	Safe between awaits
Cost of one job	A thread: about 8 MB of address space reserved for its stack	A task: a few KB
Thousands of connections	Heavy	Easy
Libraries	Any normal library	Needs async libraries (httpx, aiohttp, asyncpg)
CPU heavy work	Not with the GIL	Never: it blocks the loop

Lesson: Real network code, and blocking calls

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2621	Sleep	Easy	await asyncio.sleep	LeetCode asks this one in JavaScript. In Python asyncio the same idea is a coroutine that awaits asyncio.sleep(millis / 1000).
2723	Add Two Promises	Easy	Await both at the same time	A JavaScript problem. In Python, gather awaits both coroutines at the same time and returns their values, so the total wait is the longer one, not the sum.
2648	Generate Fibonacci Sequence	Easy	An endless generator	A JavaScript generator problem. In Python, a while True loop with yield gives the numbers one at a time, and itertools.islice takes as many as you need.
2649	Nested Array Generator	Medium	yield from for the nested lists	A JavaScript generator problem. Walk the list, and for a nested list hand over to a recursive generator with yield from.
2637	Promise Time Limit	Medium	asyncio.wait_for	A JavaScript problem. Wrap the coroutine in asyncio.wait_for with the limit in seconds, and turn the TimeoutError into the message the problem asks for.
2721	Execute Asynchronous Functions in Parallel	Medium	gather keeps the order	A JavaScript problem (write Promise.all without Promise.all). In Python, gather starts all of them, keeps the results in the order given and raises the first error.
2622	Cache With Time Limit	Medium	Store the value with its end time	A JavaScript design problem. Keep key: (value, expires_at). A key counts only while the clock is before its end time. The clock is passed in so the tests do not have to wait.
2627	Debounce	Medium	Cancel the waiting call and schedule a new one	A JavaScript problem. With asyncio, loop.call_later schedules the call. A new call cancels the handle of the one still waiting.
2636	Promise Pool	Medium	n workers share one iterator	A JavaScript problem. Start n worker coroutines that all take the next function from the same iterator. At most n functions run at any moment.
2650	Design Cancellable Function	Hard	Drive a generator by hand, race each wait against cancel	A JavaScript problem, and exactly how await works inside: send each result into the generator, throw errors into it, and throw a Cancelled error if cancel() wins the race.

Lesson: LeetCode practice

TIPS AND TRAPS

COMMON MISTAKE Calling make_tea(...) only creates a coroutine object, it runs nothing. The code runs when it is awaited, or when it is wrapped in a task (gather, create_task, TaskGroup). Forgetting await is the most common asyncio bug: Python warns "coroutine ... was never awaited".

Lesson: asyncio: many waits in one thread

COMMON MISTAKE Keep a reference to every task you create with create_task (in a list or a set), or use a TaskGroup. The loop holds tasks only weakly, so a task nobody refers to can disappear before it finishes.

Lesson: Tasks, TaskGroup and as_completed

IN THE EXAM Cancelling a task raises CancelledError inside it at its current await. Clean up in finally blocks, and do not swallow CancelledError without raising it again, or the task cannot be stopped.

Lesson: Timeouts, cancelling and limits

REMEMBER A Semaphore limits how many tasks do something at once. Without it, gathering 10,000 downloads opens 10,000 connections at the same moment, which servers often punish by refusing or blocking you.

Lesson: Timeouts, cancelling and limits

REMEMBER asyncio.Queue is like queue.Queue, but put and get are awaited. With maxsize=2 the producer waits when two items are waiting. This **back pressure** stops a fast producer from filling memory while a slow consumer catches up.

Lesson: Pipelines with asyncio.Queue

REMEMBER 20 × 0.2 s = 4 s one by one. With asyncio all 20 waits overlap, so it takes about one wait: 0.26 s, 16 times faster, in one thread.

Lesson: Real network code, and blocking calls

COMMON MISTAKE A normal blocking call (time.sleep, requests.get, a slow calculation) inside a coroutine freezes the whole loop. Use the async version (asyncio.sleep, httpx.AsyncClient, aiofiles), or push the blocking call to a thread with asyncio.to_thread, or to a process pool with loop.run_in_executor.

Lesson: Real network code, and blocking calls

STEP 6

Advanced Python · The standard library toolbox: text, data, databases and GUIs

6.1 Regular expressions

regex syntax, the re functions, groups, sub and split, flags, greedy and lazy, lookarounds.

KEY IDEAS

- **Syntax** . \d \w \s [abc] [^abc] ^ $ \b, repeats * + ? {m,n}, lazy *? +?, | and ( ). Always raw strings.
- **Functions** search (anywhere), match (start), fullmatch (whole), findall (strings), finditer (match objects), sub, subn, split.
- **Groups** m.group(n), m.groups(), (?P<name>...), m.groupdict(), (?:...) non-capturing, \1 backreference, \g<name> in sub.
- **Flags** IGNORECASE, MULTILINE, DOTALL, VERBOSE. re.compile for named patterns, re.escape for literal text.
- **Care** Greedy versus lazy, lookarounds (?=) (?!) (?<=) (?

FORMULAS

Choices for a character class

Exam favourite

patterns of length n over a class of size k match k^n strings

\d{4} matches 10^4 = 10,000 strings, 0000 to 9999.

Lesson: The maths: languages and automata

A range of repeats

[ab]{1,3} matches 2 + 4 + 8 = 14 strings

$\sum k^i$ for i from min to max.

Lesson: The maths: languages and automata

Positions tried by search

up to n + 1 starting places

Text of length 17: starts 0 to 17.

Lesson: The maths: languages and automata

A regular language

Exam favourite

built from single characters with union (|), concatenation and star (*)

Every such pattern can be matched by a finite automaton with no memory.

Lesson: The maths: languages and automata

Catastrophic backtracking

Exam favourite

ways to split n letters among nested repeats grows like 2^n

(a+)+$ on 30 a's followed by "!": about $2^{30} \approx 10^9$ tries.

Lesson: The maths: languages and automata

Linear time engines

time = O(m × n) for pattern size m and text size n

RE2 and Rust's regex guarantee this, Python's re does not.

Lesson: The maths: languages and automata

TABLES

Table: pattern · matches · pattern · matches

Pattern	Matches	Pattern	Matches
.	Any character except a newline	\d, \D	A digit, not a digit
\w, \W	A word character (letter, digit, _), not one	\s, \S	Whitespace, not whitespace
[abc], [a-z]	One of these characters, a range	[^0-9]	Any character NOT in the set
^, $	Start, end of the text (or line with MULTILINE)	\b	A word edge
*, +, ?	0 or more, 1 or more, 0 or 1	{3}, {2,5}	Exactly 3, from 2 to 5
a|b	a or b	(...)	A group, also captured
\. \$ \(	A real dot, dollar, bracket	*?, +?	The lazy versions

Lesson: Regular expressions: patterns for text

Table: · str methods (in, find, split, replace) · re module

	str methods (in, find, split, replace)	re module
Fixed text	Best: simple and fast	Works, but more than needed
Patterns (any digits, optional parts, either or)	Many lines of code	One pattern
Readability	Very clear	Can be cryptic: use re.VERBOSE and comments
Nested structures (HTML, JSON, brackets)	No	No: use a real parser

Lesson: Regular expressions: patterns for text

Table: function · looks · returns

Function	Looks	Returns
re.search(p, s)	Anywhere, first match	A match object or None
re.match(p, s)	Only at the start	A match object or None
re.fullmatch(p, s)	The whole string	A match object or None
re.findall(p, s)	Everywhere	A list of strings (or tuples of groups)
re.finditer(p, s)	Everywhere	An iterator of match objects
m.group(), m.start(), m.end(), m.span()	On a match object	The text and where it is

Lesson: search, match, fullmatch, findall, finditer

Table: syntax · meaning

Syntax	Meaning
(...)	Capture this part as group 1, 2, ...
(?P<name>...)	A named group, read with m["name"] or m.group("name")
(?:...)	Group without capturing, for | or repeats
\1, (?P=name)	The same text that group 1 or name matched
m.groups(), m.groupdict()	All groups as a tuple, named groups as a dict

Lesson: Groups: pulling parts out

Table: flag · effect

Flag	Effect
re.IGNORECASE (re.I)	Upper and lower case match each other
re.MULTILINE (re.M)	^ and $ match at the start and end of every line
re.DOTALL (re.S)	. matches newlines too
re.VERBOSE (re.X)	Spaces and # comments in the pattern are ignored
re.ASCII (re.A)	\w, \d and \s match only ASCII characters

Lesson: sub, split, flags and compile

Table: syntax · meaning

Syntax	Meaning
.*, .+	Greedy: take as much as possible, then back off
.*?, .+?	Lazy: take as little as possible, then take more
(?=...), (?!...)	Lookahead: what follows must (not) match. Uses up no characters
(?<=...), (?<!...)	Lookbehind: what comes before must (not) match
*+, ++, (?>...)	Possessive and atomic (3.11+): never give characters back

Lesson: Greedy, lazy, anchors and lookarounds

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1805	Number of Different Integers in a String	Easy	findall the digit runs	r"\d+" finds every run of digits. int() removes leading zeros, and a set keeps the different values.
2114	Maximum Number of Words Found in Sentences	Easy	Count words with split	split() with no argument splits on any run of whitespace, which is the same as re.split(r"\s+", s.strip()).
2788	Split Strings by Separator	Easy	re.split with an escaped separator	The separator may be a character that is special in regex, such as "." or "|", so pass it through re.escape. Drop empty pieces.
1910	Remove All Occurrences of a Substring	Medium	A stack of characters	Removing one copy can create a new one, so a single re.sub is not enough. Build the result character by character and cut the part off whenever it ends with it.
722	Remove Comments	Medium	Join the lines, then one regex	Join the source with newlines. r"//.*|/*(.|\n)*?*/" matches a line comment or a block comment (lazy, across lines). Remove them, split again and drop empty lines.
1023	Camelcase Matching	Medium	Build a regex from the pattern	Between and around the pattern letters, only lower case letters may be added: "FB" becomes ^[a-z]*F[a-z]*B[a-z]*$.
1451	Rearrange Words in a Sentence	Medium	A stable sort by length	Lower case the words, sort by length (sorted is stable, so equal lengths keep their order), then capitalise the first letter of the result.
10	Regular Expression Matching	Hard	Dynamic programming over the two strings	match(i, j) asks whether s[i:] matches p[j:]. With a * after p[j], either skip "x*" or use it once if the first characters match. Memoise with lru_cache.
44	Wildcard Matching	Hard	The same dynamic programming with ? and *	? matches one character, * any run (also empty). A * can either match nothing (move in the pattern) or one more character (move in the text).
591	Tag Validator	Hard	Regex for the pieces, a stack for the nesting	Regex cannot check nesting, so use it only for the pieces: remove CDATA blocks, then find tags with a regex and match open and close tags with a stack.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM Use fullmatch to check input such as a postcode or a username. search would also accept "xx12345yy" because it finds 12345 inside it.

Lesson: search, match, fullmatch, findall, finditer

COMMON MISTAKE findall changes what it returns when the pattern has groups: with one group it returns that group's text, with several it returns tuples. Use (?:...) for groups you only need for | or repeats.

Lesson: Groups: pulling parts out

REMEMBER re.compile(pattern) gives a pattern object with the same methods. re caches recent patterns, so compiling is mostly about clarity: give a long pattern a name and write it once.

Lesson: sub, split, flags and compile

COMMON MISTAKE Some patterns make the engine try a huge number of ways before failing, for example (a+)+$ on "aaaaaaaaaaaaaaaaaaaaaaaaaaa!". This is **catastrophic backtracking**, and attackers use it to freeze servers (ReDoS). Avoid nested repeats of the same thing, or use possessive quantifiers.

Lesson: Greedy, lazy, anchors and lookarounds

REMEMBER

A named, verbose pattern reads like documentation. fullmatch makes sure the line has exactly the expected shape, and broken lines are skipped instead of giving wrong data.

Lesson: Parsing real text

6.2 CSV, JSON, XML and settings files

CSV, JSON with custom types, XML with ElementTree, INI and TOML settings, pickle safety.

KEY IDEAS

- **CSV** csv.DictReader, DictWriter(f, fieldnames), writeheader, open(..., newline=""), delimiter, quoting. Values are strings.
- **JSON** dumps and loads, dump and load, indent, sort_keys, ensure_ascii, default=, object_hook, JSONDecodeError. Tuples become lists, keys strings.
- **XML** ET.fromstring, ET.parse, find, findall, findtext, iter, .//tag[@attr="v"], SubElement, indent, tostring. defusedxml for untrusted XML.
- **Settings** configparser: sections, getint, getboolean, fallback=, %(name)s. tomllib.load(binary file) and loads, read only.
- **pickle** dumps and loads any object, Python only, NEVER load untrusted data (__reduce__ runs code).

FORMULAS

CSV file size Exam favourite

$$\text{bytes} \approx \text{rows} \times (\text{sum of value lengths} + \text{separators} + 2 \text{ for } \backslash r\backslash n)$$

1000 rows of 3 values of 6 characters: 1000 × (18 + 2 + 2) = 22,000 bytes.

Lesson: The maths: sizes and overheads

JSON overhead for a list of dicts

every row repeats every key with quotes, a colon and a comma

Keys "name", "city", "age" add about 25 bytes to each row, often more than the data itself.

Lesson: The maths: sizes and overheads

Base64 for binary data in JSON or XML Exam favourite

$$\text{size out} = 4 \times \lceil n / 3 \rceil$$

A 300 KB image becomes 400 KB of text.

Lesson: The maths: sizes and overheads

Unicode escapes

ensure_ascii=True writes a non ASCII letter as 6 characters

"Zürich" grows from 7 bytes (UTF-8) to 11 characters with \u00fc.

Lesson: The maths: sizes and overheads

Sniffing a separator

pick the character that appears the same number of times on every line

`"id;price"` and `"1;2,50"`: the `;` appears once on each line, the comma does not.

Lesson: The maths: sizes and overheads

TABLES

Table: · csv · json · xml

	CSV	JSON	XML	TOML / INI	pickle
Shape	Flat table	Nested lists and dicts	Nested tags with attributes	Settings in sections	Any Python object
Types kept	None, all strings	str, numbers, bool, null, list, dict	All strings	TOML: numbers, bools, dates, lists	All of them
Read by people	Yes, and spreadsheets	Yes	Yes, wordy	Yes, made for it	No (bytes)
Other languages	Every one	Every one	Every one	Many	Python only
Safe with untrusted data	Yes	Yes	With care (defusedxml)	Yes	NO: loading can run code
Typical use	Exports, reports, data science	Web APIs, settings, logs	Documents, older APIs, office files	pyproject.toml, app config	Caches inside one trusted program

Lesson: Data formats: saving and sharing data

Table: csv tool · does

csv tool	Does
csv.reader(f), csv.writer(f)	Rows as lists
csv.DictReader(f), csv.DictWriter(f, fieldnames)	Rows as dicts keyed by the header
`delimiter=";", quotechar='"'`	Other separators and quote characters
quoting=csv.QUOTE_ALL, QUOTE_MINIMAL, QUOTE_NONNUMERIC	When values get quotes
csv.Sniffer().sniff(sample)	Guess the separator of an unknown file

Lesson: CSV: tables as text

Table: python · json · back to python

Python	JSON	Back to Python
dict	object {...}	dict (keys always str)
list, tuple	array [...]	list
str	string "..."	str
int, float	number	int or float
True, False	true, false	True, False
None	null	None
set, date, Decimal, your classes	TypeError unless default= handles them	

Lesson: JSON: nested data for the web

Table: elementtree · does

ElementTree	Does
ET.parse(path), ET.fromstring(text)	Read a file or a string into a tree
el.tag, el.attrib, el.get(name), el.text	The tag name, attributes and text of an element
el.find(path), el.findall(path), el.findtext(path)	Children by a simple path, such as "book/title"
el.iter(tag)	Every element with that tag, at any depth
.//book[@lang="de"]	The XPath subset: any depth, attribute tests, positions
ET.SubElement, el.set, ET.indent, ET.tostring, tree.write	Build, change and write XML

Lesson: XML with ElementTree

Table: · ini (configparser) · toml (tomllib) · json

	INI (configparser)	TOML (tomllib)	JSON	Environment variables
Types	Strings, with getint, getboolean	Real numbers, bools, dates, lists, tables	Numbers, bools, lists, dicts	Strings
Comments	Yes	Yes	No	No
Write with the standard library	Yes	No, read only (tomli-w writes)	Yes	os.environ
Typical use	Older tools, setup.cfg	pyproject.toml, new apps	Web app settings	Secrets and per server values (12 factor apps)

Lesson: Settings: INI and TOML

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
171	Excel Sheet Column Number	Easy	Base 26 with letters	Spreadsheet columns count A to Z, then AA. Each letter is a digit from 1 to 26: result = result × 26 + value.
168	Excel Sheet Column Title	Easy	Base 26 the other way, with a shift	There is no zero digit, so subtract 1 before each divmod by 26.
1507	Reformat Date	Easy	Split and look up the month	Split "20th Oct 2052" into three parts. Strip the two letter ending from the day, look the month up in a list, and pad with zfill(2).
2194	Cells in a Range on an Excel Sheet	Easy	Two nested loops over letters and rows	The range "K1:L2" gives column letters K to L and rows 1 to 2. Columns go in the outer loop.
1410	HTML Entity Parser	Medium	Replace the entities, the ampersand one last	Replace each named entity with its character. Do the ampersand entity last, or the text you just made could be read as a new entity.
385	Mini Parser	Medium	A stack of lists while reading characters	LeetCode uses a NestedInteger class. Here plain ints and lists stand for it. Push a new list at [, finish a number at , or], and pop at].
831	Masking Personal Information	Medium	Two formats, two rules	An email keeps the first and last letter of the name and lower cases everything. A phone keeps only digits, shows the last 4, and stars for a country code.
640	Solve the Equation	Medium	A regex for the terms on each side	The pattern [+-]?\d*x|[+-]?\d+ finds terms like +2x, -x or 5. Add up x coefficients and numbers on each side, then move everything to one side.
726	Number of Atoms	Hard	A stack of Counters	Read an atom name (capital letter and lower case letters) and its count. At (push a new Counter, at) multiply the top one by the number after it and add it to the one below.
1096	Brace Expansion II	Hard	A recursive parser for union and product	A comma means union of sets, writing things next to each other means every combination (product). Parse with a function that reads until } and calls itself at {.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM Always open CSV files with newline="". The csv module handles line endings itself (rows end with \r\n by default), and quotes inside values. Without newline="" you can get blank lines between rows on Windows.

Lesson: CSV: tables as text

REMEMBER dump and load work with files, dumps and loads (s for string) with strings. indent=2 makes it readable, sort_keys=True makes the output stable for tests and diffs, and ensure_ascii=False keeps letters like ü instead of \u00fc.

Lesson: JSON: nested data for the web

COMMON MISTAKE Never parse XML from strangers with the standard library alone: specially made files can use huge amounts of memory (the "billion laughs" attack). Use the defusedxml package for untrusted XML.

Lesson: XML with ElementTree

COMMON MISTAKE pickle can save almost any Python object, but pickle.loads can run any code the creator of the data chose, as the Evil class shows with a harmless print. Never unpickle data from the internet, uploads or anyone you do not fully trust. Use JSON for data you exchange.

Lesson: pickle: Python objects as bytes

6.3 Databases with sqlite3 and SQL

sqlite3, SQL queries and joins, placeholders against injection, transactions, constraints and indexes.

KEY IDEAS

- **Basics** sqlite3.connect(path or ":memory:"), execute, executemany, executescript, commit, close, fetchone, fetchall, lastrowid, rowcount.
- **SQL** CREATE TABLE, INSERT, SELECT, UPDATE and DELETE with WHERE, ORDER BY, GROUP BY, HAVING, JOIN, LEFT JOIN, COUNT, SUM, COALESCE, LIMIT.
- **Safety** Values only through ? or :name. Never f-strings in SQL. Check table and column names against a fixed list.
- **Transactions** "with conn:" commits or rolls back (does not close). IntegrityError from PRIMARY KEY, UNIQUE, NOT NULL, CHECK, foreign keys (PRAGMA foreign_keys = ON).
- **Speed** sqlite3.Row, CREATE INDEX, EXPLAIN QUERY PLAN (SCAN vs SEARCH), one commit for many inserts, B-tree height log base f of N.

FORMULAS

Full scan

$$\text{cost} \approx N \text{ rows read}$$

Finding one user among 1,000,000 without an index reads 1,000,000 rows.

Lesson: The maths: indexes, joins and estimates

B-tree height Exam favourite

$$h = \lceil \log_f N \rceil, \; f = \text{keys per page (fan out)}$$

N = 1,000,000 and f = 100: h = 3 page reads instead of a million rows.

Lesson: The maths: indexes, joins and estimates

Binary search in a sorted index

$$\text{comparisons} \approx \lceil \log_2 N \rceil$$

$\log_2$ 1,000,000 ≈ 20.

Lesson: The maths: indexes, joins and estimates

Selectivity of WHERE a = v Exam favourite

$$\text{rows} \approx N / V(a), \; V(a) = \text{number of different values}$$

1,000,000 rows and 100 cities: about 10,000 rows per city.

Lesson: The maths: indexes, joins and estimates

Equi join size estimate

$$|R \bowtie S| \approx |R| \times |S| / \max(V(R, a), V(S, a))$$

1000 customers, 5000 orders, joined on 1000 customer ids: 1000 × 5000 / 1000 = 5000 rows.

Lesson: The maths: indexes, joins and estimates

Cost of commits

$$\text{time} \approx \text{commits} \times \text{time per disk sync}$$

1000 inserts with 1000 commits at 5 ms each: 5 s. One commit: about 5 ms.

Lesson: The maths: indexes, joins and estimates

Table: · json or csv file · sqlite (sqlite3) · postgresql or mysql

	JSON or CSV file	SQLite (sqlite3)	PostgreSQL or MySQL	pandas DataFrame
Setup	None	None, one file	A server to install and run	pip install pandas
Find one record in a million	Read everything	Milliseconds with an index	Milliseconds with an index	Fast, but all data in memory
Safe when a program crashes	No	Yes, transactions	Yes, transactions	No, memory only
Many writers at once	No	One writer at a time	Yes, many	No
Rules on the data	None	Types, NOT NULL, UNIQUE, CHECK, foreign keys	The same and more	Only in your code
Typical use	Small settings and exports	Apps, phones, browsers, tests, prototypes	Web sites with many users	Analysis and reports

Lesson: Databases and SQL from Python

Table: step · code · meaning

Step	Code	Meaning
Open	conn = sqlite3.connect("shop.db")	Opens the file, or makes it. ":memory:" gives a database in memory only.
Cursor	cur = conn.cursor()	An object that runs SQL and walks through results. conn.execute() makes one for you.
Run SQL	cur.execute(sql, values)	Runs one statement. Values go in a tuple, matching the ? marks.
Save	conn.commit()	Makes INSERT, UPDATE and DELETE permanent. Without it they are lost on close.
Read	fetchone(), fetchall(), fetchmany(n), for row in cur	Rows come back as tuples.
Close	conn.close()	Frees the file. Uncommitted changes are thrown away.

Lesson: Connect, execute, commit, fetch

Table: sql · does · runs in step

SQL	Does	Runs in step
FROM a JOIN b ON ...	Takes rows from tables and pairs them up	1
WHERE condition	Keeps rows where the condition is true	2
GROUP BY column	Puts rows with the same value in one group	3
HAVING condition	Keeps groups (it can use COUNT, SUM, AVG)	4
SELECT columns	Picks and names the output columns	5
ORDER BY column DESC	Sorts the result	6
LIMIT n OFFSET k	Keeps n rows after skipping k	7

Lesson: SQL in one lesson: SELECT, WHERE, JOIN, GROUP BY

Table: change data · example

Change data	Example
Make a table	CREATE TABLE t (id INTEGER PRIMARY KEY, name TEXT NOT NULL UNIQUE)
Add rows	INSERT INTO t (name) VALUES (?)
Change rows	UPDATE t SET name = ? WHERE id = ?
Remove rows	DELETE FROM t WHERE id = ?
Remove the table	DROP TABLE IF EXISTS t

Lesson: SQL in one lesson: SELECT, WHERE, JOIN, GROUP BY

Table: · inner join (join) · left join · cross join

	INNER JOIN (JOIN)	LEFT JOIN	CROSS JOIN
Keeps	Only pairs that match	Every left row, with NULL when nothing matches	Every left row with every right row
Rows for 4 customers, 3 with orders	One per matching order	Matching orders plus 1 row for the customer without orders	4 × number of orders
Use it for	Orders with their customer	Customers and their orders, even with none	All combinations (rarely)

Lesson: SQL in one lesson: SELECT, WHERE, JOIN, GROUP BY

Table: style · sql · values

Style	SQL	Values
qmark	WHERE name = ? AND age > ?	a tuple: ("ann", 30)
named	WHERE name = :name AND age > :age	a dict: {"name": "ann", "age": 30}
Many rows	executemany("INSERT INTO t VALUES (?, ?)", rows)	a list of tuples
A script	`executescript("CREATE ...; INSERT ...")`	no values: only for your own fixed SQL

Lesson: Placeholders and SQL injection

Table: letter · means · in the example

Letter	Means	In the example
A, atomic	All the changes happen, or none	The failed transfer did not give Ann 500
C, consistent	Rules such as CHECK and UNIQUE always hold	No balance is ever below 0
I, isolated	Others do not see half done changes	Another connection sees the old balances until commit
D, durable	After commit the data survives a crash or power cut	SQLite writes a journal before it changes the file

Lesson: Transactions and rules on the data

Table: rule in create table · stops · error

Rule in CREATE TABLE	Stops	Error
PRIMARY KEY, UNIQUE	Two rows with the same value	IntegrityError: UNIQUE constraint failed
NOT NULL	A missing value	IntegrityError: NOT NULL constraint failed
CHECK (balance >= 0)	Values that break your rule	IntegrityError: CHECK constraint failed
REFERENCES other(col)	A link to a row that does not exist	IntegrityError: FOREIGN KEY constraint failed (after PRAGMA foreign_keys = ON)

Lesson: Transactions and rules on the data

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
175	Combine Two Tables	Easy	LEFT JOIN keeps people without an address	Every person must appear, even without an address, so use LEFT JOIN. Missing columns come back as None.
182	Duplicate Emails	Easy	GROUP BY with HAVING COUNT	Group the rows by email and keep the groups that have more than one row.
183	Customers Who Never Order	Easy	NOT IN a subquery, or LEFT JOIN ... IS NULL	Find customers whose id is not in the list of customer ids of the orders.
197	Rising Temperature	Easy	Join the table with itself on yesterday	Pair each day with the day before using DATE(day, '-1 day'), then keep pairs where the temperature went up.
196	Delete Duplicate Emails	Easy	DELETE rows that are not the smallest id	Keep MIN(id) of each email and delete every other row.
176	Second Highest Salary	Medium	DISTINCT, ORDER BY, LIMIT 1 OFFSET 1 inside a subquery	The outer SELECT turns "no row" into NULL, which is None in Python.
178	Rank Scores	Medium	The DENSE_RANK window function	DENSE_RANK() OVER (ORDER BY score DESC) gives equal scores the same rank and leaves no gaps after ties.
184	Department Highest Salary	Medium	Join with the maximum of each department	A subquery finds MAX(salary) per department. Join it back to keep every employee who earns that much, ties included.
185	Department Top Three Salaries	Hard	DENSE_RANK per department with PARTITION BY	PARTITION BY departmentId restarts the ranking in each department. Keep rank 3 or better. Ties share a rank.
262	Trips and Users	Hard	Join twice to the users table, then average a condition	Join Users once for the client and once for the driver and keep trips where neither is banned. AVG(status != 'completed') is the share of cancelled trips, because a true test counts as 1.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM — The most common beginner bug: INSERT works, the program ends, and the data is gone. You forgot conn.commit(). Using "with conn:" commits for you.

Lesson: Connect, execute, commit, fetch

REMEMBER — SQL types in SQLite are flexible: INTEGER, REAL, TEXT, BLOB and NULL. Python int, float, str, bytes and None map to them. Store dates as ISO text such as "2026-11-07", because the automatic date converters are deprecated since Python 3.12.

Lesson: Connect, execute, commit, fetch

COMMON MISTAKE — An UPDATE or DELETE without WHERE changes EVERY row. Write the WHERE first, and try it as a SELECT before you run it.

Lesson: SQL in one lesson: SELECT, WHERE, JOIN, GROUP BY

COMMON MISTAKE — Never put values into SQL with f-strings, +, % or .format(). Use ? or :name every time. Placeholders stop SQL injection, handle quotes in names like O'Neil, and let the database reuse the prepared statement.

Lesson: Placeholders and SQL injection

IN THE EXAM — Placeholders are for VALUES only. A table or column name cannot be a ?. If a user picks the column to sort by, check it against a fixed list such as {"name", "price"} before you put it into the SQL.

Lesson: Placeholders and SQL injection

REMEMBER "with conn:" commits or rolls back, but it does NOT close the connection. Since Python 3.12 you can also pass autocommit=False to sqlite3.connect for the transaction rules of the database standard (PEP 249).

Lesson: Transactions and rules on the data

REMEMBER An index is a sorted copy of one column with a pointer to each row, kept in a B-tree. It turns a scan of every row into a few page reads. It costs disk space and makes INSERT and UPDATE a little slower, so index the columns you search, join or sort by often.

Lesson: Friendlier rows and fast lookups with indexes

6.4 Desktop windows with tkinter

tkinter widgets, layouts, variables, events, Canvas animation with after() and responsive apps.

KEY IDEAS

- **First window** root = tk.Tk(), title, geometry("WxH"), Label, Button(command=f), pack, root.mainloop(). command=f without brackets.
- **Layout** grid(row, column, sticky="nsew", columnspan, padx, pady), columnconfigure(i, weight=1), pack(side, fill, expand), place(x, y). One manager per parent.
- **Input and variables** Entry.get, insert, delete. StringVar, IntVar, BooleanVar, textvariable, variable, trace_add("write", f). messagebox, filedialog.
- **Events and Canvas** bind("", f) with event.x, event.y, event.keysym. create_line, create_oval, move, coords. after(ms, f) for animation.
- **Responsive apps** Callbacks return fast. after() for timers, threads plus queue.Queue for slow work, widgets only from the main thread. Model apart from the view.

FORMULAS

Screen coordinates

x grows to the right, y grows DOWN, (0, 0) is the top left corner

A point 50 pixels from the top has y = 50, not y = height − 50.

Lesson: The maths: pixels, frames and how fast people click

Centre a window Exam favourite

x = (screen width − window width) // 2, y = (screen height − window height) // 2

A 300 × 200 window on 1920 × 1080: x = 810, y = 440, geometry "300x200+810+440".

Lesson: The maths: pixels, frames and how fast people click

Frames and speed

delay = 1000 / fps ms, speed = pixels per step × 1000 / delay

after(20, ...) gives 50 frames a second. 4 pixels a step is 200 pixels a second.

Lesson: The maths: pixels, frames and how fast people click

Sharing extra space

extra for column i = extra × weight_i / sum of weights

300 extra pixels, weights 1 and 2: 100 and 200 pixels.

Lesson: The maths: pixels, frames and how fast people click

Fitts's law Exam favourite

$$\text{time} = a + b \times \log_2(D / W + 1)$$

D = distance to the target, W = its width. Big, close buttons are faster to hit.

Lesson: The maths: pixels, frames and how fast people click

Hick's law

$$\text{time} = b \times \log_2(n + 1)$$

n equal choices. 7 menu items: $\log_2 8 = 3$ times b.

Lesson: The maths: pixels, frames and how fast people click

TABLES

Table: · tkinter · pyside6 or pyqt6 · kivy

	tkinter	PySide6 or PyQt6	Kivy	Web app (Flask, Django)	Textual
Install	Comes with Python	pip, about 100 MB	pip	pip, plus a browser	pip
Look	Simple, native with ttk	Modern, very rich	Its own touch style	Anything HTML and CSS can do	Text in the terminal
Runs on	Desktop	Desktop	Desktop, Android, iOS	Any device with a browser	Any terminal, also over SSH
Licence	Python licence	LGPL (PySide6) or GPL / paid (PyQt6)	MIT	Your choice	MIT
Best for	Small tools, teaching, the PCPP exam	Large desktop applications	Touch and mobile apps	Apps for many users	Developer tools

Lesson: Windows, buttons and the event loop

Table: step · code · what it does

Step	Code	What it does
1	root = tk.Tk()	Make the main window
2	w = tk.Label(root, text=...)	Make a widget inside a parent
3	w.pack() or w.grid(...)	Place it, or it stays invisible
4	command=function, w.bind(event, function)	Connect events to your functions
5	w.config(text=...) or w["text"] = ...	Change a widget later
6	root.mainloop()	Start the event loop

Lesson: Your first window

Table: · pack · grid · place

	pack	grid	place
Idea	Stack widgets from one side (top, left ...)	A table of rows and columns	Exact x, y position or relative position
Good for	Toolbars, simple columns of widgets	Forms, dialogs, almost every real layout	Overlays, games, special cases
Grows with the window	fill and expand	sticky and weight	Only with relx, rely, relwidth
Main options	side, fill, expand, padx, pady, anchor	row, column, sticky, columnspan, rowspan, padx, pady	x, y, relx, rely, anchor, width, height

Lesson: Placing widgets: pack, grid and place

Table: widget · use it for · read or set

Widget	Use it for	Read or set
ttk.Label, tk.Message	Text or an image	config(text=...) or textvariable
ttk.Button	An action	command=
ttk.Entry	One line of text	get(), insert(0, s), delete(0, "end")
tk.Text	Many lines of text	get("1.0", "end"), insert("end", s)
ttk.Checkbutton, ttk.Radiobutton	Yes or no, one of several	variable= BooleanVar or StringVar
ttk.Combobox, tk.Listbox	Pick from a list	get(), curselection()
ttk.Scale, ttk.Spinbox	A number in a range	variable= or get()
tk.Canvas	Drawing and animation	create_line, create_oval, move, coords
ttk.Frame, ttk.LabelFrame	Group other widgets	a parent for others
tk.Menu, filedialog, messagebox	Menus and standard dialogs	askopenfilename(), showinfo()

Lesson: Widgets and control variables

Table: event string · happens when · useful event fields

Event string	Happens when	Useful event fields
<Button-1>, <Button-3>	Left or right mouse button pressed	event.x, event.y
<Double-Button-1>	Double click	event.x, event.y
<B1-Motion>, <Motion>	Mouse moved with or without the button down	event.x, event.y
<Key>, <Return>, <Escape>, a	Any key, Enter, Esc, the a key	event.keysym, event.char
<Control-s>	Ctrl and S together	event.state
<Configure>	The widget changed size	event.width, event.height
<FocusIn>, <FocusOut>	The widget got or lost the keyboard	event.widget

Lesson: Events, the Canvas and animation with after()

Table: · everything in the callbacks · model and view apart

	Everything in the callbacks	Model and view apart
Testing	Only by clicking by hand	Normal unit tests with pytest
Changing the look	Touches the rules too	Only the view changes
Moving to another toolkit or the web	Rewrite everything	Keep the model, write a new view

Lesson: A real app: classes, scrollbars and saving

Table: long task · what to use

Long task	What to use
Waiting or a timer	root.after(ms, function)
Many small steps (a counter, an animation)	Do one step, then after(ms, next_step)
Slow input and output (download, database)	A thread that puts results in a queue.Queue, checked with after()
Heavy CPU work	A process (concurrent.futures.ProcessPoolExecutor), results checked with after()
Never	time.sleep() or a long loop inside a callback

Lesson: Long tasks without freezing the window

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
657	Robot Return to Origin	Easy	Count the moves in each direction	Like moving an item on a canvas: R adds to x, L takes away. Back at the start means x and y are both 0.
733	Flood Fill	Easy	The paint bucket tool: breadth first search	Start at the clicked pixel and paint every neighbour (up, down, left, right) that has the old colour. Stop at once if the new colour equals the old one.
836	Rectangle Overlap	Easy	Overlap on both axes	Two boxes overlap when the left edge of each is before the right edge of the other, on x AND on y. Touching edges do not count. Games use this for collisions.
1232	Check If It Is a Straight Line	Easy	Compare slopes with a cross product	Points are on one line when (y - y0) × (x1 - x0) equals (y1 - y0) × (x - x0) for every point. Multiplying avoids dividing by zero for vertical lines.
223	Rectangle Area	Medium	Both areas minus the overlap	Add the two areas and take away the overlap. The overlap width is max(0, min of right edges minus max of left edges), the same for the height.
593	Valid Square	Medium	Six distances between four points	A square has 4 equal sides and 2 equal, longer diagonals. Use squared distances to stay with whole numbers, and reject a length of 0.
874	Walking Robot Simulation	Medium	Directions as a list, obstacles as a set	Keep the four directions in a list and turn by moving the index. Store obstacles as a set of tuples so each step is checked in O(1).
1041	Robot Bounded In Circle	Medium	Run the instructions once and look at the result	After one run the robot is either back at the start or facing a new direction. In both cases it stays in a circle. Only "away and still facing north" escapes.
391	Perfect Rectangle	Hard	Areas add up and only four corners stay	The small areas must add up to the big box. Toggle every corner in a set: inner corners appear an even number of times and cancel out, so exactly the four outer corners must be left.
850	Rectangle Area II	Hard	Coordinate compression on a grid	Collect every x and y edge, sort them and mark which small cells between them are covered. Add up the covered cells and take the result modulo $10^9 + 7$.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM command=say_hello() (with brackets) is the classic beginner bug: it calls the function ONCE while the button is made and stores its result, None, as the command. Then clicks do nothing. To pass arguments, use command=lambda: say(name).

Lesson: Your first window

COMMON MISTAKE Never mix pack and grid for widgets with the same parent: Tk raises TclError ("cannot use geometry manager grid inside . which already has slaves managed by pack"). Put each group in its own Frame instead, and each Frame can use its own manager.

Lesson: Placing widgets: pack, grid and place

REMEMBER ttk widgets (from tkinter import ttk) use the look of the operating system and are styled with themes. The older tk widgets use options such as bg and fg. Prefer ttk where it has the widget. Listbox, Text, Canvas and Menu exist only in tk.

Lesson: Widgets and control variables

REMEMBER bind on a widget catches events in that widget. root.bind catches them anywhere in the window, and bind_all in every window of the program. For the close button use root.protocol("WM_DELETE_WINDOW", function).

Lesson: Events, the Canvas and animation with after()

COMMON MISTAKE Tk is not thread safe. Create and change widgets only in the thread that runs mainloop. Other threads hand their results over through a queue, and the main thread picks them up with after().

Lesson: Long tasks without freezing the window

STEP 8 Advanced Python · Exam practice

8.1 Exam practice: PCPP1 style papers

Four PCPP1 style papers, a bug hunt, the traps table, exam technique and a mock quiz.

KEY IDEAS

- **Exam facts** PCPP1 (PCPP-32-101) as listed by training providers: 45 questions, 65 minutes, 70 %. Confirm on pythoninstitute.org/pcpp1.
- **Object traps** __eq__ needs __hash__, NotImplemented for unknown types, super() follows the MRO, wraps keeps the name, slots remove __dict__.
- **Iteration traps** Generators run lazily and once, send resumes a yield, yield from returns the inner value, groupby needs sorted input.
- **Error traps** __exit__ True swallows, raise from sets __cause__, except* splits groups, never except Exception: pass.
- **Library traps** Raw strings for regex, json makes lists and str keys, csv gives str, commit() in sqlite3, command=f in tkinter.

FORMULAS

Safety margin

$$\text{aim} = \text{pass mark} + 15 \text{ points in practice}$$

Practise until you score about 85 %, because exam nerves cost a few points.

Lesson: Exam technique and a study plan

TABLES

Table: · pcpp1 (pcpp-32-101), as listed by training providers

	PCPP1 (PCPP-32-101), as listed by training providers
Questions	45
Time	65 minutes
Pass mark	70 %
Before it	PCAP (Certified Associate) is recommended
Blocks	Advanced object oriented programming. Coding conventions, best practices and standards (PEP 8, PEP 257). GUI programming with tkinter. Network programming (sockets, REST). File processing and the program environment (sqlite3, CSV, XML, logging, configparser).

Lesson: The PCPP1 exam and how to use this chapter

Table: exam block · chapters in this course

Exam block	Chapters in this course
Advanced OOP	1.1 to 1.5 (special methods, decorators, class design, exceptions and copies, metaclasses)
Conventions and best practices	3.1 (style, PEP 8, PEP 257, type hints), 3.2 testing
GUI programming	6.4 tkinter
Network programming	Web and APIs part (HTTP, sockets, requests, REST)
Files and environment	3.3 logging, 6.2 CSV, JSON, XML and settings, 6.3 sqlite3
Extra (not in the exam, used every day)	2.1 to 2.3 generators, itertools, context managers, 4.1 and 4.2 speed, 5.1 and 5.2 concurrency, 6.1 regex

Lesson: The PCPP1 exam and how to use this chapter

Table: paper · covers · lesson

Paper	Covers	Lesson
A	Special methods, MRO, class and static methods, properties, ABCs, decorators, metaclasses, slots, copies	2
B	Generators, send, yield from, iterators, itertools, context managers, exception chaining and groups	3
C	Regular expressions, json, csv, collections, heapq, sqlite3	4
D	functools, dataclasses, enum, parameter kinds, type hints, sorting, the walrus, match	5

Lesson: The PCPP1 exam and how to use this chapter

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	V(3) True False	+ calls __add__, == calls __eq__, but is compares identity: two different objects (1.1)
2	TypeError	A class that defines __eq__ without __hash__ gets __hash__ = None, so its objects cannot go in a set (1.1)
3	BCA ['D', 'B', 'C', 'A', 'object']	super() follows the MRO of the object, not the parent of the class. In B it calls C, not A (1.3)
4	2 hi	The classmethod gets the class, the staticmethod gets nothing. Both can be called on the class (1.3)
5	25 too cold	The setter refused -300 before changing _c, so the old value stays (1.3)
6	cannot make a Shape	A class with an abstract method that is not overridden cannot be instantiated (1.3)
7	10 wrapper	The decorator doubles the result. Without functools.wraps the function now reports the name wrapper (1.2)
8	['x', 'x', 'x']	repeat(3) runs first and returns the real decorator (1.2)
9	thing Meta type	The metaclass __new__ added tag. The type of a class is its metaclass, and the type of type is type (1.5)
10	False	__slots__ removes the per object __dict__, so new attributes are refused (1.3)
11	[1, 2, 9] [1, 2] False True	A shallow copy is a new outer list that shares the inner lists. deepcopy copies them too (1.4)

Lesson: Paper A: advanced object oriented programming

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	made, start, 1, middle, [2]	Calling a generator function runs nothing. Each next runs to the next yield (2.1)
2	got 10, then 5 20	send(10) resumes the paused yield with the value 10. The print inside runs before the Q2 line can print (2.1)
3	[1, 2, 'done']	yield from passes the inner values on and gives back the inner return value (2.1)
4	True [3]	in on an iterator uses it up until it finds 2, so only 3 is left (2.1)
5	[10, 15, 20] [1, 3, 6]	count is endless, islice takes 3. accumulate gives running totals (2.2)
6	['a', 'b', 1] ['a', 'b', 'a'] 10	groupby groups only NEIGHBOURS that are equal, so a appears twice. 5 choose 2 is 10 (2.2)
7	, B,	The code before yield is the set up, the finally part is the clean up (2.3)
8	exit saw ZeroDivisionError, after the with block	__exit__ returning True swallows the exception (2.3)
9	KeyError True	raise ... from err sets __cause__ and hides the automatic context (1.4)
10	V 2, T 1	except* splits the group: each handler gets the matching part (1.4)

Lesson: Paper B: generators, context managers and exceptions

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	['1', '22', '333'] home at ann	findall returns strings. In the replacement, \2 and \1 are the groups (6.1)
2	None True True	match only tries at the start, search anywhere, fullmatch the whole text (6.1)
3	['<a>'] ['<a>', '']	.+ is greedy and takes as much as it can, .+? is lazy (6.1)
4	{"a": null, "b": [1, 2]} [1, 2.0, True]	None becomes null, tuples become lists, true becomes True (6.2)
5	[['x', 'y, z', '3']] str	The quoted field keeps its comma. csv gives every value as a string (6.2)
6	[('a', 3), ('n', 2)] {'a': 1}	most_common sorts by count. A defaultdict is a dict (4.2)
7	[0, 2, 3]	maxlen=3: append 4 pushes out 1, appendleft 0 pushes out 4 from the right end (4.2)
8	1 [1, 2] 1 [2, 5, 4]	heapify puts the smallest first. After heappop the list is still a heap, not sorted (4.2)
9	3 Point(x=9, y=2) {'x': 1, 'y': 2}	A namedtuple works by name and by index. _replace makes a new one (4.2)
10	(3, 3, 6)	Aggregates give one row: COUNT, MAX and SUM (6.3)

Lesson: Paper C: the standard library

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	832040 31	With the cache every fib(n) from 0 to 30 is worked out once: 31 misses (1.2, 4.1)
2	25 24	partial fixes exp=2. reduce multiplies 1 × 2 × 3 × 4 (2.2)
3	greet Say hi.	functools.wraps copies the name and the docstring to the wrapper (1.2)
4	True Card(rank=2, suit='hearts', tags=[]) True	order=True compares the fields in order. The dataclass writes __repr__ and __eq__ (1.3)
5	Color.RED GREEN 1 2	Color(2) finds by value, Color["RED"] by name (1.3)
6	a is positional only, then 6	Parameters before / cannot be passed by name. After * they must be (1.2)
7	6 {'x': <class 'int'>, 'return': <class 'str'>}	Type hints are not checked when the code runs. Use mypy for that (3.1)
8	['C', 'a', 'b'] ['a', 'b', 'C'] 0	Upper case letters sort before lower case ones. key=str.lower ignores case. max needs default= for an empty list
9	True 3 1 2 3	:= assigns inside an expression. *x unpacks the list into separate arguments
10	x is 1	The first case needs x == 0. The second captures the value in x0

Lesson: Paper D: functions, data classes and modern syntax

Table: trap · looks like · really · chapter

Trap	Looks like	Really	Chapter
__eq__ only	Objects still work in sets	They become unhashable	1.1
NotImplemented	Return False when the type is wrong	Return NotImplemented, so Python can try the other side	1.1
super()	Calls the parent class	Calls the next class in the MRO of the object	1.3
Decorator without wraps	Keeps the name and docstring	The function is now called wrapper	1.2
Decorator with arguments	@repeat is enough	@repeat(3): three levels of functions	1.2
Mutable class attribute	Each object has its own list	All objects share one list	1.3
Shallow copy	copy.copy copies everything	Inner objects are shared	1.4
Generator	Can be looped over twice	It is empty after the first loop	2.1
groupby	Groups all equal items	Groups only neighbours: sort first	2.2
__exit__ return value	Return True to say "done"	True swallows the exception	2.3
raise from	Same as a plain raise	Sets __cause__ and shows "direct cause"	1.4
Type hints	Python checks them	Only tools such as mypy check them	3.1
Regex without r	"\d" and r"\d" are the same	Some escapes, like \b, change meaning	6.1
pickle	Safe like JSON	Loading can run code	6.2
sqlite3	Changes are saved at once	Call commit() or use with conn	6.3
tkinter callback	command=f() is right	command=f, without brackets	6.4
pack and grid	Can be mixed in one parent	TclError: use one per parent	6.4

Lesson: The traps in one table

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
304	Range Sum Query 2D - Immutable	Medium	A class with a table of prefix sums	Build sums[r + 1][c + 1] = the total of the rectangle from (0, 0) to (r, c) once in __init__. Any rectangle is then four lookups.
438	Find All Anagrams in a String	Medium	A sliding window of Counters	Keep a Counter of the last len(p) letters. Add the new letter, remove the oldest, and compare with the Counter of p.
39	Combination Sum	Medium	A recursive generator	A generator yields each combination as soon as it is found. Pass the start index so a number is never used before a smaller one, which avoids repeats.
131	Palindrome Partitioning	Medium	yield from for every first piece	Try every palindrome as the first piece, then partition the rest the same way. A generator with yield from keeps the code short.
1143	Longest Common Subsequence	Medium	Recursion with lru_cache	best(i, j) compares text1[i:] and text2[j:]. Equal letters add 1, otherwise skip one letter on either side. functools.cache stores every answer.
221	Maximal Square	Medium	Grow squares from three neighbours	size[r][c] is the side of the biggest all "1" square ending at (r, c): 1 + the smallest of the sizes above, to the left and diagonal.
72	Edit Distance	Medium	A table of smaller answers	dist[i][j] is the cost to turn the first i letters of word1 into the first j letters of word2. Equal letters cost nothing, otherwise 1 + the best of insert, delete or replace.
42	Trapping Rain Water	Hard	Two pointers with the highest wall on each side	Water above a bar is the lower of the highest walls on its left and right, minus the bar. Move the pointer on the lower side, because that side decides the water level.
76	Minimum Window Substring	Hard	Grow the window, then shrink it	Move the right end until the window holds every letter of t (count with a Counter). Then move the left end while it still does, and remember the shortest window.
460	LFU Cache	Hard	A dict of OrderedDicts, one per use count	Keep the value and the use count of each key, and an OrderedDict of keys for every count. Evict the oldest key in the group with the smallest count.

Lesson: LeetCode review set

TIPS AND TRAPS

COMMON MISTAKE Check pythoninstitute.org/pcpp1 before you book. Exam versions, prices and weights change, and this page could not confirm them from the official site.

Lesson: The PCPP1 exam and how to use this chapter

REMEMBER Cover the answers, write down what you think each line prints, then open the answers. A wrong guess tells you exactly which chapter to reread.

Lesson: The PCPP1 exam and how to use this chapter

IN THE EXAM In the exam: read the whole snippet first, then the options. Trace it on paper and watch for the answer "the code raises an exception". Mark hard questions and come back, because every question is worth the same.

Lesson: Exam technique and a study plan

1.1 HTTP and REST: how the web talks

HTTP messages, URLs, methods, status codes, headers, cookies, caching and REST design.

KEY IDEAS

- **Messages** Request line (method, path, version), headers, blank line, body. Status line, headers, blank line, body. \r\n line ends.
- **Methods and codes** GET read, POST create (201 + Location), PUT replace, PATCH change, DELETE (204). 2xx ok, 3xx redirect, 4xx client, 5xx server.
- **URLs** urlsplit, parse_qs, parse_qsl, urlencode, quote, urljoin. Percent encoding: one byte becomes %XX.
- **Headers** Content-Type, Accept, Authorization, Set-Cookie (HttpOnly, Secure, SameSite), Cache-Control, ETag and If-None-Match for 304.
- **Python** http.server.BaseHTTPRequestHandler (do_GET, send_response, send_header, end_headers, wfile), urllib.request, HTTPError for 4xx and 5xx.

FORMULAS

Time to first byte Exam favourite

about (TCP 1 + TLS 1 + request 1) round trips × RTT

RTT 50 ms: 3 × 50 = 150 ms before the first byte of the answer, however small it is.

Lesson: The maths: latency, bandwidth and caching

Transfer time

time = size × 8 / bandwidth

2 MB at 40 Mbit/s: 16 Mbit / 40 = 0.4 s.

Lesson: The maths: latency, bandwidth and caching

Average time with a cache Exam favourite

average = hit rate × hit time + (1 − hit rate) × miss time

80 % hits at 5 ms, misses at 200 ms: 0.8 × 5 + 0.2 × 200 = 44 ms.

Lesson: The maths: latency, bandwidth and caching

Little's law

requests in progress = arrival rate × time per request

200 requests a second, 50 ms each: 10 requests at the same moment.

Lesson: The maths: latency, bandwidth and caching

Bandwidth delay product

bits in flight = bandwidth × RTT

100 Mbit/s and 80 ms: 8 Mbit (1 MB) on the wire at once.

Lesson: The maths: latency, bandwidth and caching

Percent encoding size

a byte outside the allowed set becomes 3 characters

"é" is 2 bytes in UTF-8, so it becomes 6 characters: %C3%A9.

Lesson: The maths: latency, bandwidth and caching

Table: · rest over http · graphql · grpc

	REST over HTTP	GraphQL	gRPC	WebSocket
Idea	URLs name resources, methods say what to do	One URL, the client writes a query for exactly the fields it wants	Call functions on a server, messages in Protocol Buffers	One connection that stays open, both sides send at any time
Data	Usually JSON	JSON	Binary, typed	Text or binary frames
Good for	Public APIs, most web apps	Apps that need flexible queries	Fast calls between internal services	Chat, live prices, games
Caching	Built into HTTP	Harder	Do it yourself	Not for messages
In Python	requests, Flask, Django, FastAPI	graphene, strawberry	grpcio	websockets

Lesson: How the web works: one request, one response

Table: part · example · meaning

Part	Example	Meaning
scheme	https	How to talk: http, https, ftp, mailto ...
host	api.example.com	Which computer. DNS turns it into an IP address
port	8443	Which program on that computer. Default 80 for http, 443 for https
path	/v1/books/42	Which resource on the server
query	fields=title,year&lang=en	Extra options as name=value pairs joined with &
fragment	reviews	A place inside the page. The browser keeps it and never sends it

Lesson: URLs: the address of a resource

Table: method · means · safe (changes nothing) · idempotent (same result if repeated)

Method	Means	Safe (changes nothing)	Idempotent (same result if repeated)	Has a body
GET	Read a resource	Yes	Yes	No
HEAD	Like GET, but only the headers	Yes	Yes	No
POST	Create something, or run an action	No	No	Yes
PUT	Replace the whole resource	No	Yes	Yes
PATCH	Change part of a resource	No	Not always	Yes
DELETE	Remove a resource	No	Yes	Usually no
OPTIONS	Ask what is allowed (used by CORS)	Yes	Yes	No

Lesson: Inside an HTTP message

Table: range · meaning · common codes

Range	Meaning	Common codes
1xx	Wait, more is coming	101 Switching Protocols (WebSocket)
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Look somewhere else	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	The client made a mistake	400 Bad Request, 401 Unauthorized (not logged in), 403 Forbidden (not allowed), 404 Not Found, 409 Conflict, 422 invalid data, 429 Too Many Requests
5xx	The server failed	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

Lesson: Inside an HTTP message

Table: action · method and path · success · typical errors

Action	Method and path	Success	Typical errors
List books	GET /books	200 with a JSON list	
Read one book	GET /books/2	200 with the book	404 if it does not exist
Create a book	POST /books with JSON	201 Created and a Location header	400 or 422 for bad data
Replace a book	PUT /books/2 with JSON	200 or 204	404, 400
Change a book	PATCH /books/2 with some fields	200	404, 400
Delete a book	DELETE /books/2	204 No Content	404

Lesson: REST: designing an API with resources

Table: header · sent by · does

Header	Sent by	Does
Host	Client	Which site on a server with many sites
Accept, Content-Type	Both	Which format is wanted, which format is sent
Authorization	Client	Proves who you are, for example Bearer and a token
User-Agent	Client	Names the program that sends the request
Set-Cookie, Cookie	Server, then client	The server stores a small value in the browser, which sends it back on every request
Cache-Control, ETag, If-None-Match	Both	How long a copy may be kept, its version, and "is my version still right?"
Location	Server	Where a new resource is, or where to go for a redirect
Access-Control-Allow-Origin	Server	Which other websites may call this API from a browser (CORS)

Lesson: Headers, cookies and caching

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1961	Check If String Is a Prefix of Array	Easy	Join pieces until the text is long enough	Like building a URL path from its parts. Add the words one by one and stop as soon as the joined text is as long as s.
1309	Decrypt String from Alphabet to Integer Mapping	Easy	Look two places ahead for #	This is decoding, like reading %20 in a URL. If the character two places ahead is #, read two digits, otherwise one.
1773	Count Items Matching a Rule	Easy	Pick the column by its name	Like a query string such as ?color=silver filtering records. Map the rule key to a column index with a dict.
2185	Counting Words With a Given Prefix	Easy	str.startswith	Routers match URL paths by prefix in the same way. Count the words that start with the prefix.
811	Subdomain Visit Count	Medium	Every suffix of a host name	A visit to discuss.leetcode.com also counts for leetcode.com and com. Split on dots and add the count to every ending with a Counter.
93	Restore IP Addresses	Medium	Try every split into four parts	Each part is 0 to 255 with no leading zero, like a real IPv4 address. Try lengths 1, 2 and 3 for each of the four parts.
1268	Search Suggestions System	Medium	Sort once, then filter by the growing prefix	This is what a search box API returns while you type. Sort the products, and for each typed prefix keep the first three that start with it.
2284	Sender With Largest Word Count	Medium	A Counter, then max with a tuple key	Count the words each sender wrote. Pick the largest count, and on a tie the name that is largest in alphabetical order: max by (count, name).
140	Word Break II	Hard	Recursion with a cache of endings	All the ways to cut a string into dictionary words. sentences(start) returns every sentence for s[start:], and functools.cache remembers each answer.
301	Remove Invalid Parentheses	Hard	Breadth first search by number of removals	Check all strings with one character removed, then two, and so on. The first level that has valid strings gives the answer with the fewest removals.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

URLs may only contain a small set of characters. Others, like spaces, é or & inside a value, must be **percent encoded**: each byte becomes % and two hex digits, so a space is %20 and é (2 bytes in UTF-8) is %C3%A9. Use urlencode or let requests do it with params=, never glue query text together by hand.

Lesson: URLs: the address of a resource

REMEMBER

BaseHTTPRequestHandler answers with HTTP/1.0 unless you set protocol_version = "HTTP/1.1" in the class. Every line ends with \r\n (carriage return and new line), and an empty line separates the headers from the body.

Lesson: Inside an HTTP message

IN THE EXAM

urllib.request.urlopen raises HTTPError for 4xx and 5xx answers. The requests library in the next chapters does not: it returns the response, and you call raise_for_status() when you want an exception.

Lesson: REST: designing an API with resources

COMMON MISTAKE

HTTP itself has no memory between requests. Logins work because the server gives the browser a session cookie or a token, and the client sends it back every time. Mark session cookies HttpOnly, Secure and SameSite, as in the example.

Lesson: Headers, cookies and caching

1.2 Sockets, TCP and UDP: under HTTP

TCP and UDP sockets, bytes and struct, message framing, many clients with asyncio, IP addresses and subnets.

KEY IDEAS

- **Sockets** socket(AF_INET, SOCK_STREAM) for TCP, SOCK_DGRAM for UDP. An address is (host, port). Ports 0 to 65,535. 127.0.0.1 local only, 0.0.0.0 every network card.
- **TCP calls** Server: bind, listen, accept (new socket per client). Client: create_connection(addr, timeout). sendall, recv(n) gives 1 to n bytes, b"" means closed.
- **Framing** TCP keeps no message borders. Length prefix with struct.pack("!I", n), or an end marker such as a new line. Loop until a message is complete.
- **UDP** sendto and recvfrom, no handshake, borders kept, may lose or reorder. Used for DNS, games, video. 8 byte header.
- **Numbers** MSS = MTU − 40. throughput ≤ window / RTT. window = bandwidth × RTT. /n network has 2^{32-n} addresses.

FORMULAS

Addresses in a network Exam favourite

$$\text{addresses} = 2^{32 - \text{prefix}},\ \text{usable} = \text{addresses} - 2$$

/24: 256 addresses, 254 usable. /20: 4,096 addresses.

Lesson: The maths: windows, round trips and loss

Data in one packet

$$\text{MSS} = \text{MTU} - \text{IP header} - \text{TCP header}$$

1,500 − 20 − 20 = 1,460 bytes of your data.

Lesson: The maths: windows, round trips and loss

Window limit on speed Exam favourite

$$\text{throughput} \le \text{window} / \text{RTT}$$

A 64 KB window with RTT 100 ms: 65,535 × 8 / 0.1 ≈ 5.2 Mbit/s, however fast the line is.

Lesson: The maths: windows, round trips and loss

Window needed

$$\text{window} = \text{bandwidth} \times \text{RTT (the bandwidth delay product)}$$

100 Mbit/s and 50 ms: 100,000,000 × 0.05 / 8 = 625,000 bytes.

Lesson: The maths: windows, round trips and loss

TCP with loss (Mathis formula) Exam favourite

$$\text{throughput} \approx (\text{MSS} / \text{RTT}) \times 1.22 / \sqrt{p}$$

MSS 1,460 B, RTT 100 ms, loss p = 0.0001: about 14.2 Mbit/s.

Lesson: The maths: windows, round trips and loss

Chance of at least one loss

$$P = 1 - (1 - p)^n$$

100 packets with p = 0.01: $1 - 0.99^{100} \approx 0.634$.

Lesson: The maths: windows, round trips and loss

TABLES

Table: · tcp socket · udp socket · unix domain socket

	TCP socket	UDP socket	Unix domain socket	QUIC
Setup	Three way handshake	None	None, a file path on one computer	One round trip, with TLS built in
Delivery	Every byte, in order, no duplicates	May lose, repeat or reorder packets	Reliable	Reliable, per stream
Message borders	No: a stream of bytes	Yes: one datagram is one message	Stream or datagram, you choose	Streams inside one connection
Good for	HTTP/1 and HTTP/2, databases, SSH, email	DNS, video calls, games, metrics	Programs on the same computer, such as Docker or PostgreSQL	HTTP/3
In Python	socket.SOCK_STREAM, asyncio streams	socket.SOCK_DGRAM	socket.AF_UNIX	aioquic library

Lesson: What a socket is

Table: struct code · c type · bytes · range

struct code	C type	Bytes	Range
B	unsigned char	1	0 to 255
H	unsigned short	2	0 to 65,535 (a port number fits here)
I	unsigned int	4	0 to about 4.29 billion (an IPv4 address fits here)
Q	unsigned long long	8	0 to about 1.8×10^{19}
h, i, q	signed versions	2, 4, 8	Negative numbers too
!	byte order mark		Network order: the most important byte first (big endian), no padding

Lesson: Bytes: what really travels

Table: error · when · what to do

Error	When	What to do
ConnectionRefusedError	Nothing listens on that port	Check the host and port, is the server running
TimeoutError (socket.timeout)	No answer within the timeout	Retry later or give up with a clear message
ConnectionResetError	The other side crashed or closed hard	Reconnect, and send again only if that is safe
OSError: Address already in use	Another program, or your old one, has the port	Pick another port, or wait. SO_REUSEADDR helps after a restart
socket.gaierror	The host name could not be found (DNS)	Check the spelling and the network

Lesson: A TCP client and server

Table: framing method · how the receiver finds the end · used by

Framing method	How the receiver finds the end	Used by
Length prefix	Read a fixed size length first, then exactly that many bytes	PostgreSQL, Kafka, gRPC, WebSocket frames
End marker	Read until a special byte sequence, such as a new line	SMTP email, Redis, HTTP headers (end with an empty line)
Fixed size	Every message has the same size	Simple sensor data, some games
Close the connection	The message ends when the sender closes	HTTP/1.0 without Content-Length

Lesson: TCP is a stream: framing messages

Table: · tcp · udp

	TCP	UDP
Connection	connect and accept first	None, just sendto an address
Lost packets	Sent again automatically	Gone, the program must notice
Order	Always the order they were sent	Can arrive in any order
Message borders	Not kept	Kept: one sendto is one recvfrom
Header size	20 to 60 bytes	8 bytes
Speed of the first message	After one round trip	At once
Python calls	connect, sendall, recv	sendto, recvfrom

Lesson: UDP: single packets, no promises

Table: · one at a time · a thread per client · selectors

	One at a time	A thread per client	selectors	asyncio
Idea	accept, serve, then accept the next	Each accepted socket gets its own thread	One loop asks the OS which sockets are ready	Coroutines that wait with await, on one event loop
Many slow clients	Everyone else waits	Fine up to a few thousand	Fine, tens of thousands	Fine, tens of thousands
Code	Very simple	Simple, but shared data needs locks	Hard to write by hand	Looks like normal code with async and await
In Python	socket	socketserver.ThreadingTCPServer	selectors.DefaultSelector	asyncio.start_server

Lesson: Serving many clients at once

Table: layer · job · example · in python

Layer	Job	Example	In Python
Application	What the program means	HTTP, DNS, SMTP, your own protocol	requests, http.server, your code
Transport	Which program, and how reliable	TCP, UDP (ports)	socket.SOCK_STREAM, socket.SOCK_DGRAM
Internet	Which computer, and the route there	IPv4, IPv6 (IP addresses)	socket.AF_INET, socket.AF_INET6, ipaddress
Link	The next hop on the local network	Ethernet, Wi-Fi (MAC addresses)	The operating system does it

Lesson: IP addresses, ports and layers

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
190	Reverse Bits	Easy	Take the last bit, push it in from the right	Byte order and bit order matter on the wire. Take the lowest bit of n 32 times and add it to the result from the other end.
1971	Find if Path Exists in Graph	Easy	Breadth first search from the source	Can a packet get from one computer to another? Keep a set of visited computers and a queue of computers to visit.
2073	Time Needed to Buy Tickets	Easy	Count what each person buys before k is done	A queue served in turns, like a server that sends one piece to each client in turn. People before k buy at most tickets[k], people after k at most tickets[k] minus 1.
743	Network Delay Time	Medium	Dijkstra with a heap	A signal spreads from computer k over links with delays. Dijkstra finds the fastest time to each computer, and the answer is the slowest of those.
1319	Number of Operations to Make Network Connected	Medium	Union find counts the groups	You need at least n minus 1 cables. If there are enough, the answer is the number of separate groups minus 1.
1514	Path with Maximum Probability	Medium	Dijkstra on the largest product	Each link delivers a packet with some probability. Use a heap of negative probabilities so the most likely path comes out first.
787	Cheapest Flights Within K Stops	Medium	Bellman Ford for k + 1 rounds	Like a packet with a hop limit (TTL). Each round allows one more flight. Copy the prices first so one round uses only one more flight.
1615	Maximal Network Rank	Medium	Degrees of both cities, minus 1 if they are linked	The rank of two cities is the number of roads touching either one. Try every pair and do not count a shared road twice.
1192	Critical Connections in a Network	Hard	Tarjan: discovery time and lowest reachable time	A cable is critical if cutting it splits the network. In a depth first search, the link to a child is critical when the child cannot reach back above itself.
1579	Remove Max Number of Edges to Keep Graph Fully Traversable	Hard	Two union finds, shared edges first	Use the edges both Alice and Bob can use first, then each person's own edges. Every edge that joins nothing new can be removed.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

A socket only knows bytes. Turn text into bytes with .encode("utf-8") before sending and back with .decode("utf-8") after receiving. "é" is one letter but 2 bytes, so always measure the length of the bytes, never of the text.

Lesson: Bytes: what really travels

COMMON MISTAKE

Host "127.0.0.1" (localhost) only accepts connections from the same computer. "0.0.0.0" accepts them from every network card, so other computers can connect. Use 127.0.0.1 while you learn.

Lesson: A TCP client and server

COMMON MISTAKE

A very common bug: the sender calls sendall twice and the receiver expects two recv calls to return the two messages. On a fast local test it often works, and then it fails on a real network. recv only promises "some bytes", so always loop until you have a whole message.

Lesson: TCP is a stream: framing messages

REMEMBER

A UDP datagram larger than the path allows (about 1,472 bytes of data on normal Ethernet) gets split by IP, and if one piece is lost the whole datagram is lost. Keep UDP messages small.

Lesson: UDP: single packets, no promises

IN THE EXAM

A /24 network has $2^{32-24} = 256$ addresses. The first is the network address and the last is the broadcast address, so 254 are left for computers. Ports go from 0 to 65,535, and ports below 1024 usually need admin rights.

Lesson: IP addresses, ports and layers

STEP 2 Web and APIs · Calling web APIs

2.1 Calling web APIs with requests

requests: GET and POST, params and json, timeouts, errors, sessions, auth and JSON answers.

KEY IDEAS

- **Calls** requests.get, post, put, patch, delete(url, params=, json=, data=, headers=, timeout=). Response: status_code, ok, headers, text, content, json().
- **Errors** raise_for_status() for 4xx and 5xx. HTTPError, ConnectionError, Timeout, JSONDecodeError, all under RequestException (catch it last).
- **Timeouts** No default timeout. timeout=5 or timeout=(connect, read). Always pass one.
- **Session** Reuses connections, keeps cookies and headers. Auth: Authorization: Bearer token, auth=(user, pw), API key header. Secrets come from environment variables.
- **JSON** object to dict, array to list, null to None. [] for required keys, .get() for optional. datetime.fromisoformat for dates.

FORMULAS

Pages to fetch — Exam favourite

$$\text{pages} = \lceil \text{items} / \text{page size} \rceil$$

95 items, 20 per page: ⌈4.75⌉ = 5 requests.

Lesson: The maths: round trips, timeouts and limits

Time without a Session

$$n \times (\text{handshake RTTs} + 1) \times \text{RTT}$$

10 calls, 2 handshake RTTs, RTT 50 ms: 10 × 3 × 50 = 1,500 ms.

Lesson: The maths: round trips, timeouts and limits

Time with a Session — Exam favourite

$$(\text{handshake RTTs} + n) \times \text{RTT}$$

(2 + 10) × 50 = 600 ms, so 900 ms saved.

Lesson: The maths: round trips, timeouts and limits

Longest wait with timeout=(c, r)

$$\text{up to } c + r \text{ for each silent stretch}$$

timeout=(3, 2): up to 3 s to connect, then 2 s with no data before ReadTimeout.

Lesson: The maths: round trips, timeouts and limits

Rate limit

$$\text{minimum time} = \text{requests} / \text{allowed rate}$$

300 calls at 60 per minute: 5 minutes.

Lesson: The maths: round trips, timeouts and limits

Chance one of k tries works — Exam favourite

$$P = 1 - (1 - p)^k$$

p = 0.9, k = 3: 1 − 0.001 = 0.999.

Lesson: The maths: round trips, timeouts and limits

TABLES

Table: · urllib.request · requests · httpx

	urllib.request	requests	httpx	aiohttp
Comes with Python	Yes	No, pip install requests	No, pip install httpx	No, pip install aiohttp
A JSON call	Encode, build a Request, read, json.loads	requests.get(url).json()	httpx.get(url).json()	Inside async with blocks
4xx and 5xx	Raises HTTPError	Returns the response, raise_for_status() raises	Returns, raise_for_status() raises	Returns, raise_for_status() raises
async and HTTP/2	No	No	Both	async only
Best for	Scripts that must not install anything	Most scripts and apps	The same as requests, plus async code	Many calls at once in async servers

Lesson: What a web API is, and why requests

Table: on the response · gives · example

On the Response	Gives	Example
r.status_code, r.reason	The status number and its words	200 and OK
r.ok	True when the status is below 400	False for 404
r.headers	A dict of headers that ignores upper and lower case	r.headers["content-type"]
r.text	The body as a str, decoded with r.encoding	'[{"id": 3 ...'
r.content	The body as raw bytes, for images and files	b'\x89PNG...'
r.json()	The body parsed from JSON into Python	a list or a dict
r.url, r.request	The final URL, and the request that was really sent	r.request.headers

Lesson: Your first GET request

Table: argument · sends · content-type it sets · use it for

Argument	Sends	Content-Type it sets	Use it for
params=dict	The dict in the URL query string	None	Search words, filters, page numbers
json=obj	The object as JSON text in the body	application/json	Almost every modern API
data=dict	The dict as a web form in the body	application/x-www-form-urlencoded	HTML forms, OAuth token requests
files=dict	Files as multipart form data	multipart/form-data	Uploading pictures and documents
headers=dict	Extra headers	Whatever you set	API keys, Accept, a custom User-Agent

Lesson: Sending data: POST, PUT, PATCH and DELETE

Table: exception · means · parent

Exception	Means	Parent
requests.HTTPError	raise_for_status() found a 4xx or 5xx status	RequestException
requests.ConnectionError	No connection: the server is down, the name is wrong, no network	RequestException
requests.Timeout (ConnectTimeout, ReadTimeout)	The connect or a read took longer than timeout	RequestException
requests.TooManyRedirects	Redirects went round in a circle	RequestException
requests.JSONDecodeError	r.json() on a body that is not JSON	RequestException and ValueError
requests.RequestException	The parent of all of them	OSError

Lesson: When a call fails

Table: way to log in · how requests sends it · notes

Way to log in	How requests sends it	Notes
API key in a header	headers={"X-API-Key": key}	Simple. Keep the key out of your code, read it from an environment variable
Bearer token	headers={"Authorization": f"Bearer {token}"}	Most modern APIs, often an OAuth 2 access token
Basic auth	auth=("user", "password")	Name and password in base64, only safe over HTTPS
Session cookie	A Session keeps Set-Cookie and sends it back	Websites that have a login form
OAuth 2	Get a token first, then send it as Bearer	Libraries such as requests-oauthlib do the steps for you

Lesson: Sessions, cookies and authentication

Table: json · python after json.loads · watch out

JSON	Python after json.loads	Watch out
object {...}	dict	Keys are always str
array [...]	list	
string	str	Dates stay strings, parse them with datetime.fromisoformat
number	int or float	1 becomes int, 1.0 becomes float
true, false	True, False	
null	None	A missing key and a null value are different. dict.get covers both

Lesson: Working with JSON answers

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2418	Sort the People	Easy	Sort pairs by one field	Like sorting API records by a field. Zip names with heights and sort by height, largest first.
1854	Maximum Population Year	Easy	Count people alive in each year	Records with a start and an end, like user sessions in logs. Years are only 1950 to 2050, so count each year directly.
1700	Number of Students Unable to Eat Lunch	Easy	Only the counts matter	A queue of requests waiting for a matching resource. Count how many want each kind, then serve the stack until no one wants the top item.
2456	Most Popular Video Creator	Medium	Group records by creator	Typical API analytics: total views per creator, and each creator's best video (most views, then the smallest id).
1366	Rank Teams by Votes	Medium	Sort by a tuple of vote counts	Count how often each team gets each place. Sort by the counts, most first, and by the letter on a tie.
355	Design Twitter	Medium	Dicts of lists and sets, heapq.nlargest	A small social API. Store each user's tweets with a counter as the time. The feed is the 10 newest tweets of the user and the people they follow.
1348	Tweet Counts Per Frequency	Medium	Put each time into a bucket	API dashboards count events per minute, hour or day. The bucket of a time t is (t – start) // size.
1606	Find Servers That Handled Most Number of Requests	Hard	Two heaps: busy servers and free servers	A load balancer. Request i prefers server i % k, or the next free one after it. Store a free server j as i + (j – i) % k so the smallest number in the heap is the right server.
2102	Sequentially Ordinal Rank Tracker	Hard	Two heaps split at the i-th best	Keep the places already returned in one heap with the worst on top, and the rest in another with the best on top. get() moves one place across.
352	Data Stream as Disjoint Intervals	Hard	A set of numbers, joined into runs when asked	Like tracking which parts of a file have arrived. Keep the numbers in a set, and on getIntervals walk them in order and join neighbours into ranges.

Lesson: LeetCode practice

TIPS AND TRAPS

COMMON MISTAKE Always pass **timeout=**. requests has no timeout by default, so a server that never answers keeps your program waiting for ever. A number such as timeout=5 is used for connecting and again for each wait while reading.

Lesson: Your first GET request

REMEMBER For a big download use `requests.get(url, stream=True, timeout=10)` and write `for chunk in r.iter_content(64_000)` into a file opened with "wb". Then the whole file never has to fit in memory.

Lesson: Sending data: POST, PUT, PATCH and DELETE

IN THE EXAM A **Session** keeps cookies and default headers, and it reuses the TCP (and TLS) connection for every request to the same host. The example shows 3 requests over 1 connection. That saves one or two round trips each time, so use a Session whenever you make more than a couple of calls.

Lesson: Sessions, cookies and authentication

COMMON MISTAKE Never put a real token or password in code that you share or commit to git. Read it with os.environ["API_TOKEN"] and keep it in a .env file that git ignores.

Lesson: Sessions, cookies and authentication

REMEMBER

Use data["key"] when the key must be there (a missing key is a bug you want to see), and data.get("key", default) when it is optional. The next chapter shows pydantic, which checks the whole shape of an answer at once.

Lesson: Working with JSON answers

2.2 Real API work: pages, retries, limits and validation

Pagination, retries with backoff and jitter, rate limits, pydantic validation and async calls with httpx.

KEY IDEAS

- **Pages** Loop with a generator until no next page or cursor. Cursors stay correct when data changes. pages = ⌈items / size⌉.
- **Retries** Only timeouts, connection errors, 429, 502, 503, 504, and only idempotent requests. wait = random(0, min(cap, base × 2^n)). Stop after k tries.
- **Limits** 429 plus Retry-After (seconds or a date). Token bucket: capacity for bursts, rate for the average. X-RateLimit-Remaining.
- **pydantic** class M(BaseModel), Field(min_length=, ge=, le=), M.model_validate(dict), model_dump(), TypeAdapter(list[M]).validate_json(text), ValidationError.errors().
- **httpx** httpx.Client like a Session, httpx.AsyncClient with await, asyncio.gather for many, asyncio.Semaphore(n) to limit them.

FORMULAS

Backoff wait — Exam favourite

$$\text{wait}_n = \min(\text{cap}, \text{base} \times 2^n)$$

base 0.5 s: 0.5, 1, 2, 4, 8 s for n = 0 to 4.

Lesson: The maths: backoff, limits and pages

Total backoff time

$$\text{base} \times (2^k - 1) \text{ for k waits}$$

base 1 s, 4 waits: 1 + 2 + 4 + 8 = 15 s.

Lesson: The maths: backoff, limits and pages

Full jitter

$$\text{wait} = \text{random}(0, \text{base} \times 2^n),\ \text{mean} = \text{base} \times 2^n / 2$$

n = 3, base 1 s: random between 0 and 8 s, 4 s on average.

Lesson: The maths: backoff, limits and pages

Token bucket in time T — Exam favourite

$$\text{at most capacity} + \text{rate} \times T \text{ calls}$$

rate 5/s, capacity 20, 10 s: 20 + 50 = 70 calls.

Lesson: The maths: backoff, limits and pages

Rows skipped by an offset

$$\text{skipped} = (\text{page} - 1) \times \text{size}$$

page 50, size 20: 980 rows read and thrown away.

Lesson: The maths: backoff, limits and pages

Calls in flight (Little's law) — Exam favourite

$$L = \text{rate} \times \text{time per call}$$

100 calls/s at 0.25 s each: 25 at the same moment.

Lesson: The maths: backoff, limits and pages

TABLES

Table: · no retry · fixed wait · exponential backoff

	No retry	Fixed wait	Exponential backoff	Backoff with jitter
Waits	None	1, 1, 1 s	1, 2, 4, 8 s	random(0, 1), random(0, 2), random(0, 4) s
A short outage	Fails	Recovers	Recovers	Recovers
Many clients at once	Fail together	Retry together, keep the server down	Less load, still together	Spread out, the server recovers
Use it for	Requests that must not repeat	Rarely a good choice	A single client	Almost everything

Lesson: What changes when an API is real

Table: style · the request · good · weak

Style	The request	Good	Weak
Page number	?page=3&size=50	Simple, can jump to any page	Items shift when data changes, so you can see one twice or miss one
Offset and limit	?offset=100&limit=50	The same as pages, in rows	Slow for big offsets, the database still skips every row
Cursor	?after=item3 or ?cursor=eyJpZCI6...	Stable while data changes, fast	Cannot jump to page 40
Link header	`Link: <...?page=4>; rel="next"`	The server gives the next URL	You must read the header (requests: r.links["next"]["url"])

Lesson: Pagination: getting every page

Table: retry? · status or error · why

Retry?	Status or error	Why
Yes	ConnectionError, Timeout	The network or the server had a short problem
Yes	429 Too Many Requests	Wait as long as Retry-After says, then try again
Yes	502, 503, 504, sometimes 500	A server or a gateway is busy or restarting
No	400, 401, 403, 404, 422	The request itself is wrong, it will fail the same way again
Careful	POST and PATCH	Not idempotent: the first try may have worked. Retry only with an idempotency key

Lesson: Retries with backoff, jitter and Retry-After

Table: header · meaning

Header	Meaning
Retry-After: 30	Wait 30 seconds before the next call (it can also be a date)
X-RateLimit-Limit: 5000	How many calls you may make in the current window
X-RateLimit-Remaining: 12	How many are left
X-RateLimit-Reset: 1793000000	When the window starts again, often as a Unix time
RateLimit-Policy, RateLimit	Newer standard names for the same ideas (an IETF draft)

Lesson: Rate limits: the token bucket

Table: · trust it · checks by hand · dataclass

	Trust it	Checks by hand	dataclass	pydantic
Wrong type, such as "seven" for an int	Crashes later, far away	Found if you wrote the check	Not found	Found at once with a clear message
"7" for an int	Stays a str	Your choice	Stays a str	Turned into 7 (lax mode)
Missing field	KeyError somewhere	Found if you wrote the check	TypeError when you build it	Listed with its path, such as author.name
Nested objects and lists	Plain dicts	Much code	By hand	Built in
Cost	None	Many lines	Few lines	One class, a fast core written in Rust

Lesson: Checking the data with pydantic

Table: need · tool

Need	Tool
A few calls in a script	requests or httpx.Client
Many calls at once in a script	httpx.AsyncClient with asyncio.gather and a Semaphore, or a ThreadPoolExecutor with requests
Calls inside an async web app (FastAPI)	httpx.AsyncClient, never requests, which would block the event loop
HTTP/2	httpx.Client(http2=True) after pip install httpx[http2]

Lesson: Many calls at once with httpx

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
21	Merge Two Sorted Lists	Easy	Walk both lists, always take the smaller head	Like merging two sorted pages of results from two APIs. A dummy first node keeps the code short.
350	Intersection of Two Arrays II	Easy	Counter and the & operator	Which ids do two API answers share, counting repeats? Counter(a) & Counter(b) keeps the smaller count of each value.
1450	Number of Students Doing Homework at a Given Time	Easy	Check each time range	Like counting the sessions that were active at a given moment in API logs. A range counts when start <= time <= end.
703	Kth Largest Element in a Stream	Easy	A min heap of size k	Keep only the k largest values seen so far in a heap. Its smallest value is the k-th largest, like the slowest of the top k response times.
57	Insert Interval	Medium	Before, overlapping, after	Add a time range to a sorted list of ranges. Copy the ranges that end before it, merge the ones that overlap, then copy the rest.
729	My Calendar I	Medium	Two ranges overlap when each starts before the other ends	A booking API must refuse a slot that overlaps an existing one. [s1, e1) and [s2, e2) overlap when s1 < e2 and s2 < e1.
731	My Calendar II	Medium	Keep the bookings and the double booked parts	Now two bookings may overlap but never three. Keep a list of the parts that are already double booked. A new booking that touches one of them must be refused.
435	Non-overlapping Intervals	Medium	Greedy: keep the range that ends first	Sort by end time. Keep a range when it starts after the last kept one ends, otherwise remove it. Ending early leaves the most room.
480	Sliding Window Median	Hard	Keep the window sorted with bisect	Monitoring shows the median response time of the last k calls. Keep the window sorted: insort the new value, remove the old one, and read the middle.
732	My Calendar III	Hard	Sweep line: +1 at a start, −1 at an end	How many bookings overlap at the busiest moment? Add +1 at each start and −1 at each end, then walk the times in order and keep a running sum.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Write the loop as a **generator** with yield from. The caller sees one long stream of items and can stop early, and pages are only fetched when they are needed.

Lesson: Pagination: getting every page

IN THE EXAM

A token bucket holds up to capacity tokens and refills rate tokens every second. Each call takes one token, and with no token the call must wait or gets 429. So it allows short bursts but keeps the average at the rate. Servers use it to protect themselves, and clients can use it to stay polite.

Lesson: Rate limits: the token bucket

REMEMBER

By default pydantic ignores fields it does not know (like "extra" above). Set model_config = ConfigDict(extra="forbid") to reject them, or strict=True to refuse "7" for an int.

Lesson: Checking the data with pydantic

3.1 Flask basics: routes, templates, forms and JSON

Flask routes, request and response, Jinja templates, forms and sessions, a JSON API and running the app.

KEY IDEAS

- **Routes** @app.route(rule, methods=[...]) or @app.get / post. Converters string, int, float, path, uuid. url_for(endpoint, **values). 404 no match, 405 wrong method.
- **Request** request.args.get(name, default, type=int), request.form, request.get_json(silent=True), request.files, headers, cookies.
- **Responses** str is HTML, dict or list is JSON, (body, status, headers), redirect(url_for(...)), abort(code), make_response, @app.errorhandler(404).
- **Templates and sessions** render_template, {{ }} escaped, {% for %}, filters, extends and block. session is a signed cookie, needs secret_key. flash and get_flashed_messages.
- **Run and test** app.test_client() for tests. flask --app app run --debug only on your computer. gunicorn -w 4 "app:app" in production.

FORMULAS

Requests in progress (Little's law) Exam favourite

$$L = \lambda \times W$$

120 requests/s, 0.2 s each: 24 requests at the same moment, so at least 24 workers or threads.

Lesson: The maths: workers, time and limits

Gunicorn starting point

$$\text{workers} = 2 \times \text{CPU cores} + 1$$

4 cores: 9 worker processes.

Lesson: The maths: workers, time and limits

Throughput of sync workers Exam favourite

$$\text{requests/s} \le \text{workers} / \text{time per request}$$

9 workers, 50 ms each: 9 / 0.05 = 180 requests/s.

Lesson: The maths: workers, time and limits

Python time limit with threads (GIL)

$$\text{requests/s} \le 1 / \text{CPU time per request, per process}$$

10 ms of Python per request: at most 100 requests/s for one process, however many threads.

Lesson: The maths: workers, time and limits

Cookie size

$$\text{base64 makes data } 4 / 3 \text{ as big}$$

A 4,096 byte cookie holds about 3,072 bytes of session data.

Lesson: The maths: workers, time and limits

Error rate

$$\text{rate} = \text{errors} / \text{requests} \times 100\ \%$$

30 errors out of 600 requests: 5 %.

Lesson: The maths: workers, time and limits

TABLES

Table: · http.server · flask · django

	http.server	Flask	Django	FastAPI
Size	Standard library, very basic	Small core, add what you need	Batteries included: ORM, admin, auth, forms	Small core focused on APIs
Routing	Write if statements on self.path	@app.route("/books/<int:id>")	urls.py with path("books/<int:id>/", view)	@app.get("/books/{id}")
Data checks	By hand	By hand or an extension	Forms and serializers	Automatic from type hints (pydantic)
Interface	Its own	WSGI (sync), async views possible	WSGI and ASGI	ASGI (async)
Good for	Learning, tiny tools	Small and medium sites, APIs, prototypes	Big sites with users, admin and a database	Fast JSON APIs with automatic docs

Lesson: What Flask is

Table: converter · matches · gives the view

Converter	Matches	Gives the view
<name> or <string:name>	Any text without a slash	str
<int:id>	Whole numbers such as 42	int
<float:price>	Numbers such as 9.5	float
<path:rest>	Text that may contain slashes	str
<uuid:key>	A UUID	uuid.UUID

Lesson: Routes and URLs

Table: on request · holds · example

On request	Holds	Example
request.args	The query string, like a dict	request.args.get("page", 1, type=int)
request.form	Fields of an HTML form	request.form["user"]
request.get_json()	The JSON body	silent=True gives None instead of an error
request.files	Uploaded files	request.files["photo"].save(path)
request.headers, cookies	Headers and cookies	request.headers["User-Agent"]
request.method, path, url	The method and the address	"POST", "/login"

Lesson: The request and the response

Table: a view returns · flask sends

A view returns	Flask sends
A str	200 with Content-Type text/html
A dict or a list	200 with JSON (keys sorted by default)
(body, status) or (body, status, headers)	Your status and extra headers
redirect(url)	302 with a Location header
abort(code)	Stops at once with that error status
make_response(...)	A Response object you can change: headers, cookies

Lesson: The request and the response

Table: jinja · does · example

Jinja	Does	Example
{{ value }}	Prints a value, HTML escaped	{{ user }}
{% for ... %} ... {% endfor %}	A loop, with loop.index and an else part for empty lists	for book in books
{% if ... %}	A condition	if book.year < 1900
value|filter	Changes a value	upper, length, default("none"), round(2)
{% extends %} and {% block %}	One base page, other pages fill its blocks	extends "base.html"
{% include %}, macros	Reuse a part of a page	include "nav.html"

Lesson: Templates with Jinja

Table: request · view · success · errors

Request	View	Success	Errors
GET /api/books	list_books	200 and a JSON list	
GET /api/books/7	get_book	200 and the book	404 with a JSON error
POST /api/books	create_book	201, the book and Location	422 for bad data
DELETE /api/books/1	delete_book	204 with an empty body	404

Lesson: A JSON API in Flask

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
434	Number of Segments in a String	Easy	str.split() with no argument	Like counting the words a user typed into a search box. split() with no argument skips any amount of spaces, so empty parts never appear.
1662	Check If Two String Arrays are Equivalent	Easy	Join and compare	A page built from template parts must give the same text however it was cut. Join both lists and compare the results.
1544	Make The String Great	Easy	A stack of letters	Cleaning user input with a stack. Push each letter, but pop instead when it is the same letter in the other case as the top.
2696	Minimum String Length After Removing Substrings	Easy	A stack removes pairs as they appear	Removing "AB" and "CD" again and again is the same as a stack that pops when the new letter completes a pair with the top.
648	Replace Words	Medium	Check the shortest roots first	Like a router that finds the shortest registered prefix of a path. Put the roots in a set and try prefixes from short to long.
1209	Remove All Adjacent Duplicates in String II	Medium	A stack of [letter, count]	Keep each letter with how many times it repeats on top of the stack. When the count reaches k, pop the whole group.
820	Short Encoding of Words	Medium	Drop words that are endings of other words	A word that is the end of another word needs no space of its own. Remove every ending of every word from a set, then add len(word) + 1 for each word that is left.
916	Word Subsets	Medium	Merge the needs into one Counter	A word is universal when it has every letter of every word in words2, as often as needed. Merge words2 into one Counter of the largest counts, then check each word once.
30	Substring with Concatenation of All Words	Hard	Cut each window into words and compare Counters	Find every place where all the words appear joined in any order, like spotting a set of template parts in a page. Each window has a fixed length, so cut it into word sized pieces and compare counts.
224	Basic Calculator	Hard	A stack of (result, sign) for each bracket	Templates and config files evaluate small expressions. Walk the text, build numbers digit by digit, and push the result so far when a bracket opens.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Build links with `url_for("hello", name="Bob")` and never by hand. If you rename a URL later, every link still works, and url_for escapes special characters for you. Extra arguments become the query string, as with page=3 above.

Lesson: Routes and URLs

IN THE EXAM

A route answers only GET (and HEAD, OPTIONS) unless you pass methods=. A wrong method gives **405 Method Not Allowed**, a URL that matches no rule gives **404 Not Found**.

Lesson: Routes and URLs

REMEMBER

`request` looks like a global variable, but each request in each thread sees its own. Flask does this with context variables, so two users at the same moment never see each other's data.

Lesson: The request and the response

COMMON MISTAKE

Jinja **escapes** every value by default in .html templates, so a name such as <script> is shown as text and never runs. This stops cross site scripting (XSS). Only mark text as safe with |safe when you made it yourself.

Lesson: Templates with Jinja

COMMON MISTAKE The session cookie is signed, NOT encrypted: the example decodes it and reads the user name. Never put secrets in it, keep secret_key secret and long, and read it from an environment variable.

Lesson: Forms, sessions and flash messages

IN THE EXAM Real forms also need protection against cross site request forgery (CSRF), so another website cannot post forms as your user. The extension Flask-WTF adds a hidden token to every form and checks it.

Lesson: Forms, sessions and flash messages

REMEMBER @app.get, @app.post, @app.put, @app.patch and @app.delete are short forms of @app.route with one method. They make an API easier to read.

Lesson: A JSON API in Flask

COMMON MISTAKE A global dict like BOOKS is fine for learning, but it is lost when the server restarts, and each Gunicorn worker process has its own copy. Real apps keep data in a database, as the next chapter shows.

Lesson: A JSON API in Flask

3.2 Real Flask apps: structure, database, login and tests

Application factory, blueprints, sqlite3, login with hashed passwords, pytest, config and deployment.

KEY IDEAS

- **Structure** create_app(settings) returns a new app. Blueprint(name, __name__, url_prefix=...), app.register_blueprint(bp). url_for("bp.view"). current_app, not a global app.
- **Database** get_db() on g, sqlite3.Row, ? placeholders, commit(), app.teardown_appcontext(close_db), @app.cli.command("init-db").
- **Auth** generate_password_hash / check_password_hash (salted, slow). session.clear() then session["user_id"]. before_app_request loads g.user. login_required with functools.wraps.
- **Tests** pytest fixture in conftest.py builds create_app({"TESTING": True}) and returns app.test_client(). app.test_cli_runner() for commands.
- **Production** Config from environment (from_prefixed_env), app.logger, errorhandler passing HTTPException through, gunicorn "shop:create_app()" behind HTTPS.

FORMULAS

Guesses per second against a slow hash

Exam favourite

$$\text{guesses/s} = \text{raw hashes per second} / \text{iterations}$$

10 million SHA-256 per second, 1,000,000 PBKDF2 iterations: 10 guesses per second.

Lesson: The maths: hashing cost, commits and capacity

Number of passwords

$$N = (\text{letters in the alphabet})^{\text{length}}$$

8 lower case letters: $26^8 \approx 2.09 \times 10^{11}$.

Lesson: The maths: hashing cost, commits and capacity

Average time to guess Exam favourite

$$\text{time} \approx N / 2 / \text{guesses per second}$$

$2.09 \times 10^{11} / 2 / 10$ per second ≈ 331 years.

Lesson: The maths: hashing cost, commits and capacity

Commits

$$\text{many commits cost n} \times \text{commit time, one transaction costs one}$$

200 inserts with a 5 ms commit each: 1,000 ms. In one transaction: about 15 ms.

Lesson: The maths: hashing cost, commits and capacity

Salt collisions (birthday bound)

$$n \approx 1.18 \times \sqrt{(\text{number of salts})} \text{ for a 50 \% chance}$$

32 bit salts: 1.18 × 65,536 ≈ 77,000 users. Werkzeug uses 16 random characters, far more.

Lesson: The maths: hashing cost, commits and capacity

Test suite time

$$\text{time} = \text{tests} \times \text{time per test}$$

120 tests at 15 ms: 1.8 s, fast enough to run on every save.

Lesson: The maths: hashing cost, commits and capacity

TABLES

Table: · one app.py file · factory and blueprints · django project

	One app.py file	Factory and blueprints	Django project
Where code lives	Everything in one file	A package: __init__.py, auth.py, books.py, db.py	Apps with models.py, views.py, urls.py
Tests	Share one global app	Each test builds its own app	A test runner with a test database
Settings	Often written in the code	Defaults, then settings, then environment	settings.py, often read from the environment
Database and login	By hand	sqlite3 by hand, or Flask-SQLAlchemy and Flask-Login	Built in: ORM, migrations, auth, admin
Good for	Demos	Most Flask apps	Big apps that need everything

Lesson: From one file to a real app

Table: file or folder · holds

File or folder	Holds
shop/__init__.py	create_app(): config, database setup, register_blueprint for each part
shop/db.py	get_db(), close_db(), the init-db command
shop/auth.py	Blueprint auth: register, login, logout, login_required
shop/books.py	Blueprint books: the pages of the shop
shop/api.py	Blueprint api with url_prefix="/api": the JSON API
shop/templates/, shop/static/	Jinja templates, CSS, JavaScript and images
instance/	Files that belong to one installation: the SQLite file, a local config. Not in git
tests/conftest.py, tests/test_*.py	pytest fixtures and tests
pyproject.toml	The package name and its dependencies

Lesson: The application factory and blueprints

Table: · sqlite3 by hand · flask-sqlalchemy · django orm

	sqlite3 by hand	Flask-SQLAlchemy	Django ORM
You write	SQL text with ? placeholders	Python classes, queries as method calls	Python classes, queries as method calls
Changing tables later	Write ALTER TABLE yourself	Flask-Migrate (Alembic) makes migration files	makemigrations and migrate are built in
Other databases	Change the code for PostgreSQL	Change one URL in the config	Change the settings
Good for	Learning SQL, small apps	Most Flask apps that grow	Django apps

Lesson: A database with sqlite3

Table: rule · why

Rule	Why
Store generate_password_hash(...), never the password	A stolen database must not reveal passwords
Use a slow hash with a salt (scrypt or PBKDF2, Werkzeug picks scrypt by default)	Slow makes guessing expensive, the salt makes the same password give different hashes
Say "wrong name or password", not which one	Attackers cannot test which names exist
session.clear() before storing the user	No data from before the login survives (session fixation)
Set SESSION_COOKIE_SECURE and SESSION_COOKIE_HTTPONLY in production	The cookie only travels over HTTPS and JavaScript cannot read it
Limit login attempts, for example 5 a minute per user	Stops fast guessing

Lesson: Users, passwords and login_required

Table: kind of test · checks · tool

Kind of test	Checks	Tool
Unit test	One function, such as bad_book(data)	pytest
Route test	One URL: status code, JSON, redirects	app.test_client()
Command test	flask init-db and other commands	app.test_cli_runner()
End to end test	A real browser clicking through pages	Playwright or Selenium (see those subjects)
Coverage	Which lines the tests ran	coverage run -m pytest, then coverage report

Lesson: Testing a Flask app

Table: source · example · use it for

Source	Example	Use it for
Defaults in create_app	app.config.from_mapping(PAGE_SIZE=10)	Values that are safe everywhere
Settings passed to create_app	create_app({"TESTING": True})	Tests
A file in instance/	app.config.from_pyfile("config.py", silent=True)	One installation
Environment variables	FLASK_SECRET_KEY=... with from_prefixed_env()	Secrets and production values

Lesson: Configuration, errors and logging

Table: way to host · you manage · example

Way to host	You manage	Example
A platform (PaaS)	Only the code and settings	Render, Railway, Fly.io, Google App Engine
A container	A Dockerfile, the platform runs it	Docker image on Cloud Run, AWS ECS, Kubernetes
Your own server	Everything: Linux, nginx, Gunicorn, updates	A virtual machine with systemd

Lesson: Deploying a Flask app

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
595	Big Countries	Easy	WHERE with OR	A country is big when its area is at least 3,000,000 or its population at least 25,000,000. One WHERE with OR does it.
1757	Recyclable and Low Fat Products	Easy	WHERE with AND	Both columns must be "Y". The kind of filter a shop page runs for every search.
584	Find Customer Referee	Easy	NULL needs IS NULL	referee_id != 2 is not true for NULL, because NULL compared with anything is unknown. Add OR referee_id IS NULL.
1148	Article Views I	Easy	DISTINCT with a column compared to another	Authors who viewed their own article: author_id = viewer_id. DISTINCT removes repeats.
570	Managers with at Least 5 Direct Reports	Medium	GROUP BY with HAVING, then join back	Count the people per managerId, keep the ones with 5 or more, and join with the table again to get the names.
1934	Confirmation Rate	Medium	LEFT JOIN, AVG of a CASE, IFNULL	Every user who signed up must appear, even with no confirmations, so use a LEFT JOIN. The rate is the average of 1 for confirmed and 0 otherwise.
180	Consecutive Numbers	Medium	Join the table with itself twice	Pair each row with the next two ids. If all three have the same number, that number appears at least three times in a row.
550	Game Play Analysis IV	Medium	First day per player, then look for the next day	Find each player's first login with MIN, check whether they came back the day after, and divide by the number of players.
601	Human Traffic of Stadium	Hard	Window function: id minus row number groups a run	Keep the days with 100 or more people. Within a run of consecutive ids, id minus ROW_NUMBER() stays the same, so it names the run. Keep runs of 3 or more.
432	All O`one Data Structure	Hard	A dict of counts (simple version)	Count page views per key and ask for the most and least viewed key. This simple version keeps a dict and uses max and min. The O(1) version keeps a linked list of buckets, one bucket for each count.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Inside a blueprint, link with the blueprint name: url_for("api.status") or url_for(".status") for the same blueprint. Use **current_app** instead of a global app variable, because with a factory there is no global app.

Lesson: The application factory and blueprints

REMEMBER

login_required is a normal decorator (Advanced step 2). functools.wraps keeps the view name, which Flask needs, because the endpoint name comes from the function name.

Lesson: Users, passwords and login_required

IN THE EXAM

For bigger apps, Flask-Login manages the logged in user and "remember me", and Flask-WTF adds CSRF tokens to forms. They do the same jobs as the code here, with more cases covered.

Lesson: Users, passwords and login_required

REMEMBER

With pytest the helper becomes a fixture in tests/conftest.py, and pytest passes it to every test that has a parameter called client:

```
import pytest
from notes import create_app

@pytest.fixture
def client():
    app = create_app({"TESTING": True})
    return app.test_client()
```

Then run `python3 -m pytest -q`. A real run of exactly this printed "2 passed in 0.18s".

Lesson: Testing a Flask app

REMEMBER

Flask turns an exception in a view into a 500 answer. An errorhandler(Exception) can log it and send a friendly message, but must pass HTTPExceptions such as 404 through unchanged, as the example does. Use app.logger, never print, so logs get levels and can go to files or a log service.

Lesson: Configuration, errors and logging

STEP 4 Web and APIs · Django: the framework with everything built in

4.1 Django basics: project, URLs, views, templates, models and admin

Project and apps, urlpatterns, views, templates, models with QuerySets, and the admin site.

KEY IDEAS

- **Project** django-admin startproject mysite, python manage.py startapp books, add "books" to INSTALLED_APPS. manage.py runserver, makemigrations, migrate, createsuperuser, shell, check --deploy.
- **URLs and views** urlpatterns = [path("books/<int:book_id>/", views.book_detail, name="book-detail")], include("books.urls"). A view takes request and returns HttpResponse or JsonResponse. Http404, redirect, reverse.
- **Templates** render(request, "books/list.html", context). {{ value|filter }}, {% for %}{% empty %}{% endfor %}, {% if %}, {% url "name" arg %}, {% extends %} with {% block %}, {% csrf_token %} in POST forms. Values are escaped.
- **Models** class Book(models.Model) with CharField, DecimalField, ForeignKey(on_delete=...). Book.objects.create / all / filter(year__gte=1900) / get / update / delete. related_name for the other side.
- **Admin** @admin.register(Book) class BookAdmin(admin.ModelAdmin) with list_display, list_filter, search_fields. path("admin/", admin.site.urls). Only staff users can log in.

FORMULAS

URL matching

$$\text{checks for a URL} \le \text{number of patterns}$$

Django tries patterns from the top: with 40 patterns the worst case is 40 checks, still well under a millisecond.

Lesson: The maths: URL matching, pages, queries and hashing

Number of pages Exam favourite

$$\text{pages} = \lceil n / \text{per page} \rceil$$

1,234 books at 50 a page: ⌈24.68⌉ = 25 pages.

Lesson: The maths: URL matching, pages, queries and hashing

OFFSET of a page

Exam favourite

$$\text{OFFSET} = (\text{page} - 1) \times \text{per page}$$

Page 3 at 50 a page: LIMIT 50 OFFSET 100.

Lesson: The maths: URL matching, pages, queries and hashing

The N + 1 problem

Exam favourite

$$\text{queries} = 1 + n$$

A list of 50 books that reads book.author in the template: 1 + 50 = 51 queries. select_related makes it 1.

Lesson: The maths: URL matching, pages, queries and hashing

Hashing cost

$$\text{guesses/s} = \text{hashes per second} / \text{iterations}$$

Django 6.1 uses PBKDF2 with 1,500,000 iterations: 10^9 hashes per second give about 667 guesses per second.

Lesson: The maths: URL matching, pages, queries and hashing

Workers needed (Little's law)

$$\text{busy workers} = \text{requests per second} \times \text{seconds per request}$$

200 requests per second at 0.05 s each keep 10 workers busy.

Lesson: The maths: URL matching, pages, queries and hashing

TABLES

Table: · flask · django · fastapi

	Flask	Django	FastAPI
Style	Small core, add what you need	Everything built in	Built for JSON APIs, uses type hints
Database	Your choice (sqlite3, SQLAlchemy)	The Django ORM with migrations	Your choice (SQLAlchemy, SQLModel)
Users and admin	Extensions such as Flask-Login	Built in: auth and a full admin site	Write them or add libraries
HTML pages	Jinja templates	Django templates	Possible, but not its focus
Good for	Small apps, services, learning	Shops, news sites, internal tools, big projects	Fast APIs with automatic docs
Used by	Pinterest (early), many services	Instagram, Mozilla, Disqus	Many new API services

Lesson: What Django is and why it has so much built in

Table: file · what it is for

File	What it is for
manage.py	Runs every command: runserver, startapp, makemigrations, migrate, createsuperuser, shell, test
mysite/settings.py	All settings: INSTALLED_APPS, MIDDLEWARE, DATABASES, TEMPLATES, DEBUG, SECRET_KEY, ALLOWED_HOSTS
mysite/urls.py	The first URL list. It sends parts of the site to the apps with include()
mysite/wsgi.py, asgi.py	The entry point for Gunicorn (WSGI) or Uvicorn (ASGI) in production
books/models.py	The models of the app, one class for each table
books/views.py, books/urls.py	The views of the app and their URL patterns (you make urls.py yourself)
books/admin.py	Which models show up in the admin site, and how
books/migrations/	The migration files that makemigrations writes. Keep them in git
books/templates/books/	The HTML templates of the app. The second books folder stops name clashes between apps

Lesson: A project and its apps

Table: path converter · matches · gives the view

Path converter	Matches	Gives the view
<str:name> (the default)	Any text without a slash	str
<int:book_id>	0 or a positive whole number	int
<slug:slug>	Letters, digits, hyphens and underscores, such as dune-messiah	str
<uuid:token>	A UUID such as 075194d3-6885-417e-a8a8-6c931e272f00	uuid.UUID
<path:file>	Any text, slashes too	str
re_path(r"...")	A regular expression, for special cases	str for each group

Lesson: URLs and views

Table: · function based view · class based view

	Function based view	Class based view
Looks like	def book_list(request): ...	class BookList(ListView): model = Book
HTTP methods	if request.method == "POST": ... or @require_http_methods	A method for each: get(), post()
Reuse	Helpers and decorators	Inheritance and mixins, many generic views ready made
Good for	Clear, special logic	Standard pages: lists, details, create, update, delete

Lesson: URLs and views

Table: template code · what it does

Template code	What it does
`{{ book.title }}`	Shows a value. The dot tries a dict key, then an attribute, then a list index
`{{ name|default:"guest"|title }}`	Filters change a value, and you can chain them
`{% for book in books %} ... {% empty %} ... {% endfor %}`	A loop, with a part for an empty list. forloop.counter counts from 1
`{% if book.year < 1900 %} ... {% endif %}`	A condition
`{% url "book-detail" book.id %}`	Builds a link from the URL name, like reverse()
`{% extends "base.html" %}` and `{% block content %}`	Template inheritance: a page fills the blocks of a base layout
`{% csrf_token %}`	The hidden token that every POST form needs
`{# a comment #}`	A comment that never reaches the browser

Lesson: Templates

Table: · django template language · jinja (used by flask)

	Django template language	Jinja (used by Flask)
Calling functions	No arguments allowed, keeps logic out of the page	Full expressions such as f(x, y)
Speed	Fast enough for most sites	Faster for heavy templates
Escaping	On by default	On by default in Flask
In Django	The default	Possible, as a second backend in TEMPLATES

Lesson: Templates

Table: field · column in sqlite · use it for

Field	Column in SQLite	Use it for
CharField(max_length=200)	varchar(200)	Short text such as a title
TextField()	text	Long text such as a description
IntegerField(), PositiveIntegerField()	integer	Whole numbers
DecimalField(max_digits=6, decimal_places=2)	decimal	Money. Never use FloatField for money
BooleanField(default=True)	bool	Yes or no
DateField(), DateTimeField(auto_now_add=True)	date, datetime	Dates. auto_now_add fills in the time of creation
ForeignKey(Author, on_delete=models.CASCADE)	author_id plus an index	Many books for one author
EmailField(), URLField(), SlugField()	varchar	Text with a check in forms

Lesson: Models: Python classes that become tables

Table: modeladmin option · what it changes

ModelAdmin option	What it changes
list_display = ["title", "year"]	The columns of the list page
list_filter = ["in_stock"]	A filter box at the side
search_fields = ["title"]	A search box. ?q=dune becomes WHERE title LIKE %dune%
ordering = ["-year"]	The order of the list
list_per_page = 50	How many rows on each page
readonly_fields, fields, fieldsets	What the edit form shows and how
inlines = [ReviewInline]	Edit related rows, such as reviews, on the same page

Lesson: The admin site

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
257	Binary Tree Paths	Easy	Depth first search that carries the path so far	Like listing every URL of a project whose urls.py files include each other. Walk down the tree, add each value to the path, and save the path at every leaf.
228	Summary Ranges	Easy	Walk once and close a range when the next number is not one more	The admin and paginators show ranges such as 1-3. Keep where the current range starts and close it when the numbers stop following each other.
599	Minimum Index Sum of Two Lists	Easy	A dict from name to position	Like joining two lists of favourites in the ORM. Store the positions of the first list in a dict, then look up each name of the second list once.
1441	Build an Array With Stack Operations	Medium	Write down the operations, like makemigrations does	makemigrations writes a list of operations that turn the old state into the new one. Here: read the numbers 1, 2, 3 ... in order, Push each one, and Pop it again when it is not in target.
1003	Check If Word Is Valid After Substitutions	Medium	A stack, the same way template tags must close	A Django template is only valid when every {% block %} has its {% endblock %} in the right order. Here "abc" plays that part: push letters and remove "abc" whenever it appears on top.
1487	Making File Names Unique	Medium	A dict with the next free number for each name	When two users upload photo.jpg, Django storage saves the second one under a new name. Keep the next number to try for each name, so no name is checked again and again.
1048	Longest String Chain	Medium	Sort by length, then dynamic programming on a dict	Like a migration history where every migration adds one field. Sort the words by length and, for each word, try removing one letter to find the best chain that ends just before it.
1912	Design Movie Rental System	Hard	Sorted lists play the part of ORDER BY with an index	This is a small ORM: one table of copies, a query "cheapest 5 shops for a movie" and a report "cheapest 5 rented copies". Keep sorted lists of (price, shop) and (price, shop, movie) so each query only reads the first 5.
736	Parse Lisp Expression	Hard	A recursive parser with a stack of scopes	Like the context of a Django template: {% with %} adds names that live only inside its block, and inner names hide outer ones. Parse the tokens recursively and pass a new dict of names into every let.
472	Concatenated Words	Hard	Word break with a set and memory	Like checking that a URL slug such as "redbookshop" is made only of known words. For each word, test every cut point and remember which endings were already solved.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Django calls its pattern **MTV**: the **Model** holds the data, the **Template** shows it, and the **View** decides what to show. It is the same idea as MVC, only the names differ: the Django view does the job of an MVC controller.

Lesson: What Django is and why it has so much built in

REMEMBER

A **project** is the whole website with its settings. An **app** is one feature, such as books, accounts or payments. A good app does one thing, so you could even reuse it in another project.

Lesson: A project and its apps

IN THE EXAM

The test client does not check CSRF tokens unless you ask it to, so the POST in the example works without one. In a real project CsrfViewMiddleware is on, and every POST form needs {% csrf_token %}. The animation above shows what happens without it.

Lesson: URLs and views

REMEMBER

Every value is **escaped**, so the script tag in the note shows up as harmless text. Only mark text as safe (the safe filter or mark_safe) when you made it yourself, never for text a user typed.

Lesson: Templates

IN THE EXAM

In a project you never call schema_editor yourself. You change models.py, run makemigrations to write a migration file, and migrate to run it. Chapter 4.2 goes much deeper into the ORM and migrations.

Lesson: Models: Python classes that become tables

REMEMBER

The admin is for trusted staff, not for customers. Use a long random password for the superuser, keep DEBUG off in production, and consider a less obvious address than /admin/.

Lesson: The admin site

4.2 Django in depth: ORM, migrations, forms, users and tests

Q, F, aggregate, annotate, the N + 1 problem, migrations, forms, users and permissions, transactions and TestCase.

KEY IDEAS

- **QuerySets** Lazy until read. filter, exclude, order_by, get, first, exists, count. Lookups: __gte, __icontains, __in, author__name. Q(a) | Q(b), ~Q(c). F("col") inside filter and update.
- **Aggregation and speed** aggregate(Avg("price")) gives a dict, annotate(n=Count("books")) adds n per row. select_related for ForeignKey, prefetch_related for ManyToMany. only, values_list, bulk_create, indexes.
- **Migrations** makemigrations then migrate. Commit the files. New fields need a default or null=True. Data migrations use RunPython with apps.get_model. migrate app 0001 goes back. sqlmigrate shows the SQL.
- **Forms** form = BookForm(request.POST), form.is_valid(), cleaned_data, form.save(). clean_<field> for one field, clean() with add_error for several. ModelForm with Meta.model and Meta.fields.
- **Users, transactions, tests** create_user, check_password, authenticate, login_required, permission_required, Group. transaction.atomic() all or nothing. TestCase rolls back each test, setUpTestData, self.client, assertNumQueries.

FORMULAS

Queries in a loop Exam favourite

$$\text{N + 1 without help, 1 with select_related, 2 with prefetch_related}$$

20 books: 21, 1 and 2 queries.

Lesson: The maths: query counts, indexes and transactions

Time of a page Exam favourite

$$\text{time} \approx \text{queries} \times \text{round trip} + \text{work}$$

21 queries at 1.5 ms: 31.5 ms. 1 query: 1.5 ms plus a bigger row.

Lesson: The maths: query counts, indexes and transactions

Search without an index

$$\text{rows read} = n$$

1,000,000 books: the database reads all of them.

Lesson: The maths: query counts, indexes and transactions

Search with a B-tree index Exam favourite

$$\text{steps} \approx \log_b(n)$$

With about 100 keys per page: $\log_{100}(1{,}000{,}000) = 3$ page reads.

Lesson: The maths: query counts, indexes and transactions

Rows from a join table (many to many)

$$\text{pairs} = \text{books} \times \text{average tags}$$

20 books with 2 tags each: 40 rows in book_tags.

Lesson: The maths: query counts, indexes and transactions

Weighted average (LeetCode 1251)

$$\text{average} = \Sigma(\text{price} \times \text{units}) \,/\, \Sigma\text{units}$$

(5 × 100 + 20 × 15) / 115 = 800 / 115 ≈ 6.96.

Lesson: The maths: query counts, indexes and transactions

TABLES

Table: · raw sql (sqlite3) · django orm · sqlalchemy

	Raw SQL (sqlite3)	Django ORM	SQLAlchemy
You write	SQL text with ? placeholders	Book.objects.filter(...)	select(Book).where(...) or the ORM
Changes to tables	ALTER TABLE by hand	makemigrations and migrate	Alembic
Joins	Written by hand	author__name follows the ForeignKey	join() or relationships
Hard reports	Full SQL power	annotate, Subquery, Window, or raw() when needed	Very close to SQL
Used with	Small scripts	Django	Flask, FastAPI, scripts

Lesson: The ORM: asking the database in Python

Table: tool · example · sql it becomes

Tool	Example	SQL it becomes
filter and exclude	filter(year__gte=1900).exclude(stock=0)	WHERE year >= 1900 AND NOT stock = 0
Lookups	title__icontains="dune", year__in=[1965, 1969], author__country="UK"	LIKE, IN, a JOIN to author
Q objects	Q(a) | Q(b), ~Q(c)	OR and NOT
F expressions	filter(price__gt=F("stock") * 2), update(stock=F("stock") + 10)	Compare or change columns inside the database
aggregate	aggregate(Avg("price"))	One answer for the whole QuerySet: a dict
annotate	annotate(n=Count("books"))	One extra value per row, with GROUP BY
values, values_list	values("title", "year"), values_list("title", flat=True)	Only these columns, as dicts or tuples
Relations	ForeignKey, ManyToManyField, OneToOneField	author_id column, a join table, a unique author_id

Lesson: QuerySets: filter, Q, F, aggregate and annotate

Table: · select_related("author") · prefetch_related("tags")

	select_related("author")	prefetch_related("tags")
Works for	ForeignKey and OneToOne (one related row)	ManyToMany and the reverse of a ForeignKey (many rows)
How	One query with a JOIN	One extra query with WHERE id IN (...), joined in Python
Queries for 20 books	1	2
Watch out	Joining many tables makes wide rows	Filtering inside needs a Prefetch object

Lesson: Fast queries: the N + 1 problem

Table: more ways to make queries cheaper · why

More ways to make queries cheaper	Why
only("title") or defer("description")	Read fewer columns
values_list("title", flat=True)	No model objects at all, only the values
exists() instead of len(qs) > 0	The database stops at the first row
count() instead of len(qs)	SELECT COUNT(*), not every row
bulk_create, bulk_update	Many rows in one query
db_index=True or Meta.indexes	An index makes filters and order_by on that column fast
iterator()	Streams a huge QuerySet without keeping it all in memory

Lesson: Fast queries: the N + 1 problem

Table: command · what it does

Command	What it does
makemigrations	Compares the models with the migration files and writes a new file with the changes
makemigrations --empty --name fill_slugs	An empty migration for your own code, such as a data migration
migrate	Runs every migration that is not applied yet, in dependency order
migrate books 0001	Goes back: undoes every later migration of books
sqlmigrate books 0002	Shows the SQL of a migration without running it
showmigrations	Lists the migrations with [X] for applied
makemigrations --check	Fails when a model change has no migration yet, useful in CI

Lesson: Migrations: changing tables safely

Table: step · what runs · where your code goes

Step	What runs	Where your code goes
1. Bind	form = BookForm(request.POST)	In the view
2. Field checks	to_python, required, max_length, min_value, unique (ModelForm)	Field options
3. One field	clean_isbn() after the field checks pass	def clean_<field>(self): return the value
4. Many fields	clean() with all cleaned values	def clean(self): use self.add_error
5. Result	is_valid() is True: cleaned_data, or save() for a ModelForm	Redirect after saving
6. Errors	form.errors, shown next to each field in the template	Render the form again

Lesson: Forms: checking what users send

Table: · forms.form · forms.modelform

	forms.Form	forms.ModelForm
Fields come from	You list them	The model (Meta.model, Meta.fields)
Saves to the database	No, you do it	form.save() makes or updates the row
Model rules	Not used	max_length, unique and choices are checked for you
Good for	Search, contact, login	Create and edit pages for a model

Lesson: Forms: checking what users send

Table: part · what django gives you

Part	What Django gives you
User model	username, email, password (a hash), is_active, is_staff, is_superuser, groups, user_permissions
Passwords	create_user and set_password store PBKDF2 with 1,500,000 iterations and a random salt, check_password compares
Login	authenticate(username=..., password=...) then login(request, user), or the ready LoginView
In views	@login_required, @permission_required, LoginRequiredMixin, PermissionRequiredMixin
In templates	user.is_authenticated, user.username, perms.books.change_book
Permissions	add, change, delete and view for every model, grouped with Group
Your own user	AUTH_USER_MODEL = "accounts.User", set before the first migrate

Lesson: Users, logins and permissions

Table: tool · use it for

Tool	Use it for
transaction.atomic()	Changes that must all happen or none: a with block or a decorator
F("balance") + amount	Change a value inside the database, safe when two requests run at the same time
select_for_update()	Lock rows until the transaction ends (PostgreSQL, MySQL)
TestCase	Every test runs in a transaction that is rolled back, so tests never see each other's data
setUpTestData	Rows made once for the whole class, which is fast
self.client	The test client inside a TestCase: get, post, login, force_login
assertContains, assertRedirects, assertNumQueries	Check a page, a redirect, or the number of queries
python manage.py test	Finds tests.py and test_*.py, makes a test database, runs everything

Lesson: Transactions and tests

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1683	Invalid Tweets	Easy	annotate(Length(...)) then filter	A computed column: annotate each tweet with the length of its text, then keep the ones longer than 15.
1378	Replace Employee ID With The Unique Identifier	Easy	Subquery with OuterRef, a LEFT JOIN without a ForeignKey	The two tables share an id but have no ForeignKey. A Subquery that looks up the unique id for each employee works like a LEFT JOIN, and gives None when there is no match.
1068	Product Sales Analysis I	Easy	Follow a ForeignKey inside values()	With a ForeignKey, product__product_name in values() makes Django write the JOIN for you.
1661	Average Time of Process per Machine	Easy	Conditional aggregation with Sum(Case(When(...)))	Group by machine. Add the end times and subtract the start times with Case and When, then divide by the number of processes.
1280	Students and Examinations	Easy	Count per group, then fill in the zeros	Every student must appear with every subject, even with 0 exams. Count the exams per (student, subject) with values() and annotate(), then loop over all pairs.
1251	Average Selling Price	Easy	Subquery for the price on the sale date, then a weighted average	Each sale needs the price that was valid on its date. A Subquery with OuterRef and two date lookups finds it, then Sum(price times units) / Sum(units) gives the weighted average.
1211	Queries Quality and Percentage	Easy	Avg of a computed value and Count with a filter	Group by query name. Avg(rating / position) gives the quality, and Count with filter=Q(rating__lt=3) counts the poor results.
607	Sales Person	Easy	exclude with a subquery of ids	Find the sales people with an order for the company RED, as a QuerySet of ids, and exclude them. Django puts the inner query into one SQL statement with NOT IN.
1193	Monthly Transactions I	Medium	Group by a computed month, Count and Sum with filters	Annotate each row with its month (the first 7 characters of the date), group by month and country, and use filter=Q(state="approved") inside Count and Sum.
1174	Immediate Food Delivery II	Medium	Subquery for the first order of each customer	Find each customer's first order date with a Subquery that sorts by date and takes one row, keep only those first orders, then count how many were delivered on the day they were ordered.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

The QuerySet keeps the rows it read. Looping twice over the same QuerySet runs one query, but calling filter() again builds a new QuerySet and runs new SQL. Save a QuerySet in a variable when you need it twice.

Lesson: QuerySets: filter, Q, F, aggregate and annotate

IN THE EXAM

In tests, assertNumQueries(n) fails when a change adds queries, so an N + 1 problem cannot sneak back in. In development, django-debug-toolbar shows every query of a page.

Lesson: Fast queries: the N + 1 problem

REMEMBER

SQLite cannot change most column types in place, so the SQL above makes a new table, copies the rows and renames it. PostgreSQL runs a short ALTER TABLE instead. Django picks the right way for each database.

Lesson: Migrations: changing tables safely

IN THE EXAM

The usual view: if request.method == "POST", bind and check the form, save and redirect when it is valid. Otherwise make an empty form. Either way render the template, which shows the errors. In the template put {% csrf_token %} inside the form tag.

Lesson: Forms: checking what users send

REMEMBER

Start every new project with a custom user model (class User(AbstractUser): pass) and AUTH_USER_MODEL. Changing it later, when tables already point to auth.User, is a painful migration.

Lesson: Users, logins and permissions

IN THE EXAM

login_required sends a person who is not logged in to LOGIN_URL with ?next=, as the 302 above shows. permission_required with raise_exception=True gives 403 instead, which is right for a logged in user who simply may not do this.

Lesson: Users, logins and permissions

IN THE EXAM

Look at the query count: inside a test the atomic() of transfer becomes SAVEPOINT and RELEASE, because TestCase already opened a transaction around the test. Outside tests it is BEGIN and COMMIT.

Lesson: Transactions and tests

STEP 5 Web and APIs · Building APIs: Django REST framework and FastAPI

5.1 Django REST framework: JSON APIs on top of Django

Serializers, ViewSets and routers, pagination and filters, authentication, permissions, throttles and API tests.

KEY IDEAS

- **Serializers** class BookSerializer(serializers.ModelSerializer) with Meta.model, Meta.fields. serializer.data, Serializer(data=...).is_valid(raise_exception=True), validated_data, save(). validate_<field>, validate, SerializerMethodField, read_only.
- **Views and routers** @api_view(["GET", "POST"]) with Response(data, status=201). class BookViewSet(viewsets.ModelViewSet) with queryset and serializer_class. router.register("books", BookViewSet). @action. get_queryset, perform_create.
- **Security** DEFAULT_AUTHENTICATION_CLASSES (Token, Session, JWT). permission_classes: IsAuthenticated, IsAuthenticatedOrReadOnly, has_object_permission. 401 no credentials, 403 not allowed. Throttles answer 429 with Retry-After.
- **Lists** DEFAULT_PAGINATION_CLASS PageNumberPagination with PAGE_SIZE: count, next, previous, results. SearchFilter ?search=, OrderingFilter ?ordering=-year. Always order the queryset. CursorPagination for feeds.
- **Tests and design** APITestCase, self.client.post(url, data, format="json"), force_authenticate, assertNumQueries. Nouns in URLs, right status codes, one error shape, versioning, idempotent retries.

FORMULAS

Number of pages — Exam favourite

$$\text{pages} = \lceil \text{count} / \text{page size} \rceil$$

5 books at 2 a page: ⌈2.5⌉ = 3 pages.

Lesson: The maths: pages, limits and payload size

Requests allowed — Exam favourite

$$\text{rate} = \text{requests} / \text{window}$$

3/minute: the 4th request in 60 s gets 429.

Lesson: The maths: pages, limits and payload size

Wait time in a sliding window

Exam favourite

$$\text{Retry-After} = \text{oldest} + \text{window} - \text{now}$$

Requests at 0, 10 and 20 s, a 4th at 30 s: 0 + 60 − 30 = 30 s.

Lesson: The maths: pages, limits and payload size

Requests per day at a limit

$$\text{per day} = \text{rate} \times \text{windows per day}$$

100/hour: 100 × 24 = 2,400 a day.

Lesson: The maths: pages, limits and payload size

Payload size

$$\text{bytes} \approx \text{rows} \times \text{bytes per row}$$

20 books of 120 bytes: 2,400 bytes. All 50,000: 6 MB.

Lesson: The maths: pages, limits and payload size

Requests to read everything

$$\text{requests} = \lceil \text{count} / \text{page size} \rceil$$

50,000 rows at 100 a page: 500 requests.

Lesson: The maths: pages, limits and payload size

TABLES

Table: · plain django views · django rest framework · fastapi

	Plain Django views	Django REST framework	FastAPI
Input checks	By hand or with forms	Serializers	Pydantic models from type hints
CRUD endpoints	Write each view	ModelViewSet and a router	Write each function
Logins, permissions, limits	Django sessions only	Token, session, JWT (add on), permissions, throttles	Dependencies you write or add
Docs	None	Browsable API, OpenAPI with an add on	OpenAPI and Swagger UI built in
Good for	One or two JSON endpoints	APIs on top of a Django project and its models	New API only services, async work

Lesson: What Django REST framework adds

Table: serializer part · what it does

Serializer part	What it does
ModelSerializer with Meta.model and Meta.fields	Makes the fields from the model, with its rules
serializer.data	Object to dict, ready to become JSON
Serializer(data=...).is_valid()	Checks incoming data. errors holds the messages, validated_data the clean values
validate_<field>(self, value)	A rule for one field
validate(self, data)	A rule for several fields. Errors go to non_field_errors
SerializerMethodField	A read only value from a get_<name> method
read_only=True, write_only=True	Shown but never written, or accepted but never shown (passwords)
save()	Calls create() or update(), which you can override

Lesson: Serializers and API views

Table: http request · viewset action · answer

HTTP request	ViewSet action	Answer
GET /api/books/	list	200, a page of books
POST /api/books/	create	201, the new book
GET /api/books/7/	retrieve	200 or 404
PUT /api/books/7/	update	200, every field replaced
PATCH /api/books/7/	partial_update	200, only the fields sent
DELETE /api/books/7/	destroy	204, empty body
GET /api/books/oldest/	@action(detail=False)	Your own extra endpoint

Lesson: ViewSets, routers, filters and pages

Table: · apiview or @api_view · generic views · viewsets

	APIView or @api_view	Generic views	ViewSets
You write	Each method yourself	queryset and serializer_class	queryset and serializer_class
Example	def get(self, request)	ListCreateAPIView	ModelViewSet
URLs	path() for each	path() for each	One router.register
Good for	Special endpoints	A few endpoints	Full CRUD of a model

Lesson: ViewSets, routers, filters and pages

Table: setting or class · job

Setting or class	Job
TokenAuthentication	Reads "Authorization: Token ..." and finds the user
SessionAuthentication	Uses the Django login cookie, for a front end on the same site (needs CSRF)
JWT (djangorestframework-simplejwt)	Signed tokens that expire, no database lookup per request
IsAuthenticated, IsAuthenticatedOrReadOnly, IsAdminUser	Who may use the view
has_object_permission	Rules for one object, such as "only the owner may change it"
AnonRateThrottle, UserRateThrottle	Limits such as 100/hour per IP or per user, answered with 429
perform_create(serializer)	Adds server side values such as owner=request.user

Lesson: Authentication, permissions and throttling

Table: · 401 unauthorized · 403 forbidden

	401 Unauthorized	403 Forbidden
Means	We do not know who you are	We know who you are, and you may not
Example	No token, or a wrong token	Bob deletes ann's note
The client should	Log in and try again	Not try again

Lesson: Authentication, permissions and throttling

Table: way to show a relation · gives · good for

Way to show a relation	Gives	Good for
PrimaryKeyRelatedField (the default)	"author": 2	Small answers, the client looks up more if needed
StringRelatedField or source="author.name"	"author": "Jane Austen"	Showing a name, read only
A nested serializer	"author": {"id": 2, "name": ...}	Fewer requests for the client
HyperlinkedRelatedField	"author": "http://.../authors/2/"	APIs that link resources together

Lesson: Nested data and API tests

Table: rule · good · avoid

Rule	Good	Avoid
Nouns in URLs, HTTP methods as verbs	POST /api/orders/	POST /api/createOrder
Plural names, ids in the path	/api/books/7/	/api/book?id=7
Filters, search and order in the query string	/api/books/?author=2&ordering=-year	A different URL for each filter
Always paginate lists	count, next, previous, results	Returning 100,000 rows
Use the right status code	201 with the new object, 204 after delete	200 with {"error": ...}
Same error shape everywhere	{"field": ["message"]} or {"detail": "..."}	Plain text errors
Version when you break things	/api/v2/ or an Accept header	Changing fields under existing clients
Make retries safe	PUT and DELETE are idempotent, POST can take an Idempotency-Key	Charging a card twice on retry

Lesson: Designing a good API

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1408	String Matching in an Array	Easy	Check each word against the others with in	Like the SearchFilter of an API: a word is a match when it appears inside another word. Two loops and the in operator are enough for small lists.
1207	Unique Number of Occurrences	Easy	Counter, then compare the counts with a set	A validation rule in a serializer often counts things. Count each value with Counter, then the counts are unique when a set of them has the same size.
1640	Check Array Formation Through Concatenation	Easy	A dict from first number to piece	Like joining pages of API results back into one list. Each piece starts with a different number, so a dict from first number to piece finds the next piece at once.
911	Online Election	Medium	Precompute the leader after each vote, then binary search the time	Like an API that answers "what was true at time t". Work out the leader after every vote once, then each question is a binary search on the times.
2080	Range Frequency Queries	Medium	Positions per value, then two binary searches	Like an API filter ?value=4&from=1&to=2. Keep the sorted positions of each value, then count the positions inside the range with bisect.
2707	Extra Characters in a String	Medium	Dynamic programming from the end of the string	Like splitting a URL into known resource names. best[i] is the fewest extra letters from position i. Either skip the letter at i, or jump over a dictionary word that starts at i.
381	Insert Delete GetRandom O(1) - Duplicates allowed	Hard	A list plus a dict of index sets, swap with the last item to delete	Like a load balancer that picks a random server where some servers count twice. The list gives random choice in O(1), the dict of positions finds an item to remove, and swapping with the last item keeps removal O(1).
715	Range Module	Hard	A sorted list of disjoint intervals	Like a cache that remembers which ranges of ids it already fetched from an API. Keep the ranges sorted and merged, and use bisect to find the ones that touch a new range.
2276	Count Integers in Intervals	Hard	Merge intervals as they arrive and keep a running total	Like counting how many ids a set of API keys may read. Keep disjoint intervals sorted by start, merge every new one with the ones it touches, and update the total on the way.
1622	Fancy Sequence	Hard	Store one lazy formula a × x + b for every value	Like update(price=F("price") * 2 + 1) for every row at once: never touch each value. Keep one global formula and store each new value "undone" with the modular inverse, so the formula applies to it correctly later.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Response(data) is not JSON yet. The renderer turns it into JSON for programs and into the browsable HTML page for a person with a browser. That page lets you try the API by hand.

Lesson: Serializers and API views

IN THE EXAM

SearchFilter and OrderingFilter come with DRF. For filters such as ?year=1965&author=2, the django-filter add on reads them from a filterset_fields list. Always order the queryset, otherwise pages can show a row twice.

Lesson: ViewSets, routers, filters and pages

REMEMBER

Never trust ids or owners from the request body. The example ignores "owner": "bob" in the body and sets the owner from the token in perform_create. Mark such fields read only in the serializer.

Lesson: Authentication, permissions and throttling

5.2 FastAPI: fast, typed APIs with automatic docs

Pydantic models, typed parameters, response_model, Depends, async def and the event loop, docs and pytest.

KEY IDEAS

- **Endpoints** @app.get("/books/{book_id}") def get_book(book_id: int). Query parameters are plain parameters, Query(10, le=100) adds limits. A Pydantic model parameter is the body. response_model, status_code, HTTPException(404, "...").
- **Pydantic** class BookIn(BaseModel) with Field(gt=0), Field(min_length=...), Literal, nested models, @field_validator, @model_validator. model_dump(), model_dump_json(), model_json_schema(). Bad input is a 422 with locations.
- **Dependencies** Depends(get_db) and Annotated[T, Depends(f)]. yield for clean up. Header() for tokens, OAuth2PasswordBearer for logins. app.dependency_overrides in tests.
- **Async** async def only with awaitable libraries (httpx.AsyncClient, asyncpg). Plain def for blocking code, which runs in a thread pool. Never time.sleep or requests.get in async def. BackgroundTasks, lifespan.
- **Run and test** fastapi dev main.py, fastapi run or uvicorn main:app --workers 4. /openapi.json, /docs, /redoc. TestClient(app) with pytest fixtures and parametrize.

FORMULAS

Time with await — Exam favourite

$$\text{total} \approx \text{the longest single wait}$$

10 requests that each wait 0.2 s: about 0.2 s.

Lesson: The maths: concurrency, workers and timeouts

Time with a blocked loop — Exam favourite

$$\text{total} \approx n \times \text{wait}$$

10 × 0.2 = 2.0 s.

Lesson: The maths: concurrency, workers and timeouts

Requests one process can hold (Little's law) — Exam favourite

$$\text{in flight} = \text{rate} \times \text{time per request}$$

500 requests a second at 0.2 s each: 100 in flight at once.

Lesson: The maths: concurrency, workers and timeouts

Thread pool limit

$$\text{requests per second} \le \text{threads} / \text{time per request}$$

40 threads at 0.2 s: at most 200 plain def requests a second per process.

Lesson: The maths: concurrency, workers and timeouts

Workers

$$\text{workers} \approx 2 \times \text{CPU cores} + 1 \text{ (a common start for sync servers)}$$

4 cores: 9 workers. Async servers often use one worker per core.

Lesson: The maths: concurrency, workers and timeouts

Timeout budget

$$\text{total} \le \text{sum of the calls in a row}$$

Three calls of 0.3 s in a row: 0.9 s. In parallel with gather: about 0.3 s.

Lesson: The maths: concurrency, workers and timeouts

TABLES

Table: · fastapi · flask · django rest framework

	FastAPI	Flask	Django REST framework
Input checks	Pydantic from type hints	By hand or with an extension	Serializers
Docs	OpenAPI, Swagger UI and ReDoc built in	Extensions	Browsable API, OpenAPI with an add on
Async	Built on ASGI, async def everywhere	Mainly sync (WSGI)	Mainly sync, async views possible
Database and admin	Your choice (SQLAlchemy, SQLModel)	Your choice	Django ORM and admin
Good for	New API services, many waiting calls	Small apps and services	APIs next to a Django site

Lesson: What FastAPI is and why type hints matter

Table: tool · example · effect

Tool	Example	Effect
Type hint	quantity: int	"3" becomes 3, "abc" is an error
Field limits	Field(gt=0, le=10), Field(min_length=2), Field(pattern=...)	Ranges, lengths, patterns
Literal	size: Literal["S", "M", "L"] = "M"	Only these values, with a default
Nested model	address: Address	A dict is checked as an Address
field_validator	@field_validator("customer")	Your own rule or clean up for one field
model_validator	@model_validator(mode="after")	A rule across several fields
Output	model_dump(), model_dump_json()	A dict or JSON text
Schema	model_json_schema()	The JSON Schema that FastAPI puts in the docs

Lesson: Pydantic: data that checks itself

Table: where the value comes from · how you write it · example

Where the value comes from	How you write it	Example
Path	A name in {} in the route and a parameter	@app.get("/books/{book_id}") def get_book(book_id: int)
Query string	A simple parameter that is not in the path	min_year: int = 0, limit: int = Query(10, le=100)
JSON body	A parameter with a Pydantic model type	def add_book(book: BookIn)
Header or cookie	Header() or Cookie()	authorization: Annotated[str, Header()]
Answer shape	response_model and status_code	@app.post(..., response_model=BookOut, status_code=201)
Errors	raise HTTPException	HTTPException(404, "Book not found")

Lesson: Your first API: parameters, bodies and errors

Table: use of depends · example

Use of Depends	Example
A database session per request, closed after	def get_db(): db = … yield db … close
The current user from a token	def current_user(authorization: str = Header()) …
Shared query parameters	def paging(skip: int = 0, limit: int = 20)
A permission check	def admin_only(user = Depends(current_user)): if not user.is_admin: raise HTTPException(403)
Settings loaded once	def get_settings(): return Settings() with lru_cache
Fakes in tests	app.dependency_overrides[get_db] = fake_db

Lesson: Dependencies

Table: · async def with await · def (plain) · async def with a blocking call

	async def with await	def (plain)	async def with a blocking call
Example	await asyncio.sleep(0.2), await client.get(…)	time.sleep(0.2), requests.get(…)	time.sleep(0.2) inside async def
Runs on	The event loop	A thread pool	The event loop, which it blocks
10 requests at once	0.2 s	0.2 s	2.0 s
Verdict	Best with async libraries	Fine with normal libraries	Never do this

Lesson: async def, background tasks and start up

Table: tool · use it for

Tool	Use it for
BackgroundTasks	Small work after the answer, such as an email
A task queue (Celery, RQ, arq)	Heavy or important work that must survive a restart
lifespan	Open pools and load models once at start up, close them at shut down
httpx.AsyncClient with ASGITransport	Calling the app directly from async tests
WebSocket routes (@app.websocket)	Two way messages such as chat or live prices

Lesson: async def, background tasks and start up

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1122	Relative Sort Array	Easy	Sort with a key: (position in the order, value)	Like an API that lists fields in a fixed order and the rest A to Z after them. A dict gives each wanted value its position, and the sort key puts the others at the end.
1200	Minimum Absolute Difference	Easy	Sort, then only neighbours matter	After sorting, the closest pair is always next to each other, so one pass finds the smallest gap and the pairs that have it.
1287	Element Appearing More Than 25% In Sorted Array	Easy	Compare with the item a quarter further on	In a sorted list a value that fills more than a quarter must still be there a quarter of the length later. One loop, no counting.
1013	Partition Array Into Three Parts With Equal Sum	Easy	Walk once and count parts that reach a third	The total must split into three equal parts. Add numbers from the left and count each time the running sum reaches one third, then start again.
1237	Find Positive Integer Solution for a Given Equation	Medium	Two pointers on a function you can only call	You can only call the hidden function f(x, y), like a remote API that you want to call as few times as possible. f grows with x and with y, so start at x = 1 and y = 1000 and move one of them each call.
2069	Walking Robot Simulation II	Medium	Moves wrap around the border, so use the remainder	The robot only walks around the edge of the grid. One full lap is a fixed number of steps, so a huge step count shrinks to steps % lap, like a cursor that wraps around.
1801	Number of Orders in the Backlog	Medium	Two heaps: highest buy and lowest sell	An order book API: a buy matches the cheapest sell, a sell matches the highest buy. A min heap of sells and a max heap of buys (negated prices) give the best match at once.
1032	Stream of Characters	Hard	A trie of reversed words, checked from the newest letter	Like a WebSocket that gets one letter at a time and must say at once if the latest letters end a banned word. Store the words backwards in a trie and walk it from the newest letter back.
2642	Design Graph With Shortest Path Calculator	Hard	Dijkstra with a heap on every query	Like a gateway that finds the cheapest chain of services. Keep the edges in a dict of lists, let addEdge append, and run Dijkstra with a heap for each shortestPath question.
2286	Booking Concert Tickets in Groups	Hard	A segment tree with the max and the sum of free seats per range	A booking API must answer "first row with k free seats together" and "book k scattered seats" fast. A segment tree keeps the most free seats in a row and the total free seats for every range of rows.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Pydantic converts when it is safe: "3" becomes 3, but "3.5" for an int is an error, and so is the 13th month. Every problem is listed with its location, such as address.zip_code, which is exactly what FastAPI sends back in a 422 answer.

Lesson: Pydantic: data that checks itself

IN THE EXAM

Run the real server with `fastapi dev main.py` while you develop (it reloads on every save) and `fastapi run main.py` or `uvicorn main:app --workers 4` in production. Then open http://127.0.0.1:8000/docs.

Lesson: Your first API: parameters, bodies and errors

REMEMBER

For real logins FastAPI has OAuth2PasswordBearer, which reads "Authorization: Bearer ..." and also adds an Authorize button to /docs. The token is usually a JWT signed with a secret key (chapter 6.1).

Lesson: Dependencies

6.1 API security: passwords, JWT, OAuth 2.0, CORS and OWASP

Password hashing, JWTs by hand and with PyJWT, OAuth 2.0 scopes in FastAPI, CORS and CSRF, and the OWASP API Top 10.

KEY IDEAS

- **Passwords** hashlib.pbkdf2_hmac("sha256", pw, salt, 600_000) or Argon2id. Random salt per user, stored as algorithm$rounds$salt$hash. hmac.compare_digest. secrets.token_urlsafe(32), never random. Keys from os.environ, never in git.
- **JWT** header.payload.signature in base64url, HS256 = HMAC-SHA256. Readable by anyone. jwt.encode(claims, key, algorithm="HS256"). jwt.decode(token, key, algorithms=["HS256"], audience=..., issuer=...). Claims sub, exp, iat, nbf, aud, iss, jti. Short access, rotated refresh, kid for keys.
- **FastAPI auth** OAuth2PasswordBearer(tokenUrl="token", scopes=...), OAuth2PasswordRequestForm, Security(current_user, scopes=[...]). 401 with WWW-Authenticate: Bearer, 403 for missing scope, 404 for another user's object.
- **Browser rules** CORSMiddleware(allow_origins=[exact list], allow_credentials=True, allow_methods, allow_headers). Preflight OPTIONS. Never * with credentials. CSRF: SameSite cookies and CSRF tokens. Headers: HSTS, nosniff, Content-Security-Policy.
- **OWASP API Top 10** API1 object level checks, API2 authentication, API3 property level (mass assignment, extra="forbid"), API4 resource limits, API5 function level, API7 SSRF. ? placeholders against injection. Rate limits with 429 and Retry-After.

FORMULAS

Entropy of a random secret — Exam favourite

$$\text{bits} = \text{length} \times \log_2(\text{alphabet size})$$

12 random lower case letters: $12 \times \log_2 26 \approx 56.4$ bits.

Lesson: The maths: guessing, entropy, hashing cost and rate limits

Guesses to try every value — Exam favourite

$$\text{guesses} = 2^{\text{bits}}$$

A 6 digit code: $10^6 = 1{,}000{,}000$ values.

Lesson: The maths: guessing, entropy, hashing cost and rate limits

Time to crack — Exam favourite

$$\text{time} = \text{guesses} \times \text{cost per guess} / \text{machines}$$

With PBKDF2 at 0.12 s, one million guesses take 120,000 s on one core.

Lesson: The maths: guessing, entropy, hashing cost and rate limits

Chance to guess with a rate limit

$$P = \text{attempts} / \text{values}$$

5 tries on a 6 digit code: $5 / 1{,}000{,}000 = 0.0005$ %.

Lesson: The maths: guessing, entropy, hashing cost and rate limits

Token bucket

$$\text{allowed in t seconds} \le \text{capacity} + \text{rate} \times t$$

Capacity 10, rate 2 per second: at most $10 + 2 \times 60 = 130$ requests in a minute.

Lesson: The maths: guessing, entropy, hashing cost and rate limits

Birthday bound for random ids

$$P(\text{any repeat}) \approx n^2 / 2^{\text{bits} + 1}$$

1 billion random 128 bit ids: about $10^{18} / 2^{129}$, practically zero.

Lesson: The maths: guessing, entropy, hashing cost and rate limits

TABLES

Table: · session cookie · jwt bearer token · api key

	Session cookie	JWT bearer token	API key	OAuth 2.0
What the client holds	A random session id in a cookie	A signed token with claims	A long random string	An access token from an authorization server
What the server keeps	A session table	Only the secret key	A table of hashed keys	Depends on the token type
Log out at once	Easy: delete the session	Hard: wait for exp or keep a deny list	Delete the key	Revoke at the authorization server
Watch out for	CSRF, so use SameSite and CSRF tokens	Long lifetimes, alg confusion, storing in localStorage	Keys leaked in code or logs	Using the wrong flow
Good for	Websites with server pages (Django, Flask)	APIs and mobile apps	Server to server calls	Letting another app act for a user

Lesson: What API security is and why every endpoint needs it

Table: way to store a password · verdict · why

Way to store a password	Verdict	Why
Plain text	Never	One leak shows every password, and people reuse them on other sites
Encrypted	Never	Whoever has the key can read them all
sha256(password)	Never	Billions of guesses a second on a graphics card, and the same password gives the same hash
sha256(salt + password)	Weak	The salt stops shared tables, but the hash is still fast
PBKDF2, bcrypt, scrypt, Argon2id	Yes	Salted and slow on purpose. Argon2id and scrypt also need memory

Lesson: Storing passwords and secrets

Table: part · what it holds · example

Part	What it holds	Example
Header	The algorithm and the type	{"alg": "HS256", "typ": "JWT"}
Payload (claims)	Who, what and until when	sub (user), exp (expiry), iat (issued), nbf (not before), aud (audience), iss (issuer), jti (id), scope
Signature	HMAC-SHA256 of header.payload with the secret key	base64url of 32 bytes
Encoding	base64url without = padding, parts joined with dots	eyJhbGci... . eyJzdWIi... . Q46Cf6DE...

Lesson: JSON Web Tokens from scratch

Table: practice · why

Practice	Why
Pin algorithms=["HS256"] when decoding	Stops alg confusion and alg none tricks
Short access tokens (5 to 15 minutes)	A stolen token is useful only for minutes
Long refresh tokens that work once (rotation)	A second use shows theft, so all sessions can be ended
Check aud and iss	A token for another API or from another issuer is refused
kid in the header with a key table	Keys can be swapped without logging everyone out
jti and a deny list for log out	A JWT cannot be deleted, so remember the ids of revoked ones until they expire
Keep tokens out of URLs and logs	URLs end up in browser history, proxies and log files

Lesson: Real tokens with PyJWT: expiry, refresh and key rotation

Table: oauth 2.0 flow · who uses it · notes

OAuth 2.0 flow	Who uses it	Notes
Authorization code with PKCE	Web, mobile and single page apps logging users in	The standard choice today. PKCE stops a stolen code from being used
Client credentials	A server calling another server, with no user	The client id and secret give a token
Device code	TVs and command line tools	The user types a short code on another device
Password (resource owner)	Only your own first party app	Legacy. The FastAPI tutorial uses it because it is simple
Implicit	Nobody	Deprecated. Tokens in the URL leaked

Lesson: OAuth 2.0 logins, scopes and owner checks in FastAPI

Table: · 401 unauthorized · 403 forbidden

	401 Unauthorized	403 Forbidden
Meaning	We do not know who you are	We know who you are, and the answer is no
Typical cause	No token, bad signature, expired	Missing scope or role
Client should	Log in or refresh the token	Not retry, ask for access

Lesson: OAuth 2.0 logins, scopes and owner checks in FastAPI

Table: · cors · csrf

	CORS	CSRF
What it is	Rules that let a server allow other sites to READ its answers in a browser	An attack where another site makes your browser SEND a request with your cookies
Who enforces it	The browser, from the server's headers	The server, with tokens and cookie settings
When it matters	Any API called from a page on another origin	When the login is a cookie the browser adds by itself
The fix or setting	An exact list of origins, never * with credentials	SameSite=Lax or Strict cookies plus a CSRF token (Django has it built in)
Bearer tokens	Still needed	Not a risk: the browser never adds the Authorization header by itself

Lesson: CORS, CSRF and security headers

Table: owasp api 2023 · what goes wrong · python fix

OWASP API 2023	What goes wrong	Python fix
API1 Broken Object Level Authorization	Change the id in the URL and read someone else's data	Check the owner on every object (filter by user in the query)
API2 Broken Authentication	Weak passwords, no rate limit on login, tokens that never expire	Slow hashes, login limits, short exp, pinned algorithms
API3 Broken Object Property Level Authorization	Mass assignment (is_admin from the body) or returning secret fields	Input models with extra="forbid", response_model for output
API4 Unrestricted Resource Consumption	Huge pages, huge uploads, no rate limit	Cap limit, body size and requests per client
API5 Broken Function Level Authorization	A normal user can call an admin endpoint	Role and scope checks as dependencies
API6 Unrestricted Access to Sensitive Business Flows	Bots buy every ticket or create many accounts	Limits per account, device and payment, CAPTCHA
API7 Server Side Request Forgery	The API fetches a user URL that points inside the network	https only, block internal addresses, an allowlist
API8 Security Misconfiguration	Debug pages, open CORS, missing headers, default keys	DEBUG off, exact CORS list, security headers
API9 Improper Inventory Management	An old /v1 or a test server nobody patches	A list of every API version and host, retire old ones
API10 Unsafe Consumption of APIs	Trusting data from a partner API without checks	Validate it with models, use timeouts, follow no redirects blindly

Lesson: The OWASP API Security Top 10

Table: control · how in python · notes

Control	How in Python	Notes
HTTPS everywhere	TLS at the proxy (nginx, a cloud load balancer), HSTS header	Tokens and passwords travel in every request
Rate limits	A token bucket per client, often in Redis, or slowapi for FastAPI and DRF throttles	Send 429 Too Many Requests with Retry-After
Login limits	Count failures per account and per IP, slow down or lock for a while	Stops password guessing
Secrets	os.environ, a vault, never in git, rotate on a schedule	Scan commits with a secret scanner in CI
Logging	Log user id, path, status and time, never passwords or tokens	Mask Authorization headers in error reports
Dependencies	pip-audit or the GitHub dependency alerts	Most vulnerabilities come from old libraries
Least privilege	A database user that can only do what the app needs	A leak then exposes less

Lesson: Rate limits, HTTPS, logging and secrets in production

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2325	Decode the Message	Easy	Build a substitution table from first appearances	This is a simple substitution cipher. The first time each letter appears in the key it stands for the next letter of the alphabet. A dict holds the table, and spaces stay as they are. Ciphers like this are easy to break, which is why real systems use keys and HMAC instead.
1844	Replace All Digits with Characters	Easy	Shift a letter by a digit	Every digit says how far to move the letter before it, the same idea as a Caesar shift. chr(ord(letter) + digit) does the move.
1528	Shuffle String	Easy	Put each character at its target index	A transposition: indices[i] says where s[i] belongs. Fill a list of the right length, then join it.
1461	Check If a String Contains All Binary Codes of Size K	Medium	A set of every window of length k	There are 2^k codes of length k, the same count an attacker must try to guess a k bit token. Put every window of the string in a set and compare its size with 2^k.
187	Repeated DNA Sequences	Medium	Two sets: seen once and seen again	Slide a window of 10 letters and remember each one in a set. A window that is already there is a repeat, like a replayed request id that a server must refuse. The answer is sorted here so the tests can compare it.
1980	Find Unique Binary String	Medium	Cantor diagonal: differ from string i at place i	Take bit i of string i and flip it. The new string differs from every given string in at least one place, so it cannot be any of them. The same trick proves that some sets cannot be listed.
721	Accounts Merge	Medium	Union find over e-mail addresses	Two accounts are one person when they share an e-mail, like linking identities in a login system. Union find joins every e-mail with the first e-mail of its account, then each group becomes one sorted account. The list is sorted at the end so the tests can compare it.
420	Strong Password Checker	Hard	Count runs, then spend deletions where they save a replacement	A strong password has 6 to 20 characters, a lower case letter, an upper case letter and a digit, and never three equal characters in a row. Short passwords only need inserts. Long ones need deletes, and a delete in a run whose length is a multiple of 3 saves one replacement, so spend deletes on those runs first, then on runs with remainder 1 (two deletes) and then three at a time.
2156	Find Substring With Given Hash Value	Hard	Rolling hash, walking from the right	The hash gives the first letter the power p^0, so it is easy to add a letter on the LEFT: multiply by p and add the new value. Walk from the end of the string, drop the letter that falls out of the window, and remember the leftmost window that matches. Rolling hashes also power rsync and duplicate file checks.
1392	Longest Happy Prefix	Hard	The KMP failure table	A happy prefix is also a suffix. The prefix function of KMP stores, for each position, the longest proper prefix that ends there, and its last value is the answer. It runs in O(n), with no hash collisions to worry about.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Keep secret keys out of the code: read them with `os.environ["SECRET_KEY"]`, keep a local .env file in .gitignore, and use a vault (AWS Secrets Manager, HashiCorp Vault) in production. A key that was ever pushed to git is burnt: rotate it, because deleting the commit does not remove copies.

Lesson: Storing passwords and secrets

REMEMBER In a browser, an HttpOnly, Secure, SameSite cookie is the safest place for a token, because JavaScript (and so an XSS bug) cannot read it. localStorage is readable by any script on the page.

Lesson: Real tokens with PyJWT: expiry, refresh and key rotation

6.2 Web revision exam: HTTP, Flask, Django, FastAPI and security

Four papers that run in the page, eight bugs, the web traps, a design answer and a mock quiz.

KEY IDEAS

- **HTTP** Status classes 2xx, 3xx, 4xx, 5xx. 201 with Location, 204, 301, 304, 400, 401, 403, 404, 405, 412, 422, 429. Idempotent: GET, PUT, DELETE. requests.get(url, timeout=5) and raise_for_status().
- **Flask** @app.get("/books/<int:book_id>"), request.args.get("page", 1, type=int), request.get_json(), return dict, status, headers. g, session, Blueprint(url_prefix=...), url_for, errorhandler, test_client().
- **Django and DRF** Lazy QuerySets, Q, F, annotate, select_related and prefetch_related, makemigrations and migrate, reverse and resolve, autoescape. Serializer.is_valid() then validated_data, ModelViewSet with a router.
- **FastAPI** Type hints decide path, query and body. Pydantic gives 422 with locations. response_model, Depends with yield, async def only with await, TestClient and dependency_overrides.
- **Security** Slow salted hashes, JWT with pinned algorithms and short exp, scopes give 403, owner checks give 404, CORS for browsers, ? placeholders, caps on page size, secrets in the environment.

FORMULAS

Codes available Exam favourite

$$\text{codes} = \text{alphabet}^{\text{length}}$$

$62^7 \approx 3.52 \times 10^{12}$.

Lesson: A system design answer: a link shortener API

Storage per year

$$\text{rows} = \text{writes per second} \times \text{seconds per year}$$

$100 \times 31{,}536{,}000 \approx 3.2 \times 10^9$ rows.

Lesson: A system design answer: a link shortener API

Load on the database with a cache

$$\text{reads to the database} = \text{reads} \times (1 - \text{hit rate})$$

$10{,}000 \times (1 - 0.99) = 100$ a second.

Lesson: A system design answer: a link shortener API

TABLES

Table: paper · covers · chapters

Paper	Covers	Chapters
A	URLs, query strings, encoding, status codes, idempotency, cookies, JSON and raw HTTP	1.1 HTTP, 1.2 sockets, 2.1 requests, 2.2 API data
B	Flask routes, converters, 404 and 405, JSON bodies, query arguments, g, sessions, blueprints, url_for	3.1 Flask, 3.2 Flask apps
C	Django QuerySets, Q and F, annotate, JOINs, URLs, templates and autoescape, DRF serializers	4.1 Django, 4.2 ORM, 5.1 DRF
D	Pydantic conversion and errors, nested models, model_dump, PyJWT checks, hashes, secrets and IP checks	5.2 FastAPI, 6.1 API security

Lesson: How to use this revision chapter

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	https api.shop.example 8443 /v2/books q=python&page=2 top	urlsplit gives each part. The port is an int, and the fragment after # is never sent to the server (1.1)
2	{'tag': ['a', 'b']} ['']	parse_qs always gives lists and drops empty values unless keep_blank_values=True (2.1)
3	q=tea+%26+cake&page=1 /a%20b/ %2Fa%20b%2F	urlencode writes a space as + and & as %26. quote keeps / unless safe="" (2.1)
4	https://x.example/api/v1/books https://x.example/api/books https://x.example/books	Without the trailing slash, the last part of the base is replaced. A leading / starts again at the root (2.1)
5	[201, 404] Too Many Requests [2, 3, 4, 5]	The class of a status code is code // 100: 2 success, 3 redirect, 4 client error, 5 server error (1.1)
6	['PATCH', 'POST']	GET, HEAD, OPTIONS, PUT and DELETE are idempotent: repeating them changes nothing more. POST and PATCH are not (1.1)
7	abc123 ['session', 'theme']	A Cookie header holds name=value pairs separated by semicolons, and SimpleCookie reads them (1.1)
8	{"price": 9.5, "tags": ["a", "b"], "ok": null, "1": "one"} one	JSON turns a tuple into a list, None into null and the int key 1 into the string "1" (2.2)
9	['HTTP/1.1', '301', 'Moved Permanently'] /new	A raw response is a status line, headers and a blank line. A 301 sends the new address in Location (1.2)
10	5 4 True	é takes 2 bytes in UTF-8, and Content-Length counts bytes. Percent encoding also works on the UTF-8 bytes (1.2)

Lesson: Paper A: HTTP, URLs and JSON

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	200 {'id': 2, 'title': 'Dune'}	A view that returns a dict is sent as JSON (3.1)
2	404 {'error': 'not found', 'path': '/books/9'}	"two" does not fit int:, so no route matches and the answer is 404, not 400. The 404 handler formats both cases (3.1)
3	405 405	The path exists but not for DELETE or POST: 405 Method Not Allowed (3.1)
4	201 {'id': 3} /books/3	A view can return (body, status, headers). url_for builds the Location from the view name (3.1)
5	{'error': 'title is required'}	get_json(silent=True) returns None for a body that is not JSON instead of failing (3.1)
6	{'page': 1, 'q': 'tea'} {'page': 3, 'q': None}	type=int falls back to the default when the value is not a number. A missing argument is None. Flask sorts the keys of JSON answers (3.1)
7	{'user': 'ann', 'visits': 2}	before_request fills g for each request. The session cookie from the first call came back with the second (3.2)
8	1	A new test client has no cookie, so it starts a new session (3.2)
9	{'books': 3} 404	The blueprint adds the prefix /admin, so /stats alone does not exist (3.2)
10	/books/7 /admin/stats /search?q=a+b&page=2	Blueprint views are named blueprint.view. Arguments that are not in the rule become the query string (3.1)

Lesson: Paper B: Flask

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	QuerySet 2 ['Dune', 'Ubik']	filter gives a lazy QuerySet. year__gt is a field lookup, and SQL runs only when the values are needed (4.2)
2	<QuerySet ['Dune', 'Emma']>	Q objects joined with | make an OR. A slice of a QuerySet is still a QuerySet (4.2)
3	[2, 0, 4]	update with F("stock") - 1 runs one SQL UPDATE, only on the rows with stock above 0 (4.2)
4	[('Bob', 2), ('Ann', 1)]	annotate(Count("books")) adds a count per author, using the related_name (4.2)
5	True 1	Filtering across the foreign key adds a JOIN. all() is a single SELECT (4.2)
6	/books/7/ {'pk': 7} book-detail	reverse turns a name into a URL and resolve turns a URL back into the view and its arguments (4.1)
7	EMMA, DUNE hi	upper is a filter, forloop.last skips the last comma, and autoescape turns < into < so user text cannot inject HTML (4.1)
8	False ['title', 'year']	Every field is checked and all errors are collected: the title is longer than 10 and 1200 is below 1450 (5.1)
9	True {'title': 'Emma', 'year': 1815, 'stock': 0}	"1815" becomes the int 1815, and the default fills the missing stock (5.1)
10	200 {'pk': 42} 404	int: converts 42 and refuses x, which gives 404 (4.1)

Lesson: Paper C: Django and Django REST framework

Table: q · output · why (chapter)

Q	Output	Why (chapter)
1	3 False	"3" becomes 3. Pydantic copies a mutable default, so each object gets its own list (5.2)
2	2 ['qty', 'size']	"3.5" is not a whole number and XL is not one of the Literal values. Pydantic lists every error (5.2)
3	('ship_to', 'zip_code')	Errors in nested models have the full path, which FastAPI puts after body in a 422 answer (5.2)
4	{'id': 1, 'ship_to': {'city': 'Bonn', 'zip_code': '53111'}} {"id":1}	exclude_none drops note. include keeps only the named fields, and model_dump_json writes compact JSON (5.2)
5	eyJ 2 eyJh	Every JWT starts with eyJ because it is base64 of {" and has three parts, so two dots (6.1)
6	DecodeError, InvalidAudienceError, ok	PyJWT 2 refuses to decode without algorithms=. A token with aud needs audience= as well (6.1)
7	32 64 32 22	SHA-256 gives 32 bytes, which is 64 hex digits. token_hex(16) gives 32 characters and token_urlsafe(16) gives 22 (6.1)
8	True True	The same key and message always give the same HMAC. compare_digest only changes how long the comparison takes (6.1)
9	[True, False, True, False, True]	172.16.0.0/12 ends at 172.31.255.255, so 172.32.0.1 is public. ::1 is the IPv6 loopback (6.1)
10	True False	A hash is the same for the same input. Different salts give different results, which defeats precomputed tables (6.1)

Lesson: Paper D: Pydantic, tokens and security

Table: trap · looks like · really · chapter

Trap	Looks like	Really	Chapter
URL fragment	Sent to the server	Stays in the browser	1.1
PUT and PATCH	The same	PUT replaces the whole resource and is idempotent, PATCH changes part of it	1.1
requests.get(url)	Gives up after a while	Waits for ever without timeout=	2.1
A 404 or 500 from requests	Raises an error	Only raise_for_status() raises	2.1
JSON keys	Keep their type	Always become strings	2.2
Flask int: converter	Bad values give 400	They give 404 because no route matches	3.1
A route without methods=	Accepts every method	Only GET (and HEAD, OPTIONS)	3.1
Flask debug=True	Fine on a server	The debugger can run code: never in production	3.2
QuerySet	Runs SQL when made	Runs when looped, sliced with a step, counted or printed	4.2
book.author in a loop	One query	One query per book (N + 1): use select_related	4.2
|safe in a template	Just formatting	Turns off autoescape: XSS if the text came from a user	4.1
serializer.data before is_valid()	Works	Raises an AssertionError for input data	5.1
time.sleep in async def	Waits only this request	Blocks every request on the event loop	5.2
A JWT payload	Secret	Readable by anyone, only signed	6.1
CORS	Protects the API	Only tells browsers which sites may read answers	6.1
A valid token	Can read anything	Still needs the owner check on every object	6.1

Lesson: The traps in one table

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1629	Slowest Key	Easy	One pass over the gaps between times	Like finding the slowest request in an access log: each duration is the gap to the previous time. Keep the longest, and on a tie the larger letter.
1701	Average Waiting Time	Medium	Simulate one worker with a clock	One server handles requests in the order they arrive. The clock is the time the server becomes free. A request starts at max(clock, arrival), and its waiting time is the finish time minus the arrival.
210	Course Schedule II	Medium	Topological sort with in-degrees (Kahn)	The same problem as running database migrations or loading apps in dependency order. Start with every item that depends on nothing, and each time one is done, lower the count of the items waiting for it. A cycle leaves items behind, so the answer is empty.
399	Evaluate Division	Medium	A weighted graph and a search	Like a currency API that only knows some rates: a / b = 2 is an edge from a to b with weight 2 and from b to a with 0.5. A query multiplies the weights along any path, or gives -1.0 when there is none.
1942	The Number of the Smallest Unoccupied Chair	Medium	Two heaps: free chairs and busy chairs by leaving time	This is exactly how a connection pool hands out the lowest free connection number. Go through arrivals in time order, first free every chair whose guest has left, then give the smallest free one.
1438	Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit	Medium	Sliding window with a max deque and a min deque	Like finding the longest stretch where response times stayed within a band. Two deques keep the window's maximum and minimum in O(1). When max minus min is too big, move the left edge.
752	Open the Lock	Medium	Breadth first search over 10,000 states	A 4 digit lock has only 10^4 codes, which is why short codes need attempt limits. BFS from 0000 turns one wheel up or down at each step and skips the dead ends, so the first time it reaches the target is the fewest turns.
2050	Parallel Courses III	Hard	Longest path in a DAG, in topological order	Like a deployment pipeline where jobs run in parallel once their needs are done: a job finishes at its own time plus the latest finish among the jobs it waits for. The answer is the latest finish of all.
1235	Maximum Profit in Job Scheduling	Hard	Sort by end time, DP with binary search	Like choosing which paid batch jobs a single worker should run. Sort jobs by end time. The best profit up to a job is either the best without it, or its profit plus the best among jobs that end by its start, found with bisect.
630	Course Schedule III	Hard	Greedy by deadline with a max heap of durations	Take tasks in order of their deadline. When the total time passes a deadline, drop the longest task taken so far, because that frees the most time while keeping the count. A max heap (negative numbers in heapq) finds it.

Lesson: LeetCode review set

TIPS AND TRAPS

REMEMBER

Cover the answers, write down what you think each line prints, then open the answers. Every wrong guess points to the chapter in brackets. All four papers run in this page, so press Run to see the real output too.

Lesson: How to use this revision chapter

IN THE EXAM

In a written exam, trace each snippet on paper before you look at the options, and watch for the answers "404", "405" and "the code raises". In an interview, say your assumptions out loud, start with the simplest working API, then add caching, security and limits.

Lesson: Exam technique and a study plan